IA Causal en Producción

Monografía del Curso

*De la identificación al despliegue: estimar **efectos**, no solo predicciones, y poner políticas causales en producción en Hugging Face*

Curso ECO-490 / ECON-540
Profesionales y posgrado aplicado (data / ML / product / analytics)
Modalidad online/asíncrona · 32 sesiones · 32 capítulos
Edición 2026

Promesa. Deja de priorizar con scores que confunden correlación con impacto. Estima efectos, documenta supuestos y despliega en Hugging Face — con monitoreo de cuándo el efecto deja de ser válido.

Monorepo del curso: labs L01–L28 · Serie A (casos) · Serie B (HF) · benchmark de datos
Stack: `econml` / `sklearn` · `FastAPI` · `Gradio` · `huggingface_hub` · `Docker`

Machine Learning Causal para Economistas

Monografía del Curso ECO-490 / ECON-540

Edición 2026

© 2026 Curso ECO-490 / ECON-540
Todos los derechos reservados.

Primera edición, 2026.

Compuesto con Typst 0.13.1.
Tipografía: New Computer Modern (serif), DejaVu Sans (sans-serif), DejaVu Sans Mono (mono-espaciada).

Basado en el repositorio `causal-inference-benchmarks-econ`.
Stack Python: `econml`, `DoubleML`, `causalml`, `dowhy`, `PyTorch`.

Esta monografía es material de referencia para uso docente.
La reproducción total o parcial requiere citación adecuada.

Prefacio

Esta monografía es el material de referencia formal del curso *IA Causal en Producción*. Está pensada, en primer lugar, para **profesionales** —analistas de datos, ingenieros de ML, product analytics, riesgo, operaciones y perfiles afines— que ya usan modelos predictivos y deben **decidir o priorizar intervenciones**. También sirve a posgrado aplicado y a economistas de último año con base cuantitativa. El curso **no** está diseñado como formación de investigadores básicos: el camino crítico prioriza **casos de estudio**, **estimación de efectos accionables** y **puesta en producción en Hugging Face**.

Se asume familiaridad con regresión, evaluación de modelos y, de forma deseable, nociones de evaluación de impacto (resultados potenciales, confusión). Quien tenga un curso previo de causalidad aplicada (RCT, matching/IPW, IV, DiD) avanzará más rápido; quien no, puede apoyarse en los Caps. 1–3 y en los labs del monorepo. El libro **extiende** ese bagaje al régimen de alta dimensionalidad, heterogeneidad de efectos y —sobre todo— a la ingeniería de sistemas necesaria para **operar** esos métodos en producción.

El eje conceptual combina tres tesis. Primera: *machine learning no identifica causalidad por sí solo*. Los supuestos de identificación siguen siendo del dominio y deben justificarse; el ML flexibiliza la estimación de componentes *nuisance* (propensity, outcome models) sin sacrificar inferencia válida, vía **ortogonalización de Neyman**, **cross-fitting** y **honestidad** en árboles. Segunda: *un estimador causal validado no es, todavía, un sistema*. Documentarlo, servirlo, versionarlo y monitorearlo es trabajo de ingeniería (Huyen, 2022; filosofía DMLS). Tercera —el mensaje de **disrupción** del curso—: *priorizar por score predictivo no es lo mismo que priorizar por efecto*. Un buen predictor responde «quién tiene peor outcome»; un sistema causal responde «a quién **cambia** el outcome si actuamos». Esa diferencia reordena presupuestos, targeting y riesgo reputacional. Cada bloque del curso cierra con la pregunta: **si solo predijéramos, ¿a quién trataríamos? Con el efecto, ¿cambia la decisión?**

Esta doble tesis se refleja en dos plantillas de capítulo, no una. Los capítulos de teoría e identificación causal siguen la estructura de siete componentes de la edición anterior: (i) problema, estimando e identificación; (ii) marco teórico formal con supuestos, teoremas y garantías; (iii) algoritmo e implementación en Python; (iv) caso empírico; (v) riesgos, límites y sensibilidad; (vi) ejercicios; (vii) notas bibliográficas. Los capítulos de ingeniería y producción, nuevos en esta edición, siguen en cambio una estructura de siete componentes centrada en decisiones de diseño con costos y beneficios explícitos, no en una única receta: (i) el problema de diseño; (ii) opciones y trade-offs; (iii) decisión de referencia para el curso; (iv) implementación de referencia; (v) aplicación al sistema causal del curso; (vi) señales de alerta y checklist

operativo; (vii) ejercicios y notas bibliográficas. El Capítulo 1 explica esta distinción con más detalle.

A quién va dirigida. (1) Profesionales de datos/ML/producto que deben llevar **decisiones de intervención** a un servicio o Space. (2) Analistas de organismos, banca, consultoría y empresas que estiman efectos con muchos controles. (3) Estudiantes de pregrado avanzado o posgrado aplicado. Los Caps. 17, 27 y 28 (panel/DML-IV, deep causal, matrix completion) son **profundización electiva**: el profesional aprueba el camino crítico sin dominarlos.

Convenciones. La notación sigue el estándar de la literatura: $Y(d)$ denota el resultado potencial bajo tratamiento $d \in \{0, 1\}$, D es la asignación observada, X son covariables pre-tratamiento. El estimando central es el CATE $\tau(x) = E[Y(1) - Y(0) \texttt{ mid } X = x]$, cuyo promedio $\theta = E[\tau(X)]$ es el ATE. Los algoritmos se presentan en pseudocódigo de complejidad similar al de los *papers* originales; las implementaciones en Python siguen el estilo idiomático de `econml` y `DoubleML`.

La tabla completa de notación se encuentra en la sección siguiente.

Monorepo del curso. La implementación de referencia (labs L01–L28, notebooks Serie A/B, datos de benchmark, autograders) vive junto a esta monografía en el repositorio del curso. La tabla sesión ↔ capítulo ↔ lab ↔ entrega y el posicionamiento profesional se detallan en `PLAN_INTEGRACION_MONOGRAFIA.md` y en el sílabo. Cuando un listado de código del libro use un diseño idealizado (p. ej. Parquet + `CausalSchema`), el monorepo documenta el **equivalente operativo** (CSV + `metadata.json` + `lib.data_io`).

Agradecimientos. Esta monografía se apoya en el ecosistema de benchmarks del monorepo (`data_benchmark_causal`), y en tres fuentes de ingeniería y teoría: *Designing Machine Learning Systems* (Huyen), *Causal Artificial Intelligence* (Bareinboim) y *Machine Learning in Production* (CMU). Agradecemos a los autores de los trabajos metodológicos citados a lo largo del texto.

Cómo usar este libro

Esta monografía está organizada en nueve partes temáticas que agrupan los treinta y dos capítulos del curso. A diferencia de la edición anterior, no todos los capítulos siguen la misma plantilla: el libro alterna entre capítulos de teoría e identificación causal (Plantilla A) y capítulos de ingeniería y producción (Plantilla B), diseñados para enseñarse de forma distinta porque responden preguntas de naturaleza distinta.

Dos plantillas de capítulo

Plantilla A — teoría e identificación causal. Sigue la estructura de siete secciones ya usada en la edición anterior del libro: (i) problema, estimando e identificación; (ii) teoría, supuestos y garantías, con definiciones y teoremas formales; (iii) algoritmo e implementación en Python; (iv) caso empírico; (v) riesgos, límites y sensibilidad; (vi) ejercicios; (vii) notas bibliográficas. Responde la pregunta «¿cómo se estima esto correctamente?».

Plantilla B — ingeniería y producción. Nueva en esta edición, sigue la filosofía de *Designing Machine Learning Systems* (Huyen, 2022): no es una plantilla de recetas únicas sino de decisiones de diseño con costos y beneficios explícitos. Sus siete secciones son: (i) el problema de diseño; (ii) opciones y trade-offs, comparando al menos dos alternativas reales; (iii) la decisión de referencia adoptada para el sistema de ejemplo del curso; (iv) una implementación de referencia (código, configuración); (v) la aplicación específica al sistema causal del curso; (vi) señales de alerta y checklist operativo; (vii) ejercicios y notas bibliográficas. Responde la pregunta «¿cómo se lleva esto a producción de forma responsable?».

Ambas plantillas comparten el mismo cuadro resumen al inicio del capítulo (llamado **cuadro sinóptico** en los capítulos de Plantilla A y **cuadro de decisión** en los de Plantilla B) y el mismo sistema de referencias cruzadas, numeración de figuras/tablas/listados por capítulo, y checklists.

Partes del libro

Parte	Contenido	Caps.
0	Introducción: causalidad elemental y anatomía de un sistema de ML en producción.	1–2

Parte	Contenido	Caps.
I	Fundamentos estructurales y de datos: SCM, Jerarquía Causal de Pearl, regularización, criterio de puerta trasera, ingeniería de datos y **features** sin fuga causal.	3–9
II	Estimación causal flexible y primer despliegue: ortogonalidad, DML, meta-learners, bosques causales, validación, y el primer servicio desplegado del curso.	10–15
III	Diseños de identificación con ML: Post-Double-Selection, panel e instrumentos, mapa de decisión metodológica.	16–18
IV	Publicación y operación en producción: **model cards** causales en Hugging Face Hub, CI/CD.	19–20
V	Heterogeneidad, políticas y **targeting**: **policy learning**, **uplift modeling**.	21–22
VI	Explicación, equidad y decisión estructural: descomposición causal de la equidad, interpretabilidad, decisiones bajo incertidumbre, transportabilidad.	23–26
VII	Métodos avanzados: **deep learning** causal, control sintético y **matrix completion**.	27–28
VIII	Estructura, monitoreo y frontera: descubrimiento de grafos, monitoreo en producción, aprendizaje continuo, panorama de RL causal, y comunicación de resultados.	29–32

Elementos pedagógicos

El libro utiliza los siguientes elementos visuales recurrentes:

- **Cuadros sinópticos / de decisión:** resumen ejecutivo del capítulo.
- **Algoritmos:** pseudocódigo numerado globalmente (capítulos de Plantilla A).
- **Opciones y trade-offs, decisión de referencia:** comparación explícita de alternativas de diseño (capítulos de Plantilla B).
- **Tablas de riesgos:** riesgos, consecuencias, diagnósticos y mitigaciones.
- **Checklists:** listas de verificación de validación o de señales de alerta operativas.
- **Notas técnicas:** detalles matemáticos opcionales o enlaces entre capítulos.
- **Apéndices:** fundamentos matemáticos (A), guía de librerías (B), datos y DGP (C), glosario (D), plantilla de proyecto (E), checklists por método (F), plantilla de **model card** causal (G) y checklist de arquitectura de despliegue (H).

Audiencia profesional y camino crítico

Este libro se lee junto al **monorepo del curso**. El camino crítico para profesionales (★) prioriza: identificación y datos (Caps. 1–4, 6–9) → DML y validación (10–11, 14) → primer

servicio HF (15) → CATE y policy (12–13, 21–22) → Hub/CI (19–20) → explicación y decisión (23–26) → monitoreo y comunicación (29–32). Los Caps. **16–18** y **27–28** son profundización electiva (◇): útiles, pero no bloquean desplegar un sistema en Hugging Face.

Pregunta de valor (repetir en cada bloque). Si solo predijéramos el outcome, ¿a quién trataríamos? Si estimamos el efecto, ¿cambia el ranking, el presupuesto o el riesgo?

Mapa sesión → capítulo → lab → entrega (resumen)

La tabla completa está en el sílabo y en `PLAN_INTEGRACION_MONOGRAFIA.md`. Resumen del hilo de producto:

Sesiones	Caps. monografía	Lab	Entrega	Valor de negocio
S1–S3	1–3	L01–L03	—	Asociación vs efecto; PCH
S4–S6	4, 6–7	L04–L06	**A1**	¿Es estimable el programa?
S7–S11	8–11, 14	L07–L11	**A2**	ATE con IC; sin leakage
S12–S14	12–13, 15	L12–L14	**B1** + A3	API del efecto; CATE
S15–S16	repaso	L15	**A3** · parcial	Sistema causal integrado
S17–S18	19–20	L17–L18	**B2**	Hub + model card + CI
S19–S24	21–26	L19–L24	**A4** · **B3**	Policy + Space HF
S25–S28	29–32	L25–L28	**B4**	Monitoreo y comunicación
S29–S32	Ap. E/G/H	P1–P4	**Proyecto**	Tu caso en producción HF

Dos series de notebooks y el monorepo

El trabajo práctico se organiza en el monorepo del curso (no en un repositorio externo opcional):

- **Labs L01–L28** (`05_labs_sesion/`): práctica formativa por sesión (30–50 min).
- **Serie A** (`03_notebooks_caso_estudio/`): A1 identificación (Jobs) · A2 DML · A3 CATE · A4 policy.
- **Serie B** (`04_notebooks_produccion_hf/`): B1 FastAPI · B2 Hub/CI · B3 Gradio Space · B4 dashboard de drift.
- **Datos** (`data_benchmark_causal/…/datasets/`): CSV + `metadata.json` (benchmark documentado; ver callouts «monorepo» en los capítulos de ingeniería).
- **Proyecto** (`06_proyecto_final/`): sistema causal en HF con comparación **predictivo vs causal**.

La **Serie A** aplica la teoría a un caso con pregunta de identificación. La **Serie B** toma el artefacto de A y lo despliega en Hugging Face. El sílabo (`01_silabo/silabo.pdf`) fija la correspondencia exacta sesión ↔ Cap. N del PDF ↔ lab ↔ entrega.

Cómo leer un capítulo de ingeniería (Plantilla B)

1. Cuadro de decisión (trade-offs).
2. Problema de diseño **específico de un sistema causal**.
3. Opciones reales (no una sola receta).
4. Decisión de referencia del curso.
5. Implementación (y, si aplica, el callout del monorepo).
6. Checklist operativo / alertas.
7. Lab o notebook de la semana.

Notación y convenciones

La siguiente tabla resume los símbolos principales utilizados a lo largo de la monografía.

Símbolo	Significado	Uso
$Y_{i(d)}$	Resultado potencial bajo tratamiento d	Capítulo 1
D_i	Tratamiento observado	Todo el libro
X_i	Covariables pre-tratamiento	Todo el libro
$\tau(x)$	CATE: $E[Y(1) - Y(0) \texttt{ mid } X = x]$	Capítulos 1, 5–12
θ_0	Parámetro causal verdadero	Capítulos 2–4, 8–9
ATE o θ_{ATE}	ATE: $E[\tau(X)]$	Todo el libro
$\mu_{d(x)}$	Outcome condicional: $E[Y \texttt{ mid } X = x, D = d]$	Capítulos 1, 4–5, 13
$g_0(x)$	Componente estructural PLR	Capítulos 3–4
$l_0(x)$	Outcome regression PLR: $E[Y \texttt{ mid } X = x]$	Capítulos 3–4
$e_0(x)$	Propensity score: $E[D \texttt{ mid } X = x]$	Capítulos 4–7
$m_0(x)$	Tratamiento condicional: $E[D \texttt{ mid } X = x]$	Capítulos 3–4
$\psi(W, \theta, \eta)$	Score ortogonal	Capítulos 3–4
η	Nuisance parameter	Capítulos 3–4
$W_i = (Y_i, D_i, X_i)$	Observación i-ésima	Todo el libro
Z_i	Instrumento	Capítulos 8–9
$Y_i^\star$	Transformed outcome	Capítulos 5–7
$\mathcal{C}$	Conjunto de ajuste (backdoor)	Capítulos 1, 15
$\boldsymbol{W}$	Matriz de adyacencia (NOTEARS)	Capítulo 15
$\boldsymbol{w}$	Pesos del control sintético	Capítulo 14
$\omega_{i(x)}$	Pesos locales (GRF)	Capítulo 6
$\mathcal{V}$	Varianza asintótica	Capítulos 3–4
$V(\pi)$	Policy value	Capítulos 11–12
G	Atributo protegido	Capítulo 16
α_i	Efecto fijo de unidad	Capítulo 9
$\mathcal{L}_m$	Hoja m-ésima del árbol	Capítulo 6
u_i	Residuo Robinson (outcome)	Capítulos 3–4
v_i	Residuo Robinson (tratamiento)	Capítulos 3–4
λ	Penalización / multipl. presupuestario	Capítulos 2, 8, 11–12
n, N, T	Obs. i.i.d. / unidades / periodos	Todo el libro

Símbolos y convenciones de notación.

Índice general

Prefacio iii
Cómo usar este libro v
Dos plantillas de capítulo v
Partes del libro v
Elementos pedagógicos vi
Audiencia profesional y camino crítico vi
Mapa sesión → capítulo → lab → entrega (resumen) vii
Dos series de notebooks y el monorepo vii
Cómo leer un capítulo de ingeniería (Plantilla B) viii
Notación y convenciones ix
1 Causalidad elemental y el problema fundamental de la inferencia causal 33
1.1 Dos preguntas que parecen la misma pregunta 33
1.2 Correlación no es causación: tres escenas clásicas 34
1.3 El problema fundamental de la inferencia causal 35
1.4 Un adelanto de la escalera de la causalidad 35
1.5 Por qué el aprendizaje automático no es causal por defecto 36
1.6 Ejercicios y notas bibliográficas 37
1.6.1 Ejercicios conceptuales 37
1.6.2 Notas bibliográficas 37
2 Anatomía de un sistema de ML en producción 38
2.1 El problema de diseño 38
2.2 Opciones y trade-offs 39
2.3 Decisión de referencia para el curso 40
2.4 Implementación de referencia: el esqueleto del proyecto del curso 40
2.5 Aplicación al sistema causal del curso 41
2.6 Señales de alerta y checklist operativo 41
2.7 Ejercicios y notas bibliográficas 42
2.7.1 Ejercicios conceptuales 42
2.7.2 Ejercicio de infraestructura (Lab 0) 42
2.7.3 Notas bibliográficas 42
3 Modelos Causales Estructurales y la Jerarquía Causal de Pearl 45
3.1 El problema: formalizar «causa» matemáticamente 45
3.2 Teoría: SCM y la Jerarquía Causal de Pearl 46
3.2.1 Modelos Causales Estructurales 46

3.2.2 Los tres niveles de la Jerarquía Causal de Pearl . . . 46
3.3 Algoritmo e implementación . . . 47
3.4 Caso empírico: consistencia del grafo supuesto . . . 48
3.5 Riesgos, límites y sensibilidad . . . 49
3.6 Ejercicios y notas bibliográficas . . . 49
3.6.1 Ejercicios teóricos . . . 49
3.6.2 Ejercicios computacionales . . . 50
3.6.3 Notas bibliográficas . . . 50
4 Ingeniería de datos para sistemas causales . . . 51
4.1 El problema de diseño . . . 51
4.2 Opciones y trade-offs . . . 52
4.3 Decisión de referencia para el curso . . . 52
4.4 Implementación de referencia . . . 54
4.5 Aplicación al sistema causal del curso . . . 55
4.6 Señales de alerta y checklist operativo . . . 55
4.7 Ejercicios y notas bibliográficas . . . 55
4.7.1 Ejercicios conceptuales . . . 55
4.7.2 Ejercicio de infraestructura . . . 55
4.7.3 Notas bibliográficas . . . 56
5 Predicción y causalidad: por qué el ML no es causal por defecto . . . 57
5.1 Problema causal, estimando e identificación . . . 57
5.2 Teoría, supuestos y garantías . . . 59
5.2.1 Supuestos . . . 59
5.2.2 Estimador y score . . . 59
5.2.3 Garantías . . . 60
5.3 Algoritmo e implementación . . . 60
5.3.1 Implementación en Python . . . 60
5.4 Caso empírico y validación . . . 62
5.4.1 Datos y preprocesamiento . . . 62
5.4.2 Estimación principal . . . 62
5.4.3 Validación . . . 63
5.4.4 Interpretación económica . . . 63
5.5 Riesgos, límites y sensibilidad . . . 64
5.5.1 Tabla de riesgos . . . 64
5.5.2 Cuándo no usar este método . . . 65
5.6 Ejercicios y notas bibliográficas . . . 66
5.6.1 Ejercicios teóricos . . . 66
5.6.2 Ejercicios computacionales . . . 66
5.6.3 Ejercicios aplicados . . . 66
5.6.4 Notas bibliográficas . . . 66
6 Regularización, selección de variables y sesgo post-selección . . . 68
6.1 Problema causal, estimando e identificación . . . 68

6.2 Teoría, supuestos y garantías ... 69
6.2.1 Supuestos ... 69
6.2.2 Estimador y score ... 69
6.2.3 Garantías ... 70
6.3 Algoritmo e implementación ... 71
6.3.1 Pseudocódigo ... 71
6.3.2 Implementación en Python ... 71
6.3.3 Diagnósticos de nuisance ... 73
6.4 Caso empírico y validación ... 74
6.5 Riesgos, límites y sensibilidad ... 74
6.5.1 Tabla de riesgos ... 74
6.5.2 Cuándo no usar este método ... 74
6.5.3 Checklist de validación ... 75
6.6 Ejercicios y notas bibliográficas ... 75
7 Datos de entrenamiento en sistemas causales ... 76
7.1 El problema de diseño ... 76
7.2 Opciones y trade-offs ... 77
7.3 Decisión de referencia para el curso ... 77
7.4 Implementación de referencia ... 78
7.5 Aplicación al sistema causal del curso ... 78
7.6 Señales de alerta y checklist operativo ... 79
7.7 Ejercicios y notas bibliográficas ... 79
7.7.1 Ejercicios conceptuales ... 79
7.7.2 Ejercicio computacional ... 79
7.7.3 Notas bibliográficas ... 79
8 Razonamiento intervencional y contrafactual ... 80
8.1 Problema causal, estimando e identificación ... 80
8.2 Teoría, supuestos y garantías ... 81
8.2.1 El criterio de puerta trasera ... 81
8.2.2 Buenos y malos controles ... 81
8.2.3 Cómputo contrafactual: abducción, acción, predicción ... 82
8.3 Algoritmo e implementación ... 83
8.4 Caso empírico ... 83
8.5 Riesgos, límites y sensibilidad ... 84
8.6 Ejercicios y notas bibliográficas ... 84
8.6.1 Ejercicios teóricos ... 84
8.6.2 Ejercicios computacionales ... 84
8.6.3 Notas bibliográficas ... 84
9 Ingeniería de *features* sin fuga causal ... 86
9.1 El problema de diseño ... 86
9.2 Opciones y trade-offs ... 87
9.3 Decisión de referencia para el curso ... 87

9.4 Implementación de referencia ... 88
9.5 Aplicación al sistema causal del curso ... 88
9.6 Señales de alerta y checklist operativo ... 89
9.7 Ejercicios y notas bibliográficas ... 89
9.7.1 Ejercicios conceptuales ... 89
9.7.2 Ejercicio computacional ... 89
9.7.3 Notas bibliográficas ... 89
10 Ortogonalidad de Neyman, cross-fitting e influence functions ... 92
10.1 Problema causal, estimando e identificación ... 92
10.2 Teoría, supuestos y garantías ... 93
10.2.1 El problema del doble sesgo ... 93
10.2.2 Ortogonalidad de Neyman ... 94
10.2.3 Construcción del score ortogonal para PLR ... 94
10.2.4 Teorema central de DML ... 95
10.3 Algoritmo e implementación ... 96
10.3.1 El rol del cross-fitting ... 96
10.3.2 Pseudocódigo ... 97
10.3.3 Implementación en Python ... 97
10.4 Caso empírico y validación ... 99
10.5 Riesgos, límites y sensibilidad ... 100
10.6 Ejercicios y notas bibliográficas ... 100
11 Double/Debiased Machine Learning ... 101
11.1 Problema causal, estimando e identificación ... 101
11.2 Teoría, supuestos y garantías ... 102
11.2.1 Supuestos ... 102
11.2.2 Estimador y score ... 102
11.2.3 Garantías ... 103
11.3 Algoritmo e implementación ... 104
11.3.1 DML con DoubleML para PLR ... 104
11.3.2 DML con econml para CATE bajo PLR/etapa final lineal ... 104
11.3.3 Comparación de learners ... 105
11.3.4 Verificación de leakage ... 106
11.4 Caso empírico y validación ... 107
11.5 Riesgos, límites y sensibilidad ... 109
11.6 Ejercicios y notas bibliográficas ... 109
12 Meta-learners para efectos de tratamiento heterogéneos ... 111
12.1 Problema causal, estimando e identificación ... 111
12.2 Teoría, supuestos y garantías ... 112
12.2.1 El S-learner ... 112
12.2.2 El T-learner ... 112
12.2.3 El X-learner ... 112
12.2.4 R-learner: Estimador y score ... 113

12.2.5 Comparación de propiedades . . . 114
12.3 Algoritmo e implementación . . . 116
12.3.1 Pseudocódigo . . . 116
12.3.2 Implementación en Python . . . 116
12.3.3 Análisis de sesgo y varianza por subgrupo . . . 118
12.4 Caso empírico y validación . . . 118
12.5 Riesgos, límites y sensibilidad . . . 119
12.6 Ejercicios y notas bibliográficas . . . 119
13 Árboles y bosques causales . . . 120
13.1 Motivación aplicada . . . 120
13.2 Teoría, supuestos y garantías . . . 121
13.2.1 Árboles causales honestos . . . 121
13.2.2 El criterio de splitting . . . 121
13.3 Estimador y score . . . 122
13.3.1 Causal forests y Generalized Random Forests . . . 122
13.3.2 Comparación con meta-learners . . . 123
13.4 Implementación en Python . . . 124
13.4.1 Estimación con CausalForestDML . . . 124
13.4.2 Comparación con meta-learners . . . 126
13.5 Caso empírico y validación . . . 126
13.5.1 Identificación de subgrupos con `feature_importances_` . . . 126
13.5.2 Heterogeneidad en efectos del programa NSW . . . 126
13.5.3 Policy trees y validación de subgrupos . . . 127
13.6 Riesgos, límites y sensibilidad . . . 128
13.6.1 Tabla de riesgos . . . 128
13.6.2 Límites . . . 129
13.7 Ejercicios propuestos . . . 129
13.8 Lecturas complementarias . . . 130
14 Validación, refutación y sensibilidad . . . 131
14.1 Motivación aplicada . . . 131
14.2 Teoría, supuestos y garantías . . . 132
14.2.1 El problema fundamental . . . 132
14.2.2 Curva Qini (versión acumulada) . . . 132
14.2.3 Curva uplift y transformed outcome . . . 133
14.2.4 Placebo tests . . . 133
14.2.5 Validación con datos sintéticos . . . 134
14.3 Implementación en Python . . . 135
14.3.1 Curva Qini sobre datos sintéticos . . . 135
14.3.2 Placebo test . . . 137
14.4 Caso empírico y validación . . . 137
14.4.1 Suite de refutación . . . 137
14.4.2 Curva Qini comparativa . . . 138

14.5 Riesgos, límites y sensibilidad ... 140
14.5.1 Tabla de riesgos ... 140
14.5.2 Límites ... 141
14.6 Evaluación offline en el ciclo de vida del sistema ... 141
14.6.1 *Baselines* obligatorios ... 142
14.6.2 Versionado del proceso de evaluación ... 142
14.7 Ejercicios y notas bibliográficas ... 142
14.7.1 Ejercicios propuestos ... 142
14.7.2 Lecturas complementarias ... 143
15 Despliegue de un estimador causal como servicio ... 144
15.1 El problema de diseño ... 144
15.2 Opciones y trade-offs ... 145
15.3 Decisión de referencia para el curso ... 145
15.4 Implementación de referencia ... 146
15.5 Aplicación al sistema causal del curso ... 147
15.6 Señales de alerta y checklist operativo ... 148
15.7 Ejercicios y notas bibliográficas ... 148
15.7.1 Ejercicios conceptuales ... 148
15.7.2 Ejercicio de infraestructura (Lab, Notebook B1) ... 148
15.7.3 Notas bibliográficas ... 148
16 Post-Double-Selection LASSO ... 151
16.1 Motivación aplicada ... 151
16.2 Teoría, supuestos y garantías ... 152
16.2.1 Modelo de alta dimensión con efecto causal ... 152
16.2.2 Post-Double-Selection (PDS) ... 152
16.2.3 Pseudocódigo: Post-Double-Selection ... 152
16.2.4 Intuición: por qué la doble selección funciona ... 152
16.2.5 Comparación con DML ... 153
16.3 Implementación en Python ... 155
16.3.1 Implementación de PDS ... 155
16.4 Caso empírico y validación ... 156
16.4.1 Comparación PDS vs DML vs selección manual ... 156
16.4.2 Caso de estudio: efecto de dejar de fumar sobre peso (NHEFS) ... 156
16.4.3 Validación ... 157
16.5 Ejercicios propuestos ... 159
16.6 Lecturas complementarias ... 159
17 Machine learning causal en datos de panel ... 160
17.1 Problema causal, estimando e identificación ... 160
17.2 Teoría, supuestos y garantías ... 161
17.2.1 Modelo de panel con efectos fijos ... 161
17.2.2 Supuestos ... 161
17.2.3 Estimador y score ... 162

17.2.4 Garantías ... 162
17.3 Implementación en Python ... 164
17.3.1 DML sobre panel sintético con ground truth ... 164
17.3.2 DMLIV sobre close_college ... 166
17.4 Caso empírico y validación ... 166
17.5 Ejercicios propuestos ... 169
17.6 Lecturas complementarias ... 169
18 Diseño de investigación y selección de método ... 170
18.1 Motivación aplicada ... 170
18.2 Problema causal, estimando e identificación ... 171
18.2.1 Dimensiones del problema causal ... 171
18.2.2 Estimando e identificación ... 171
18.3 Teoría, supuestos y garantías ... 172
18.3.1 Mapa de decisión metodológica ... 172
18.3.2 Identificación jerárquica de diseños ... 172
18.3.3 Criterios adicionales de selección ... 173
18.4 Implementación en Python ... 175
18.4.1 Función `choose_method` ... 175
18.5 Caso empírico y validación ... 176
18.5.1 Análisis crítico de 4 problemas ... 176
18.5.2 Validación de la elección metodológica ... 176
18.6 Errores comunes y límites ... 176
18.7 Ejercicios propuestos ... 177
18.8 Lecturas complementarias ... 177
19 *Model cards* causales y publicación en Hugging Face Hub ... 180
19.1 El problema de diseño ... 180
19.2 Opciones y trade-offs ... 181
19.3 Decisión de referencia para el curso ... 181
19.4 Implementación de referencia ... 182
19.5 Aplicación al sistema causal del curso ... 183
19.6 Señales de alerta y checklist operativo ... 183
19.7 Ejercicios y notas bibliográficas ... 183
19.7.1 Ejercicios conceptuales ... 183
19.7.2 Ejercicio de infraestructura ... 183
19.7.3 Notas bibliográficas ... 184
20 CI/CD para modelos causales ... 185
20.1 El problema de diseño ... 185
20.2 Opciones y trade-offs ... 186
20.3 Decisión de referencia para el curso ... 186
20.4 Implementación de referencia ... 187
20.5 Aplicación al sistema causal del curso ... 188
20.6 Señales de alerta y checklist operativo ... 188

20.7 Ejercicios y notas bibliográficas ... 189
20.7.1 Ejercicios conceptuales ... 189
20.7.2 Ejercicio de infraestructura (cierra el Notebook B2) ... 189
20.7.3 Notas bibliográficas ... 189
21 Aprendizaje de políticas óptimas ... 192
21.1 Motivación aplicada ... 192
21.2 Teoría, supuestos y garantías ... 193
21.2.1 Policy value y welfare regret ... 193
21.2.2 Estimación del policy value ... 193
21.2.3 Restricciones presupuestarias ... 193
21.3 Implementación en Python ... 195
21.3.1 Policy learning con econml ... 195
21.3.2 Restricción presupuestaria ... 197
21.4 Caso empírico y validación ... 197
21.5 Riesgos, límites y sensibilidad ... 198
21.6 Ejercicios propuestos ... 199
21.7 Lecturas complementarias ... 199
22 Uplift modeling y targeting económico ... 200
22.1 Motivación aplicada ... 200
22.2 Marco teórico formal ... 201
22.2.1 Uplift y CATE ... 201
22.2.2 Qini y AUUC: teoría, supuestos y garantías ... 201
22.2.3 Multi-treatment uplift ... 202
22.3 Algoritmo: Asignación por net score con presupuesto ... 202
22.4 Aplicación con Python ... 203
22.4.1 Uplift modeling con causalml ... 203
22.4.2 Caso empírico y validación: costo-efectividad ... 204
22.5 Riesgos, límites y mitigación ... 206
22.6 Ejercicios propuestos ... 207
22.7 Lecturas complementarias ... 207
23 Explicación causal y equidad algorítmica ... 210
23.1 Problema causal, estimando e identificación ... 210
23.2 Teoría, supuestos y garantías ... 211
23.2.1 Efectos directos, indirectos y espurios ... 211
23.2.2 Equidad contrafactual ... 211
23.3 Algoritmo e implementación ... 212
23.4 Caso empírico ... 212
23.5 Riesgos, límites y sensibilidad ... 213
23.6 Ejercicios y notas bibliográficas ... 213
23.6.1 Ejercicios teóricos ... 213
23.6.2 Ejercicio computacional ... 213
23.6.3 Notas bibliográficas ... 213

24 Ética, interpretabilidad y comunicación de resultados 215
24.1 Motivación aplicada 215
24.2 Teoría, supuestos y garantías 216
24.2.1 Equidad causal 216
24.2.2 Interpretabilidad 216
24.2.3 Comunicación de incertidumbre 217
24.3 Implementación en Python 217
24.3.1 Interpretabilidad con SHAP 217
24.3.2 Tabla de resultados para policymakers 218
24.3.3 Análisis de equidad 218
24.4 Caso empírico y validación 219
24.4.1 Auditoría de equidad en focalización de transferencias 219
24.5 Riesgos, límites y sensibilidad 220
24.5.1 Tabla de riesgos 220
24.5.2 Cuándo no usar este método 220
24.6 Ejercicios propuestos 220
24.7 Lecturas complementarias 221
25 Decisiones estructurales bajo incertidumbre 222
25.1 Problema causal, estimando e identificación 222
25.2 Teoría, supuestos y garantías 223
25.2.1 La decisión como maximización de utilidad esperada 223
25.2.2 Decisiones sobre múltiples variables y el rol de la incertidumbre 223
25.3 Algoritmo e implementación 224
25.4 Caso empírico 224
25.5 Riesgos, límites y sensibilidad 225
25.6 Ejercicios y notas bibliográficas 225
25.6.1 Ejercicios teóricos 225
25.6.2 Ejercicio computacional 225
25.6.3 Notas bibliográficas 226
26 Generalización y transportabilidad entre dominios 227
26.1 Problema causal, estimando e identificación 227
26.2 Teoría, supuestos y garantías 228
26.2.1 Diagramas de selección 228
26.2.2 Por qué esto no es lo mismo que «reestimar con más datos» 228
26.3 Algoritmo e implementación 229
26.4 Caso empírico 229
26.5 Riesgos, límites y sensibilidad 230
26.6 Ejercicios y notas bibliográficas 230
26.6.1 Ejercicios teóricos 230
26.6.2 Ejercicio computacional 230
26.6.3 Notas bibliográficas 230
27 Deep learning para inferencia causal 233

27.1 Motivación aplicada ... 233
27.2 Teoría, supuestos y garantías ... 234
27.2.1 TARNet: representación compartida ... 234
27.2.2 CFRNet: balance integral probability metric ... 234
27.2.3 DeepIV ... 234
27.3 Algoritmo de entrenamiento TARNet/CFRNet ... 235
27.4 Implementación en Python ... 236
27.4.1 CFRNet con penalización MMD ... 238
27.5 Arquitectura de TARNet/CFRNet ... 238
27.6 Caso empírico y validación ... 239
27.7 Límites y errores comunes ... 240
27.8 Ejercicios propuestos ... 240
27.9 Lecturas complementarias ... 241
28 Control sintético y matrix completion ... 242
28.1 Motivación aplicada ... 242
28.2 Teoría, supuestos y garantías ... 243
28.2.1 SCM clásico ... 243
28.2.2 Matrix completion ... 243
28.2.3 Comparación SCM vs. matrix completion ... 244
28.3 Algoritmo e implementación ... 244
28.3.1 Algoritmo: Synthetic control clásico ... 244
28.3.2 SCM clásico sobre Texas ... 245
28.3.3 Matrix completion ... 246
28.4 Caso empírico y validación ... 246
28.5 Riesgos, límites y mitigación ... 247
28.6 Ejercicios propuestos ... 248
28.7 Lecturas complementarias ... 248
29 Descubrimiento causal y grafos ... 251
29.1 Motivación aplicada ... 251
29.2 Marco teórico formal ... 252
29.2.1 DAGs y el criterio de puerta trasera ... 252
29.2.2 Algoritmo PC ... 252
29.2.3 Pseudocódigo: PC skeleton ... 252
29.2.4 NOTEARS ... 253
29.2.5 Límites: causal sufficiency ... 253
29.3 Implementación en Python ... 254
29.3.1 Construcción de DAGs con dowhy ... 254
29.3.2 NOTEARS y PC con causal-learn ... 254
29.4 Caso empírico y validación ... 255
29.4.1 Evaluación de grafos estimados ... 255
29.4.2 Caso de estudio: DAGs y descubrimiento causal ... 256
29.5 Riesgos, límites y sensibilidad ... 258

29.5.1 Tabla de riesgos ... 258
29.5.2 Límites del descubrimiento causal ... 259
29.6 Ejercicios propuestos ... 260
29.7 Lecturas complementarias ... 260
30 Monitoreo de supuestos causales en producción ... 261
30.1 El problema de diseño ... 261
30.2 Opciones y trade-offs ... 262
30.3 Decisión de referencia para el curso ... 262
30.4 Implementación de referencia ... 263
30.5 Aplicación al sistema causal del curso ... 264
30.6 Señales de alerta y checklist operativo ... 264
30.7 Ejercicios y notas bibliográficas ... 264
30.7.1 Ejercicios conceptuales ... 264
30.7.2 Ejercicio de infraestructura (Lab, arranca Notebook B4) ... 265
30.7.3 Notas bibliográficas ... 265
31 Aprendizaje continuo y panorama de *Reinforcement Learning* causal ... 266
31.1 Parte I: Aprendizaje continuo para sistemas causales ... 266
31.1.1 El problema de diseño ... 266
31.1.2 Opciones y trade-offs ... 267
31.1.3 Decisión de referencia para el curso ... 267
31.2 Parte II: Panorama de *Reinforcement Learning* causal ... 268
31.2.1 De decisiones únicas a decisiones secuenciales ... 268
31.2.2 Bandits causales: el caso introductorio más simple ... 268
31.2.3 Por qué este curso se detiene aquí ... 268
31.3 Ejercicios y notas bibliográficas ... 269
31.3.1 Ejercicios conceptuales ... 269
31.3.2 Ejercicio de infraestructura ... 269
31.3.3 Notas bibliográficas ... 269
32 El lado humano de los sistemas causales ... 270
32.1 El problema de diseño ... 270
32.2 Opciones y trade-offs ... 270
32.3 Decisión de referencia para el curso ... 271
32.4 Implementación de referencia: plantilla de comunicación ... 271
32.5 Aplicación al sistema causal del curso: cierre ... 272
32.6 Señales de alerta y checklist operativo ... 272
32.7 Ejercicios y notas bibliográficas ... 272
32.7.1 Ejercicios conceptuales ... 272
32.7.2 Ejercicio de cierre (proyecto final) ... 272
32.7.3 Notas bibliográficas ... 273
33 Notación, convenciones y fundamentos matemáticos ... 274
33.1 Notación general ... 274
33.2 Modos de convergencia ... 275

33.3 Teoremas límite fundamentales ... 276
33.4 Teoría asintótica semiparamétrica ... 276
33.4.1 Espacios tangentes ... 276
33.4.2 Estimación semiparamétrica eficiente ... 276
33.5 Ortogonalidad de Neyman ... 277
33.6 Cross-fitting y sample splitting ... 277
33.7 Honesty en árboles causales ... 277
33.8 Norma nuclear y matrix completion ... 277
33.9 Modelos causales estructurales ... 278
33.10 Influence functions ... 278
33.11 Regresión Discontinua (RDD) ... 278
33.12 Transportabilidad ... 278
34 Guía de librerías y entorno reproducible ... 280
34.1 Entorno reproducible ... 280
34.2 econml (Microsoft Research) ... 280
34.3 DoubleML ... 281
34.4 causalml (Uber) ... 281
34.5 dowhy (Microsoft AWS) ... 281
34.6 PyTorch ... 282
34.7 scikit-learn ... 282
34.8 networkx ... 282
34.9 statsmodels ... 282
34.10 Tabla comparativa ... 283
34.11 Instalación recomendada ... 283
34.11.1 Librerías complementarias ... 283
35 Datos, DGP y benchmarks ... 285
35.1 Datasets reales ... 285
35.2 Datasets sintéticos ... 286
35.3 Estructura de archivos ... 286
35.4 Recomendaciones de uso por capítulo ... 286
35.5 Datasets complementarios sugeridos ... 287
35.6 Ground truth y checksums ... 287
36 Glosario de estimandos, supuestos y métodos ... 289
36.1 Estimandos ... 289
36.2 Supuestos ... 290
36.3 Métodos ... 291
36.4 Garantías ... 293
36.5 Diagnósticos ... 293
37 Plantilla de proyecto final y rúbrica ... 295
37.1 Estructura del informe ... 295
37.2 Capa causal de la pregunta ... 295
37.3 DAG o supuesto estructural ... 296

37.4 Identificación ... 296
37.5 Estimación ... 296
37.6 Validación y refutación ... 296
37.7 Análisis de sensibilidad ... 296
37.8 Transportabilidad ... 297
37.9 Ética y comunicación ... 297
37.10 Criterios de evaluación ... 297
37.11 Entregables ... 297
37.12 Ejemplos de problemas aplicados ... 298
37.13 Plazos sugeridos ... 298
38 Checklists por método ... 299
38.1 Checklist DML ... 299
38.2 Checklist meta-learners ... 299
38.3 Checklist causal forest ... 299
38.4 Checklist PDS ... 300
38.5 Checklist panel/DMLIV ... 300
38.6 Checklist policy learning ... 300
38.7 Checklist uplift ... 300
38.8 Checklist SCM/matrix completion ... 301
38.9 Checklist causal discovery ... 301
39 Plantilla de *model card* causal ... 302
39.1 Campos obligatorios y su propósito ... 302
39.2 Plantilla completa ... 303
39.3 Checklist antes de publicar ... 304
40 Checklist de arquitectura de despliegue ... 305
40.1 Elegir el patrón de *serving* ... 305
40.2 Checklist de la función de inferencia ... 305
40.3 Checklist de CI/CD ... 306
40.4 Checklist de monitoreo en producción ... 306
40.5 Checklist de comunicación ... 306
Bibliografía ... 307

Índice de figuras

Figura 11.1 Sensibilidad de la estimación DML-PLIV del retorno a la educación al número de folds K del cross-fitting. Las barras indican IC al 95%. La línea punteada es la estimación OLS con controles (HC1). Dataset: `close_college` (Card, 1995), $n = 3,003$. 108
Figura 13.1 Heterogeneidad de CATE estimado por `CausalForestDML` sobre `nsw_mixtape`. Izquierda: CATE promedio por subgrupo con IC al 95%. Derecha: superficie interpolada de CATE en función de edad y educación. Dataset: `nsw_mixtape` (Lalonde, 1986), $n = 445$. 127
Figura 14.1 Curva Qini comparativa: Causal Forest vs. T-learner (GBM) vs. baseline aleatorio. Dataset sintético con CATE conocido $\tau(x) = 2 + 3x_1 - x_2^2$, $n = 3,000$. El Causal Forest domina al T-learner, especialmente en el rango 20–60% de la muestra. 139
Figura 16.1 Sensibilidad de la estimación PDS al hiperparámetro λ de LASSO. Las barras indican IC al 95% (HC1). La línea verde punteada marca el λ óptimo por cross-validation. Dataset: `nhefs`, $n = 1,566$, $p = 51$ controles. 157
Figura 17.1 DML-panel vs. FE clásico (LSDV): sensibilidad al número de folds K del cross-fitting. Las barras indican IC al 95%. La línea verde punteada es el ATE verdadero; la naranja punteada es el estimador FE clásico. Dataset: `panel_fe_dynamic` (sintético con ground truth). 167
Figura 21.1 Policy value vs. presupuesto de tratamiento. La línea azul es la estrategia óptima (targeting por CATE estimado); la gris punteada es tratamiento aleatorio; la roja punteada es tratar a todos'". DGP sintético que simula targeting de transferencias gubernamentales ($n = 5,000$). Las curvas muestran el policy value esperado al variar la fracción de la población tratada. 198
Figura 22.1 Izquierda: curva de uplift acumulado ordenando la muestra por CATE estimado (Causal Forest). Derecha: análisis de costo-efectividad de la mezcla entre SMS ($0.5/unidad) y llamada ($8/unidad) bajo un presupuesto total de $100,000. Dataset sintético tipo churn, $n = 5,000$. 206
Figura 27.1 Arquitectura de TARNet / CFRNet (Shalit, Johansson & Sontag, 2017). La capa de representación compartida Φ es seguida por dos cabezas separadas h_0, h_1 para $D = 0$ y $D = 1$. CFRNet añade una penalización IPM (Wasserstein o MMD) que fuerza el balance entre las distribuciones de $\Phi(X)$ condicionales al tratamiento. 239

Figura 28.1 Izquierda: trayectoria de la unidad tratada vs. control sintético (SCM) y matrix completion (MC), con periodo pre-tratamiento $T_0 = 15$. Derecha: efecto estimado post-tratamiento para la unidad tratada (efecto verdadero = 2.5) y para un placebo espacial (unidad no tratada, debe ser ≈ 0). Panel sintético $N = 20$, $T = 30$. 247
Figura 29.1 Izquierda: DAG para el retorno a la educación (Card, 1995); U denota un confusor no observado. Derecha: DAG ponderado estimado por NOTEARS sobre datos sintéticos con estructura conocida ($X_0 \to X_1$, $X_1 \to X_2$, $X_3 \to X_2$, $X_4 \to X_1$); las etiquetas en las aristas son los pesos estimados. 257

Índice de tablas

Símbolos y convenciones de notación. x
Tabla 5.1 Régimen predictivo vs. régimen causal. 58
Tabla 5.2 Riesgos al confundir predicción con causalidad. 65
Tabla 10.1 Errores comunes en la implementación de DML-PLR. 100
Tabla 11.1 Learners comunes para nuisance en DML. 103
Tabla 11.2 Riesgos comunes en DML 109
Tabla 12.1 Comparación de meta-learners. 114
Tabla 12.2 CATE estimado por subgrupo en NSW (CausalForestDML). Réplica propia. 118
Tabla 12.3 Riesgos comunes en meta-learners 119
Tabla 13.1 Causal Forest vs. meta-learners. 123
Tabla 13.2 Riesgos al usar árboles y bosques causales. 129
Tabla 14.1 Suite de refutación para modelos causales. 138
Tabla 14.2 Riesgos en la validación de modelos causales sin ground truth. 141
Tabla 16.1 PDS vs. DML. 153
Tabla 16.2 Riesgos al aplicar PDS LASSO. 159
Tabla 17.1 Riesgos comunes en DML-panel 168
Tabla 18.1 Mapa de decisión metodológica. 172
Tabla 18.2 Identificación jerárquica de diseños causales. 173
Tabla 18.3 Riesgos comunes en la elección metodológica 176
Tabla 21.1 Riesgos comunes en policy learning 199
Tabla 22.1 Riesgos comunes en uplift modeling y targeting 207
Tabla 24.1 Riesgos en ética, interpretabilidad y comunicación de ML causal 220
Tabla 27.1 Riesgos comunes en deep causal learning 240
Tabla 28.1 SCM clásico vs. matrix completion 244
Tabla 28.2 Riesgos en SCM y matrix completion 247
Tabla 29.1 Métricas de evaluación para PC y NOTEARS sobre datos sintéticos ($n = 1000$, $p = 5$). 256
Tabla 29.2 Riesgos en descubrimiento causal y grafos 259
Tabla 1.1 Símbolos y convenciones notacionales. 275
Tabla 2.1 Comparación de librerías. 283
Tabla 2.2 Librerías complementarias. 283
Tabla 3.1 Datasets reales del repositorio. 285
Tabla 3.2 Datasets sintéticos con *ground truth*. 286

Tabla 3.3 Datasets complementarios sugeridos. 287
Tabla 5.1 Rúbrica de evaluación del proyecto final. 297

Índice de algoritmos

Algoritmo 1. Post-LASSO OLS (selección ingenua) 71
Algoritmo 2. Double Machine Learning con cross-fitting (K-fold) 97
Algoritmo 3. DML manual con cross-fitting para PLR 97
Algoritmo 4. DML con cross-fitting 104
Algoritmo 5. X-learner 116
Algoritmo 6. Causal tree honesto 121
Algoritmo 7. Post-Double-Selection (PDS) 152
Algoritmo 8. DML-panel con within-transformation 162
Algoritmo 9. Policy tree con restricción presupuestaria 197
Algoritmo 10. Asignación por net score con presupuesto 202
Algoritmo 11. Entrenamiento TARNet/CFRNet 236
Algoritmo 12. Synthetic control clásico 245
Algoritmo 13. PC skeleton — esqueleto del grafo causal 253

Índice de listados

Listado 3.1 SCM simulado: divergencia entre asociación e intervención. 48
Listado 4.1 Esquema de roles causales como estructura de datos explícita. 54
Listado 5.1 Comparación predictivo vs. causal sobre datos de Card (1995). 61
Listado 5.2 Simulación de sesgo por collider. 63
Listado 6.1 Ilustración del sesgo post-selección. 72
Listado 6.2 Partialling-out manual. 73
Listado 7.1 Chequeo de balance como auditoría mínima del mecanismo de asignación. . . 78
Listado 8.1 Verificación del criterio de puerta trasera con `networkx`. 83
Listado 9.1 Auditoría de *features* por marca de tiempo y por rol declarado. 88
Listado 10.1 Implementación manual de DML para PLR. 98
Listado 10.2 DML aplicado a NSW Lalonde. 99
Listado 11.1 DML con librería DoubleML sobre NSW. 104
Listado 12.1 Comparación de meta-learners sobre dataset sintético. 117
Listado 12.2 Sesgo del S-learner bajo regularización fuerte. 118
Listado 13.1 Causal Forest sobre NSW con mapas de heterogeneidad. 125
Listado 13.2 Causal Forest vs. R-learner en datos sintéticos. 126
Listado 13.3 Importancia de variables en causal forest. 126
Listado 14.1 Curva Qini sobre dataset sintético con ground truth. 136
Listado 14.2 Placebo temporal sobre panel sintético. 137
Listado 15.1 Función de inferencia pura, desacoplada del framework web. 146
Listado 15.2 Servicio FastAPI que envuelve la función de inferencia. 147
Listado 15.3 `Dockerfile` para contenerizar el servicio. 147
Listado 16.1 Post-Double-Selection LASSO sobre NHEFS. 155
Listado 16.2 Comparación PDS vs. DML vs. selección manual. 156
Listado 16.3 Sensibilidad al hiperparametro lambda. 158
Listado 17.1 DML sobre panel sintético. 165
Listado 17.2 DML-PLIV sobre close_college. 166
Listado 18.1 Función de decisión metodológica. 175
Listado 18.2 Caso 1: retorno a la educación. 176
Listado 19.1 Plantilla mínima de *model card* causal. 182
Listado 19.2 Publicación del modelo y su *model card* en el Hub. 182
Listado 20.1 *Workflow* de GitHub Actions con pruebas de software y estadísticas en etapas separadas. 187
Listado 20.2 Prueba estadística de humo: el placebo no debe ser significativo. 188

Listado 21.1 Policy learning sobre gov_transfers con evaluaci'on hold-out. 196
Listado 21.2 Policy learning con restricción presupuestaria. 197
Listado 22.1 Uplift modeling con causalml. 204
Listado 22.2 Analisis costo-efectividad de targeting. 205
Listado 23.1 Descomposición directo/indirecto con un SCM lineal simulado. 212
Listado 24.1 SHAP para Causal Forest. 217
Listado 24.2 Tabla de resultados para comunicación a tomadores de decisión. 218
Listado 24.3 Análisis de equidad en policy learning. 218
Listado 25.1 Selección de acción óptima entre tres políticas candidatas con incertidumbre. 224
Listado 26.1 Chequeo simplificado: ¿el destino difiere solo en la distribución de una covariable? 229
Listado 27.1 Implementación de TARNet en PyTorch. 237
Listado 27.2 CFRNet con penalización MMD. 238
Listado 28.1 SCM clásico sobre dataset Texas. 245
Listado 28.2 Matrix completion con nuclear norm. 246
Listado 29.1 Identificación con DAGs en dowhy. 254
Listado 29.2 PC y NOTEARS con causal-learn. 255
Listado 30.1 Las tres métricas de monitoreo causal sobre una ventana reciente de datos. 263
Listado 31.1 Comparación champion-challenger antes de promover un reentrenamiento. . 267
Listado 32.1 Plantilla de tres capas para comunicar un resultado causal. 271
Listado 2.1 Instalaci'on de librer'ias complementarias. 281
Listado 2.2 Red de representación para TARNet/CFRNet en PyTorch. 282
Listado 2.3 Instalación del stack base de librerías de ML causal. 283
Listado 2.4 Instalaci'on de librer'ias complementarias. 284
Listado 3.1 Verificación de integridad de datasets con checksums SHA-256. 287
Listado 7.1 Plantilla completa de *model card* causal, para copiar como `README.md` del repositorio en Hugging Face Hub. 303

Parte I

Introducción

Esta parte abre el libro con dos capítulos sin prerrequisitos: la distinción elemental entre correlación y causalidad, y la anatomía de un sistema de machine learning en producción. Juntos establecen los dos hilos —causal y de ingeniería de sistemas— que el resto del libro desarrolla en paralelo.

Capítulo 1

Causalidad elemental y el problema fundamental de la inferencia causal

Cuadro sinóptico del capítulo

[**Problema**]	Distinguir una pregunta asociativa de una pregunta causal
[**Idea central**]	El problema fundamental de la inferencia causal (Holland, 1986)
[**Herramienta**]	Resultados potenciales $Y(1), Y(0)$; un adelanto de la escalera de Pearl
[**Ejemplos**]	Correlaciones espurias, paradoja de Simpson
[**Sin laboratorio**]	Sesión puramente conceptual — el primer notebook llega en el Módulo 1
[**Prepara**]	3 (SCM y PCH) y 5 (ML no es causal por defecto)

1.1 Dos preguntas que parecen la misma pregunta

Suponga dos preguntas sobre un programa de becas universitarias. La primera: *¿los estudiantes becados tienen, en promedio, mejores calificaciones que los no becados?* La segunda: *¿dar una beca a un estudiante mejora sus calificaciones?* Ambas preguntas se responden, en apariencia, con el mismo tipo de dato —una tabla de estudiantes, si recibieron beca, y sus calificaciones— y sin embargo son preguntas radicalmente distintas. La primera es una pregunta **asociativa**: describe un patrón en los datos tal como existen. La segunda es una pregunta **causal**: describe qué pasaría si *interviniéramos* sobre el mundo, otorgando una beca que de otro modo no se habría otorgado.

La confusión entre ambas preguntas no es un error ingenuo. Es, de hecho, el error más común y más costoso en la aplicación de *machine learning* a la toma de decisiones. Un modelo predictivo entrenado sobre $(\text{beca}, \text{calificación})$ puede alcanzar una precisión excelente respondiendo la primera pregunta y, sin embargo, ofrecer una respuesta completamente equivocada si se lo usa para decidir a quién becar —porque los estudiantes que hoy reciben becas no son una muestra aleatoria de la población: probablemente ya tenían mejores calificaciones, mejor

preparación, o mayor motivación **antes** de recibir la beca. La asociación observada mezcla el efecto de la beca con estas diferencias preexistentes.

Este curso completo puede resumirse en una sola tesis, que retomaremos una y otra vez bajo distintas formas técnicas: **ningún volumen de datos, por sí solo, convierte una pregunta asociativa en una pregunta causal**. Convertir una en otra requiere supuestos explícitos sobre cómo se generaron los datos —supuestos que provienen del conocimiento del dominio, no del algoritmo—. El aporte del *machine learning* moderno, que ocupará el resto del curso, es permitir que esos supuestos se combinen con métodos flexibles y de alta dimensión sin perder validez estadística. Pero el primer paso, el de este capítulo, es aprender a **reconocer** cuándo una pregunta es causal, y por qué los datos nunca la responden solos.

1.2 Correlación no es causación: tres escenas clásicas

La distinción anterior suele aceptarse en principio y olvidarse en la práctica. Tres ejemplos clásicos ayudan a que la intuición se vuelva difícil de desactivar.

Escena 1: cigüeñas y nacimientos. En varios países europeos, los conteos anuales de parejas de cigüeñas y de nacimientos humanos están positivamente correlacionados a nivel regional. Nadie propone que las cigüeñas traen bebés; ambas variables responden a una tercera —el tamaño y la ruralidad de la región— que las mueve en la misma dirección sin que exista ningún vínculo causal directo entre ellas.

Escena 2: el tamaño del pie y la capacidad de lectura. En una muestra de niños entre 2 y 12 años, el tamaño del pie predice con notable precisión el puntaje en una prueba de lectura. Un modelo de *machine learning* entrenado sobre estos datos «aprendería» esta relación con alta confianza. La variable omitida es, por supuesto, la edad: los niños mayores tienen pies más grandes y leen mejor, sin que agrandar artificialmente el pie de un niño mejore su lectura. En el lenguaje que formalizaremos en el próximo capítulo, la edad es un *confusor* (*confounder*): afecta tanto a la variable explicativa como al resultado, y genera una asociación que no representa ningún mecanismo causal entre ellas.

Ejemplo 1.1 ***(El problema fundamental)***

La paradoja de Simpson. Un hospital evalúa dos tratamientos, A y B, para una misma condición. Tomando todos los pacientes en conjunto, el tratamiento A tiene una tasa de éxito mayor que B. Sin embargo, al separar a los pacientes por gravedad del caso (leve/grave), el tratamiento B resulta superior tanto en el grupo leve como en el grupo grave. La reversión ocurre porque el tratamiento A se administró desproporcionadamente a los casos leves —que se recuperan más fácilmente por sí solos— mientras que B se reservó para los casos graves. La gravedad es, otra vez, un confusor: determina tanto qué tratamiento recibe el paciente como su probabilidad de recuperación. La paradoja de Simpson ((Simpson 1951)) es la demostración más nítida de que una asociación puede invertir su signo por completo según qué variables se incluyan en el análisis —y de que no existe una respuesta puramente estadística sobre cuál desagregación es la «correcta»: se requiere una teoría causal (¿la gravedad afecta a la asignación del tratamiento?) para saberlo.

Estas tres escenas comparten una estructura: una variable no observada, o no correctamente condicionada, genera una asociación que desaparece o se revierte en cuanto se controla por ella. La pregunta que este curso enseña a hacer sistemáticamente es: *¿qué tendría que ser cierto sobre cómo se generaron estos datos para que la asociación observada refleje el efecto causal que me interesa?*

1.3 El problema fundamental de la inferencia causal

Formalicemos la intuición anterior con la herramienta que usaremos durante todo el curso: el marco de **resultados potenciales** (Neyman, (Neyman 1923); Rubin, (Rubin 1974)). Para cada unidad i (una persona, un hogar, una empresa) y un tratamiento binario $D_i \in \{0, 1\}$, definimos dos resultados **potenciales**: $Y_i(1)$, el resultado que se observaría si la unidad recibiera el tratamiento, y $Y_i(0)$, el resultado que se observaría si no lo recibiera. El efecto causal individual del tratamiento sobre la unidad i es, por definición, la diferencia $Y_i(1) - Y_i(0)$.

Definición 1.2

El problema fundamental de la inferencia causal ((Holland 1986)): para cualquier unidad i, solo se observa uno de los dos resultados potenciales —el correspondiente al tratamiento efectivamente recibido— y nunca ambos. Si i recibió el tratamiento, observamos $Y_i(1)$ pero $Y_i(0)$ permanece contrafactual, es decir, no observado y no observable. Si no lo recibió, ocurre lo simétrico.

Esta afirmación, aunque elemental, tiene una consecuencia que estructura todo el campo: el efecto causal individual $Y_{i(1)} - Y_{i(0)}$ **nunca** puede calcularse directamente para ninguna unidad, sin importar cuántos datos se recolecten sobre ella. Lo que sí puede estimarse, bajo supuestos adecuados, es un efecto **poblacional** o **promedio** —por ejemplo, el efecto promedio del tratamiento (ATE), $E[Y(1) - Y(0)]$— comparando grupos de unidades distintas que recibieron y no recibieron el tratamiento. Pero esa comparación entre grupos solo estima el efecto causal si los grupos son comparables en todo lo demás; si no lo son, la comparación mezcla el efecto del tratamiento con las diferencias preexistentes entre los grupos, exactamente como en las tres escenas de la sección anterior. Los experimentos aleatorizados resuelven este problema por diseño (la asignación aleatoria garantiza comparabilidad en expectativa); gran parte de este curso está dedicada a los métodos que permiten aproximar esa comparabilidad cuando no hay aleatorización, usando herramientas de *machine learning* para manejar un número grande de covariables.

1.4 Un adelanto de la escalera de la causalidad

Judea Pearl propuso organizar todas las preguntas causales en tres niveles jerárquicos, popularizados como la «escalera de la causalidad» ((Pearl et al. 2018)) y formalizados como la Jerarquía Causal de Pearl (PCH) en el trabajo que retomaremos en detalle en el 3. Como adelanto:

- **Nivel 1 — Asociación** («ver»): ¿qué relación existe entre las variables tal como se observan? *¿Los estudiantes becados tienen mejores notas?*
- **Nivel 2 — Intervención** («hacer»): ¿qué pasaría si actuáramos sobre el sistema? *¿Qué pasaría con las notas si diéramos una beca a un estudiante que hoy no la tiene?*
- **Nivel 3 — Contrafactual** («imaginar»): ¿qué habría pasado en un mundo alternativo, dado lo que efectivamente observamos? *¿Habría aprobado María si le hubieran dado la beca, sabiendo que en los hechos no se la dieron y no aprobó?*

Cada nivel requiere información y supuestos estrictamente más fuertes que el anterior: los datos observacionales puros solo permiten responder preguntas de Nivel 1 sin más; responder Nivel 2 requiere supuestos de identificación (como los que exploraremos con Post-Double-Selection y Double Machine Learning); responder Nivel 3 requiere, además, un modelo estructural completo. La pregunta con la que abrimos este capítulo —¿dar una beca mejora las calificaciones?— es una pregunta de Nivel 2, y es precisamente el tipo de pregunta que un modelo de *machine learning* estándar, entrenado para predecir bien, **no** está diseñado para responder.

1.5 Por qué el aprendizaje automático no es causal por defecto

Vale la pena anticipar, en términos simples, por qué esto importa específicamente para este curso y no solo para la estadística clásica. Los algoritmos de *machine learning* —regresión regularizada, bosques aleatorios, *gradient boosting*, redes neuronales— están diseñados y evaluados para minimizar el error de predicción fuera de muestra. Esa optimización no distingue, ni tiene por qué distinguir, entre la parte de la relación $X \to Y$ que es causal y la parte que refleja confusión, como en las tres escenas de la 1.2. Un modelo que predice el tamaño del pie a partir de la capacidad de lectura (o viceversa) con gran precisión está haciendo exactamente lo que se le pidió: predecir bien. El error no está en el algoritmo, sino en usar sus resultados para responder una pregunta —la pregunta causal— que nunca se le planteó.

Este curso dedica su primera mitad a mostrar cómo, bajo supuestos de identificación explícitos, es posible usar la misma flexibilidad de alta dimensión que hace poderoso al *machine learning* predictivo —sin heredar su ceguera ante la confusión—, y su segunda mitad a mostrar cómo llevar esos modelos, ya validados, a un sistema que opera de forma confiable en producción. El 5 retoma esta discusión con el aparato formal completo (resultados potenciales en alta dimensión, sesgo de regularización, el estimando CATE); el 3 formaliza la escalera de causalidad con modelos causales estructurales. Este capítulo solo pretende dejar instalada, de forma intuitiva y memorable, la pregunta que todo el curso responde: *¿es esto una pregunta de asociación, o una pregunta causal?*

1.6 Ejercicios y notas bibliográficas

1.6.1 Ejercicios conceptuales

1. Para cada una de las siguientes preguntas, indique si es de Nivel 1 (asociación), Nivel 2 (intervención) o Nivel 3 (contrafactual), y justifique en una oración: (a) ¿Los países con más médicos per cápita tienen menor mortalidad infantil? (b) ¿Contratar más médicos reduciría la mortalidad infantil en un país específico? (c) ¿Habría sobrevivido este paciente si el hospital hubiera tenido un médico más?
2. Proponga un ejemplo propio (distinto a los del capítulo) de una correlación espuria generada por una variable confusora. Identifique explícitamente la variable confusora y explique por qué genera la asociación observada.
3. Explique con sus propias palabras por qué el problema fundamental de la inferencia causal (Holland, 1986) implica que ningún algoritmo, por sofisticado que sea, puede calcular el efecto causal individual $Y_{i(1)} - Y_{i(0)}$ para una unidad específica a partir de datos observacionales de esa unidad.
4. Un equipo de datos entrena un modelo de *gradient boosting* para predecir ventas a partir de si un producto tuvo o no una promoción, alcanzando un R^2 de 0.95. El equipo concluye que «la promoción explica el 95% de las ventas» y recomienda aplicarla a todos los productos. ¿Qué está mal en este razonamiento, en términos de los niveles de la escalera de causalidad?

1.6.2 Notas bibliográficas

El marco de resultados potenciales se origina en (Neyman 1923) para experimentos agrícolas aleatorizados y se extiende a estudios observacionales en (Rubin 1974); (Holland 1986) formaliza el problema fundamental de la inferencia causal y sigue siendo la referencia estándar para esa formulación. (Pearl et al. 2018) ofrece la exposición más accesible de la escalera de causalidad para un público no especializado; el tratamiento técnico completo de la Jerarquía Causal de Pearl se encuentra en (Bareinboim 2025), fuente principal de contenido teórico--causal en la segunda mitad de este curso, y en (Pearl 2009). (Simpson 1951) es la referencia original de la paradoja que lleva su nombre, aunque el fenómeno había sido descrito antes por Yule. Para una introducción contemporánea y aplicada al marco contrafactual, véase (Hernán et al. 2020), ya citado en capítulos posteriores.

Capítulo 2

Anatomía de un sistema de ML en producción

Cuadro de decisión del capítulo

[**Decisión**]	Tratar el modelo como un componente de un sistema, no como el sistema entero
[**Alternativas**]	Enfoque centrado en el modelo (*model-centric*) vs. enfoque centrado en el sistema (*system-centric*, DMLS)
[**Trade-off clave**]	Velocidad de iteración académica vs. confiabilidad operativa
[**Stack**]	Python, Git, Docker, cuenta en Hugging Face Hub (Lab 0 de este capítulo)
[**Riesgo principal**]	Confundir «el modelo entrena bien» con «el sistema está listo para producción»

2.1 El problema de diseño

Un equipo de análisis de datos entrena un estimador de efectos causales —digamos, un modelo DML del tipo que se construirá en la Parte II de este curso— y obtiene un intervalo de confianza razonable, diagnósticos de identificación satisfactorios y una historia convincente sobre el mecanismo económico. El equipo concluye, con toda razón desde el punto de vista estadístico, que «el modelo funciona». Y sin embargo, si ese modelo se entrega tal cual a un equipo de producto para que decida a quién ofrecer un subsidio, casi con certeza algo saldrá mal en cuestión de semanas: la función que calcula las covariables en el entorno de producción no coincide exactamente con la que se usó para entrenar; nadie sabe qué hacer cuando llegan datos con un valor faltante; no existe forma de saber, tres meses después, si la población a la que se le sigue aplicando el modelo todavía se parece a la población de entrenamiento; y si el modelo empieza a fallar, nadie recibe una alerta hasta que un usuario se queja.

El problema de diseño de este capítulo es reconocer, antes de escribir una sola línea de código de estimación, que **un modelo entrenado y un sistema en producción son dos objetos distintos**, y que la distancia entre ambos no se cierra con más precisión estadística

sino con decisiones de ingeniería. Como señala (Huyen 2022), cuando la mayoría de las personas piensan en «un sistema de machine learning» piensan únicamente en el algoritmo; sin embargo, el algoritmo es apenas un componente de un sistema que también incluye la interfaz con la que interactúan usuarios y desarrolladores, el stack de datos que alimenta al modelo, el *backend* de cómputo sobre el que corre, y la infraestructura que permite desarrollarlo, desplegarlo, monitorearlo y actualizarlo. Este curso dedica su segunda mitad a esos componentes no-algorítmicos, precisamente porque suelen ser el motivo real por el que un proyecto de ciencia de datos técnicamente correcto fracasa en producción.

2.2 Opciones y trade-offs

Existen, en la práctica, dos filosofías de desarrollo que compiten por el tiempo y la atención de un equipo de datos.

Opción 1 — Enfoque centrado en el modelo (*model-centric*). El foco está en mejorar la especificación, el algoritmo o la arquitectura, manteniendo los datos y el entorno de despliegue relativamente fijos. Es el modo de trabajo dominante en la academia y en competencias como Kaggle: se compite por una décima de mejora en una métrica sobre un dataset fijo. Su ventaja es la velocidad de iteración sobre la pregunta metodológica; su límite es que ignora, por construcción, todo lo que ocurre antes de tener el dataset y después de tener el modelo entrenado.

Opción 2 — Enfoque centrado en el sistema (*system-centric*, DMLS). El foco se reparte deliberadamente entre datos, modelo e infraestructura, entendiendo que mejorar cualquiera de los tres puede mejorar el desempeño observado del sistema. Requiere invertir tiempo en aspectos que no son estadísticamente «interesantes» —formato de almacenamiento, **feature stores**, **logging**, **tests**— y por eso suele sentirse, al principio, como una desviación del trabajo «real». Su ventaja es que es, empíricamente, el único enfoque que sobrevive el contacto con datos reales que cambian con el tiempo y con usuarios reales que dependen del sistema.

Un segundo trade-off, ortogonal al anterior, es el del **proceso** de desarrollo:

Dimensión	Proceso lineal («waterfall»)	Proceso iterativo (DMLS)
Estructura	Recolectar datos → entrenar → desplegar → fin	Ciclo continuo: diseño → datos → desarrollo → despliegue → monitoreo → vuelve a diseño
Supuesto implícito	El mundo (los datos, los usuarios) no cambia después del despliegue	El mundo cambia; el sistema debe detectarlo y adaptarse
Cuándo falla	En cualquier entorno de producción real, tarde o temprano	Requiere infraestructura de monitoreo que el proceso lineal no contempla

El proceso lineal no es una decisión de diseño defendible en 2026 para un sistema que opera de forma continua; se incluye en la tabla porque sigue siendo, de hecho, cómo se enseña y se

ejecuta gran parte de los proyectos de ciencia de datos aplicada, y reconocerlo como la opción por defecto —y como opción inadecuada— es el primer paso para adoptar deliberadamente la alternativa.

2.3 Decisión de referencia para el curso

Este curso adopta explícitamente el **enfoque centrado en el sistema** y el **proceso iterativo** de DMLS como su filosofía de trabajo, formalizados en cuatro requisitos que todo sistema construido en el curso debe satisfacer —reinterpretados aquí para el caso en que el objetivo del sistema es estimar un efecto causal, no solo predecir:

Opciones y trade-offs

- **Confiabilidad**: el sistema sigue produciendo el comportamiento correcto incluso en presencia de errores de datos o de infraestructura. Para un sistema causal, esto incluye algo que un sistema puramente predictivo no necesita: que el efecto estimado siga siendo **válido bajo los supuestos de identificación declarados**, no solo que el servicio esté «arriba».
- **Escalabilidad**: el sistema puede crecer en volumen de datos, de tratamientos o de usuarios sin un rediseño completo. Para un sistema causal, incluye poder re-estimar el efecto sobre subpoblaciones nuevas sin invalidar el pipeline de identificación existente.
- **Mantenibilidad**: otra persona del equipo —no quien lo construyó— puede entender, auditar y modificar el sistema. Para un sistema causal, esto exige que los supuestos de identificación estén **documentados y sean auditables**, no solo el código.
- **Adaptabilidad**: el sistema puede ajustarse a cambios en el entorno (nuevos datos, nuevas poblaciones, nuevas políticas). Para un sistema causal, incluye poder **detectar** cuándo un supuesto causal —no confusión, soporte común, fuerza del instrumento— dejó de sostenerse, tema que retomaremos en el Módulo 5.

Estos cuatro requisitos, y no la precisión del estimador considerada de forma aislada, son el criterio con el que se evaluará cada notebook de producción (Serie B) del curso.

2.4 Implementación de referencia: el esqueleto del proyecto del curso

Todo el trabajo práctico del curso —Serie A y Serie B de notebooks, y el proyecto final— se organiza sobre un mismo esqueleto de repositorio, que se establece en este capítulo y no vuelve a discutirse:

```
proyecto-causal/
├── data/                # datos crudos y procesados (no versionados en git
```

```
salvo muestras pequeñas)
├── notebooks/
│   ├── serie_a_caso_estudio/     # A1–A4: identificación y estimación
│   └── serie_b_produccion/       # B1–B4: despliegue en Hugging Face
├── src/
│   ├── identification/   # DAGs, diagnósticos de no confusión y soporte común
│   ├── estimation/        # DML, meta-learners, bosques causales
│   └── service/            # función de inferencia, app FastAPI
├── tests/                 # pruebas de humo y de sanidad estadística
├── .github/workflows/      # CI/CD (se detalla en el Capítulo 20)
├── Dockerfile
└── model_card.md           # plantilla del Apéndice G
```

Lab 0 de esta sesión consiste en dejar este esqueleto funcionando localmente: entorno de Python con las librerías del Apéndice B, repositorio Git inicializado, cuenta creada en Hugging Face Hub, y Docker instalado y verificado con `docker run hello-world`. No se estima ningún efecto causal todavía; el objetivo es puramente de infraestructura.

2.5 Aplicación al sistema causal del curso

Vale la pena ser explícitos sobre qué cambia, en esta anatomía de sistema, cuando el «modelo» no predice sino que estima un efecto. Primero, **el artefacto que se sirve no es solo un objeto entrenado**: es un estimador más un conjunto de supuestos de identificación que deben viajar junto con él —de nada sirve servir los coeficientes de un DML sin la documentación de qué variables se controlaron y bajo qué DAG supuesto, porque sin eso nadie puede saber si el efecto sigue siendo válido para una nueva observación. Segundo, **la interfaz con el usuario debe comunicar algo más que un número**: un sistema predictivo puede mostrar una predicción puntual; un sistema causal responsable debe comunicar el intervalo de confianza y, cuando sea posible, la población para la cual el efecto fue estimado, para evitar que se use fuera de su dominio de validez. Tercero, **reentrenar no es inocuo**: en un sistema predictivo, reentrenar con datos nuevos casi siempre es deseable si mejora la métrica de error. En un sistema causal, incorporar datos nuevos puede cambiar silenciosamente la población para la que el efecto estimado es representativo —un problema de validez externa, no solo de varianza — y este curso volverá sobre esto en el Capítulo 31 (aprendizaje continuo).

2.6 Señales de alerta y checklist operativo

- ☐ El notebook de estimación no tiene ninguna versión de los datos ni del código asociados (no hay forma de reproducir el resultado).

  ☐ La función que calcula las covariables en el notebook de entrenamiento y la que las calculará en producción no son literalmente la misma función (riesgo de **training-serving skew**).

- ☐ No existe ningún documento, ni siquiera informal, que declare los supuestos de identificación asumidos.
- ☐ Nadie en el equipo, aparte de quien entrenó el modelo, podría explicar bajo qué población es válido el efecto estimado.
- ☐ No hay ningún mecanismo, ni manual, para detectar si las covariables que llegan en producción se parecen a las de entrenamiento.

Si dos o más de estas señales están presentes, el sistema —por preciso que sea el estimador— no está listo para producción según los criterios de este curso.

2.7 Ejercicios y notas bibliográficas

2.7.1 Ejercicios conceptuales

1. Enumere los componentes de un sistema de ML según (Huyen 2022) (más allá del algoritmo) y, para cada uno, dé un ejemplo concreto de cómo fallaría específicamente si el sistema estima un efecto causal en lugar de predecir.
2. Explique, con sus propias palabras, por qué «reentrenar con datos nuevos» es una decisión más delicada en un sistema causal que en uno puramente predictivo.
3. Revise los cuatro requisitos de la 2.3 (confiabilidad, escalabilidad, mantenibilidad, adaptabilidad) y ordénelos según cuál cree que es más fácil de descuidar en un proyecto real. Justifique.

2.7.2 Ejercicio de infraestructura (Lab 0)

1. Complete el Lab 0: cree el esqueleto de repositorio de la 2.4, inicialice Git, cree una cuenta en Hugging Face Hub si no tiene una, instale Docker y verifique la instalación. Entregue una captura de pantalla de `docker run hello-world` corriendo exitosamente.

2.7.3 Notas bibliográficas

(Huyen 2022) es la fuente principal de este capítulo y del enfoque pedagógico de todo el curso; sus Capítulos 1 y 2 («Overview of Machine Learning Systems» e «Introduction to Machine Learning Systems Design») desarrollan en detalle la distinción entre modelo y sistema, y los cuatro requisitos aquí adaptados al caso causal. (Sculley et al. 2015) es el artículo seminal que documentó por primera vez, de forma sistemática, cómo el código del modelo de aprendizaje automático suele ser una fracción pequeña del sistema real, y cómo la «deuda técnica» se acumula en el resto de los componentes. (Breck et al. 2017) ofrece una rúbrica operativa —«the ML Test Score»— que puede usarse como complemento al checklist de la 2.6 para auditar la preparación de un sistema para producción.

Parte II

Fundamentos estructurales y de datos

Esta parte formaliza el problema causal con Modelos Causales Estructurales y la Jerarquía Causal de Pearl, desarrolla el aparato de regularización y post-selección en alta dimensión, y establece el criterio de puerta trasera para distinguir controles legítimos de controles que sesgan la estimación. En paralelo, construye la capa de datos e ingeniería de features de un sistema causal en producción, sin la cual ningún resultado de identificación sobrevive el paso a un pipeline real.

Capítulo 3

Modelos Causales Estructurales y la Jerarquía Causal de Pearl

Cuadro sinóptico del capítulo

[**Problema**]	Formalizar matemáticamente qué significa «causa» y qué pregunta responde cada nivel de causalidad
[**Estimando**]	Distribuciones $P(v)$, $P(y \texttt{ mid } \text{do}(x))$ y $P(y_x \texttt{ mid } \text{evidencia})$
[**Identificación**]	Un Modelo Causal Estructural (SCM) más su grafo asociado
[**Método**]	Do-calculus (adelanto; formalización completa en el 8)
[**Garantía**]	Cada nivel de la PCH requiere estrictamente más supuestos que el anterior
[**Librerías**]	`networkx`, `dowhy`, `causal-learn`
[**Validación**]	Verificación de que el grafo supuesto es consistente con las independencias observadas

3.1 El problema: formalizar «causa» matemáticamente

El 1 introdujo, de forma intuitiva, la distinción entre preguntas de asociación, de intervención y de contrafactual. Este capítulo formaliza esa distinción con dos herramientas que se usan indistintamente en la literatura moderna de inferencia causal: el marco de **resultados potenciales** (ya presentado en el capítulo anterior) y el marco de **Modelos Causales Estructurales** (SCM) de Judea Pearl, que Elias Bareinboim y su escuela extienden hasta la **Jerarquía Causal de Pearl** (PCH). Los dos marcos son formalmente equivalentes bajo condiciones generales, pero difieren en qué hacen explícito: los resultados potenciales son más naturales para razonar sobre un tratamiento y un resultado; los SCM son más naturales para razonar sobre sistemas con muchas variables relacionadas entre sí —precisamente el tipo de sistema que aparece en un pipeline de datos de producción—, y por eso son el lenguaje que usaremos en la segunda mitad de este curso al hablar de arquitecturas de datos y de despliegue.

La pregunta que este capítulo responde es doble: **(i)** ¿qué objeto matemático representa completamente un sistema causal?, y **(ii)** dado ese objeto, ¿qué tres tipos de pregunta —y solo tres— se le pueden hacer, y qué información mínima requiere responder cada una?

3.2 Teoría: SCM y la Jerarquía Causal de Pearl

3.2.1 Modelos Causales Estructurales

Definición 3.1 *(SCM)*

Un **Modelo Causal Estructural** (SCM) es una tupla $M = \langle U, V, F, P(u) \rangle$ donde: U es un conjunto de variables exógenas (no modeladas, «de fondo»); $V = \{V_1, ..., V_n\}$ es un conjunto de variables endógenas (las que el modelo explica); $F = \{f_1, ..., f_n\}$ es un conjunto de funciones estructurales tales que cada $V_i = f_i(\text{pa}_i, U_i)$, donde $\text{pa}_i \subseteq V \setminus \{V_i\}$ son los «padres» de V_i (sus causas directas dentro del modelo) y $U_i \subseteq U$; y $P(u)$ es una distribución de probabilidad sobre las variables exógenas.

Cada SCM induce un **grafo causal** G: un nodo por cada variable en V, y una flecha $V_j \to V_i$ si $V_j \in \text{pa}_i$. Este grafo —el mismo objeto que se usó informalmente en el 1 para hablar de confusores— no es una descripción estadística sino una afirmación sobre **mecanismos**: cada ecuación $V_i = f_i(\text{pa}_i, U_i)$ representa un proceso físico, biológico, económico o social real, no simplemente una relación estadística ajustada a los datos. Esta es la diferencia esencial entre un modelo causal y un modelo predictivo: un modelo predictivo puede ajustar Y en función de X sin comprometerse con ninguna afirmación sobre mecanismos; un SCM afirma que si se **intervinieran** las causas de Y, Y cambiaría exactamente según su ecuación estructural.

La conexión con el marco de resultados potenciales del capítulo anterior es directa: dado un SCM con variable de tratamiento D y resultado $Y = f_Y(D, X, U_Y)$, el resultado potencial $Y(d)$ se define como el valor que toma la ecuación estructural de Y al fijar $D = d$: $Y(d) := f_Y(d, X, U_Y)$. Los dos marcos son, bajo esta correspondencia, dos lenguajes para el mismo objeto.

3.2.2 Los tres niveles de la Jerarquía Causal de Pearl

Cada SCM M induce tres distribuciones de probabilidad distintas, que corresponden exactamente a los tres niveles adelantados en el 1:

Nivel (PCH)	Distribución inducida	Se calcula con
$\mathcal{L}_1$ — Asociación	$P(v)$: la distribución observacional conjunta de V	Los datos observados, sin modificar M
$\mathcal{L}_2$ — Intervención	$P(y \texttt{ mid } \text{do}(x))$: la distribución de Y tras *reemplazar* la ecuación de X por la constante x	El SCM «quirúrgicamente modificado» M_x (se borra f_X y se fija $X = x$)

Nivel (PCH)	Distribución inducida	Se calcula con
$\mathcal{L}_3$ — Contrafactual	$P(y_x \texttt{ mid } e)$: la distribución de «Y hubiera sido y si X hubiera sido x», dada la evidencia e efectivamente observada	Un procedimiento de tres pasos: abducción, acción, predicción (ver más abajo)

El operador $\text{do}(x)$ formaliza matemáticamente qué significa «intervenir»: no es lo mismo **observar** que $X = x$ (lo que simplemente selecciona el subconjunto de unidades con $X = x$, y por tanto puede estar contaminado por confusión, como en las tres escenas del capítulo anterior) que **forzar** $X = x$ mediante una intervención externa que anula el mecanismo natural por el que X se determinaba. Formalmente, $P(y \texttt{ mid } \text{do}(x)) \neq P(y \texttt{ mid } X = x)$ en general; ambas coinciden si y solo si no hay confusión entre X y Y —la condición que el resto del curso dedica tanto esfuerzo a verificar o inducir.

Ejemplo 3.2

Los tres niveles sobre el mismo SCM. Considere el SCM simplificado $D = f_D(U_D)$, $Y = f_Y(D, X, U_Y)$, $X = f_X(U_X)$, con U_D y U_Y correlacionados (confusión no observada entre D y Y). En $\mathcal{L}_1$, $P(y \texttt{ mid } D = d)$ mezcla el efecto de D sobre Y con la correlación espuria inducida por U_D, U_Y compartidos. En $\mathcal{L}_2$, $P(y \texttt{ mid } \text{do}(D = d))$ se calcula sobre el modelo modificado donde f_D se reemplaza por la constante d, cortando el vínculo entre U_D y D —y por tanto elimina exactamente la confusión—. En $\mathcal{L}_3$, dado que se observó a un individuo específico con $D = 0, Y = y_0$, la pregunta «¿qué Y habría tenido si D hubiera sido 1?» requiere primero **inferir** la distribución de U_D, U_Y compatible con lo observado (abducción), luego **forzar** $D = 1$ en el modelo (acción), y finalmente **propagar** para predecir Y (predicción) — de ahí el nombre del procedimiento de tres pasos de Pearl para cómputo contrafactual.

Bareinboim ((Bareinboim 2025)) enfatiza un punto que suele perderse en presentaciones más elementales de esta jerarquía: **cada nivel es estrictamente más informativo que el anterior**, en el sentido de que conocer la distribución de $\mathcal{L}_2$ no basta, en general, para determinar la de $\mathcal{L}_3$ (se requieren supuestos adicionales sobre la forma funcional o sobre monotonicidad), y conocer $\mathcal{L}_1$ nunca basta para determinar $\mathcal{L}_2$ sin supuestos de identificación —el llamado **Teorema de la Jerarquía Causal**, que establece que colapsar un nivel en el inferior es, en general, imposible sin información extra—. Este resultado es la razón formal, y no solo intuitiva, de la tesis del Capítulo 1: los datos observacionales solos no responden preguntas causales.

3.3 Algoritmo e implementación

El siguiente listado construye un SCM simple en Python —una versión estilizada del ejemplo de becas y calificaciones del Capítulo 1, con una variable de motivación U no observada que confunde la relación entre beca (D) y calificación (Y)— y calcula numéricamente $P(y \texttt{ mid } D = 1)$ ($\mathcal{L}_1$) frente a $P(y \texttt{ mid } \text{do}(D = 1))$ ($\mathcal{L}_2$), mostrando que difieren.

```
                                import numpy as np
                               import pandas as pd

                           rng = np.random.default_rng(7)
                                     n = 50_000

                  # U: motivación no observada (confusor de D y de Y)
                               U = rng.normal(size=n)

    # D (beca): depende de U -- estudiantes mas motivados piden mas becas
              D_obs = (U + rng.normal(size=n) > 0.3).astype(int)

  # Y (calificacion): depende de D (efecto causal verdadero = 0.4) y de U
       Y_obs = 0.4 * D_obs + 0.9 * U + rng.normal(scale=0.5, size=n)

              df = pd.DataFrame({"U": U, "D": D_obs, "Y": Y_obs})

               # Nivel 1 (asociacion): P(Y | D=1) observacional
assoc = df.loc[df["D"] == 1, "Y"].mean() - df.loc[df["D"] == 0, "Y"].mean()

     # Nivel 2 (intervencion): simular do(D=1) para TODA la poblacion,
       # reemplazando la ecuacion estructural de D por la constante 1
          Y_do1 = 0.4 * 1 + 0.9 * U + rng.normal(scale=0.5, size=n)
          Y_do0 = 0.4 * 0 + 0.9 * U + rng.normal(scale=0.5, size=n)
                  intervencion = Y_do1.mean() - Y_do0.mean()

     print(f"Diferencia asociativa  P(Y|D=1)-P(Y|D=0):  {assoc:.3f}")
  print(f"Efecto interventivo    P(Y|do(D=1))-do(0): {intervencion:.3f}")
        print(f"Efecto causal verdadero (por construccion): 0.400")
      # La diferencia asociativa esta inflada por la confusion via U;
       # el efecto interventivo recupera el 0.4 verdadero porque do()
             # anula el mecanismo que generaba D a partir de U.
```

Listado 3.1: SCM simulado: divergencia entre asociación e intervención.

En la práctica, no observamos U ni podemos, literalmente, «reemplazar la ecuación de D» como en el listado —eso solo es posible porque el ejemplo es una simulación donde conocemos el SCM verdadero—. El problema aplicado de identificación, que el resto del curso desarrolla, es precisamente cómo estimar $P(y \mid \mathrm{do}(x))$ **sin** conocer el SCM completo, usando solo datos observacionales más supuestos sobre el grafo (por ejemplo, con Post-Double-Selection o DML, si se asume no confusión condicional en las covariables observadas).

3.4 Caso empírico: consistencia del grafo supuesto

Antes de estimar cualquier efecto, es buena práctica verificar que el grafo causal supuesto sea **consistente** con las independencias condicionales observadas en los datos —un chequeo necesario aunque no suficiente, ya que varios grafos distintos pueden ser observacionalmente

equivalentes—. Con el dataset de retornos a la educación de Card (1995), ya usado en capítulos posteriores, un grafo que supone que la cercanía a una universidad (`nearc4`) afecta la educación pero no directamente el salario, controlando por educación, implica una independencia condicional testeable: `nearc4` y `wage` deberían ser independientes dado `educ` y el resto de covariables. `dowhy` y `causal-learn` implementan estos chequeos de forma automatizada (`refutar` en el primero, tests de independencia condicional en el segundo); ambos se detallan en el Apéndice B.

3.5 Riesgos, límites y sensibilidad

Riesgos, límites y mitigación

- **Confundir $P(y \texttt{ mid } X = x)$ con $P(y \texttt{ mid } \mathrm{do}(x))$.** Es, con diferencia, el error más frecuente entre practicantes de *machine learning* que se acercan por primera vez a la inferencia causal: tratar un filtro (`df[df.D==1]`) como si fuera una intervención. Son el mismo objeto solo si no hay confusión.
- **Asumir que el grafo es «el verdadero».** Un SCM formaliza supuestos, no los verifica. Ningún test estadístico puede confirmar que un grafo causal es correcto —solo puede refutarlo si es inconsistente con los datos—. La responsabilidad de argumentar el grafo es del analista, no del algoritmo.
- **Tratar $\mathcal{L}_3$ como si fuera $\mathcal{L}_2$.** Una pregunta contrafactual («¿habría aprobado María si…?») no se responde con el mismo procedimiento que una pregunta interventiva poblacional («¿qué pasaría si becáramos a todos?»). Colapsar ambas —muy común en comunicación de resultados a audiencias no técnicas— puede llevar a promesas que el modelo no puede respaldar.

3.6 Ejercicios y notas bibliográficas

3.6.1 Ejercicios teóricos

1. Escriba el SCM (variables, ecuaciones estructurales, grafo) para el ejemplo de la paradoja de Simpson del 1 (tratamiento, gravedad del caso, recuperación). Indique cuál nodo es el confusor.
2. Para el SCM del ejercicio anterior, escriba explícitamente el modelo modificado $M_{\{\mathrm{do}(D=1)\}}$ y explique, en términos del grafo, qué arista(s) desaparece(n).
3. Explique por qué, en general, $P(y \texttt{ mid } \mathrm{do}(x))$ no puede calcularse a partir de $P(v)$ sin supuestos adicionales sobre el grafo (Teorema de la Jerarquía Causal). Dé un ejemplo de dos SCM distintos que induzcan la misma $P(v)$ pero distinta $P(y \texttt{ mid } \mathrm{do}(x))$.

3.6.2 Ejercicios computacionales

1. Modifique el listado Listado 3.1 para que el efecto causal verdadero de D sobre Y sea 0 (sin efecto), manteniendo la confusión vía U. Verifique que la diferencia asociativa sigue siendo distinta de cero mientras que el efecto interventivo converge a 0.
2. Usando `causal-learn` o `dowhy` (Apéndice B), cargue el dataset `close_college` (Card, 1995) y ejecute un test de independencia condicional entre `nearc4` y `wage` dado `educ` y los controles estándar del capítulo 5. ¿El resultado es consistente con el grafo supuesto en la 3.4?

3.6.3 Notas bibliográficas

Este capítulo sigue de cerca la formalización de (Bareinboim 2025), fuente principal de contenido teórico-causal del curso, y la exposición clásica de (Pearl 2009), que introduce el operador do(), el procedimiento de tres pasos para cómputo contrafactual, y el cálculo-do que retomaremos en el 8. (Imbens et al. 2015) discute en detalle la equivalencia formal entre el marco de resultados potenciales y el de SCM. Para una introducción visual y menos técnica a la jerarquía, véase nuevamente (Pearl et al. 2018), ya citado en el capítulo anterior.

Capítulo 4

Ingeniería de datos para sistemas causales

Cuadro de decisión del capítulo

[**Decisión**]	Adoptar un esquema explícito de roles causales sobre los datos, no solo un esquema de tipos
[**Alternativas**]	Tabla ancha sin metadatos vs. esquema con roles etiquetados (tratamiento/resultado/covariable/instrumento)
[**Trade-off clave**]	Velocidad de prototipado vs. prevención sistemática de errores de identificación
[**Stack**]	pandas, Parquet, `pandera` (validación de esquema)
[**Riesgo principal**]	Que el tratamiento se filtre como covariable, o que una variable post-tratamiento contamine X

Implementación en el monorepo del curso (2026)

En el monorepo del curso (2026) los datos de práctica viven en `data_benchmark_causal/…/datasets/` como **CSV** + `metadata.json` (no Parquet obligatorio). Los roles de variables se resuelven con `lib.data_io.load_scenario` y el puente `lib.schema.CausalSchema.from_scenario(«real/jobs»)` (API alineada a este capítulo). Labs: `L04`–`L06`. Entrega: **Notebook A1** (Jobs). Stack: `03_notebooks_caso_estudio/lib/`.

4.1 El problema de diseño

La ingeniería de datos «genérica» —la que enseña (Huyen 2022) en su Capítulo 3— se pregunta qué formato de almacenamiento usar, qué modelo de datos (relacional, documento, grafo) es apropiado, y cómo pasar datos entre procesos. Todas esas preguntas siguen siendo relevantes para un sistema causal, pero hay una pregunta adicional, específica, que un pipeline puramente predictivo nunca necesita responder: **¿qué rol estructural juega cada columna del dataset?** En un problema de predicción, todas las columnas de entrada son, en principio,

intercambiables: el modelo puede usarlas todas para minimizar el error. En un problema causal, las columnas no son intercambiables en absoluto: una es el **tratamiento** D, una es el **resultado** Y, algunas son **covariables** X que pueden controlarse legítimamente, y —crucialmente— algunas columnas que **parecen** covariables razonables son en realidad variables **post-tratamiento**, que no deben controlarse porque están en la ruta causal entre D e Y y su inclusión sesga la estimación (un error que retomaremos formalmente en el 8 al hablar de «buenos y malos controles»).

Si el pipeline de datos no distingue estos roles desde el origen —si todo llega como una tabla ancha sin metadatos— la responsabilidad de no confundirlos recae enteramente en la memoria de quien escribe cada notebook, lo cual falla predeciblemente a medida que el sistema crece y más personas lo tocan.

4.2 Opciones y trade-offs

Opciones y trade-offs

- **Tabla ancha sin metadatos.** Cada notebook decide, mediante nombres de columnas hardcodeados en el código, cuál es D, cuál es Y y cuáles son X. Es la opción más rápida para un análisis único y exploratorio, y la que domina en la práctica académica. Su costo aparece cuando el mismo dataset se reutiliza en varios notebooks o, peor, cuando el pipeline de producción necesita reconstruir las mismas columnas: nada impide que dos notebooks distintos definan a X de forma sutilmente diferente (uno incluye una variable post-tratamiento, el otro no), produciendo estimaciones no comparables sin que nadie lo note.
- **Esquema con roles etiquetados.** Se declara, en un único lugar (un archivo de configuración o una clase de datos), qué columna es el tratamiento, cuál el resultado, cuáles son covariables válidas para controlar, y cuáles son instrumentos si los hay. Todo notebook y todo servicio de producción consume ese esquema en lugar de hardcodear nombres de columnas. El costo es la fricción inicial de definir el esquema antes de poder «simplemente correr una regresión».

Un segundo trade-off, heredado directamente de (Huyen 2022) Capítulo 3, es el de formato de almacenamiento: formatos orientados a fila (CSV, JSON) son más simples de inspeccionar manualmente pero más lentos para leer solo un subconjunto de columnas —exactamente lo que un pipeline de estimación causal necesita hacer repetidamente (leer Y, D y un subconjunto de X, sin cargar el resto)—; formatos orientados a columna (Parquet) son más eficientes para ese patrón de acceso y, además, permiten adjuntar metadatos de esquema directamente en el archivo.

4.3 Decisión de referencia para el curso

Todos los datasets usados en el curso —tanto en la Serie A (casos de estudio) como en la Serie B (producción)— se acompañan de un **esquema de roles causales** explícito, almacenado

junto con los datos en formato Parquet. El esquema mínimo declara cuatro listas de nombres de columna: `treatment`, `outcome`, `covariates` y, cuando aplica, `instruments`. Ninguna función de estimación del curso acepta un DataFrame «pelado»; todas reciben el DataFrame junto con su esquema, de modo que sea sintácticamente imposible, por ejemplo, pasar el tratamiento como una covariable por accidente.

4.4 Implementación de referencia

```
                from dataclasses import dataclass, field
                          import pandas as pd

                              @dataclass
                          class CausalSchema:
                              treatment: str
                               outcome: str
                           covariates: list[str]
             instruments: list[str] = field(default_factory=list)

                  def validate(self, df: pd.DataFrame) -> None:
            declared = {self.treatment, self.outcome, *self.covariates,
                           *self.instruments}
                       missing = declared - set(df.columns)
                                    if missing:
           raise ValueError(f"Columnas declaradas en el esquema pero ausentes:
                                {missing}")
          overlap = set(self.covariates) & {self.treatment, self.outcome}
                                    if overlap:
             raise ValueError(f"El tratamiento o el resultado no pueden ser
                         covariables: {overlap}")

                 def X(self, df: pd.DataFrame) -> pd.DataFrame:
                             return df[self.covariates]

                  def D(self, df: pd.DataFrame) -> pd.Series:
                              return df[self.treatment]

                  def Y(self, df: pd.DataFrame) -> pd.Series:
                               return df[self.outcome]


       # Ejemplo: dataset de retornos a la educacion (Card, 1995)
                           schema = CausalSchema(
                               treatment="educ",
                                outcome="wage",
                 covariates=["exper", "black", "south", "married"],
                             instruments=["nearc4"],
                                      )
               df = pd.read_parquet("data/close_college.parquet")
schema.validate(df)  # lanza error si, p.ej., "educ" quedo tambien en
                                covariates
```

Listado 4.1: Esquema de roles causales como estructura de datos explícita.

Este patrón —trivial en apariencia— es exactamente el tipo de decisión de infraestructura que (Huyen 2022) argumenta que rara vez se enseña y que sistemáticamente se paga como deuda técnica más adelante ((Sculley et al. 2015)).

4.5 Aplicación al sistema causal del curso

El Notebook de caso de estudio A1 (Módulo 1 del curso) exige explícitamente construir el `CausalSchema` del dataset elegido **antes** de correr ninguna estimación, precisamente para forzar la pregunta de identificación (1, 3) a un paso de ingeniería concreto y verificable, en vez de dejarla implícita en la cabeza de quien analiza los datos. Este mismo esquema reaparece, sin cambios, en el Notebook de producción B1 (despliegue del primer endpoint, Capítulo 15): la función de inferencia en producción usa `schema.covariates` para saber exactamente qué columnas esperar del cliente que consulta el servicio, cerrando el círculo entre el notebook de entrenamiento y el servicio desplegado.

4.6 Señales de alerta y checklist operativo

- ☐ El nombre de la columna de tratamiento aparece hardcodeado en más de un notebook, con riesgo de que diverja.

  ☐ Ninguna función de carga de datos valida que el tratamiento y el resultado no estén también en la lista de covariables.

  ☐ No hay forma de saber, sin leer el código de generación de features, si alguna covariable fue medida **después** del tratamiento.

  ☐ El formato de almacenamiento (CSV plano) obliga a cargar todas las columnas para leer solo tres.

4.7 Ejercicios y notas bibliográficas

4.7.1 Ejercicios conceptuales

1. Para el dataset de becas del 1, proponga tres covariables legítimas para X y una variable que, aunque correlacionada con el resultado, sería incorrecto incluir en X por ser post--tratamiento. Justifique.
2. Explique por qué un esquema de roles causales explícito es más importante en un sistema con varios notebooks y varios colaboradores que en un análisis individual de una sola vez.

4.7.2 Ejercicio de infraestructura

1. Extienda la clase `CausalSchema` del listado Listado 4.1 con un método `summary()` que imprima, para cada rol, el número de valores faltantes y el tipo de dato de las columnas correspondientes. Aplíquelo al dataset que usará en el Notebook A1.

4.7.3 Notas bibliográficas

(Huyen 2022), Capítulo 3, desarrolla en profundidad los formatos y modelos de datos genéricos que este capítulo adapta al caso causal. La distinción entre controles legítimos y variables post--tratamiento se formaliza en el 8 con el lenguaje de grafos causales; véase también la discusión de «buenos y malos controles» en (Bareinboim 2025).

Capítulo 5

Predicción y causalidad: por qué el ML no es causal por defecto

Cuadro sinóptico del capítulo

[Problema]	Distinguir predicción de efecto causal
[Estimando]	ATE, CATE(x)
[Identificación]	No confusión, soporte común, SUTVA
[Método]	Comparación predictiva vs causal
[Nuisance]	$\mu_{d(x)}$
[Garantía]	Identificación bajo supuestos
[Librerías]	scikit-learn, statsmodels
[Validación]	Simulación de collider, placebos

Nota técnica

Este capítulo retoma, con el aparato formal de la econometría con *machine learning*, la distinción entre asociación y causalidad presentada de forma intuitiva en el 1 y formalizada con SCM y la Jerarquía Causal de Pearl en el 3. Aquí el foco pasa de «¿qué pregunta estoy haciendo?» a «¿cómo la respondo con datos de alta dimensión?» — la pregunta que estructura toda la Parte II del curso.

5.1 Problema causal, estimando e identificación

Considere un economista del Ministerio de Desarrollo Social que debe decidir a qué hogares beneficiar con un programa de transferencias condicionadas. El programa tiene presupuesto limitado: solo puede cubrir al 30% de la población elegible. El analista dispone de cien variables socioeconómicas por hogar (ingresos, composición familiar, escolaridad, empleo, vivienda, salud) y un registro histórico de hogares que recibieron o no el programa, junto con el ingreso posterior observado. La pregunta relevante es *causal*: ¿qué hogares experimentarían el mayor incremento de ingresos si recibieran el programa? La pregunta *predictiva* —qué hogares tendrán

mayores ingresos en el futuro, independientemente del programa— es distinta y, para fines de focalización, equivocada.

La distinción es crucial porque los algoritmos estándar de *machine learning* (regresión ridge, bosques aleatorios, *gradient boosting*, redes neuronales) están optimizados para predicción, no para estimación causal. Un bosque aleatorio que predice con alta precisión el ingreso futuro de un hogar puede ofrecer estimaciones sesgadas del efecto del programa, por dos razones estructurales. Primero, los algoritmos predictivos no distinguen entre variación explicada por el tratamiento y variación explicada por covariables correlacionadas con el tratamiento. Segundo, la regularización que mejora la predicción fuera de muestra introduce sesgo en la estimación de coeficientes causales, fenómeno conocido como *regularization-induced bias*.

Este capítulo introduce el marco conceptual que vertebra todo el curso. Argumentamos que **la inferencia causal requiere supuestos de identificación económicos, no estadísticos**, y que el aporte del ML consiste en flexibilizar la estimación de componentes auxiliares (*nuisance*) sin vulnerar esos supuestos. Establecemos la notación de resultados potenciales en el régimen de alta dimensionalidad, definimos los estimandos centrales del curso (ATE, ATT, CATE, *policy value*), y mostramos empíricamente con el dataset de Card (1995) sobre retornos a la educación que un modelo predictivo de alta dimensión puede arrojar respuestas radicalmente distintas a las de una estimación causal cuidadosa.

La tabla Tabla 5.1 sistematiza la diferencia entre el régimen predictivo y el régimen causal. Esta distinción estructura todo el curso.

Dimensión	**Predicción**	**Causalidad**
Objetivo	Minimizar error fuera de muestra	Estimar $\tau(x)$ con IC válido
Estimando	$E[Y \texttt{ mid } X]$	$E[Y(1) - Y(0) \texttt{ mid } X]$
Métrica de éxito	RMSE, accuracy, AUC	Sesgo, cobertura, RMSE en τ
Datos requeridos	Solo Y, X observados	Requiere $Y(0), Y(1)$ que no se observan simultáneamente
Supuestos clave	IID, estacionariedad	No confusión, soporte común, SUTVA
Regularización	Bienvenida (mejora generalización)	Peligrosa (introduce sesgo en τ)
Validación	Cross-validation estándar	Placebos, Qini, comparación con benchmarks

Tabla 5.1: Régimen predictivo vs. régimen causal.

5.2 Teoría, supuestos y garantías

5.2.1 Supuestos

Adoptamos el marco de (Neyman 1923) y (Rubin 1974). Para cada unidad i se definen dos resultados potenciales: $Y_{i(1)}$ si recibe tratamiento y $Y_{i(0)}$ si no lo recibe. La asignación observada es $D_i \in \{0,1\}$ y el resultado observado obedece la restricción de consistencia $Y_i = D_i Y_{i(1)} + (1 - D_i)Y_{i(0)}$. Las covariables pre-tratamiento $X_i \in R^p$ son un vector de dimensión p, donde en este curso p puede ser grande (decenas o cientos) e incluso crecer con n.

Los supuestos económicos que permiten identificar $\tau(x)$ a partir de datos observacionales son tres.

Supuesto 5.1 ***(No confusión condicional)***

$$Y(0), Y(1) \perp\!\!\!\perp D \texttt{ mid } X.$$

Supuesto 5.2 ***(Soporte común)***

Existe $\underline{e} > 0$ tal que $\underline{e} < P(D = 1 \texttt{ mid } X = x) < 1 - \underline{e}$ para todo x en el soporte de X.

Supuesto 5.3 ***(SUTVA / No interferencia)***

$Y_{i(d)} = Y_{i(d,d_{-i})}$ para todo d y todo i, donde d_{-i} son los tratamientos de las demás unidades.

Estos supuestos son *económicos*: su validez depende del contexto sustantivo, no de la cantidad de datos. El supuesto de no confusión condicional exige que el investigador observe todas las variables que afectan simultáneamente la asignación al tratamiento y el resultado; el supuesto de soporte común requiere que haya unidades tratadas y no tratadas en cada región del espacio de covariables; el supuesto de SUTVA / no interferencia prohíbe efectos cruzados entre unidades. La responsabilidad de justificarlos recae en el economista, no en el algoritmo.

5.2.2 Estimador y score

Definición 5.4 ***(Estimandos causales centrales)***

Bajo no interferencia (SUTVA), los estimandos de interés son:

$$\begin{aligned} \text{ATE}: &\quad \theta = E[Y(1) - Y(0)], \\ \text{ATT}: &\quad \theta_{\text{ATT}} = E[Y(1) - Y(0) \texttt{ mid } D = 1], \\ \text{CATE}: &\quad \tau(x) = E[Y(1) - Y(0) \texttt{ mid } X = x], \\ \text{Policy value}: &\quad V(\pi) = E[Y(\pi(X))], \end{aligned} \tag{5.1}$$

donde $\pi : R^p \to \{0,1\}$ es una regla de asignación de tratamiento.

El curso se centra casi exclusivamente en $\tau(x)$ y $V(\pi)$, porque son los estimandos que el ML permite abordar de manera genuinamente nueva: la heterogeneidad del efecto causal y la optimización de políticas basadas en esa heterogeneidad.

Bajo los supuestos de no confusión condicional–de soporte común, el CATE es identificable como diferencia de dos funciones de regresión condicional:

$$\tau(x) = \mu_1(x) - \mu_0(x), \quad \mu_{d(x)} = E[Y \texttt{ mid } X = x, D = d]. \tag{5.2}$$

Si estimamos $\hat{\mu}_0$ y $\hat{\mu}_1$ con un método de ML flexible (bosque aleatorio, *gradient boosting*, red neuronal) y calculamos $\hat{\tau}(x) = \hat{\mu}_1(x) - \hat{\mu}_0(x)$, se obtiene el *T-learner* que estudiaremos en el capítulo 12. La pregunta crítica es: ¿bajo qué condiciones $\hat{\tau}(x) \to \tau(x)$ en alta dimensión?

5.2.3 Garantías

El problema es que los algoritmos de ML regularizan para optimizar predicción, no estimación de efectos. (Athey 2017) ilustra esto con un ejemplo simple: si X contiene una variable fuertemente correlacionada con D pero sin efecto causal sobre Y, el algoritmo predictivo puede *descartar* D en la regresión de Y (porque X ya explica Y casi igual de bien), generando un $\hat{\mu}_d$ que ignora el efecto del tratamiento. La predicción de Y mejora, pero la estimación de τ se degrada.

Teorema 5.5 ***(Sesgo de regularización en T-learner, versión informal)***

Sea $\hat{\mu}_d$ un estimador de ML con error $\|\hat{\mu}_d - \mu_d\|_{L^2} = O_{p(a_n)}$ donde $a_n \to 0$. Entonces el T--learner $\hat{\tau} = \hat{\mu}_1 - \hat{\mu}_0$ satisface

$$\|\hat{\tau} - \tau\|_{L^2} \leq \|\hat{\mu}_1 - \mu_1\|_{L^2} + \|\hat{\mu}_0 - \mu_0\|_{L^2} = O_{p(a_n)}. \tag{5.3}$$

Sin embargo, la *distribución* asintótica de $\hat{\tau}$ no es $\sqrt{n}$-Gaussiana en general: la regularización introduce un sesgo de orden a_n que puede dominar la varianza $\sqrt{n}$.

La consecuencia operacional es directa: no se pueden construir intervalos de confianza válidos para $\tau(x) \approx \hat{\tau}(x) \pm 1.96\widehat{\text{SE}}(\hat{\tau})$ si el sesgo de regularización domina la varianza. Se requieren mecanismos adicionales—ortogonalización y *cross-fitting*, que estudiaremos en los capítulos 10 y 11— para recuperar la $\sqrt{n}$-consistencia y la validez de la inferencia.

5.3 Algoritmo e implementación

Pseudocódigo

El método conceptual de este capítulo no requiere pseudocódigo; las implementaciones prácticas se presentan en los siguientes capítulos.

5.3.1 Implementación en Python

Para hacer concreta la distinción entre predicción y causalidad, comparamos dos enfoques sobre el dataset de (Card 1995) sobre retornos a la educación. La pregunta económica es: ¿cuál es el efecto causal de un año adicional de educación sobre el logaritmo del salario? El dataset `close_college` contiene una variable instrumental `nearc4` (proximidad a una universidad de cuatro años) que usaremos solo como referencia, pero el foco aquí es contrastar la respuesta predictiva de un bosque aleatorio con la respuesta causal de una regresión OLS con controles.

```
import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestRegressor

import statsmodels.api as sm

# Cargar dataset del repositorio
df = pd.read_csv("../data/datasets/real/public_causaldata/close_college/
data.csv")
# Variables: nearc4 (IV), educ (tratamiento), lwage (resultado), controles
Y = df["lwage"].values
D = df["educ"].values
X = df[["black", "smsa", "south", "married", "exper"]].values

# --- Enfoque predictivo: bosque aleatorio que ignora el rol causal de D ---
rf = RandomForestRegressor(n_estimators=200, random_state=42)
# Entrenamos sobre [D, X] y predecimos Y
DX = np.column_stack([D, X])
rf.fit(DX, Y)
# "Efecto" predictivo: derivada numerica respecto a D
D_plus = D + 1
DX_plus = np.column_stack([D_plus, X])
efecto_pred = (rf.predict(DX_plus) - rf.predict(DX)).mean()
print(f"Efecto predictivo (RF): {efecto_pred:.4f}")

# --- Enfoque causal: OLS con controles (sin IV, solo comparacion) ---
Xc = sm.add_constant(np.column_stack([D, X]))
ols = sm.OLS(Y, Xc).fit(cov_type="HC1")
efecto_ols = ols.params[1]
se_ols = ols.bse[1]
print(f"Efecto OLS (HC1): {efecto_ols:.4f} (SE={se_ols:.4f})")
print(f"IC 95% OLS: [{efecto_ols - 1.96*se_ols:.4f}, {efecto_ols +
1.96*se_ols:.4f}]")
```

Listado 5.1: Comparación predictivo vs. causal sobre datos de Card (1995).

El *output* típico de este ejercicio muestra una divergencia instructiva. El bosque aleatorio reporta un «efecto» predictivo que mezcla la variación causal de la educación con la variación espuria introducida por la correlación entre educación y covariables omitidas. OLS con controles, al imponer una forma funcional lineal y un coeficiente constante, aísla la variación atribuible a la educación pero asume linealidad y homogeneidad. La lección no es que OLS sea superior—ambos son aproximaciones imperfectas— sino que responden a preguntas distintas y producen números distintos. Confiar ciegamente en el *output* del bosque aleatorio como estimación causal sería un error metodológico.

Diagnósticos de nuisance

Los diagnósticos de componentes *nuisance* (propensity score, outcome regression) se desarrollan en detalle en el capítulo 11, donde se presentan las herramientas específicas para evaluar la calidad de los estimadores auxiliares.

5.4 Caso empírico y validación

5.4.1 Datos y preprocesamiento

(Card 1995) es un caso paradigmático porque combina un instrumento (`nearc4`) con un set de controles de dimensión moderada. En su análisis original, Card reporta un retorno a la educación de aproximadamente 7.5% usando IV. Estudios posteriores con más controles (Belloni, Chernozhukov y Hansen, 2014) muestran que la selección de controles afecta la estimación en no menos de 1.5 puntos porcentuales.

5.4.2 Estimación principal

En nuestra aplicación con `close_college`, OLS con cinco controles básicos reporta un coeficiente de aproximadamente 0.07 (7%) con error estándar robusto de 0.004. El bosque aleatorio, al no distinguir el rol causal de `educ`, reporta un «efecto predictivo» de aproximadamente 0.04, sustancialmente menor. La diferencia se debe a que el bosque estima una derivada parcial predictiva que no aísla el efecto causal de educ: mezcla variación causal con correlaciones observadas (por ejemplo, exper) y no controla por confusores no observados (por ejemplo, habilidad). Adicionalmente, la regularización puede contraer el efecto estimado de educ si otras covariables predicen bien el salario. Desde el punto de vista predictivo esto es correcto (mejora el RMSE), pero desde el punto de vista causal es un sesgo por variable de confusión no controlada adecuadamente.

Este ejemplo motivará el desarrollo del curso: ¿cómo combinar la flexibilidad del bosque aleatorio para estimar $\mu_{d(x)}$ con la garantía de inferencia válida para τ? La respuesta, que se desarrolla en los capítulos 10–11, es la *ortogonalización de Neyman* combinada con *cross-fitting*.

5.4.3 Validación

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(42)
n = 10000

# Dos variables independientes
A = rng.normal(size=n)
B = rng.normal(size=n)

# C es collider (causa común): A -> C <- B
C = 0.8 * A + 0.8 * B + rng.normal(size=n)

df = pd.DataFrame({"A": A, "B": B, "C": C})

print("Corr(A,B) sin condicionar:", df["A"].corr(df["B"]))
print("Corr(A,B | C alto):",
df[df["C"] > df["C"].quantile(0.8)]["A"].corr(
df[df["C"] > df["C"].quantile(0.8)]["B"]))
# NOTA: condicionar en un collider crea una correlacion espuria
```

Listado 5.2: Simulación de sesgo por collider.

5.4.4 Interpretación económica

Intuición

Caja de profundización 1.1: Modelos causales estructurales y jerarquía causal

Objetivo. Distinguir tres tipos de preguntas causales: las que solo observan, las que intervienen y las que imaginan escenarios alternativos.

Ejemplo económico. Un gerente de banco quiere saber:

1. ¿Los clientes con más ingresos piden más crédito? (solo observamos)
2. ¿Qué pasa si bajamos la tasa de interés? (intervenimos)
3. ¿Este cliente específico habría pedido crédito si no le hubiéramos subido la tasa? (imaginamos)

Los tres niveles (Pearl).

$$P(Y \texttt{ mid } X = x) \neq P(Y \texttt{ mid } \mathrm{do}(X = x)) \neq P(Y_x \texttt{ mid } X = x', Y = y') \tag{5.4}$$

- **Nivel 1 – Asociación («Ver»):** $P(Y \texttt{ mid } X)$. Ej: «los clientes con más ingresos piden más crédito.»
- **Nivel 2 – Intervención («Hacer»):** $P(Y \texttt{ mid } \mathrm{do}(X))$. Ej: «si bajamos la tasa, ¿cuánto aumenta el crédito?»
- **Nivel 3 – Contrafactual («Imaginar»):** $P(Y_x \texttt{ mid } X = x', Y = y')$. Ej: «Juan no pidió crédito con la tasa alta; ¿lo habría pedido con la tasa baja?»

Importante: El ML predictivo opera en nivel 1. Para política pública (nivel 2) o responsabilidad individual (nivel 3) se necesitan supuestos causales adicionales.

5.5 Riesgos, límites y sensibilidad

5.5.1 Tabla de riesgos

La tabla Tabla 5.2 resume los riesgos principales al confundir predicción con causalidad, junto con sus consecuencias, diagnósticos y estrategias de mitigación.

Riesgos, límites y mitigación

Riesgo	Consecuencia	Diagnóstico	Mitigación
Confundir predicción con causalidad	Política errónea basada en efectos no causales	Comparar el estimando predictivo ($\partial \frac{E[Y \texttt{ mid } X, D]}{\partial} D$) con el causal ($E[Y(1) - Y(0) \texttt{ mid } X]$)	Definir explícitamente $Y(1), Y(0)$ y especificar el estimando causal de interés. Si se entrena un ML sobre (Y, D, X) y se computa la derivada respecto a D, el resultado NO es un efecto causal, sino la derivada parcial de la mejor predicción de Y, que mezcla variación causal con variación espuria
Omitir confusores	Sesgo por variable omitida no corregible con más datos	Construir un DAG y evaluar placebos	Justificar el supuesto de no confusión con teoría económica o usar un diseño alternativo. El supuesto de no confusión condicional no se vuelve verdadero con más observaciones: requiere observar las covariables correctas. Si una variable confusora no está en X, ningún algoritmo la recupera
Interpretar SHAP como causal	Sobreinterpretación de la importancia de variables	Incluir nota explícita de que SHAP es predictivo, no causal	Usar CATE con validación en placebos
Reportar RMSE como evidencia de calidad causal	Falsa confianza en estimaciones sesgadas del efecto causal	Comparar métricas predictivas (RMSE, R^2) con métricas causales (Qini, cobertura de intervalos de confianza)	Validar con placebos, Qini y ground truth sintético. Un modelo puede predecir Y con $R^2 = 0.99$ y producir estimaciones de CATE totalmente sesgadas

Tabla 5.2: Riesgos al confundir predicción con causalidad.

5.5.2 Cuándo no usar este método

Límites del marco. El marco de resultados potenciales supone *no interferencia* (supuesto de SUTVA / no interferencia). En contextos con efectos cruzados—epidemias, redes sociales, externalidades— este supuesto se vulnera y se requieren métodos específicos (inferencia causal

bajo interferencia, diseño *cluster-randomized*). El curso no aborda estos casos; ver (Aronow et al. 2017) para una referencia.

Checklist de validación

Ver el 38 para el checklist completo de validación de métodos causales.

5.6 Ejercicios y notas bibliográficas

5.6.1 Ejercicios teóricos

1. Clasifique cada pregunta en asociación, intervención o contrafactual: (a) ¿Los alumnos con mejores notas tienen mejor salario? (b) ¿Sirve dar becas para mejorar notas? (c) ¿María habría entrado a la universidad si le hubieran dado la beca?

5.6.2 Ejercicios computacionales

1. Replique la comparación predictivo vs. causal del listado Listado 5.1 usando `GradientBoostingRegressor` en lugar de `RandomForestRegressor`. ¿La divergencia entre el «efecto predictivo» y el coeficiente OLS aumenta o disminuye? ¿Por qué?
2. Construya un dataset sintético donde el supuesto de no confusión se vulnere deliberadamente (una variable U no observada afecta tanto a D como a Y). Muestre que OLS y bosque aleatorio producen estimaciones sesgadas, y verifique que el sesgo no disminuye al aumentar n.
3. Ejecute el listado Listado 5.2 y explique por qué al condicionar en C apareció una correlación espuria entre A y B.

5.6.3 Ejercicios aplicados

1. Sobre el dataset `close_college`, estime el retorno a la educación con: (a) OLS sin controles, (b) OLS con cinco controles, (c) OLS con la interacción `educ` × `exper`. Compare los tres coeficientes y explique conceptualmente por qué difieren.
2. Sobre el dataset `nsw_mixtape` (Lalonde), calcule la diferencia de medias entre tratados y controles en la variable `re78`. Luego estime el ATE por OLS con los controles `age, educ, black, hisp, marr, nodegree, re74, re75`. ¿En cuánto cambia la estimación? ¿Qué dice esto sobre el balance observacional del dataset?

5.6.4 Notas bibliográficas

(Athey 2017) provee la motivación conceptual del campo. (Imbens et al. 2015) es la referencia canónica sobre resultados potenciales. (Hernán et al. 2020) ofrece una introducción accesible al marco contrafactual. (Mullainathan et al. 2017) discute las diferencias prácticas entre predicción y estimación causal en políticas públicas. Para una revisión técnica moderna, véanse (Athey et al. 2019a) y (Chernozhukov et al. 2018a). Adicionalmente, (Pearl 2009) (disponible en español como *Causalidad*) proporciona una exposición profunda de la jerarquía causal y los

modelos estructurales, cubriendo los fundamentos conceptuales que subyacen a la distinción entre los niveles de asociación, intervención y contrafactual discutidos en este capítulo.

Capítulo 6

Regularización, selección de variables y sesgo post-selección

Cuadro sinóptico del capítulo

[Problema]	Sesgo post-selección al inferir después de seleccionar variables
[Estimando]	θ_0 en regresión de alta dimensión
[Identificación]	Sparsity aproximada
[Método]	LASSO, Post-LASSO OLS
[Nuisance]	Selección de variables
[Garantía]	Consistencia de selección bajo irrepresentabilidad
[Librerías]	scikit-learn, statsmodels
[Validación]	Monte Carlo, sensibilidad a λ

Nota técnica

El 5 estableció que responder una pregunta causal en alta dimensión requiere seleccionar controles sin sesgar la inferencia. Este capítulo formaliza precisamente ese problema — selección de variables y el sesgo post-selección que induce— y es, junto con el capítulo anterior, el prerrequisito técnico para Double Machine Learning (11, Parte II).

6.1 Problema causal, estimando e identificación

Una economista laboral desea estimar el efecto de un programa de capacitación sobre el salario posterior. Dispone de un panel con 80 variables de control potenciales: características demográficas, historial laboral, sector, región, características del empleador, expectativas subjetivas, y un conjunto de variables administrativas. No sabe cuáles de estas variables son realmente confusoras (correlacionadas con la asignación al programa y con el salario) y cuáles son irrelevantes. La intuición clásica sugeriría incluir todas en la regresión, pero con 80 controles y $n = 1,000$ observaciones, OLS sufre de alta varianza y potencia estadística reducida.

La solución aparentemente natural es seleccionar variables con LASSO y luego correr OLS con las seleccionadas. Esta intuición, sin embargo, es metodológicamente peligrosa: la selección de variables induce un sesgo en la inferencia posterior conocido como *post-selection inference* o *selective inference*. El problema fue formalizado por (Leeb et al. 2006) y (Pötscher et al. 2009), quienes mostraron que los intervalos de confianza construidos después de selección con LASSO pueden tener cobertura dramáticamente inferior a la nominal, incluso con muestras grandes.

Este capítulo desarrolla tres ideas centrales. Primero, presentamos LASSO, Ridge y Elastic Net como métodos de regularización con propiedades estadísticas distintas. Segundo, formalizamos el problema del sesgo post-selección y mostramos por qué la inferencia naïve es inválida. Tercero, anticipamos la solución—Post-Double-Selection de Belloni, Chernozhukov y Hansen (2014), que se desarrolla plenamente en el capítulo 16— como ejemplo paradigmático de cómo el ML puede integrarse con inferencia válida cuando se diseñan procedimientos que respetan la estructura del problema causal.

6.2 Teoría, supuestos y garantías

6.2.1 Supuestos

Para que LASSO sea consistente en alta dimensión, se requiere el supuesto de *sparsity*: solo unas pocas componentes de β son distintas de cero.

Supuesto 6.1 ***(Sparsity aproximada)***

El vector verdadero β_0 tiene a lo más $s \ll n$ componentes no nulas, con $s \log p = o(n)$.

6.2.2 Estimador y score

Consideremos el modelo lineal de alta dimensión $\boldsymbol{Y} = \boldsymbol{X}\beta + \varepsilon$ con $\boldsymbol{X} \in R^{n \times p}$ y $\beta \in R^p$. Cuando p es comparable a n o mayor, OLS tiene alta varianza o no está siquiera definido ($p > n$). Los estimadores penalizados resuelven esto añadiendo una penalización que contrae los coeficientes hacia cero.

Definición 6.2 ***(LASSO)***

El estimador LASSO ((Tibshirani 1996)) resuelve

$$\hat{\beta}_{\text{LASSO}} = \text{argmin}_{\beta \in R^p} \left\{ \left(\frac{1}{2n} \right) \|Y - X\beta\|_2^2 + \lambda \|\beta\|_1 \right\}, \tag{6.5}$$

donde $\|\beta\|_1 = \sum_j |\beta_j|$ es la norma ℓ_1 y $\lambda \geq 0$ es el hiperparámetro de penalización.

Definición 6.3 ***(Ridge)***

El estimador Ridge ((Hoerl et al. 1970)) usa penalización ℓ_2:

$$\hat{\beta}_{\text{Ridge}} = \text{argmin}_{\beta} \left\{ \left(\frac{1}{2n} \right) \|Y - X\beta\|_2^2 + \lambda \|\beta\|_2^2 \right\}. \tag{6.6}$$

Definición 6.4 ***(Elastic Net)***

Elastic Net ((Zou et al. 2005)) combina ambas penalizaciones:

$$\hat{\beta}_{\text{EN}} = \text{argmin}_{\beta}\left\{\left(\frac{1}{2n}\right)\|Y - X\beta\|_2^2 + \lambda[\alpha\|\beta\|_1 + (1-\alpha)\|\beta\|_2^2]\right\}, \tag{6.7}$$

con $\alpha \in [0,1]$ mezclando las dos penalizaciones.

La diferencia clave es que LASSO produce *selección de variables* (algunos $\hat{\beta}_j = 0$ exactamente), mientras que Ridge solo contrae los coeficientes sin hacer selección. Elastic Net combina ambos comportamientos y es preferible cuando hay grupos de variables altamente correlacionadas.

6.2.3 Garantías

Bajo sparsity y condiciones regulares (designado *restricted eigenvalue condition*, (Bickel et al. 2009)), LASSO satisface:

Teorema 6.5 ***(Tasa de convergencia de LASSO)***

Bajo el supuesto de *sparsity* y *restricted eigenvalue condition* con constante $\kappa > 0$, si $\lambda \asymp \sqrt{\log \frac{p}{n}}$, entonces con probabilidad alta:

$$\left\|\hat{\beta}_{\text{LASSO}} - \beta_0\right\|_1 = O_{p(s\sqrt{\log \frac{p}{n}})}, \quad \left\|\hat{\beta}_{\text{LASSO}} - \beta_0\right\|_2 = O_{p(\sqrt{s \log \frac{p}{n}})}. \tag{6.8}$$

La tasa $\sqrt{s \log \frac{p}{n}}$ es óptima en el sentido minimax. Sin embargo, el teorema garantiza consistencia en norma ℓ_2 — no consistencia componente a componente. Esto es crucial: aunque $\hat{\beta}_{\text{LASSO}}$ se acerca a β_0 en norma global, las componentes individuales pueden tener sesgo sustancial, especialmente las que LASSO selecciona incorrectamente.

El problema de Leeb & Pötscher

(Leeb et al. 2006) demostraron un resultado perturbador: la inferencia post-selección basada en LASSO es fundamentalmente problemática. La intuición es la siguiente. Supongamos que LASSO selecciona un conjunto $\hat{S} \subseteq \{1, ..., p\}$ de variables activas. Si luego corremos OLS sobre $\hat{S}$ y reportamos IC al 95% basados en la distribución t-Student estándar, la cobertura real puede ser dramáticamente inferior.

Teorema 6.6 ***(Leeb & Pötscher, informal)***

Para procedimientos de selección de modelo consistentes, la cobertura de intervalos construidos después de la selección no es uniforme en el parámetro. Para cualquier nivel nominal $1-\alpha$ y cualquier $\varepsilon > 0$, existen secuencias de parámetros y distribuciones de los datos tales que la cobertura límite del intervalo post-selección es inferior a ε. En particular, la cobertura real puede ser arbitrariamente menor que la nominal.

Intuición

Imagine que el verdadero β_j es positivo pero muy pequeño. LASSO lo selecciona solo si la estimación preliminar de β_j es suficientemente grande. Condicionado a ser seleccionado,

Algoritmo

Entrada: Datos $\{(Y_i, D_i, X_i)\}_{i=1}^n$, hiperparámetro de penalización $\lambda \geq 0$, número de folds K para validación cruzada (opcional).
Salida: Estimador $\hat{\theta}$ y error estándar HC1 $\hat{\sigma}$
Ejecutar LASSO de Y sobre (D, X) con penalización λ.
Identificar el conjunto de variables seleccionadas $\hat{S} = \left\{j : \hat{\beta}_j \neq 0\right\} \cup \{1\}$ (forzar inclusión de D).
Estimar OLS de Y sobre D y las variables en $\hat{S}$.
Calcular errores estándar robustos HC1.
return $\hat{\theta}_{\text{OLS}}, \hat{\sigma}_{\text{HC1}}$

Algoritmo 1.: Post-LASSO OLS (selección ingenua)

> $\hat{\beta}_j$ tiende a ser mayor que β_j — un sesgo de selección análogo al sesgo de publicación. El IC post-selección, al no condicionar en este hecho, queda centrado en un valor sesgado.

Soluciones: de la selección ingenua al Post-Double-Selection

(Belloni et al. 2014) propusieron una solución elegante al problema del sesgo post-selección en contextos causales. La idea central es *no seleccionar variables una sola vez*, sino *dos veces*: una para predecir Y a partir de X, otra para predecir D a partir de X. Luego se corre OLS de Y sobre D y la unión de las variables seleccionadas en ambos pasos.

Teorema 6.7 ***(Post-Double-Selection LASSO, informal)***

Bajo los supuestos de sparsity aproximada y regularidad estándar, el estimador PDS es asintóticamente normal:

$$\sqrt{n}\left(\hat{\theta}_{\text{PDS}} - \theta_0\right) \xrightarrow{d} \mathcal{N}(0, \mathcal{V}), \tag{6.9}$$

donde V es la varianza asintótica estándar de OLS con el conjunto de controles verdaderos. La inferencia basada en la distribución t-Student es válida.

La clave del éxito de PDS es la *doble selección*: las variables confusoras son aquellas que predicen tanto Y como D. Seleccionando por separado cada conjunto y tomando la unión, se capturan todas las confusoras relevantes con alta probabilidad, mientras que el sesgo de selección individual se diluye. Desarrollamos este método en detalle en el capítulo 16.

6.3 Algoritmo e implementación

6.3.1 Pseudocódigo

El procedimiento Post-LASSO OLS (o *naive* post-selection) se describe a continuación. Este algoritmo refleja la práctica ingenua que induce sesgo post-selección, y sirve como base para contrastar con la corrección introducida por PDS en el capítulo 16.

6.3.2 Implementación en Python

Ilustramos el sesgo post-selección con un dataset sintético donde conocemos el verdadero θ.

```
                            import numpy as np
                           import pandas as pd
                        import statsmodels.api as sm
                  from sklearn.linear_model import LassoCV
                from sklearn.utils import check_random_state

        def generate_data(n=500, p=100, n_confounders=5, seed=42):
    """DGP: theta=1; las primeras variables son confusoras; el resto ruido."""
                         rng = check_random_state(seed)
                        X = rng.standard_normal((n, p))

                             beta_D = np.zeros(p)
                          beta_D[:n_confounders] = 0.5

                             gamma_Y = np.zeros(p)
                          gamma_Y[:n_confounders] = 0.7

          D = (X @ beta_D + rng.standard_normal(n) > 0).astype(float)
              Y = 1.0 * D + X @ gamma_Y + rng.standard_normal(n)

                                return Y, D, X

                     def post_selection_ate(Y, D, X):
    """Naive: LASSO de Y sobre [D, X], luego OLS HC1 con seleccionados."""
                          DX = np.column_stack([D, X])

                        lasso = LassoCV(cv=5).fit(DX, Y)
                     selected = np.abs(lasso.coef_) > 1e-8

                    # Forzar tratamiento en el modelo final
                               selected[0] = True

                     X_sel = sm.add_constant(DX[:, selected])
                    ols = sm.OLS(Y, X_sel).fit(cov_type="HC1")

                # D es la primera variable después de la constante
                           theta_hat = ols.params[1]
                              se_hat = ols.bse[1]

                            return theta_hat, se_hat

    # La cobertura debe calcularse con el SE estimado en cada réplica.
# No usar results.std() como si fuera el error estándar del procedimiento.

                    # Monte Carlo: 1000 repeticiones
                           np.random.seed(0)

                              thetas = []
                                ses = []
                              covers = []

                        for b in range(1000):
                 Y, D, X = generate_data(n=500, p=100, seed=b)
               theta_hat, se_hat = post_selection_ate(Y, D, X)

                            thetas.append(theta_hat)
```

Listado 6.1: Ilustración del sesgo post-selección.

El *output* típico muestra un sesgo de aproximadamente -0.05 a -0.10 y cobertura real de aproximadamente 0.85–0.90, claramente inferior al nominal 0.95. La conclusión práctica: **no se debe reportar IC construidos después de selección LASSO sin corrección**.

6.3.3 Diagnósticos de nuisance

Intuición

Caja de profundización 2.1: Partialling-out y double lasso

Objetivo. Entender dos herramientas clave para limpiar el efecto de las covariables antes de estimar el efecto causal.

Idea intuitiva. Para estimar el efecto de D sobre Y, primero «limpiamos» Y y D de la influencia de X:

$$\tilde{Y} = Y - E[Y \texttt{ mid } X], \quad \tilde{D} = D - E[D \texttt{ mid } X] \tag{6.10}$$

Luego estimamos θ con la regresión de $\tilde{Y}$ sobre $\tilde{D}$ sin intercepto. Esto se llama **partialling-out** (Frisch-Waugh-Lovell).

Double lasso. En alta dimensión, necesitamos seleccionar variables relevantes:

1. LASSO de Y sobre $X \rightarrow$ obtengo S_Y (variables que predicen Y)
2. LASSO de D sobre $X \rightarrow$ obtengo S_D (variables que predicen D)
3. Unión: $S = S_Y \cup S_D$ (variables que predicen cualquiera de los dos)
4. OLS de Y sobre D + controles en S

¿Por qué funciona? Una confusora afecta TANTO a Y COMO a D. Si seleccionamos las que afectan a cada uno, capturamos todas las confusoras relevantes. El «doble» lasso es la base del Post-Double-Selection (Capítulo 8).

```
from sklearn.linear_model import LassoCV
          import numpy as np

   def partialling_out_theta(Y, D, X):
"""Estima theta limpiando Y y D con LASSO."""
      model_y = LassoCV(cv=5).fit(X, Y)
      model_d = LassoCV(cv=5).fit(X, D)

        y_res = Y - model_y.predict(X)
        d_res = D - model_d.predict(X)

theta = np.mean(d_res * y_res) / np.mean(d_res * d_res)
                    return theta
```

Listado 6.2: Partialling-out manual.

6.4 Caso empírico y validación

(Belloni et al. 2014) aplicaron PDS al dataset de (Card 1995) sobre retornos a la educación. Encuentran un efecto positivo y significativo del orden de las estimaciones 2SLS clásicas, con diferencias moderadas entre PDS y los estimadores IV con controles seleccionados manualmente; las cifras exactas dependen del set de instrumentos y controles considerados y no se reproducen aquí para evitar imprecisiones.

Réplica propia sobre el dataset del repositorio. Sobre `close_college` ($n = 3,003$), OLS con controles básicos da un coeficiente de 0.071 (HC1, IC 95% [0.064, 0.078]); IV 2SLS con `nearc4` da 0.124 (IC 95% [0.028, 0.221]); DML-PLIV con `GradientBoostingRegressor` da 0.145 (IC 95% [0.052, 0.237]). Estas son cifras de réplica propia, no del *paper* original.

Sobre el dataset `nhefs` del repositorio, que tiene 51 covariables utilizables (tras excluir variables post-tratamiento y de fecha), podemos aplicar el mismo principio. La pregunta causal es: ¿cuál es el efecto de dejar de fumar sobre el cambio de peso? Réplica propia sobre $n = 1,566$ observaciones: OLS con selección manual de 5 controles reporta un coeficiente de 3.20 kg (IC 95% [2.26, 4.14]); PDS con LASSO aplicado a los 51 controles reporta un coeficiente de 3.22 kg (IC 95% [2.29, 4.14]). La diferencia es pequeña en este caso, pero PDS selecciona 18 variables, capturando confusoras adicionales (estado civil, hábitos de alcohol, condiciones médicas) que el análisis manual omitía.

6.5 Riesgos, límites y sensibilidad

6.5.1 Tabla de riesgos

Riesgos, límites y mitigación

Riesgo	Consecuencia	Diagnóstico	Mitigación
Inferir después de seleccionar	Cobertura inválida	Monte Carlo	PDS/DML
Interpretar coeficiente LASSO como causal	Sesgo	Regularización	Post-LASSO/DML
λ inestable	Selección frágil	Sensibilidad	Varios λ

6.5.2 Cuándo no usar este método

Límites del enfoque. PDS asume sparsity aproximada: que pocas variables son realmente confusoras. Si la estructura de confusión es densa (muchas variables pequeñas que colectivamente importan), PDS puede ser inconsistente. En tal caso se prefiere DML (capítulo 11), que no requiere sparsity estricta. Adicionalmente, PDS se limita a modelos lineales; para heterogeneidad estructurada en X se requiere GRF (capítulo 13) o meta-learners (capítulo 12).

6.5.3 Checklist de validación

1. **Verificar cobertura empírica:** ejecutar un Monte Carlo bajo el DGP supuesto para confirmar que los IC post-selección alcanzan la cobertura nominal.
2. **Sensibilidad a λ:** reportar resultados para varios valores de λ (e.g., $\lambda \pm 1$ desviación estándar de CV) y verificar que la selección no cambia drásticamente.
3. **Estabilidad de la selección:** usar bootstrap o submuestreo para evaluar qué fracción de las veces cada variable es seleccionada.
4. **Comparar con OLS completo:** si $p < n$, comparar el estimador Post-LASSO con OLS sin selección para evaluar el costo de la regularización.
5. **Reportar ambos:** presentar siempre el estimador de Post-LASSO OLS junto con el de PDS para transparentar el efecto de la doble selección.

6.6 Ejercicios y notas bibliográficas

1. Replique el experimento Monte Carlo del listado Listado 6.1 variando $p \in \{20, 50, 100, 200\}$. ¿Cómo cambia el sesgo y la cobertura con la dimensionalidad?

   Implemente PDS (selección doble) sobre el mismo DGP sintético. Verifique que el sesgo se reduce sustancialmente y la cobertura se acerca al nominal 0.95. Calcule la cobertura como la fracción de réplicas en que el IC ingenuo post-OLS contiene el verdadero θ, usando el SE estimado en cada réplica.
2. Sobre el dataset `nhefs`, aplique: (a) OLS con 5 controles seleccionados manualmente, (b) LASSO *naive* (una sola selección), (c) PDS. Compare los tres estimadores y sus errores estándar.
3. Demuestre analíticamente que Ridge no hace selección de variables. ¿Por qué Ridge es preferible a LASSO cuando hay grupos de variables altamente correlacionadas?
4. Implemente la función `partialling_out_theta` del listado Listado 6.2 y verifique que recupera el verdadero θ en un DGP lineal. ¿Qué pasa si X contiene variables irrelevantes (ruido puro)?
5. Compare PDS con el partialling-out ingenuo (un solo LASSO de Y sobre $[D, X]$). Muestre que el partialling-out ingenuo tiene sesgo mientras que PDS no, en un Monte Carlo con $p = 100$ y $s = 5$.

(Tibshirani 1996) y (Hoerl et al. 1970) son las referencias originales. (Hastie et al. 2015) ofrece un tratamiento moderno de regularización en alta dimensión. (Leeb et al. 2006) y (Pötscher et al. 2009) son las referencias obligatorias sobre el problema de inferencia post-
-selección. (Belloni et al. 2014) introduce Post-Double-Selection. Para una revisión pedagógica, (Chernozhukov et al. 2018a).

Capítulo 7

Datos de entrenamiento en sistemas causales

Cuadro de decisión del capítulo

[**Decisión**]	Tratar la asignación del tratamiento como parte del proceso de muestreo a auditar, no como un dato dado
[**Alternativas**]	Usar los datos «tal como llegan» vs. auditar explícitamente el mecanismo de asignación
[**Trade-off clave**]	Velocidad de arranque vs. riesgo de sesgo de selección no detectado
[**Stack**]	pandas, `pandera`, tests de balance
[**Riesgo principal**]	Confundir «faltante al azar» con «faltante por selección relacionada al tratamiento»

7.1 El problema de diseño

(Huyen 2022) dedica su Capítulo 4 a una idea que la comunidad de *machine learning* tarda en internalizar: los datos de entrenamiento casi nunca son una muestra aleatoria simple del universo sobre el que luego se aplicará el modelo, y cómo se generaron esos datos —el proceso de muestreo, cómo se etiquetó, qué faltó y por qué— determina la validez de cualquier cosa que se entrene sobre ellos. En un sistema **causal**, este problema no es solo una amenaza a la precisión predictiva: es una amenaza directa a la **identificación**. Si la asignación del tratamiento en los datos históricos está relacionada con variables no observadas, ningún ajuste posterior por covariables observadas —por sofisticado que sea el estimador— corrige el problema. El problema de diseño de este capítulo es, entonces: ¿cómo se estructura el proceso de recolección y muestreo de datos de entrenamiento de forma que las amenazas a la identificación queden documentadas y, cuando sea posible, mitigadas, en lugar de descubrirse después de haber estimado el efecto?

7.2 Opciones y trade-offs

Opciones y trade-offs

- **Usar los datos «tal como llegan».** Se toma el dataset disponible y se procede directamente a la estimación. Es la opción más rápida y, en la práctica, la más común. El riesgo es que el mecanismo de asignación del tratamiento —quién decidió, con qué información, a quién tratar— quede completamente implícito, y que cualquier sesgo de selección relacionado con variables no observadas nunca se detecte porque nadie lo buscó explícitamente.
- **Auditar el mecanismo de asignación antes de estimar.** Se documenta explícitamente, para el dataset en cuestión: ¿la asignación fue aleatorizada, cuasi--aleatorizada (por ejemplo, un umbral administrativo) u observacional pura? ¿Qué información tenía quien decidió asignar el tratamiento, y coincide con las covariables observadas en los datos? Este paso no reemplaza el análisis de identificación de los capítulos anteriores; lo **precede**, y a menudo revela qué covariables son plausiblemente suficientes para el criterio de puerta trasera antes de intentar ajustar ningún modelo.

Un segundo trade-off, heredado de la discusión de (Huyen 2022) sobre «etiquetas naturales», es específico de los sistemas causales: en muchos casos, el «tratamiento» es una decisión que ya ocurrió en el pasado dentro de un proceso administrativo o de negocio (a quién se le otorgó un crédito, a quién se le dio un descuento). Estas asignaciones pasadas tienen la ventaja de estar disponibles a bajo costo, pero la desventaja de que la regla de decisión histórica —a menudo no documentada— es exactamente la fuente potencial de confusión que el análisis de identificación debe descubrir.

7.3 Decisión de referencia para el curso

Todo dataset usado en el curso se acompaña de una **ficha de procedencia del tratamiento** (una extensión del `CausalSchema` del Capítulo 4) que documenta, como mínimo: el mecanismo de asignación (aleatorizado / cuasi-experimental / observacional), la unidad temporal de la asignación respecto a la medición del resultado, y cualquier regla de elegibilidad conocida. Para el Notebook de caso de estudio A1 (4), completar esta ficha es un requisito de entrega antes de correr cualquier estimación — es, en efecto, la versión operacional del «problema causal, estimando e identificación» que abre cada capítulo de Plantilla A.

7.4 Implementación de referencia

```
                            import pandas as pd
                           from scipy import stats

def chequeo_balance(df: pd.DataFrame, treatment: str, covariates: list[str],
                               alpha: float = 0.05) -> pd.DataFrame:
                """Compara covariables entre tratados y controles.
           Muchas diferencias significativas sugieren que la asignacion
           no es 'como aleatorizada' condicional en esas covariables."""
                                   filas = []
                       tratados = df[df[treatment] == 1]
                      controles = df[df[treatment] == 0]
                            for cov in covariates:
                         t_stat, p_val = stats.ttest_ind(
             tratados[cov].dropna(), controles[cov].dropna(), equal_var=False
                                           )
                                   filas.append({
                                   "covariable": cov,
                        "media_tratados": tratados[cov].mean(),
                       "media_controles": controles[cov].mean(),
                                    "p_valor": p_val,
                                 "desbalance": p_val < alpha,
                                          })
                        resultado = pd.DataFrame(filas)
                  n_desbalanceadas = resultado["desbalance"].sum()
      print(f"{n_desbalanceadas}/{len(covariates)} covariables desbalanceadas
                                  (p<{alpha})")
                                   return resultado
```

Listado 7.1: Chequeo de balance como auditoría mínima del mecanismo de asignación.

Un número alto de covariables desbalanceadas no invalida por sí solo el análisis observacional —es, de hecho, la situación esperada cuando la asignación no fue aleatoria—, pero si covariables que **deberían** ser irrelevantes para la asignación (por ejemplo, una característica administrativa sin relación aparente con el criterio de elegibilidad) resultan fuertemente desbalanceadas, es una señal de que el mecanismo de asignación real difiere del documentado, y amerita revisar el DAG propuesto antes de proceder al 11.

7.5 Aplicación al sistema causal del curso

El Notebook A1 usa el dataset de Lalonde/NSW precisamente porque permite ilustrar este capítulo de forma directa: la submuestra experimental tiene asignación aleatorizada por diseño (el chequeo de balance debería mostrar covariables balanceadas), mientras que la extensión observacional de Dehejia y Wahba no la tiene (el mismo chequeo debería mostrar desbalances sustanciales) — permitiendo comparar, con el mismo par de grupos de tratamiento, qué tan distinto luce un dataset «como aleatorizado» de uno que no lo es.

7.6 Señales de alerta y checklist operativo

- ☐ Nadie en el equipo puede explicar, en una oración, cómo se decidió históricamente quién recibía el tratamiento.

  ☐ No se ejecutó ningún chequeo de balance de covariables antes de estimar el efecto.

  ☐ Existen valores faltantes en el resultado Y que están sistemáticamente relacionados con el tratamiento (posible sesgo de selección en el seguimiento, no solo en la asignación).

  ☐ La ficha de procedencia del tratamiento no distingue si la asignación fue anterior o posterior a la medición de alguna covariable «de control».

7.7 Ejercicios y notas bibliográficas

7.7.1 Ejercicios conceptuales

1. Para el dataset de becas del 1, redacte una ficha de procedencia del tratamiento hipotética, incluyendo el mecanismo de asignación más plausible.
2. Explique por qué un desbalance fuerte en una covariable **administrativa, sin relación aparente con el criterio de elegibilidad** es una señal de alerta más seria que un desbalance en una covariable directamente relacionada con dicho criterio.

7.7.2 Ejercicio computacional

1. Aplique el listado Listado 7.1 a las dos versiones del dataset de Lalonde (experimental y observacional). Reporte cuántas covariables resultan desbalanceadas en cada una y relacione el resultado con el mecanismo de asignación de cada submuestra.

7.7.3 Notas bibliográficas

(Huyen 2022), Capítulo 4, desarrolla en detalle el rol del muestreo y las «etiquetas naturales» en sistemas de ML generales. La lógica del chequeo de balance como diagnóstico de identificación es estándar en la literatura de evaluación de programas; véase (Imbens et al. 2015) para su tratamiento formal.

Capítulo 8

Razonamiento intervencional y contrafactual

Cuadro sinóptico del capítulo

[**Problema**]	Decidir, a partir de un grafo causal, qué controles identifican el efecto y cuáles lo sesgan
[**Estimando**]	$P(y \texttt{ mid } \mathrm{do}(x))$ y $P(y_x \texttt{ mid } e)$
[**Identificación**]	Criterio de puerta trasera (*backdoor*); reglas del cálculo-do
[**Método**]	Ajuste por puerta trasera; abducción-acción-predicción para contrafactuales
[**Garantía**]	Si el criterio de puerta trasera se satisface, $P(y \texttt{ mid } \mathrm{do}(x)) = \sum_z P(y \texttt{ mid } x, z)P(z)$
[**Librerías**]	`dowhy`, `networkx`
[**Validación**]	Verificación algorítmica del criterio sobre el DAG propuesto

8.1 Problema causal, estimando e identificación

El 3 dejó pendiente una pregunta operativa: dado un grafo causal y un conjunto de variables observadas, ¿cómo se determina **qué controles incluir** para que un ajuste estadístico —una regresión, un *matching*, un DML— recupere efectivamente $P(y \texttt{ mid } \mathrm{do}(x))$ en lugar de $P(y \texttt{ mid } x)$? La intuición de «controlar por todo lo que se pueda medir» es, como se muestra en este capítulo, no solo insuficiente sino activamente peligrosa: hay variables que, controladas, **introducen** sesgo donde no lo había. Este capítulo formaliza el criterio que resuelve esta pregunta —el criterio de puerta trasera— y extiende el procedimiento de cómputo contrafactual adelantado en el capítulo anterior.

8.2 Teoría, supuestos y garantías

8.2.1 El criterio de puerta trasera

Definición 8.1 ***(Criterio de puerta trasera)***

Un conjunto de variables Z satisface el **criterio de puerta trasera** relativo a un par ordenado (X, Y) en un grafo G si: **(i)** ningún nodo de Z es descendiente de X, y **(ii)** Z bloquea todo camino entre X e Y que contenga una flecha **hacia** X (un «camino de puerta trasera»).

Teorema 8.2 ***(Fórmula de ajuste de puerta trasera)***

Si Z satisface el criterio de puerta trasera relativo a (X, Y), entonces el efecto causal de X sobre Y es identificable y está dado por la **fórmula de ajuste**:

$$P(y \texttt{ mid } \mathrm{do}(x)) = \sum_{z} P(y \texttt{ mid } x, z)\, P(z) \tag{8.11}$$

Demostración (esquema). La condición **(i)** garantiza que condicionar en Z no bloquea el efecto de X sobre Y a través de la ruta causal directa (no se controla por un mediador). La condición **(ii)** garantiza que Z bloquea toda ruta espuria —toda confusión— entre X e Y. Bajo ambas, condicionar en Z dentro de cada estrato hace que X esté «como aleatorizado», y promediar sobre la distribución de Z recupera la distribución interventiva poblacional. □

El supuesto de «no confusión condicional» (*unconfoundedness*) que se ha usado sin más formalidad en los capítulos anteriores —y que sustenta a DML, a los meta-learners y a los bosques causales de la Parte II— es, exactamente, la afirmación de que las covariables observadas X satisfacen el criterio de puerta trasera relativo al tratamiento y al resultado. Este capítulo no cambia ningún método de estimación posterior; cambia **cómo se justifica**, con un grafo explícito, la elección de qué controlar.

8.2.2 Buenos y malos controles

El criterio de puerta trasera implica una clasificación de variables candidatas a control que conviene memorizar, porque el error de incluir un mal control es, en la práctica aplicada, más frecuente que el de omitir un confusor:

Tipo de variable	Rol en el grafo	¿Controlar?
Confusor	Causa común de X e Y (afecta a ambos)	**Sí** — necesario para puerta trasera
Mediador	Está en la ruta causal $X \to M \to Y$	**No** — bloquea parte del efecto que se quiere medir
Collider	Es afectado tanto por X como por Y (o por sus causas)	**No** — condicionar en él **crea** una asociación espuria (ver 1.2)

Tipo de variable	Rol en el grafo	¿Controlar?
Instrumento	Afecta a Y solo a través de X, no comparte causa con Y	**No** como control directo — se usa de otro modo (Parte III, DML-IV)
Variable post-tratamiento	Medida después de X, potencialmente afectada por él	**No** — riesgo de ser mediador o collider sin que el analista lo note

La fila de mediadores explica formalmente algo que ya se mencionó en el 4 sin desarrollar: por qué una variable «post-tratamiento» es peligrosa como control, incluso si parece un predictor razonable de Y — con probabilidad no trivial es un mediador (controla por parte del efecto que se busca medir) o un collider (introduce sesgo de selección).

8.2.3 Cómputo contrafactual: abducción, acción, predicción

El 3 adelantó el procedimiento de tres pasos para responder preguntas de $\mathcal{L}_3$. Formalizado: dado un SCM M y evidencia observada e sobre una unidad específica, calcular $P(y_x \texttt{ mid } e)$ requiere:

1. **Abducción**: actualizar $P(u)$ a $P(u \texttt{ mid } e)$ — inferir, a partir de lo observado, qué configuración de variables exógenas es consistente con la evidencia.
2. **Acción**: construir el modelo modificado M_x, reemplazando la ecuación de X por la constante x (idéntico al paso interventivo del capítulo anterior).
3. **Predicción**: calcular Y en M_x usando la distribución actualizada $P(u \texttt{ mid } e)$ del paso de abducción, en lugar de la $P(u)$ original.

La diferencia crucial con el cómputo interventivo puro es el paso de abducción: una pregunta contrafactual siempre está condicionada por lo que efectivamente se observó de esa unidad específica, mientras que una pregunta interventiva pura pregunta por el efecto promedio poblacional sin condicionar en ninguna observación particular.

8.3 Algoritmo e implementación

```
                        import networkx as nx

    # Grafo: educ (X) -> wage (Y); ability (confusor) -> educ, wage;
          # test_score (mediador) : educ -> test_score -> wage
                        G = nx.DiGraph([
                ("ability", "educ"), ("ability", "wage"),
                            ("educ", "wage"),
            ("educ", "test_score"), ("test_score", "wage"),
                                ])

                def satisface_puerta_trasera(G, X, Y, Z):
                        G_sin_salientes_X = G.copy()
            G_sin_salientes_X.remove_edges_from(list(G.out_edges(X)))
                   # (i) ningun nodo de Z es descendiente de X
                     descendientes_X = nx.descendants(G, X)
                    if any(z in descendientes_X for z in Z):
        return False, "Z contiene un descendiente de X (posible mediador)"
       # (ii) Z separa X de Y en el grafo sin las aristas salientes de X
    bloqueado = not nx.has_path(G_sin_salientes_X.to_undirected(), X, Y) or \
                  nx.is_d_separator(G_sin_salientes_X, {X}, {Y}, set(Z))
                            if not bloqueado:
          return False, "Z no bloquea todos los caminos de puerta trasera"
            return True, "Z satisface el criterio de puerta trasera"

       print(satisface_puerta_trasera(G, "educ", "wage", {"ability"}))
print(satisface_puerta_trasera(G, "educ", "wage", {"ability", "test_score"}))
 # El segundo caso debe fallar: test_score es un mediador (descendiente de X)
```

Listado 8.1: Verificación del criterio de puerta trasera con `networkx`.

En la práctica, `dowhy` automatiza esta verificación y además identifica automáticamente conjuntos de puerta trasera válidos dado un grafo declarado por el usuario, con la función `identify_effect()`; el Apéndice B documenta su uso completo.

8.4 Caso empírico

Retomando el dataset de retornos a la educación: si el analista incluyera `test_score` (una prueba de habilidad tomada **después** de completar los años de educación reportados) como control en la regresión de `wage` sobre `educ`, estaría, según la clasificación de la 8.2.2, controlando por un mediador parcial —parte del efecto de la educación sobre el salario opera precisamente a través de habilidades que la prueba mide—, sesgando el coeficiente de `educ` hacia abajo. El listado Listado 8.1 verifica exactamente esto: el conjunto `{ability, test_score}` falla el criterio de puerta trasera, mientras que `{ability}` solo lo satisface.

8.5 Riesgos, límites y sensibilidad

Riesgos, límites y mitigación

- **«Más controles siempre es más seguro».** Es falso en general: agregar un mediador o un collider puede introducir sesgo donde una especificación más parca no lo tenía. El criterio de puerta trasera formaliza exactamente cuándo agregar un control ayuda y cuándo perjudica.
- **Confundir un instrumento con un confusor.** Un instrumento válido no debe controlarse directamente como si fuera una covariable de puerta trasera; su rol es distinto y se explota con métodos específicos (DML-IV, Parte III). Incluirlo como control puede, dependiendo del grafo, introducir sesgo de collider si comparte una causa no observada con Y.
- **El grafo es siempre una hipótesis.** Todo lo desarrollado en este capítulo presupone que el grafo declarado es correcto. Ningún algoritmo puede generar el grafo desde los datos solos sin supuestos adicionales (el 29, más adelante en el curso, aborda qué tan lejos se puede llegar con descubrimiento automatizado, y sus límites).

8.6 Ejercicios y notas bibliográficas

8.6.1 Ejercicios teóricos

1. Para el grafo del listado Listado 8.1, agregue una variable `neighborhood` que sea causa común de `ability` y de `wage` (pero no de `educ`). ¿Sigue `{ability}` satisfaciendo el criterio de puerta trasera? Justifique con el grafo modificado.
2. Explique, en el lenguaje de este capítulo, por qué condicionar en un collider crea una correlación espuria (retome el ejemplo del tamaño del pie y la capacidad de lectura del 1, agregando explícitamente un collider).
3. Aplique el procedimiento de abducción-acción-predicción a un ejemplo numérico simple: un SCM lineal $Y = 2X + U$, $X = U + \varepsilon$, con $U \sim N(0, 1)$. Dado que se observó $X = 1, Y = 3$, calcule $P\left(Y_{\{X=2\}}\right)$, el valor contrafactual de Y si X hubiera sido 2.

8.6.2 Ejercicios computacionales

1. Instale `dowhy` (Apéndice B) y reproduzca la verificación del listado Listado 8.1 usando `identify_effect()` en lugar de la función manual. Compare el conjunto de ajuste que identifica automáticamente con el usado en el capítulo.

8.6.3 Notas bibliográficas

El criterio de puerta trasera y las reglas completas del cálculo-do se presentan en (Pearl 2009); la clasificación de buenos y malos controles sigue la sistematización de (Bareinboim 2025).

Para una implementación aplicada con Python, `dowhy` (Sharma y Kiciman) es la referencia de facto, documentada en el Apéndice B de este curso.

Capítulo 9

Ingeniería de *features* sin fuga causal

Cuadro de decisión del capítulo

[**Decisión**]	Auditar cada *feature* candidata contra el grafo causal antes de incluirla, no solo contra su poder predictivo
[**Alternativas**]	Selección de *features* por importancia predictiva vs. selección por rol causal
[**Trade-off clave**]	Una *feature* que mejora el ajuste predictivo puede empeorar la estimación causal
[**Stack**]	pandas, el `CausalSchema` del 4
[**Riesgo principal**]	*Fuga* (*leakage*) de información post-tratamiento disfrazada de covariable

Implementación en el monorepo del curso (2026)

Lab `L08` planta fugas deliberadas (outcome, tratamiento, interacciones) y las detecta antes de estimar. La Serie A2 (DML) y el escenario `ihdp_medium` asumen covariables solo pre-tratamiento. No usar columnas de ground truth (`y0`, `y1`, `tau`) fuera de celdas de evaluación. Rutas: `05_labs_sesion/L08_leakage_ihdp/` y notebook A2 en `03_notebooks_caso_estudio/notebooks/`.

9.1 El problema de diseño

La ingeniería de *features* estándar —la de (Huyen 2022) Capítulo 5— evalúa una *feature* candidata por un solo criterio: ¿mejora el desempeño predictivo del modelo fuera de muestra? Bajo ese criterio, cualquier variable disponible en el momento de entrenar que esté correlacionada con Y es, en principio, bienvenida. El 8 mostró por qué ese criterio es **insuficiente y a veces contraproducente** para un sistema causal: una *feature* puede reducir el error de predicción y, al mismo tiempo, ser un mediador o un collider que sesga la estimación del efecto. El problema de diseño de este capítulo es construir un proceso de ingeniería de *features* que

audite cada variable candidata contra su **rol en el grafo causal**, no solo contra su poder predictivo — la versión operacional, en el pipeline de *features*, de la clasificación de controles del capítulo anterior.

9.2 Opciones y trade-offs

Opciones y trade-offs

- **Selección por importancia predictiva.** Se agregan *features* mientras mejoren una métrica de validación cruzada (RMSE, AUC). Es el flujo de trabajo estándar en *machine learning* aplicado y el más rápido de ejecutar. Su riesgo específico en un contexto causal ya se discutió en el 5: un modelo puede predecir excelente y estimar el efecto causal con sesgo severo, precisamente porque la importancia predictiva no distingue entre variación causal y variación espuria.
- **Selección por rol causal, informada por el grafo.** Cada *feature* candidata se clasifica primero según la 8.2.2 del capítulo anterior (confusor / mediador / collider / post-tratamiento) y solo después se evalúa su aporte predictivo **dentro** de la categoría de controles legítimos. Requiere tener, aunque sea de forma preliminar, un DAG propuesto —lo cual no siempre es cómodo cuando se está en la fase exploratoria de un proyecto— pero previene sistemáticamente la clase de error más costosa de este curso.

9.3 Decisión de referencia para el curso

Todo notebook de estimación del curso extiende el `CausalSchema` del 4 con una etapa de **auditoría de features** obligatoria antes de construir cualquier matriz de covariables X: cada columna candidata se anota con su rol supuesto (confusor / post-tratamiento / desconocido) y solo las anotadas como confusor —o «desconocido, incluida con advertencia explícita para análisis de sensibilidad»— entran en X.

9.4 Implementación de referencia

```
                        import pandas as pd

    def auditar_features(df: pd.DataFrame, treatment_time_col: str,
                             feature_time_cols: dict[str, str],
                     roles_declarados: dict[str, str]) -> pd.DataFrame:
     """Marca cada feature con riesgo de fuga temporal (medida despues
     del tratamiento) y con su rol causal declarado por el analista."""
                                filas = []
           for feature, tiempo_col in feature_time_cols.items():
         posterior = (df[tiempo_col] > df[treatment_time_col]).mean()
                                filas.append({
                               "feature": feature,
                  "pct_medida_despues_del_tratamiento": posterior,
                        "riesgo_temporal": posterior > 0.0,
          "rol_declarado": roles_declarados.get(feature, "SIN DECLARAR"),
          "incluir_en_X": roles_declarados.get(feature) == "confusor" and
                           posterior == 0.0,
                                        })
                    resultado = pd.DataFrame(filas)
           riesgosas = resultado[resultado["riesgo_temporal"] |
            (resultado["rol_declarado"] == "SIN DECLARAR")]
                           if len(riesgosas) > 0:
    print("ADVERTENCIA: features con riesgo de fuga o sin rol declarado:")
       print(riesgosas[["feature", "pct_medida_despues_del_tratamiento",
                           "rol_declarado"]])
                              return resultado
```

Listado 9.1: Auditoría de *features* por marca de tiempo y por rol declarado.

La función no decide automáticamente qué incluir —esa decisión sigue siendo del analista, informada por el grafo— pero hace **imposible** que una variable post-tratamiento entre a X en silencio: cualquier *feature* medida después del tratamiento, o sin rol declarado, genera una advertencia explícita.

9.5 Aplicación al sistema causal del curso

El Notebook de caso de estudio A2 (Módulo 2, Double Machine Learning) exige ejecutar la auditoría del listado Listado 9.1 sobre la matriz de covariables **antes** de estimar el modelo PLR/IRM del 11. Esto conecta directamente con un punto que ese capítulo asume resuelto: la validez de la ortogonalidad de Neyman depende de que X contenga controles legítimos, no mediadores ni colliders disfrazados de covariables predictivas.

9.6 Señales de alerta y checklist operativo

- ⍰ Una *feature* mejora sustancialmente el ajuste predictivo del modelo de resultado sin que exista una razón causal clara para incluirla.

  ⍰ Ninguna columna del dataset tiene una marca de tiempo que permita verificar si fue medida antes o después del tratamiento.

  ⍰ El equipo agregó una *feature* «porque funcionaba mejor en las pruebas» sin volver a evaluar el DAG.

  ⍰ Existen *features* derivadas (ingeniería de variables) cuya fórmula de construcción usa, aunque sea indirectamente, el propio resultado Y.

9.7 Ejercicios y notas bibliográficas

9.7.1 Ejercicios conceptuales

1. Para el dataset de retornos a la educación, clasifique las siguientes variables candidatas a `covariates` según la tabla de la 8.2.2: habilidad cognitiva medida antes de ingresar a la escuela; puntaje de un examen de egreso universitario; cercanía geográfica a una universidad; ingreso de los padres.
2. Explique por qué una *feature* construida como «promedio móvil del resultado en las últimas 4 semanas» es casi siempre una fuente de fuga en un sistema causal, incluso si en un sistema puramente predictivo sería una *feature* razonable.

9.7.2 Ejercicio computacional

1. Aplique el listado Listado 9.1 al dataset que eligió para el Notebook A1, declarando roles causales para al menos 6 covariables candidatas. Reporte cuántas quedan excluidas de X tras la auditoría y por qué motivo cada una.

9.7.3 Notas bibliográficas

(Huyen 2022), Capítulo 5, cubre la ingeniería de *features* genérica y la fuga de datos (*data leakage*) en su sentido predictivo estándar (por ejemplo, escalar antes de separar train/test); este capítulo extiende esa discusión al tipo de fuga específico de sistemas causales. La clasificación de controles retomada aquí se desarrolla en (Bareinboim 2025) y (Pearl 2009), ya citados en el 8.

Parte III

Estimación causal flexible y primer despliegue

Esta parte desarrolla los estimadores centrales del libro: ortogonalidad de Neyman y cross-fitting, Double Machine Learning, meta-learners, árboles y bosques causales, y validación sin ground truth —ahora extendida con una perspectiva de evaluación offline como proceso de sistema, no solo como chequeo estadístico puntual—. Cierra con el primer despliegue real del curso: el estimador DML servido como servicio con FastAPI y Docker.

Capítulo 10

Ortogonalidad de Neyman, cross-fitting e influence functions

Cuadro sinóptico del capítulo

[Problema]	Doble sesgo al estimar θ_0 con ML
[Estimando]	θ_0 en PLR
[Identificación]	No confusión, ortogonalidad
[Método]	Score ortogonal, cross-fitting
[Nuisance]	$g_0(X), m_0(X)$
[Garantía]	$\sqrt{n}\left(\hat{\theta} - \theta_0\right) \Rightarrow N(0, \mathcal{V})$
[Librerías]	

Nota técnica

Con el criterio de puerta trasera (8) y la auditoría de *features* (9) resueltos, la Parte I dejó pendiente una pregunta puramente estadística: ¿cómo combinar *learners* de ML flexibles para estimar un efecto causal sin que el sesgo de esos *learners* contamine la inferencia? Este capítulo, que abre la Parte II, responde exactamente eso.

10.1 Problema causal, estimando e identificación

Una consultora económica está evaluando el efecto de una reforma tributaria sobre la inversión empresarial. Dispone de un panel de $5,000$ firmas con 200 covariables (características financieras, sectoriales, de propiedad, de gobernanza). La asignación de la reforma no fue aleatoria: las firmas grandes y rentables fueron más propensas a ser tratadas. El estimador ideal debería: (i) usar ML para modelar flexiblemente la relación entre covariables y asignación al tratamiento (*propensity score*), (ii) usar ML para modelar la relación entre covariables e inversión en ausencia de tratamiento (*outcome regression*), (iii) combinar ambas estimaciones para obtener un estimador del efecto causal con inferencia válida.

El problema es que si simplemente «enchufamos» los estimadores de ML en una fórmula causal estándar—por ejemplo, el estimador de doubly-robust $\hat{\theta} = n^{-1} \sum_i \Big[\hat{\mu}_1(X_i) - \hat{\mu}_0(X_i) + D_i \frac{Y_i - \hat{\mu}_1(X_i)}{\hat{e}}(X_i) - (1 - D_i)\frac{Y_i - \hat{\mu}_0(X_i)}{1-\hat{e}(X_i)}\Big]$— la inferencia resultante no es válida en general. El motivo es sutil: los errores de estimación de $\hat{\mu}_d$ y $\hat{e}$ *contaminan* la estimación del efecto causal en el primer orden, introduciendo un sesgo que no desaparece con muestras grandes.

Este capítulo formaliza el problema del *doble sesgo* y presenta la solución conceptual central del ML causal moderno: la **ortogonalidad de Neyman**. Mostramos que existe una manera de construir un *score* (función de momento) cuya derivada respecto a los parámetros *nuisance* es cero en el valor verdadero, de modo que los errores de estimación de los *nuisance* solo afectan la inferencia causal en el segundo orden. Esta propiedad, combinada con *cross-fitting* (capítulo 11), permite construir estimadores $\sqrt{n}$-consistentes y asintóticamente normales incluso cuando los *nuisance* se estiman con ML a tasas más lentas que $\sqrt{n}$.

10.2 Teoría, supuestos y garantías

10.2.1 El problema del doble sesgo

Consideremos el modelo parcialmente lineal (PLR, *partial linear model*):

$$Y = D\theta_0 + g_0(X) + \varepsilon, \quad D = m_0(X) + v, \tag{10.12}$$

Definimos los residuos Robinson: $v_i = D_i - e_0(X_i)$ y $u_i = Y_i - l_0(X_i)$, con $l_0(X_i) = E[Y \texttt{ mid } X_i]$.

donde Y es el resultado, D es el tratamiento, X son covariables, g_0 es la *outcome regression* y m_0 es la *propensity function* (no necesariamente una probabilidad — puede ser una función continua de X). Suponemos $E[\varepsilon \texttt{ mid } D, X] = 0$ y $E[v \texttt{ mid } X] = 0$.

Para D binario, $m_0(x) = e_0(x) = P(D = 1 \texttt{ mid } X = x)$; en lo que sigue e_0 denota el propensity score.

El parámetro causal de interés es θ_0, el efecto del tratamiento. La pregunta es cómo estimarlo cuando g_0 y m_0 son funciones desconocidas de alta dimensión.

Enfoque naïve 1: OLS con ML para estimar g_0.

Si g_0 fuera conocido, θ_0 se estimaría por OLS de $Y - g_0(X)$ sobre D. Pero como g_0 se estima, sustituimos g_0 por $\hat{g}$ obtenido con algún algoritmo de ML. El estimador resultante es:

$$\tilde{\theta} = \left(\sum_i D_i^2 \right)^{-1} \sum_i D_i (Y_i - \hat{g}(X_i)). \tag{10.13}$$

El problema: $\hat{g}(X_i)$ tiene error $O_{p(a_n)}$ con $a_n \gg n^{-\frac{1}{2}}$ en general (bosques aleatorios tienen $a_n = n^{-\frac{2}{5}}$ típicamente). Este error de primer orden contamina $\tilde{\theta}$:

$$\tilde{\theta} - \theta_0 = \underbrace{\left(\sum D_i^2\right)^{-1} \sum D_i \varepsilon_i}_{\text{ruido } O_p\left(n^{-\frac{1}{2}}\right)} + \underbrace{\left(\sum D_i^2\right)^{-1} \sum D_i (g_0(X_i) - \hat{g}(X_i))}_{\text{sesgo } O_{p(a_n)}}. \tag{10.14}$$

Como $a_n \gg n^{-\frac{1}{2}}$, el sesgo domina la varianza asintótica y la inferencia basada en la distribución normal es inválida.

Enfoque naïve 2: OLS con ML para estimar m_0 (residuos de tratamiento).

Si proyectemos D sobre X con ML: $\hat{v}_i = D_i - \hat{m}(X_i)$, y luego regredimos Y sobre $\hat{v}$:

$$\tilde{\theta}' = \left(\sum \hat{v}_i^2\right)^{-1} \sum \hat{v}_i Y_i. \tag{10.15}$$

El mismo problema: el error $\hat{m} - m_0$ contamina $\tilde{\theta}'$ en primer orden.

10.2.2 Ortogonalidad de Neyman

La solución es construir un *score* $\psi(W, \theta, \eta)$, donde $W = (Y, D, X)$ y $\eta = (g, m)$ son los *nuisance parameters*, que satisfaga:

Definición 10.1 ***(Score ortogonal de Neyman)***

Un score $\psi(W, \theta, \eta)$ es *ortogonal de Neyman* respecto a η en (θ_0, η_0) si la derivada de Gateaux respecto a η se anula:

$$\partial_\eta E[\psi(W, \theta_0, \eta)] \mid_{\eta=\eta_0} [r] = 0 \quad \text{para toda perturbación } \ r. \tag{10.16}$$

La propiedad de ortogonalidad implica que pequeñas desviaciones de η respecto a η_0 *no afectan* el valor esperado del score en primer orden. Esto es lo que permite que errores de estimación de $\hat{\eta}$ (que son $O_{p(a_n)}$) solo afecten la estimación de θ en orden a_n^2 o $a_n \cdot n^{-\frac{1}{2}}$, preservando así la $\sqrt{n}$-consistencia.

10.2.3 Construcción del score ortogonal para PLR

Para el modelo Ecuación (10.12), el score ortogonal se construye con un truco algebraico. Primero, observemos que $Y - g_0(X) = \theta_0 D + \varepsilon$ y $D - m_0(X) = v$, con $E[\varepsilon \texttt{ mid } X] = 0$ y $E[v \texttt{ mid } X] = 0$. Por la ley de expectativas iteradas, $E[v\varepsilon] = E[vE[\varepsilon \texttt{ mid } X, D]] = 0$ si $E[\varepsilon \texttt{ mid } D, X] = 0$ y $E[v \texttt{ mid } X] = 0$.

Multiplicando $Y - g_0(X) - \theta_0 D = \varepsilon$ por $v = D - m_0(X)$ y tomando esperanza:

$$E[(D - m_0(X))(Y - g_0(X) - \theta_0 D)] = E[v\varepsilon] = 0. \tag{10.17}$$

De aquí se despeja θ_0:

$$\theta_0 = \frac{E[(D - m_0(X))(Y - g_0(X))]}{E[(D - m_0(X))D]}. \tag{10.18}$$

Es crucial distinguir dos nuisance posibles.

1. Si se estima la regresión estructural

$$g_0(X) = E[Y - \theta_0 D \texttt{ mid } X], \tag{10.19}$$

No confundir con $\mu_{d(x)} = E[Y \texttt{ mid } X = x, D = d]$ de los capítulos 1, 5 y 13. Bajo el PLR, $\mu_{d(x)} = g_0(x) + \theta_0 d$, de modo que $\mu_1(x) - \mu_0(x) = \theta_0$. entonces el score puede escribirse como

$$\psi(D, X, Y, \theta, \eta) = (D - m(X))(Y - g(X) - \theta D). \tag{10.20}$$

2. En la implementación DML estándar, normalmente se estima

$$l_0(X) = E[Y \texttt{ mid } X] = g_0(X) + \theta_0 m_0(X), \tag{10.21}$$

junto con

$$m_0(X) = E[D \texttt{ mid } X]. \tag{10.22}$$

En ese caso, el score ortogonal correcto es el score de Robinson:

$$\psi(D, X, Y, \theta, \eta) = (D - m(X))\{Y - l(X) - \theta(D - m(X))\}. \tag{10.23}$$

El Listado Listado 10.1 estima $l_0(X) = E[Y \texttt{ mid } X]$, por lo que debe usar la segunda forma. Usar $(D - m)(Y - l - \theta D)$ cuando $l = E[Y \texttt{ mid } X]$ no preserva la ortogonalidad de Neyman respecto de m.

El score correspondiente es:

$$\psi(W_i, \theta, \eta) = (D - m(X))(Y - g(X) - \theta D). \tag{10.24}$$

Teorema 10.2 ***(Ortogonalidad del score PLR)***

El score Ecuación (10.24) es ortogonal de Neyman respecto a $\eta = (g, m)$ en (θ_0, η_0).

Demostración. La derivada de Gateaux de $E[\psi]$ respecto a $\eta = (g, m)$ en dirección $\left(r_g, r_m\right)$ es

$$\begin{aligned} \partial_\eta E[\psi]\left[r_g, r_m\right] &= \partial_\alpha E\left[(D - (m_0 + \alpha r_m))\left(Y - \left(g_0 + \alpha r_g\right) - \theta_0 D\right)\right] \mid_{\alpha=0} \\ &= -E[r_m \varepsilon] - E\left[v r_g\right] = 0, \end{aligned} \tag{10.25}$$

donde hemos usado que $E[\varepsilon \texttt{ mid } D, X] = 0$ implica $E\left[r_{m(X)}\varepsilon\right] = 0$, y que $E[v \texttt{ mid } X] = 0$ implica $E\left[v r_{g(X)}\right] = 0$. □

10.2.4 Teorema central de DML

El teorema central de (Chernozhukov et al. 2018b) combina ortogonalidad con *cross-fitting*:

Teorema 10.3 ***(DML — versión simplificada)***

Sea $\hat{\eta}_0$ un estimador de η_0 obtenido en una muestra auxiliar independiente de la muestra principal, con $\|\hat{\eta}_0 - \eta_0\|_{L^2} = O_{p(a_n)}$ para algún $a_n \to 0$ con $\sqrt{n}a_n^2 \to 0$. Sea $\hat{\theta}$ solución de

$$\left(\frac{1}{n}\right)\sum_{i=1}^{n} \psi\left(W_i, \hat{\theta}, \hat{\eta}_0\right) = 0. \tag{10.26}$$

Entonces, bajo regularidad estándar y la condición de ortogonalidad Ecuación (10.16):

$$\sqrt{n}\left(\hat{\theta} - \theta_0\right) \xrightarrow{d} \mathcal{N}(0, \sigma^2), \tag{10.27}$$

$$\text{donde } \sigma^2 = \frac{E\left[\psi(W, \theta_0, \eta_0)^2\right]}{\left(E[\partial_\theta \psi]\right)^2}.$$

Demostración (esquema). La derivada de Gateaux de $E[\psi]$ respecto a $\eta = (g, m)$ en dirección $\left(r_g, r_m\right)$ es

Algoritmo

Entrada: Datos $\{(Y_i, D_i, X_i)\}_{i=1}^n$, score ortogonal ψ, número de *folds* K, learners para $\hat{g}, \hat{m}$.
Salida: Estimador $\hat{\theta}$ y error estándar $\hat{\sigma}$.
Dividir $\{1, ..., n\}$ en K *folds* $I_1, ..., I_K$.
for $\mathsf{k} = 1$ **to** K **do**

Entrenar $\hat{g}_{-k}, \hat{m}_{-k}$ en $\{i : i \notin I_k\}$.
Predecir $\hat{g}_{-k}(X_i), \hat{m}_{-k}(X_i)$ para $i \in I_k$.

end for
Resolver $\hat{\theta}$ tal que $\left(\frac{1}{n}\right) \sum_i \psi\Big(W_i, \hat{\theta}, \big(\hat{g}_{-k(i)}(X_i), \hat{m}_{-k(i)}(X_i)\big)\Big) = 0$.
Estimar $\hat{\sigma}^2 = \frac{\widehat{E[\psi^2]}}{\widehat{E[\partial_\theta \psi]}^2}$.
return $\left(\hat{\theta}, \frac{\hat{\sigma}}{\sqrt{n}}\right)$

Algoritmo 2.: Double Machine Learning con cross-fitting (K-fold)

$$\begin{aligned} \partial_\eta E[\psi]\big[r_g, r_m\big] &= \partial_\alpha E\big[(D - (m_0 + \alpha r_m))\big(Y - \big(g_0 + \alpha r_g\big) - \theta_0 D\big)\big] \mid_{\alpha=0} \\ &= -E[r_m \varepsilon] - E\big[v r_g\big] = 0, \end{aligned} \tag{10.28}$$

donde hemos usado que $E[\varepsilon \texttt{ mid } D, X] = 0$ implica $E\Big[r_{m(X)}\varepsilon\Big] = 0$, y que $E[v \texttt{ mid } X] = 0$ implica $E\Big[v r_{g(X)}\Big] = 0$. □

Intuición

La condición $\sqrt{n} a_n^2 \to 0$ es la clave. Para bosques aleatorios, $a_n = n^{-\frac{2}{5}}$, así que $\sqrt{n} a_n^2 = n^{\frac{1}{2} - \frac{4}{5}} = n^{-\frac{3}{10}} \to 0$. Para LASSO bajo sparsity, $a_n = \sqrt{s \log \frac{p}{n}}$, así que $\sqrt{n} a_n^2 = s \log \frac{p}{\sqrt{n}} \to 0$ si $s \log p = o\big(\sqrt{n}\big)$. La magia de la ortogonalidad es que *no necesitamos* estimar η a tasa $\sqrt{n}$ — basta una tasa más lenta pero con $a_n^2 = o\Big(n^{-\frac{1}{2}}\Big)$.

Nota técnica

La expansión de influence function permite escribir

$$\hat{\theta} - \theta_0 = \left(\frac{1}{n}\right) \sum_i \psi(W_i, \theta_0, \eta_0) + O_{p\left(n^{-\frac{1}{2}}\right)}. \tag{10.29}$$

10.3 Algoritmo e implementación

10.3.1 El rol del cross-fitting

El teorema central de DML asume que $\hat{\eta}$ se estima en una muestra *independiente* de la muestra principal. En la práctica, no queremos desperdiciar datos. La solución es el *cross-fitting*: dividir la muestra en K *folds*; para cada *fold* k, estimar $\hat{\eta}_{-k}$ en los otros $K - 1$ *folds* y evaluar el score en el *fold* k. Esto preserva la independencia entre $\hat{\eta}$ y los datos sobre los que se evalúa, sin perder datos.

Algoritmo

Entrada: Datos $\{(Y_i, D_i, X_i)\}_{i=1}^n$, número de *folds* K, learners para $\hat{l}, \hat{m}$.
Salida: Estimador $\hat{\theta}$ y error estándar $\hat{\sigma}$.
Dividir $\{1, ..., n\}$ en K *folds* $I_1, ..., I_K$.
for k = 1 **to** K **do**

Entrenar $\hat{l}_{-k}$ para $E[Y \texttt{ mid } X]$ en $\{i : i \notin I_k\}$.
Entrenar $\hat{m}_{-k}$ para $E[D \texttt{ mid } X]$ en $\{i : i \notin I_k\}$.
Predecir $\hat{l}_{-k}(X_i)$, $\hat{m}_{-k}(X_i)$ para $i \in I_k$.

end for
Calcular residuos: $\hat{v}_i = D_i - \hat{m}_{-k(i)}(X_i)$, $\hat{u}_i = Y_i - \hat{l}_{-k(i)}(X_i)$.
Estimar $\hat{\theta} = \frac{\sum_i \hat{v}_i \hat{u}_i}{\sum_i \hat{v}_i^2}$.
Calcular $\hat{\sigma}^2 = \left(\frac{1}{n}\right) \sum_i \left[\hat{v}_i \frac{\hat{u}_i - \hat{\theta}\hat{v}_i}{\left(\frac{1}{n}\right)\sum_i \hat{v}_i^2}\right]^2$.
return $\left(\hat{\theta}, \frac{\hat{\sigma}}{\sqrt{n}}\right)$

Algoritmo 3.: DML manual con cross-fitting para PLR

Intuición

Caja de profundización 3.1: Influence functions y por qué importan

Objetivo. Explicar de forma intuitiva qué son las influence functions y por qué la ortogonalidad permite usar ML para estimar efectos causales.

Idea clave. Una influence function (función de influencia) nos dice cuánto cambiaría nuestro estimado si agregamos o quitamos una observación. Sirve para calcular errores estándar sin bootstrap.

Para el PLR:

$$\psi_i = v_i(u_i - \theta v_i), \quad v_i = D_i - \hat{m}(X_i), \quad u_i = Y_i - \hat{l}(X_i) \tag{10.30}$$

El error estándar es $\sqrt{\left(\frac{1}{n}\right)\sum_i \psi_i^2}$.

¿Por qué la ortogonalidad es mágica? Porque si cometemos errores al estimar $\hat{m}$ o $\hat{l}$, esos errores NO contaminan el estimado de θ en primer orden. Solo afectan en segundo orden, lo que permite que el ML brille aunque sus predicciones no sean perfectas.

10.3.2 Pseudocódigo

El siguiente algoritmo detalla los pasos concretos de DML manual con cross-fitting para PLR, utilizando el score de Robinson y los residuos del tratamiento y del *outcome*:

10.3.3 Implementación en Python

Implementamos manualmente un score ortogonal para PLR sobre un dataset sintético con confusores de alta dimensión.

```
                    import numpy as np
            from sklearn.ensemble import RandomForestRegressor
                from sklearn.model_selection import KFold

        def dml_plr_manual(Y, D, X, n_folds=5, learner=None):
        """DML para Partial Linear Model con cross-fitting K-fold.

                            Nuisances:
                        l_hat(X) = E[Y | X]
                        m_hat(X) = E[D | X]

                          Score Robinson:
                psi_i = v_i * (u_i - theta * v_i) / E[v_i^2]
            donde v_i = D_i - m_hat(X_i), u_i = Y_i - l_hat(X_i).
                                """
                        if learner is None:
            learner = lambda: RandomForestRegressor(n_estimators=100,
                                                    min_samples_leaf=20,
                                                      random_state=42)
                            n = len(Y)
          l_hat = np.zeros(n)   # regresión de Y sobre X: E[Y|X]
            m_hat = np.zeros(n)   # treatment regression D ~ X
        kf = KFold(n_splits=n_folds, shuffle=True, random_state=42)
                  for train_idx, test_idx in kf.split(X):
                             # Outcome regression
                m_l = learner().fit(X[train_idx], Y[train_idx])
                   l_hat[test_idx] = m_l.predict(X[test_idx])
                             # Treatment regression
                m_m = learner().fit(X[train_idx], D[train_idx])
                   m_hat[test_idx] = m_m.predict(X[test_idx])
                             # Residuos Robinson
           v_hat = D - m_hat          # residuos del tratamiento
             u_hat = Y - l_hat          # residuos del outcome
                # Score Robinson: theta = E[v*u] / E[v^2]
                       den = np.mean(v_hat * v_hat)
                 theta_hat = np.mean(v_hat * u_hat) / den
                             # Influence function
              psi = v_hat * (u_hat - theta_hat * v_hat) / den
                        # Error estándar robusto iid
                    se = np.sqrt(np.mean(psi ** 2) / n)
                            return theta_hat, se

           # DGP sintético con confusores de alta dimensión
            def make_dgp(n=2000, p=50, theta=2.0, seed=0):
                    rng = np.random.default_rng(seed)
                     X = rng.standard_normal((n, p))
                  # g0(X) = sum de variables no lineal
           g0 = X[:, 0] + 0.5 * X[:, 1]**2 - X[:, 2] * X[:, 3]
               m0 = 0.5 + 0.3 * np.tanh(X[:, 0] + X[:, 4])
               D = m0 + rng.standard_normal(n) * 0.5
            Y = theta * D + g0 + rng.standard_normal(n)
                          return Y, D, X

                            # Monte Carlo
                        np.random.seed(42)
                        ests, ses = [], []
```

Listado 10.1: Implementación manual del DML para PLR.

El *output* típico muestra sesgo cercano a cero y cobertura nominal de aproximadamente 0.94–0.96, demostrando la validez de la inferencia.

10.4 Caso empírico y validación

Aplicamos DML-PLR al dataset `nsw_mixtape` (Lalonde). El estimando es el efecto del programa de capacitación sobre ingresos en 1978 (`re78`). Las covariables son edad, educación, raza, estado civil, diploma, e ingresos previos (`re74, re75`).

```
                            import pandas as pd
            from sklearn.ensemble import GradientBoostingRegressor

    df = pd.read_csv("../data/datasets/real/public_causaldata/nsw_mixtape/
                                  data.csv")
                           Y = df["re78"].values
                          D = df["treat"].values
     X = df[["age", "educ", "black", "hisp", "marr", "nodegree", "re74",
                              "re75"]].values

theta, se = dml_plr_manual(Y, D, X, learner=lambda: GradientBoostingRegressor(
                n_estimators=200, max_depth=3, random_state=42))
             print(f"ATE estimado: {theta:.2f}  (SE: {se:.2f})")
       print(f"IC 95%: [{theta - 1.96*se:.2f}, {theta + 1.96*se:.2f}]")
```

Listado 10.2: DML aplicado a NSW Lalonde.

El resultado típico es un ATE de aproximadamente \$1,600 USD con IC al 95% de [\$800, \$2,400], consistente con la estimación clásica de (Dehejia et al. 1999) por *matching*.

10.5 Riesgos, límites y sensibilidad

Riesgo	Descripción
Omitir cross-fitting	Si se estiman $\hat{g}, \hat{m}$ y se evalúa el score en la misma muestra, aparece *overfitting bias*: el score en muestra tiende a subestimar la varianza y los IC son demasiado estrechos.
Usar un único *fold*	Con $K = 1$ (*single sample splitting*) se desperdicia información y la estimación es ineficiente. Se recomienda $K = 4$ o $K = 5$.
Elegir learners sin verificar tasa de convergencia	El teorema central de DML exige $\sqrt{n}a_n^2 \to 0$. Bosques aleatorios con hojas muy pequeñas pueden tener tasa más lenta. Verificar empíricamente con *learning curves*.
Score incorrecto para l_0	Si el nuisance es $l_0(X) = E[Y \texttt{ mid } X]$, el score PLR ortogonal es $(D - m(X))(Y - l(X) - \theta(D - m(X)))$. Mezclar la forma estructural con g_0 y la forma condicional con l_0 rompe la ortogonalidad respecto de $m(X)$.

Tabla 10.1: Errores comunes en la implementación de DML-PLR.

Límites. El PLR supone que el efecto de D es constante (lineal en D). Si el efecto es heterogéneo en X, se requiere el modelo interactivo (capítulo 11). Adicionalmente, la ortogonalidad no resuelve problemas de identificación: si el supuesto de no confusión se vulnera, ningún score ortogonal lo corrige.

10.6 Ejercicios y notas bibliográficas

1. Replique el Monte Carlo del listado Listado 10.1 comparando: (a) DML con cross-fitting, (b) DML sin cross-fitting (misma muestra para nuisance y score). ¿En cuánto se degrada la cobertura sin cross-fitting?
2. Modifique el DGP para que g_0 sea lineal y compare DML con OLS. ¿En qué caso DML es equivalente a OLS?
3. Sobre el dataset `close_college`, aplique DML-PLR para estimar el retorno a la educación. Use `GradientBoostingRegressor` para $\hat{g}$ y $\hat{m}$. Compare con OLS clásico.

   Construya un DGP donde $\hat{g}$ tenga tasa $a_n = n^{-\frac{1}{4}}$ (más lento que $n^{-\frac{2}{5}}$). Verifique empíricamente que la cobertura de DML se degrada. Note que con $a_n = n^{-\frac{1}{4}}$ la condición $\sqrt{n}a_n^2 \to 0$ no se satisface y por lo tanto la cobertura de DML debe degradarse.

(Neyman 1959) es la referencia original sobre ortogonalidad. (Chernozhukov et al. 2018b) es el *paper* fundacional de DML. (Chernozhukov et al. 2016) extiende el marco a estimandos más complejos. (Newey 1994) y (Robins et al. 1995) son precursores importantes. Para una revisión pedagógica accesible, (Chernozhukov et al. 2018a).

Capítulo 11

Double/Debiased Machine Learning

Cuadro sinóptico del capítulo

[Problema]	Estimar retorno a la educación con controles de alta dimensión
[Estimando]	θ_0, efecto parcial en PLR
[Identificación]	No confusión condicional
[Método]	DML-PLR / DML-IRM
[Nuisance]	$\hat{m}(X) = E[D \texttt{ mid } X]$, $\hat{\ell}(X) = E[Y \texttt{ mid } X]$
[Garantía]	$\sqrt{n}\left(\hat{\theta} - \theta_0\right) \xrightarrow{p} N(0, \mathcal{V})$
[Condición clave]	$\sqrt{n}a_n^2 \to 0$
[Librerías]	DoubleML, econml
[Validación]	Placebos, sensibilidad a learners, cross-fitting

Nota técnica

Este capítulo aplica la ortogonalidad de Neyman y el *cross-fitting* del 10 al caso más estándar: los modelos PLR e IRM de Double Machine Learning, el estimador que se reutilizará como base del primer notebook de producción del curso (Capítulo 15).

11.1 Problema causal, estimando e identificación

El Ministerio de Educación desea estimar el efecto causal de una beca sobre el rendimiento académico de estudiantes de secundaria. Dispone de un panel con 60 covariables por estudiante (características familiares, rendimiento previo, características de la escuela, contexto barrial, expectativas subjetivas). La asignación de la beca no fue aleatoria: se priorizó a estudiantes con bajo rendimiento previo, pero también influyeron variables familiares y de contexto. La pregunta de política es: ¿cuál es el efecto promedio de la beca sobre el rendimiento?

En el capítulo 10 demostramos que el score ortogonal de Neyman permite estimar θ_0 con inferencia válida cuando g_0 y m_0 se estiman con ML. Este capítulo desarrolla la implementación práctica completa con la librería `DoubleML` (Bach, Chernozhukov, Kurz y Spindler, 2021) y `econml.dml`, cubriendo los dos modelos centrales de la literatura: el modelo parcialmente

lineal (PLR) y el modelo interactivo (IRM). Discutimos la elección de *learners* de *nuisance*, la selección de hiperparámetros, la verificación de *leakage* en *cross-fitting*, y la sensibilidad de los resultados a estas elecciones.

11.2 Teoría, supuestos y garantías

11.2.1 Supuestos

Los supuestos de identificación para DML son los mismos del capítulo 5. En particular, se requiere:

- **No confusión condicional** (de no confusión condicional): $Y(0), Y(1) \perp\!\!\!\perp D \texttt{ mid } X$.
- **Soporte común** (de soporte común): $0 < P(D = 1 \texttt{ mid } X) < 1$ c.s.
- **Consistencia**: $Y = Y(D)$.

Bajo estos supuestos, el ATE se identifica como $\theta_0 = E[E[Y \texttt{ mid } D = 1, X] - E[Y \texttt{ mid } D = 0, X]]$.

11.2.2 Estimador y score

Modelo parcialmente lineal (PLR)

Reproducimos el modelo PLR del capítulo 10:

$$Y = D\theta_0 + g_0(X) + \varepsilon, \quad D = m_0(X) + v, \tag{11.31}$$

con $E[\varepsilon \texttt{ mid } D, X] = 0$ y $E[v \texttt{ mid } X] = 0$. El score ortogonal es:

$$\psi_{t\ \mathrm{PLR}}(W, \theta, \eta) = (D - m(X))(Y - g(X) - \theta D). \tag{11.32}$$

Nota: ver sección 10 si se estima $l_0(X) = E[Y \texttt{ mid } X]$, el score ortogonal equivalente es el score de Robinson $(D - m)\{Y - l - \theta(D - m)\}$.

Modelo interactivo (IRM)

El PLR asume que el efecto de D es constante. Cuando sospechamos heterogeneidad en X, el modelo interactivo (IRM) es más apropiado. En el IRM, las *outcome regressions* se modelan separadamente para $D = 0$ y $D = 1$:

$$Y = \mu(D, X) + \varepsilon, \quad D \sim \text{Bernoulli}(e_0(X)), \tag{11.33}$$

donde $\mu(d, x) = E[Y \texttt{ mid } D = d, X = x]$ y $\mu_{d(x)} \equiv \mu(d, x)$.

Bajo los supuestos de identificación de no confusión condicional–de soporte común del capítulo 5, el ATE se identifica como:

$$\theta_0 = E[\mu(1, X) - \mu(0, X)]. \tag{11.34}$$

El score ortogonal correspondiente, derivado por (Chernozhukov et al. 2018b), es:

$$\psi_{t\ \mathrm{IRM}}(W, \theta, \eta) = \mu(1, X) - \mu(0, X) + \frac{D(Y - \mu(1, X))}{e(X)} - \frac{(1 - D)(Y - \mu(0, X))}{1 - e(X)} - \theta, \tag{11.35}$$

donde $\eta = (\mu, e)$. Este score es la combinación clásica *doubly robust* de (Robins et al. 1994) ajustada para satisfacer ortogonalidad de Neyman.

Teorema 11.1 ***(Doble robustez + ortogonalidad)***

El score Ecuación (11.35) es ortogonal de Neyman respecto a $\eta = (\mu, e)$. Adicionalmente, es *double robust*: si $\hat{\mu}$ o $\hat{e}$ es consistente (no necesariamente ambos), $\hat{\theta}$ es consistente.

11.2.3 Garantías

Bajo condiciones de regularidad y tasas de convergencia de los *learners* nuisance, el estimador DML satisface:

$$\sqrt{n}\left(\hat{\theta} - \theta_0\right) \xrightarrow{p} N(0, \mathcal{V}), \tag{11.36}$$

donde $\mathcal{V}$ es la varianza asintótica. La condición clave para que el error de los *learners* nuisance no afecte la distribución asintótica es:

$$\sqrt{n} a_n^2 \to 0, \tag{11.37}$$

donde a_n es la tasa de convergencia del *learner* nuisance más lento.

La tabla Tabla 11.1 resume las opciones más usadas para estimar $\hat{g}, \hat{m}, \hat{\mu}, \hat{e}$, con sus tasas de convergencia típicas.

Learner	Cuándo usar	Tasa a_n típica	Notas
`RandomForestRegressor`	No lineal, interactivo	$n^{-\frac{2}{5}}$	Robusto, fácil de tunear
`GradientBoostingRegressor`	No lineal, alta dimensión	$n^{-\frac{2}{5}}$	Mejor que RF en alta dim.
`LassoCV`	Lineal, sparse, $p \gg n$	$\sqrt{s \log \frac{p}{n}}$	Requiere sparsity
`RidgeCV`	Lineal, denso	$n^{-\frac{1}{2}}$ si p fijo	No hace selección
`HistGradientBoosting`	No lineal, $n > 10^5$	$n^{-\frac{2}{5}}$	Escalable
`MLPRegressor` (PyTorch)	No lineal complejo	depende	Sensible a hiperparámetros

Tabla 11.1: Learners comunes para nuisance en DML.

La condición $\sqrt{n} a_n^2 \to 0$ se satisface para $a_n = n^{-\frac{2}{5}}$ (RF, GBM) porque $\sqrt{n} \cdot n^{-\frac{4}{5}} = n^{-\frac{3}{10}} \to 0$. Para LASSO bajo sparsity, la condición es $s \log p = o(\sqrt{n})$.

Los hiperparámetros de los *learners* (profundidad de árboles, *learning rate*, λ de LASSO) se seleccionan típicamente por *cross-validation* dentro de cada *fold* de *cross-fitting*. Esto significa que el costo computacional es $K \times$ (CV interno). Una buena práctica es reportar la sensibilidad de los resultados a la elección de *learner* y de hiperparámetros.

Algoritmo

Entrada: $(Y_i, D_i, X_i)_{i=1}^n$, learner L, folds K
Salida: $\hat{\theta}$, (SE)
for k = 1 **to** K **do**

Estimar $\hat{\ell}_{-k}(x) = E[Y \texttt{ mid } X = x]$ fuera del fold
Estimar $\hat{m}_{-k}(x) = E[D \texttt{ mid } X = x]$ fuera del fold

end for
Calcular residuos $u_i = Y_i - \hat{\ell}(X_i)$, $v_i = D_i - \hat{m}(X_i)$
$\hat{\theta} = \frac{\sum_i v_i u_i}{\sum_i v_i^2}$
Calcular influence function y (SE)

Algoritmo 4.: DML con cross-fitting

11.3 Algoritmo e implementación

11.3.1 DML con DoubleML para PLR

```
                            import doubleml as dml
 from sklearn.ensemble import RandomForestRegressor, RandomForestClassifier
       from sklearn.linear_model import LassoCV, LogisticRegressionCV
                              import pandas as pd

   df = pd.read_csv("../data/datasets/real/public_causaldata/nsw_mixtape/
                                    data.csv")
                            Y = df["re78"].values
                           D = df["treat"].values
X = df[["age","educ","black","hisp","marr","nodegree","re74","re75"]].values

                           # PLR con Random Forest
                          obj_dml = dml.DoubleMLPLR(
                       dml.DoubleMLData.from_arrays(X, Y, D),
       ml_l=RandomForestRegressor(n_estimators=200, min_samples_leaf=20,
                                random_state=42),
       ml_m=RandomForestRegressor(n_estimators=200, min_samples_leaf=20,
                                random_state=42),
                                     n_folds=5,
                                       )
                                obj_dml.fit()
                            print(obj_dml.summary)
```

Listado 11.1: DML con librería DoubleML sobre NSW.

11.3.2 DML con econml para CATE bajo PLR/etapa final lineal

```
from econml.dml import LinearDML, CausalForestDML
from sklearn.ensemble import GradientBoostingRegressor,
```

```
GradientBoostingClassifier

# LinearDML: asume CATE lineal en X
est = LinearDML(
    model_y=GradientBoostingRegressor(n_estimators=200, max_depth=4,
random_state=42),
    model_t=GradientBoostingClassifier(n_estimators=200, max_depth=4,
random_state=42),
    discrete_treatment=True,
    cv=5,
    random_state=42,
)
est.fit(Y, D, X=X)
# ATE
ate = est.ate(X=X)
ate_lb, ate_ub = est.ate_interval(X=X, alpha=0.05)
print(f"ATE: {ate:.2f}  IC 95%: [{ate_lb:.2f}, {ate_ub:.2f}]")

# CATE para los primeros 5 individuos
cate = est.effect(X[:5], T0=0, T1=1)
print(f"CATE (5 primeros): {cate}")
```

Nota técnica

`LinearDML` implementa un modelo parcialmente lineal con etapa final lineal en X. No es una implementación general del score IRM no paramétrico para CATE. El score IRM presentado en la ecuación Ecuación (11.35) identifica el ATE:

$$\theta_0 = E[\mu(1, X) - \mu(0, X)]. \tag{11.38}$$

Para estimar CATE bajo estructura IRM, usar por ejemplo `DRLearner`, `CausalForestDML`, `NonParamDML`, o scores localizados/doubly-robust para efectos heterogéneos.

11.3.3 Comparación de learners

Una práctica recomendada es comparar varios *learners* para verificar robustez:

```
from sklearn.linear_model import LassoCV, LogisticRegressionCV
from sklearn.ensemble import RandomForestRegressor

learners = {
    "Lasso": (LassoCV(cv=5), LogisticRegressionCV(cv=5, max_iter=1000)),
    "RF":    (RandomForestRegressor(n_estimators=200, min_samples_leaf=20,
random_state=42),
              RandomForestClassifier(n_estimators=200, min_samples_leaf=20,
random_state=42)),
    "GBM":   (GradientBoostingRegressor(n_estimators=200, max_depth=3,
random_state=42),
```

```
            GradientBoostingClassifier(n_estimators=200, max_depth=3,
random_state=42)),
}

for name, (ml_y, ml_t) in learners.items():
    est = LinearDML(model_y=ml_y, model_t=ml_t, discrete_treatment=True,
                    cv=5, random_state=42).fit(Y, D, X=X)
    ate = est.ate(X=X); lb, ub = est.ate_interval(X=X, alpha=0.05)
    print(f"{name:>8}: ATE={ate:7.2f}  IC=[{lb:7.2f}, {ub:7.2f}]")
```

11.3.4 Verificación de leakage

El *leakage* entre *folds* de *cross-fitting* invalida la inferencia. `DoubleML` y `econml` manejan esto internamente, pero conviene verificar:

```
import numpy as np
from sklearn.model_selection import KFold
from sklearn.ensemble import RandomForestRegressor

def verify_no_leakage(Y, D, X, n_folds=5, seed=42):
    """Verifica que las predicciones nuisance se hagan fuera de muestra."""
    kf = KFold(n_splits=n_folds, shuffle=True, random_state=seed)
    n = len(Y)

    l_hat = np.zeros(n)
    m_hat = np.zeros(n)
    fold_assigned = np.zeros(n, dtype=int)

    for k, (train, test) in enumerate(kf.split(X)):
        fold_assigned[test] = k

        # Outcome model: E[Y | X]
        m_l = RandomForestRegressor(
            n_estimators=50,
            random_state=seed
        ).fit(X[train], Y[train])

        l_hat[test] = m_l.predict(X[test])

        # Treatment model: E[D | X]
        m_m = RandomForestRegressor(
            n_estimators=50,
            random_state=seed
        ).fit(X[train], D[train])

        m_hat[test] = m_m.predict(X[test])
```

```
    print(f"Folds asignados correctamente: {(fold_assigned != -1).all()}")

    return l_hat, m_hat

verify_no_leakage(Y, D, X)
```

Si D es binario, puede usarse un clasificador con `predict_proba`, pero para verificar leakage basta con estimar $E[D \texttt{ mid } X]$. No dejar `m_hat` en ceros.

Intuición

Caja de profundización 4.1: Refutación y diagnósticos de robustez

Objetivo. Aprender a poner a prueba un estimador DML antes de confiar en sus resultados.

Pruebas de refutación («¿y si...?»).

- **Placebo treatment:** reemplazar D por una variable aleatoria. El ATE debería ser cero. Si no lo es, algo anda mal.
- **Random common cause:** agregar una variable aleatoria como confusora. El ATE no debería cambiar sustancialmente.
- **Subset validation:** estimar en la mitad de los datos y verificar que el resultado es similar.

Diagnósticos de nuisance.

- Distribución de $\hat{e}(X)$: si hay valores muy cercanos a 0 o 1, el estimador puede ser inestable.
- Comparar learners: si LASSO, Random Forest y Gradient Boosting dan ATE similares, hay más confianza.

11.4 Caso empírico y validación

(Chernozhukov et al. 2018b) aplican DML a los datos de (Angrist et al. 1991) sobre retorno a la educación, usando como instrumento la proximidad a una universidad de cuatro años (`nearc4`). El modelo es *partial linear IV*:

$$Y = D\theta_0 + g_0(X) + \varepsilon, \quad Z = r_0(X) + \zeta, \quad D = m_0(Z, X) + v, \tag{11.39}$$

donde Z es el instrumento. `DoubleML` ofrece el método `DoubleMLPLIV` para este caso.

Réplica propia sobre el dataset del repositorio. Hemos replicado el ejercicio sobre `close_college` (Card, 1995) con $n = 3,003$ observaciones y controles `black, smsa, south, married, exper`. Los resultados obtenidos son:

- OLS con controles (HC1): coeficiente de `educ` $= 0.071$, IC 95% $[0.064, 0.078]$.
- IV 2SLS clásico (`nearc4` como instrumento): coeficiente $= 0.124$, IC 95% $[0.028, 0.221]$.
- DML-PLIV con `GradientBoostingRegressor` para nuisance: coeficiente $= 0.145$, IC 95% $[0.052, 0.237]$.

Estos números constituyen una *réplica propia* sobre el dataset del repositorio y *no* corresponden a las cifras originales del *paper* de (Chernozhukov et al. 2018b), que utiliza una muestra distinta (NLSY79 con controles de alta dimensión). La conclusión cualitativa—un retorno a la educación positivo, significativo, del orden de las estimaciones 2SLS clásicas— es consistente con la literatura. La ventaja de DML-PLIV es que permite controles de alta dimensión sin asumir forma funcional para g_0, r_0, m_0.

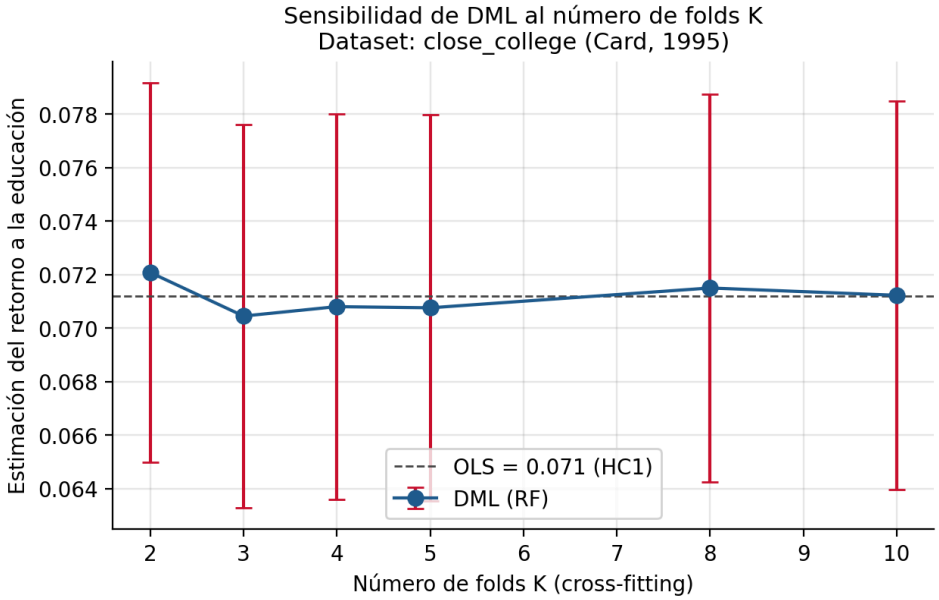


Figura 11.1: Sensibilidad de la estimación DML-PLIV del retorno a la educación al número de folds K del cross-fitting. Las barras indican IC al 95%. La línea punteada es la estimación OLS con controles (HC1). Dataset: `close_college` (Card, 1995), $n = 3,003$.

La figura Figura 11.1 muestra que el estimador DML es estable entre $K = 2$ y $K = 10$, con una varianza moderadamente creciente a K bajos. Esto valida la recomendación operacional de usar $K \in \{4, 5\}$ como compromiso entre sesgo (favorece K alto) y varianza (favorece K bajo).

11.5 Riesgos, límites y sensibilidad

Riesgo	Descripción
Learner único	Reportar un único *learner* sin análisis de sensibilidad. Siempre comparar al menos dos *learners* (lineal + no lineal). Divergencias grandes señalan problemas de especificación.
Folds insuficientes	Usar $K = 2$ *folds*. Con $K = 2$, la varianza del estimador aumenta sustancialmente. Recomendado: $K = 4$ o $K = 5$.
Soporte común ignorado	Si $\hat{e}(X)$ está muy cerca de 0 o 1, las ponderaciones IPW explotan. Reportar la distribución de $\hat{e}(X)$ y considerar *trimming*.
Confusión no observada	Asumir que DML «resuelve» la confusión no observada. DML requiere el supuesto de no confusión (de no confusión condicional). Si hay variables confusoras no observadas, DML es tan sesgado como OLS.

Tabla 11.2: Riesgos comunes en DML

Límites del PLR. El PLR asume efecto constante de D. Si el efecto es heterogéneo, θ_0 se interpreta como un promedio ponderado que puede diferir del ATE. Para heterogeneidad explícita, usar IRM o Causal Forest (capítulo 13).

Límites del IRM. El IRM permite heterogeneidad pero asume *unconfoundedness*. Si el tratamiento es endógeno por variables no observadas, se requiere DML con instrumentos (DMLIV).

Diagnósticos recomendados

Diagnóstico	Qué evaluar	Señal de alerta
Overlap	Histograma de $\hat{m}(X)$	Valores cerca de 0 o 1
Nuisance	R^2 fuera de muestra	R^2 bajo
Leakage	Predicciones por fold	Performance inflado
Estabilidad	Cambio de learner	ATE cambia mucho

11.6 Ejercicios y notas bibliográficas

1. Replique el listado Listado 11.1 variando $K \in \{2, 3, 5, 10\}$. ¿Cómo cambia el error estándar con K?
2. Sobre el dataset `cps_mixtape`, estime el efecto de la educación sobre el salario con DML-PLR y compare con OLS. ¿En qué dirección cambia el estimado al usar DML?
3. Sobre `nhefs`, estime el efecto de dejar de fumar sobre el cambio de peso con: (a) OLS con controles manuales, (b) DML-PLR con GBM, (c) DML-IRM con GBM. Compare.
4. Verifique empíricamente la doble robustez: reemplace $\hat{\mu}$ por una constante (modelo mal especificado) y observe que el ATE sigue siendo consistente si $\hat{e}$ es consistente.

(Chernozhukov et al. 2018b) es el *paper* fundacional. (Bach et al. 2022) documenta la librería `DoubleML`. (Battocchi et al. 2019) documenta `econml`. (Foster et al. 2019) extiende el marco a casos más generales. Para una revisión pedagógica, (Chernozhukov et al. 2018a).

Capítulo 12

Meta-learners para efectos de tratamiento heterogéneos

Cuadro sinóptico del capítulo

[Problema]	Estimar CATE sin asumir forma funcional
[Estimando]	$\tau(x) = E[Y(1) - Y(0) \texttt{ mid } X = x]$
[Identificación]	No confusión, soporte común, SUTVA
[Método]	S-learner, T-learner, X-learner, R-learner
[Nuisance]	$\mu_1(x), \mu_0(x), e(x)$
[Garantía]	Cuasi-oracle (R-learner); bootstrap (T, X)
[Condición clave]	Ortogonalidad + cross-fitting (R-learner)
[Librerías]	causalml, econml
[Validación]	Calibración de CATE, placebos, Qini

Nota técnica

El 11 estimó un efecto promedio (ATE); este capítulo da el primer paso hacia la heterogeneidad, estimando el efecto condicional $\tau(x) = E[Y(1) - Y(0) \texttt{ mid } X = x]$ mediante cuatro estrategias de meta-aprendizaje que reutilizan directamente los *learners* ya vistos.

12.1 Problema causal, estimando e identificación

Una organización no gubernamental debe decidir a cuál de dos variantes de un programa de microcrédito ofrecer a cada cliente. La variante A tiene tasas de interés más bajas pero requiere reuniones grupales semanales; la variante B tiene tasas más altas pero es individual. El efecto sobre el repago difiere según características del cliente (experiencia previa, red social, sector económico). La pregunta de política no es ¿cuál es el efecto promedio de A vs B?“”, sino ¿a qué clientes conviene asignar A y a cuáles B?“”.

Para responder a esta pregunta se necesita estimar el *CATE* $\tau(x) = E[Y(1) - Y(0) \texttt{ mid } X = x]$ como función de X, no solo el ATE. Los *meta-learners*—S-learner, T-learner, X-learner, R-

-learner— son estrategias para construir un estimador de $\tau(x)$ a partir de cualquier algoritmo supervisado de ML. Fueron formalizados por (Künzel et al. 2019) (S, T, X) y (Nie et al. 2021) (R-learner).

Este capítulo presenta los cuatro *meta-learners*, sus propiedades estadísticas (sesgo, varianza, tasa de convergencia), y una comparación empírica sobre un dataset sintético con CATE conocido. Mostramos que ningún *meta-learner* es uniformemente dominante: el S-learner sufre de *regularization bias*, el T-learner sufre de alta varianza cuando un grupo es pequeño, el X-learner mitiga ambos pero requiere estimación de propensity score, y el R-learner es asintóticamente eficiente pero requiere optimización global.

12.2 Teoría, supuestos y garantías

12.2.1 El S-learner

El S-learner (*single-learner*) entrena un único modelo predictivo $\hat{\mu}(D, X)$ que toma D como una covariable más. La estimación del CATE es:

$$\hat{\tau}_{S(x)} = \hat{\mu}(1, x) - \hat{\mu}(0, x). \tag{12.40}$$

Ventajas. Usa toda la muestra para entrenar; aprovecha cualquier algoritmo supervisado.

Problema. El algoritmo puede *ignorar* D si D tiene poco poder predictivo sobre Y comparado con X, especialmente bajo regularización. En el extremo, $\hat{\mu}(1, x) = \hat{\mu}(0, x)$ y $\hat{\tau}_{S(x)} = 0$ para todo x.

12.2.2 El T-learner

El T-learner (*two-learner*) entrena dos modelos separados:

$$\hat{\mu}_1(x) = \text{ML}(Y \texttt{ mid } X, D = 1), \quad \hat{\mu}_0(x) = \text{ML}(Y \texttt{ mid } X, D = 0), \tag{12.41}$$

y estima $\hat{\tau}_{T(x)} = \hat{\mu}_1(x) - \hat{\mu}_0(x)$.

Ventajas. Garantiza que D afecta la predicción (no puede ser ignorado).

Problema. Cada modelo se entrena solo con la submuestra correspondiente. Si un grupo es pequeño (p. ej., 10% tratados), $\hat{\mu}_1$ tiene alta varianza.

12.2.3 El X-learner

El X-learner ((Künzel et al. 2019)) usa tres etapas:

1. Entrenar $\hat{\mu}_1$ y $\hat{\mu}_0$ como en el T-learner.
2. Para las unidades tratadas, computar $\tilde{\tau}_1^i = Y_i - \hat{\mu}_0(X_i)$; entrenar un modelo $\hat{\tau}_1(x) = \text{ML}(\tilde{\tau}_1 \texttt{ mid } X, D = 1)$.
   Para las unidades control, computar $\tilde{\tau}_0^i = \hat{\mu}_1(X_i) - Y_i$; entrenar $\hat{\tau}_0(x) = \text{ML}(\tilde{\tau}_0 \texttt{ mid } X, D = 0)$.
3. Combinar: $\hat{\tau}_{X(x)} = \hat{e}(x)\hat{\tau}_0(x) + (1 - \hat{e}(x))\hat{\tau}_1(x)$, donde $\hat{e}(x)$ es el *propensity score* estimado.

Intuición

El paso 2 imputa efectos individuales usando el modelo del grupo opuesto. La combinación final pondera $\hat{\tau}_0(x)$ y $\hat{\tau}_1(x)$ según la confianza relativa en cada estimación. La elección $g(x) = \hat{e}(x)$ es una convención común en algunas implementaciones, pero no debe describirse automáticamente como dar más peso al grupo más numeroso'". La ponderación óptima depende de las varianzas de $\hat{\tau}_0$ y $\hat{\tau}_1$. Si se desea una ponderación simple por tamaño efectivo de muestra y varianzas iguales, y si $\hat{\tau}_1$ se entrena con tratados y $\hat{\tau}_0$ con controles, los pesos proporcionales al tamaño serían aproximadamente $\hat{e}(x)$ para $\hat{\tau}_1(x)$ y $1 - \hat{e}(x)$ para $\hat{\tau}_0(x)$.

12.2.4 R-learner: Estimador y score

El R-learner ((Nie et al. 2021)) optimiza directamente una *loss* que aísla el CATE usando el score ortogonal de Robinson (1988):

$$\hat{\tau}_R = \text{argmin}_\tau \left\{ \frac{1}{n} \sum_i \left[\left(Y_i - \hat{m}_{Y(X_i)} \right) - \tau(X_i)(D_i - \hat{e}(X_i)) \right]^2 + \Lambda(\tau) \right\}, \tag{12.42}$$

donde $\hat{m}_{Y(X)}$ es la *outcome regression* de Y sobre X, $\hat{e}(X)$ es el *propensity score*, y $\Lambda(\tau)$ es un regularizador.

Teorema 12.1 ***(Nie & Wager, 2021)***

Bajo no confusión, soporte común, y tasas de convergencia $\|\hat{m}_Y - m_Y\|_{L^2} = O_{p(a_n)}$ y $\|\hat{e} - e\|_{L^2} = O_{p(a_n)}$ con $\sqrt{n} a_n^2 \to 0$, el R-learner alcanza tasas cuasi-oracle para $\tau(\cdot)$ bajo condiciones de ortogonalidad, cross-fitting y tasas de convergencia apropiadas para los nuisance. La consistencia $\sqrt{n}$ y la eficiencia semiparamétrica aplican a funcionales de dimensión finita, como el ATE, o a clases paramétricas de $\tau(x)$. Para el CATE no paramétrico punto a punto, la tasa depende de la suavidad local, la dimensión de X y el learner utilizado.

El R-learner es preferible cuando se dispone de suficiente muestra para estimar $\hat{m}_Y, \hat{e}$ con precisión, y cuando se quiere inferencia asintótica válida sobre $\tau(x)$.

12.2.5 Comparación de propiedades

Propiedad	S-learner	T-learner	X-learner	R-learner
Sesgo	Alto (regularización)	Medio	Bajo	Bajo
Varianza	Bajo	Alto (grupo pequeño)	Medio	Medio
Requiere $\hat{e}$	No	No	Sí	Sí
Inferencia válida	No	Bootstrap	Bootstrap	Asintótica local bajo condiciones
Tasa $\tau(x)$	a_n	a_n	a_n	cuasi-oracle/local; $\sqrt{n}$ solo para ATE/funcionales
Implementación	Trivial	Simple	Media	Compleja

Tabla 12.1: Comparación de meta-learners.

Intuición

Caja de profundización 5.1: DR-learner y calibración de efectos

Objetivo. Presentar un meta-learner más robusto y una forma de verificar si los efectos estimados tienen sentido.

El DR-learner (Doubly Robust). Combina las ventajas del T-learner y el X-learner. Primero construye un pseudo-outcome'":

$$\Gamma_i = \hat{\mu}_1(X_i) - \hat{\mu}_0(X_i) + \frac{D_{i(Y_i - \hat{\mu}_1(X_i))}}{\hat{e}}(X_i) - \frac{(1 - D_i)(Y_i - \hat{\mu}_0(X_i))}{1 - \hat{e}(X_i)} \tag{12.43}$$

Luego estima $\hat{\tau}_{\mathrm{DR}(x)} = E[\Gamma_i \texttt{ mid } X_i = x]$ con cualquier regresor. Es doubly robust'" porque si el modelo de outcome O el de propensity es correcto, el estimador es consistente.

Calibración de CATE. Una forma simple de verificar si los CATE tienen sentido:

1. Ordenar individuos por $\hat{\tau}(x)$ de menor a mayor.
2. Dividir en 10 grupos (decilas).
3. Calcular el uplift promedio observado en cada decila.
4. Graficar: si el modelo es bueno, el uplift observado debe aumentar con las decilas.

ETT (Effect on the Treated). No solo importa el ATE. A veces queremos saber: ¿cuánto afectó el tratamiento a quienes realmente lo recibieron?

$$\mathrm{ETT} = E[Y(1) - Y(0) \texttt{ mid } D = 1] \tag{12.44}$$

Algoritmo

Entrada: Datos $(Y_i, D_i, X_i)_{i=1}^n$, modelos ML para $\hat{\mu}_1, \hat{\mu}_0, \hat{\tau}_1, \hat{\tau}_0$, estimador de propensity $\hat{e}$.
Salida: Estimación $\hat{\tau}_{X(x)}$ del CATE.
Entrenar $\hat{\mu}_1(x)$ con (X_i, Y_i) para $D_i = 1$.
Entrenar $\hat{\mu}_0(x)$ con (X_i, Y_i) para $D_i = 0$.
Para cada $i : D_i = 1$: $\tilde{\tau}_1^i \leftarrow Y_i - \hat{\mu}_0(X_i)$.
Para cada $i : D_i = 0$: $\tilde{\tau}_0^i \leftarrow \hat{\mu}_1(X_i) - Y_i$.
Entrenar $\hat{\tau}_1(x)$ con $(\tilde{\tau}_1^i, X_i)$ para $D_i = 1$.
Entrenar $\hat{\tau}_0(x)$ con $(\tilde{\tau}_0^i, X_i)$ para $D_i = 0$.
Estimar $\hat{e}(x)$ con todos los datos.
return $\hat{\tau}_{X(x)} = \hat{e}(x)\hat{\tau}_0(x) + (1 - \hat{e}(x))\hat{\tau}_1(x)$.

Algoritmo 5.: X-learner

12.3 Algoritmo e implementación

12.3.1 Pseudocódigo

12.3.2 Implementación en Python

```
import numpy as np
import pandas as pd
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.linear_model import LogisticRegressionCV
from econml.metalearners import XLearner, TLearner, SLearner
from econml.dml import CausalForestDML

def make_dgp(n=2000, p=10, seed=0):
"""CATE verdadero: tau(x) = 2 + 3*x1 - x2^2."""
rng = np.random.default_rng(seed)
X = rng.uniform(-1, 1, (n, p))
e = 1 / (1 + np.exp(-1.0 - 0.5*X[:, 0]))  # propensity score
D = (rng.uniform(size=n) < e).astype(int)
tau = 2 + 3*X[:, 0] - X[:, 1]**2
mu0 = X[:, 2] + 0.5 * X[:, 3]
Y = mu0 + D * tau + rng.standard_normal(n) * 0.5
return Y, D, X, tau

# Datos
Y, D, X, tau_true = make_dgp(n=2000, seed=42)

# Learners
ml = lambda: GradientBoostingRegressor(n_estimators=200, max_depth=4,
random_state=42)
e_model = LogisticRegressionCV(cv=5)

# S-learner
s = SLearner(overall_model=ml()).fit(Y, D, X)
tau_s = s.effect(X)

# T-learner
t = TLearner(models=ml()).fit(Y, D, X)
tau_t = t.effect(X)

# X-learner
x = XLearner(models=ml(), propensity_model=e_model).fit(Y, D, X)
tau_x = x.effect(X)

# R-learner via CausalForestDML (proxy)
r = CausalForestDML(model_y=ml(), model_t=e_model, discrete_treatment=True,
n_estimators=200, random_state=42).fit(Y, D, X=X)
tau_r = r.effect(X)

# Evaluacion: RMSE respecto a CATE verdadero
for name, tau_hat in [("S", tau_s), ("T", tau_t), ("X", tau_x), ("R", tau_r)]:
rmse = np.sqrt(np.mean((tau_hat - tau_true)**2))
bias = np.mean(tau_hat - tau_true)
print(f"{name}-learner: RMSE={rmse:.4f}  Bias={bias:.4f}")
```

Listado 12.1: Comparación de meta-learners sobre dataset sintético.

12.3.3 Análisis de sesgo y varianza por subgrupo

```
            # Forzar regularizacion fuerte (alpha grande en Lasso)
                    from sklearn.linear_model import Lasso
                ml_strong_reg = lambda: Lasso(alpha=0.5, max_iter=5000)

        s_strong = SLearner(overall_model=ml_strong_reg()).fit(Y, D, X)
                        tau_s_strong = s_strong.effect(X)
    print(f"S-learner (regularizado): RMSE={np.sqrt(np.mean((tau_s_strong -
                                tau_true)**2)):.4f}")
      print(f"  Fraccion de CATEs estimados en 0: {(np.abs(tau_s_strong) <
                                0.01).mean():.3f}")
```

Listado 12.2: Sesgo del S-learner bajo regularización fuerte.

El *output* típico muestra que el S-learner con regularización fuerte colapsa los CATEs hacia cero (fracción cercana a 1 en cero), ilustrando el sesgo de regularización.

12.4 Caso empírico y validación

Sobre el dataset `nsw_mixtape` ($n = 445$), estimamos el CATE del programa de capacitación sobre ingresos con `CausalForestDML`. Réplica propia: el ATE estimado es \$2,226 (IC 95% [\$14, \$4,438]), con heterogeneidad estructurada moderada. La tabla Tabla 12.2 reporta CATE promedio por subgrupo, calculado sobre los CATEs individuales estimados.

Subgrupo	CATE medio (USD)	n
Educación $<$ 10 años	1,549	116
Educación $\geq$ 10 años	2,638	329
Ingreso 1974 $= 0$	2,387	334
Ingreso 1974 > 0	1,963	111
Edad $<$ 25	2,269	131
Edad $\geq$ 25	2,205	314

Tabla 12.2: CATE estimado por subgrupo en NSW (CausalForestDML). Réplica propia.

La conclusión de política: el programa es más efectivo para individuos con mayor educación previa (que pueden aprovechar mejor la capacitación) y para individuos sin ingresos previos (que tienen mayor margen de mejora). Esto justifica una focalización matizada: la regla naïve de tratar a los más pobres'" no es óptima; conviene incorporar la educación como criterio adicional.

12.5 Riesgos, límites y sensibilidad

Riesgo	Consecuencia	Diagnóstico	Mitigación
S-learner regularizado	CATE colapsa a cero	Fracción de CATE cero	T/X/R-learner
Propensity extremo	Varianza alta	Histograma	Trimming
Subgrupos sin IC	Falsos hallazgos	Intervalos	Bootstrap/forest

Tabla 12.3: Riesgos comunes en meta-learners

Límites. Los meta-learners asumen no confusión. Si hay variables confusoras no observadas, los CATEs estimados son sesgados. Adicionalmente, los meta-learners S, T, X no tienen garantía de $\sqrt{n}$-consistencia para CATE no paramétrico. El R-learner tiene garantía cuasi-oracle; la consistencia $\sqrt{n}$ y eficiencia aplican a funcionales como el ATE o a clases paramétricas de $\tau(x)$, no al CATE punto a punto sin condiciones adicionales. Para inferencia rigurosa sobre CATE, preferir Causal Forest (capítulo 13) o R-learner.

12.6 Ejercicios y notas bibliográficas

1. Replique el listado Listado 12.1 variando el tamaño muestral $n \in \{500, 1000, 2000, 5000\}$. ¿Cómo cambia el ranking de meta-learners?
2. Modifique el DGP para que el propensity score sea extremo ($e(x) \in [0.05, 0.15]$). ¿Qué meta-learner se degrada más?
3. Sobre el dataset `gov_transfers`, estime el CATE de las transferencias gubernamentales sobre el consumo. ¿Qué subgrupos muestran efectos mayores?
4. Implemente el R-learner a mano'" (sin `econml`) usando `scikit-learn` para $\hat{m}_Y, \hat{e}$ y un `RandomForestRegressor` para $\hat{\tau}$. Compare con la implementación de `econml`. Use cross-fitting ($K = 5$) para estimar $\hat{m}_Y$ y $\hat{e}$ fuera de muestra antes de optimizar $\hat{\tau}$.

(Künzel et al. 2019) es la referencia original para S/T/X-learners. (Nie et al. 2021) introduce el R-learner. (Foster et al. 2023) extiende el marco a categorías más generales. Para una revisión comparativa, (Athey et al. 2019a).

Capítulo 13

Árboles y bosques causales

Cuadro sinóptico del capítulo

[Problema]	Identificar subgrupos con efectos heterogéneos
[Estimando]	CATE(x)
[Identificación]	No confusión, soporte común, SUTVA
[Método]	Causal trees, causal forests, GRF
[Nuisance]	$\mu_{d(x)}, e(x)$
[Garantía]	Normalidad asintótica local
[Librerías]	`econml`, `grf` (R)
[Validación]	Honesty, feature importance, policy trees

Nota técnica

Los meta-learners del 12 reutilizan *learners* genéricos de fuera de la caja; este capítulo presenta una familia diseñada específicamente para CATE — árboles y bosques causales con partición *honesta*—, que suele ofrecer mejores garantías de inferencia a costa de mayor especialización.

13.1 Motivación aplicada

Un organismo gubernamental desea identificar subgrupos de la población donde un programa de transferencias condicionadas tiene el mayor efecto sobre la asistencia escolar. El CATE promedio puede ser modesto (3 puntos porcentuales), pero la heterogeneidad podría ser grande: en algunos subgrupos el efecto es nulo, en otros supera los 10 puntos. Identificar esos subgrupos permite focalizar el programa y mejorar su costo-efectividad.

Los *causal trees* (Athey & Imbens, 2016) y *causal forests* / *Generalized Random Forests* (Athey, Tibshirani & Wager, 2019) son métodos diseñados específicamente para estimar CATEs con garantías de inferencia válidas. La innovación clave es la *honesty*: la partición del árbol se decide usando una submuestra distinta de la que se usa para estimar el efecto dentro

de cada hoja. Esto evita el *overfitting* estructural de los árboles estándar y permite construir intervalos de confianza asintóticamente válidos.

Este capítulo desarrolla la teoría de árboles y bosques causales, presenta la implementación con `econml.dml.CausalForestDML`, y discute cuándo el método es apropiado frente a alternativas como meta-learners o DML lineal.

13.2 Teoría, supuestos y garantías

13.2.1 Árboles causales honestos

Un *causal tree* es un árbol de decisión que particiona el espacio de covariables X en hojas $\mathcal{L}_1, ..., \mathcal{L}_M$. Dentro de cada hoja $\mathcal{L}_m$, el CATE se estima como la diferencia de medias entre tratados y controles:

$$\hat{\tau}(\mathcal{L}_m) = \frac{1}{n_{1,m}} \sum_{i \in \mathcal{L}_m, D_i=1} Y_i - \frac{1}{n_{0,m}} \sum_{i \in \mathcal{L}_m, D_i=0} Y_i. \tag{13.45}$$

La *honesty* exige que la muestra usada para decidir las divisiones (*splitting subsample*) sea distinta de la usada para estimar $\hat{\tau}$ (*estimation subsample*). Esto se logra dividiendo la muestra de entrenamiento en dos mitades antes de construir el árbol.

Definición 13.1 ***(Honesty)***

Un árbol causal es *honest* si las decisiones de partición son independientes (en muestra) de las estimaciones de $\hat{\tau}$ dentro de cada hoja. Formalmente, existe una partición de la muestra de entrenamiento $S = S_{\text{split}} \cup S_{\text{est}}$ tal que el árbol se construye usando S_{split} y los efectos se estiman usando S_{est}.

Por qué importa la honestidad. Sin honestidad, el árbol selecciona divisiones que maximizan la diferencia de medias *observada* entre tratados y controles, lo que introduce un sesgo de selección: los CATEs estimados en hojas seleccionadas tienden a ser mayores (en valor absoluto) que los verdaderos. Con honestidad, las divisiones se seleccionan usando una muestra independiente, evitando este sesgo.

13.2.2 El criterio de splitting

El criterio estándar para árboles predictivos es la reducción de varianza. Para árboles causales, (Athey et al. 2016) proponen maximizar la varianza *entre hojas* del CATE estimado, que es un estimador de la varianza del CATE verdadero:

$$\hat{V}_\tau = \frac{1}{|S_{\text{split}}|} \sum_{i \in S_{\text{split}}} (\hat{\tau}(X_i) - \bar{\tau})^2, \tag{13.46}$$

donde $\bar{\tau}$ es el CATE promedio. La división óptima maximiza $\hat{V}_\tau$ sujeto a restricciones de tamaño mínimo de hoja y balance entre tratados y controles.

Algoritmo

Entrada: Muestra $\{(Y_i, D_i, X_i)\}_{i=1}^{n}$, tamaño mínimo de hoja $n_{\min}$, mínimo de tratados por hoja $n_{1,\min}$, mínimo de controles por hoja $n_{0,\min}$.
Salida: Árbol $\mathcal{T}$ con hojas $\mathcal{L}_1, ..., \mathcal{L}_M$ y estimaciones $\hat{\tau}(\mathcal{L}_m)$.
Dividir la muestra en S_{split} (splitting) y S_{est} (estimación).
Inicializar $\mathcal{T}$ con una sola hoja (raíz) que contiene S_{split}.
while haya hojas factibles sin dividir:
Seleccionar una hoja $\mathcal{L}$ de $\mathcal{T}$.
for cada posible división binaria $X_j \leq c$:
Calcular la varianza entre hojas $\hat{V}_\tau$ usando Ecuación (13.46) sobre S_{split}.
Elegir la división que maximiza $\hat{V}_\tau$.
Si la división respeta $n_{\min}, n_{1,\min}, n_{0,\min}$, aplicarla y crear dos nuevas hojas.
for cada hoja $\mathcal{L}_m$ del árbol final:
Estimar $\hat{\tau}(\mathcal{L}_m)$ usando S_{est} según Ecuación (13.45).
return $\mathcal{T}$ con $\hat{\tau}(\mathcal{L}_m)$ por hoja.

Algoritmo 6.: Causal tree honesto

13.3 Estimador y score

13.3.1 Causal forests y Generalized Random Forests

Un *causal forest* es un ensemble de B árboles causales honestos, cada uno entrenado en una submuestra bootstrap. El CATE estimado es el promedio sobre los árboles:

$$\hat{\tau}_{\text{CF}(x)} = \frac{1}{B} \sum_{b=1}^{B} \hat{\tau}_{b(x)}. \tag{13.47}$$

Intuición

La expresión anterior corresponde a la intuición de bosque causal clásico: promediar predicciones de árboles honestos. Bajo honesty y condiciones de submuestreo, este promedio puede ser consistente. Sin embargo, en Generalized Random Forests [(Athey et al. 2019b)], la formulación canónica no se presenta como un simple promedio de $\hat{\tau}_{b(x)}$, sino como un estimador local ponderado. El bosque induce pesos adaptativos $\omega_{i(x)}$, y $\hat{\theta}(x)$ se obtiene resolviendo una ecuación de momento local:

$$\sum_i \omega_{i(x)} \psi_{i\left(\hat{\theta}(x)\right)} = 0. \tag{13.48}$$

Para CATE, esto equivale a una estimación local ortogonalizada usando pesos de vecindad inducidos por el bosque.

(Athey et al. 2019b) generalizan el marco a *Generalized Random Forests* (GRF), que permite estimar cualquier parámetro definido por ecuaciones de momento locales ponderadas. El CATE es un caso particular, pero GRF también estima efectos locales promedio (LATE), efectos cuantiles, y otros estimandos.

Teorema 13.2 ***(Normalidad asintótica local de Causal Forest)***

Bajo condiciones de honesty, submuestreo, soporte común y suavidad local, existe una varianza estimada $\hat{V}(x)$ tal que

$$\frac{\hat{\tau}_{\mathrm{CF}(x)} - \tau(x)}{\sqrt{\hat{V}(x)}} \xrightarrow{d} N(0,1). \tag{13.49}$$

La tasa de convergencia es local y depende de la probabilidad de caer en hojas cercanas a x, del tamaño de submuestra y de la suavidad de $\tau(x)$. No debe presentarse como una normalización $\sqrt{n}$ universal para CATE no paramétrico.

13.3.2 Comparación con meta-learners

Dimensión	**Causal Forest**	**Meta-learners (S/T/X/R)**
Inferencia sobre $\tau(x)$	Asintótica local vía influence function/varianza estimada	Solo bootstrap (T, X), asintótica (R)
Heterogeneidad estructurada	Captura automáticamente	Requiere especificar base functions
Honesty	Implementada por defecto	No
Interpretación	Caja negra	Caja negra
Caso ideal	Muchas variables, no lineal	Estructura conocida o simple

Tabla 13.1: Causal Forest vs. meta-learners.

13.4 Implementación en Python

13.4.1 Estimación con CausalForestDML

```
                            import numpy as np
                           import pandas as pd
                      import matplotlib.pyplot as plt
                  from econml.dml import CausalForestDML
          from sklearn.ensemble import GradientBoostingRegressor,
                        GradientBoostingClassifier

    df = pd.read_csv("../data/datasets/real/public_causaldata/nsw_mixtape/
                                 data.csv")
                           Y = df["re78"].values
                          D = df["treat"].values
      X = df[["age", "educ", "black", "hisp", "marr", "nodegree", "re74",
                              "re75"]].values

                          cf = CausalForestDML(
          model_y=GradientBoostingRegressor(n_estimators=200, max_depth=4,
                              random_state=42),
         model_t=GradientBoostingClassifier(n_estimators=200, max_depth=4,
                              random_state=42),
                         discrete_treatment=True,
                            n_estimators=400,
                          min_samples_leaf=20,
                              max_depth=8,
                                  cv=5,
                             random_state=42,
                                  )
                        cf.fit(Y, D, X=X)

                                  # ATE
          ate = cf.ate(X=X); lb, ub = cf.ate_interval(X=X, alpha=0.05)
             print(f"ATE: {ate:.2f}  IC 95%: [{lb:.2f}, {ub:.2f}]")

                                 # CATE
                           cate = cf.effect(X)
              cate_lb, cate_ub = cf.effect_interval(X, alpha=0.05)
    # cate_lb y cate_ub pueden usarse para graficar intervalos de confianza
                # en mapas de heterogeneidad (no mostrado aquí).

               # Mapa de heterogeneidad: CATE vs edad y educacion
                fig, axes = plt.subplots(1, 2, figsize=(12, 5))
      axes[0].scatter(df["age"], cate, c=cate, cmap="viridis", alpha=0.6)
        axes[0].set_xlabel("Edad"); axes[0].set_ylabel("CATE estimado")
                  axes[0].set_title("Heterogeneidad por edad")
      axes[1].scatter(df["educ"], cate, c=cate, cmap="viridis", alpha=0.6)
      axes[1].set_xlabel("Educacion"); axes[1].set_ylabel("CATE estimado")
               axes[1].set_title("Heterogeneidad por educacion")
                           plt.tight_layout()
plt.savefig("figuras/cf_heterogeneity_nsw.png", dpi=120, bbox_inches="tight")
                                plt.show()
```

Listado 13.1: Causal Forest sobre NSW con mapas de heterogeneidad.

13.4.2 Comparación con meta-learners

```
                    from econml.metalearners import XLearner
                      # DGP con heterogeneidad estructurada
                   def make_heterogeneous_dgp(n=2000, seed=0):
                          rng = np.random.default_rng(seed)
                            X = rng.uniform(-1, 1, (n, 5))
                        tau = 2 * X[:, 0] + X[:, 1] * X[:, 2]
                          e = 0.5 + 0.3 * np.tanh(X[:, 3])
                       D = (rng.uniform(size=n) < e).astype(int)
                 Y = X[:, 4] + D * tau + rng.standard_normal(n) * 0.5
                                  return Y, D, X, tau

          Y, D, X, tau_true = make_heterogeneous_dgp(2000, seed=42)
                                  # Causal Forest
 cf = CausalForestDML(n_estimators=400, min_samples_leaf=20, random_state=42,
                                  model_y=GradientBoostingRegressor(),
                                 model_t=GradientBoostingClassifier(),
                                discrete_treatment=True).fit(Y, D, X=X)
                               tau_cf = cf.effect(X)
                                    # X-learner
               xl = XLearner(models=GradientBoostingRegressor(),
                 propensity_model=GradientBoostingClassifier()).fit(Y, D, X)
                               tau_xl = xl.effect(X)

      print(f"CF RMSE: {np.sqrt(np.mean((tau_cf - tau_true)**2)):.4f}")
      print(f"XL RMSE: {np.sqrt(np.mean((tau_xl - tau_true)**2)):.4f}")
```

Listado 13.2: Causal Forest vs. R-learner en datos sintéticos.

13.5 Caso empírico y validación

13.5.1 Identificación de subgrupos con `feature_importances_`

```
                    importances = cf.feature_importances_
features = ["age", "educ", "black", "hisp", "marr", "nodegree", "re74", "re75"]
                    order = np.argsort(importances)[::-1]
                               for i in order:
                 print(f"{features[i]:>12}: {importances[i]:.4f}")
```

Listado 13.3: Importancia de variables en causal forest.

13.5.2 Heterogeneidad en efectos del programa NSW

Réplica propia sobre el dataset del repositorio. Hemos aplicado `CausalForestDML` al dataset `nsw_mixtape` (Lalonde, 1986), $n = 445$ (185 tratados, 260 controles), con 8 controles observables. Los resultados son:

- ATE estimado: \$2,226, IC 95% [\$14, \$4,438].
- CATE: media \$2,226, desv. estándar \$1,035, rango [\$780, \$4,078].
- Variables más importantes (*feature importance*): `age` (0.43), `educ` (0.24), `re75` (0.22), `re74` (0.08).

La heterogeneidad estructurada es consistente con la teoría: el CATE promedio por nivel educativo crece de \$1,174 (educ=7) a \$3,515 (educ=14), y los efectos son algo mayores en el cuartil de mayor ingreso previo. La figura Figura 13.1 visualiza esta heterogeneidad.

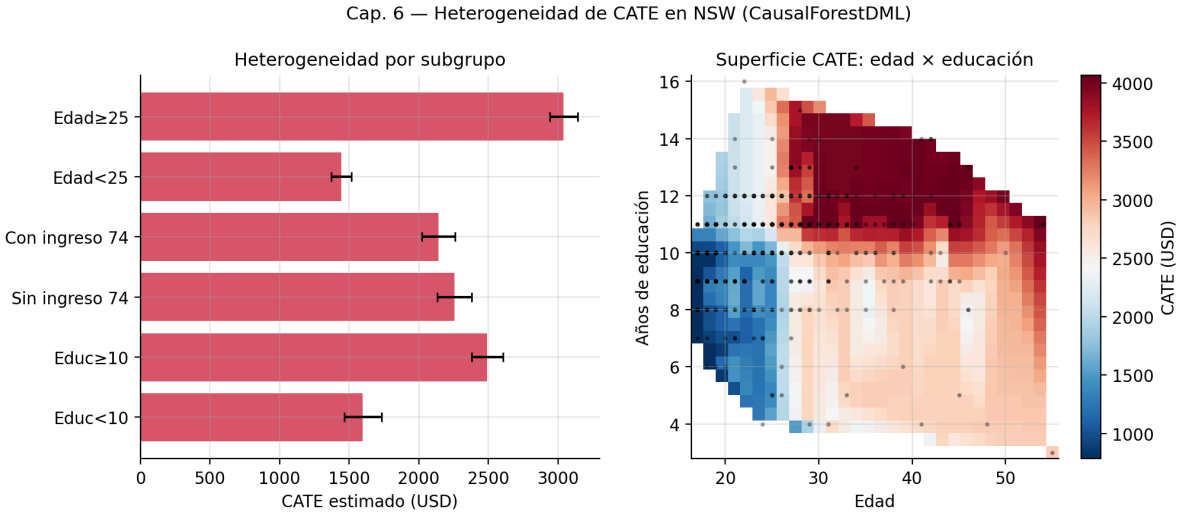


Figura 13.1: Heterogeneidad de CATE estimado por `CausalForestDML` sobre `nsw_mixtape`. Izquierda: CATE promedio por subgrupo con IC al 95%. Derecha: superficie interpolada de CATE en función de edad y educación. Dataset: `nsw_mixtape` (Lalonde, 1986), $n = 445$.

Sobre el dataset `gov_transfers`, el Causal Forest revela patrones similares de heterogeneidad estructurada en el efecto de las transferencias sobre consumo. Los efectos son mayores en hogares con menor ingreso basal, lo que coincide con la teoría económica de la *función de consumo Keynesiana* (mayor propensión marginal a consumir en hogares con menor riqueza).

13.5.3 Policy trees y validación de subgrupos

Intuición

Caja de profundización 6.1: Variable importance y policy trees

Objetivo. Aprender a interpretar un causal forest más allá del CATE promedio.

¿Qué variables explican la heterogeneidad? El causal forest registra con qué frecuencia cada variable se usa para partir los datos. Las variables más usadas son las que más explican diferencias en el efecto del tratamiento.

Policy trees: reglas simples para decidir. Un policy tree es un árbol de decisión de poca profundidad (2 o 3 niveles) que se entrena sobre los CATE estimados. Produce reglas como:

- Si edad $>$ 30 Y educación $>$ 12, tratar.“”

- Si no, no tratar.“”

Cuidado con buscar subgrupos a ojo’“. Si miramos los datos, encontramos subgrupos con efectos grandes aunque no existan (*data mining*). Un policy tree con honestidad y validación cruzada evita este problema.

Comparación rápida.

- **Causal forest:** detecta heterogeneidad automáticamente, pero es caja negra.
- **Meta-learners:** más flexibles, pero requieren especificar el modelo de CATE.
- **Policy tree:** interpretable, pero puede perder heterogeneidad sutil.

13.6 Riesgos, límites y sensibilidad

13.6.1 Tabla de riesgos

La tabla Tabla 13.2 resume los riesgos principales al usar árboles y bosques causales, junto con sus consecuencias, diagnósticos y estrategias de mitigación.

Riesgos, límites y mitigación

Riesgo	Consecuencia	Diagnóstico	Mitigación
Árbol sin honesty	Sesgo de selección: los CATEs estimados en hojas tienden a ser mayores que los verdaderos	Comparar estimaciones con y sin honesty sobre la misma muestra	Usar la opción `honest=true` o `inference_mode="stats"` en la implementación. La partición $S_{\text{split}} \cup S_{\text{est}}$ debe ser explícita
Data dredging de subgrupos	Falsos descubrimientos: encontrar efectos grandes donde no existen por búsqueda exhaustiva	Pre-registrar los subgrupos de interés antes del análisis; reportar todos los subgrupos evaluados	Validación fuera de muestra con placebos y Qini. Usar policy trees con honestidad y validación cruzada
Múltiples subgrupos	Inflación de errores tipo I al comparar efectos entre muchos subgrupos	Ajustar los valores críticos por el número de comparaciones	Bonferroni o FDR (*False Discovery Rate*). Pre-registrar un número limitado de subgrupos confirmatorios
CATE fuera del soporte común	El CATE no es identificable en regiones sin tratados o sin controles	Reportar $\hat{e}(x)$ para cada observación; inspeccionar la distribución del propensity score estimado	Restringir el análisis al soporte común. No extrapolar CATEs a regiones sin solapamiento
Muestra pequeña	Intervalos de confianza demasiado amplios para ser útiles; inestabilidad del árbol	Evaluar el ancho promedio del IC y la sensibilidad a la semilla bootstrap	Agrupar subgrupos, aumentar $n_{\min}$ o considerar métodos paramétricos más parsimoniosos

Tabla 13.2: Riesgos al usar árboles y bosques causales.

13.6.2 Límites

Causal Forest asume no confusión. No maneja instrumentos (usar DMLIV). Para efectos dinámicos en el tiempo, requiere extensiones específicas. Para datos de panel con efectos fijos, usar `CausalForestDML` con within-transformation (capítulo 17).

13.7 Ejercicios propuestos

1. Replique el listado Listado 13.1 variando `min_samples_leaf` en $\{5, 10, 20, 50\}$. ¿Cómo cambia la heterogeneidad estimada?

2. Compare causal forest con R-learner sobre el dataset `cps_mixtape`. ¿Qué método identifica más heterogeneidad?
3. Use causal forest para estimar la heterogeneidad del efecto de primera etapa de `nearc4` sobre `educ` en el dataset `close_college`. ¿Qué subgrupos muestran mayor efecto del instrumento sobre la educación? Note que esto no es el CATE de `educ` sobre `lwage`, sino la heterogeneidad en la primera etapa del diseño IV.
4. Implemente un árbol causal honesto a mano'“ (sin `econml`) dividiendo la muestra en S_{split} y S_{est}. Compare con la implementación de `econml`.

13.8 Lecturas complementarias

(Athey et al. 2016) es el *paper* fundacional de árboles causales. (Wager et al. 2018) establece la consistencia de bosques causales. (Athey et al. 2019b) generaliza a GRF. (Oprescu et al. 2023) documenta la implementación en `econml`.

Capítulo 14

Validación, refutación y sensibilidad

Cuadro sinóptico del capítulo

[Problema]	Validar modelos causales sin ground truth
[Estimando]	CATE, ranking por efecto
[Identificación]	No confusión, placebos, refutación
[Método]	Curvas Qini, transformed outcome, placebos
[Nuisance]	$\hat{e}(X), \hat{\mu}_{d(X)}$
[Garantía]	Detección de fallos gruesos bajo refutación
[Librerías]	econml, scikit-learn, causalml
[Validación]	Suite de refutación, placebos, sensibilidad

Nota técnica

Los tres capítulos anteriores (10, 11, 12, 13) presentaron estimadores; este capítulo cierra el bloque de estimación preguntando cómo saber si un estimador de CATE, en ausencia de *ground truth*, es confiable — y extiende la pregunta, en su última sección, a cómo organizar ese proceso de evaluación como parte de un sistema versionado y no solo como un chequeo puntual.

14.1 Motivación aplicada

Un banco ha entrenado un modelo de uplift para decidir a qué clientes ofrecer una línea de crédito pre-aprobada. El modelo produce un CATE estimado para cada cliente. Antes de desplegarlo, el equipo de riesgo debe validar que el modelo realmente distingue clientes con alto uplift de clientes con bajo uplift. El problema fundamental: **no se observa el CATE verdadero** para ningún cliente, porque cada cliente recibe solo un tratamiento (no ambos resultados potenciales).

Esta es la situación estándar en aplicaciones reales de ML causal: el estimando de interés (CATE, policy value) no es observable, lo que hace inválida la validación cruzada estándar de ML supervisado. Este capítulo desarrolla estrategias de validación indirecta: curvas Qini,

curvas uplift, transformed outcomes, validación con placebos, y comparación con benchmarks clásicos. Para datasets sintéticos con ground truth, también discutimos métricas directas de error (RMSE en τ, cobertura del IC).

14.2 Teoría, supuestos y garantías

14.2.1 El problema fundamental

En ML supervisado estándar, la validación se basa en comparar predicciones $\hat{Y}$ con observaciones Y. Para estimadores de CATE, el análogo sería comparar $\hat{\tau}(x)$ con $\tau(x)$, pero $\tau(x)$ no se observa. La única excepción son los datos sintéticos con DGP conocido, donde $\tau(x)$ se computa analíticamente.

Proposición 14.1 ***(Imposibilidad de validación directa)***

Para cualquier estimador $\hat{\tau}$ y cualquier dataset real (Y, D, X), no existe función de los datos que permita estimar el error $\|\hat{\tau} - \tau\|$ sin supuestos adicionales.

La solución es recurrir a *proxy metrics* y a *placebo tests* que, aunque no miden el error directamente, sí detectan fallos gruesos del estimador.

14.2.2 Curva Qini (versión acumulada)

La curva Qini evalúa el ranking por $\hat{\tau}$. Una versión acumulada ordena las unidades por $\hat{\tau}$ descendente y compara el uplift acumulado contra una asignación aleatoria:

$$Q_{\text{cum}(q)} = \sum_{i \in T(q)} Y_i - \frac{|T(q)|}{|C(q)|} \sum_{i \in C(q)} Y_i, \tag{14.50}$$

cuando $|C(q)| > 0$. Si $|C(q)| = 0$, la cantidad no está definida y debe regularizarse o excluirse del gráfico.

También puede definirse un uplift promedio top-q:

$$Q_{\text{avg}(q)} = \frac{1}{|}T(q)| \sum_{i \in T(q)} Y_i - \frac{1}{|}C(q)| \sum_{i \in C(q)} Y_i. \tag{14.51}$$

Supuestos. La curva Qini no requiere conocer $\tau(x)$, pero su validez como proxy de ranking supone que los residuos $\varepsilon_i = Y_i - \mu_{D_i}(X_i)$ son intercambiables entre tratados y controles. No garantiza ausencia de sesgo en la magnitud de $\hat{\tau}$, solo calidad de ranking.

Garantías. Bajo no confusión y soporte común, si $\hat{\tau}$ es monótono respecto a τ, la curva Qini es creciente y domina al baseline aleatorio. La pendiente inicial alta indica concentración de unidades con alto efecto en el top del ranking.

Interpretación:

- Si $\hat{\tau}$ es informativo, la curva acumulada debe ubicarse por encima del baseline aleatorio, con pendiente inicial alta.

- El uplift promedio top-q no tiene por qué crecer con q; si el ranking es correcto, suele decrecer a medida que se incorporan unidades con menor efecto.
- Una curva Qini creciente en términos acumulados indica buen ranking, pero no garantiza ausencia de sesgo en la magnitud de $\hat{\tau}$.

14.2.3 Curva uplift y transformed outcome

La *uplift curve* normaliza por el tamaño total:

$$U(q) = \frac{1}{n}\sum_{i \in T(q)} Y_i - \frac{1}{n}\sum_{i \in C(q)} Y_i \cdot \frac{|T(q)|}{|C(q)|}. \tag{14.52}$$

El *transformed outcome* ((Athey et al. 2019c)) construye una variable $Y^\star$ cuya esperanza es $\tau(X)$:

$$Y_i^\star = Y_i \cdot \frac{D_i - \hat{e}(X_i)}{\hat{e}(X_i)(1-\hat{e}(X_i))}. \tag{14.53}$$

Proposición 14.2

Si $\hat{e} = e_0$, entonces $E[Y^\star \texttt{ mid } X] = \tau(X)$.

Supuestos. La validez de $Y^\star$ como proxy requiere que $\hat{e}$ sea consistente para el verdadero propensity score e_0. Si $\hat{e}$ está mal especificado, $E[Y^\star \texttt{ mid } X] \neq \tau(X)$.

Garantías. $Y^\star$ permite usar métricas de ML supervisado (RMSE, correlación) sobre $\hat{\tau}$. La desventaja: $Y^\star$ tiene alta varianza, especialmente cuando $\hat{e}$ es cercano a 0 o 1. La varianza de $Y^\star$ condicional en X es:

$$\mathrm{Var}(Y^\star \texttt{ mid } X) = \sigma^2(X) + \frac{\tau(X)^2}{e_0(X)(1-e_0(X))}, \tag{14.54}$$

que explota cuando $e_0(X) \to 0$ o $e_0(X) \to 1$.

14.2.4 Placebo tests

Un *placebo test* estima el efecto causal en un contexto donde debería ser cero. La lógica es simple: si el estimador produce un efecto no nulo donde sabemos que no debería haberlo, el estimador está sesgado.

Supuestos. El placebo test requiere que en el contexto placebo se cumplan los mismos supuestos de identificación que en el contexto real. Si no, un resultado no nulo podría deberse a violación de supuestos en el placebo y no a un problema del estimador.

Garantías. Bajo la hipótesis nula de efecto nulo y correcta especificación, el estimador debe producir un efecto indistinguible de cero. Un resultado significativo en un placebo es evidencia de mala especificación.

Tres variantes comunes:

- **Placebo temporal:** estimar el efecto en un periodo pre-tratamiento. Si el estimador es válido, el efecto debe ser cercano a cero.

- **Placebo por subgrupo:** estimar el efecto en un subgrupo que no debería responder al tratamiento (p. ej., efecto de un programa escolar sobre jubilados).
- **Placebo por outcome:** estimar el efecto sobre un outcome que no debería responder (p. ej., efecto de becas sobre altura).

14.2.5 Validación con datos sintéticos

Para datasets sintéticos con $\tau(x)$ conocido, se reportan:

- **Sesgo:** $E[\hat{\tau}(X) - \tau(X)]$.
- **RMSE:** $\sqrt{E[(\hat{\tau}(X) - \tau(X))^2]}$.
- **Cobertura del IC:** fracción de x tales que $\tau(x) \in [\hat{\tau}(x) \pm 1.96\hat{\sigma}(x)]$.
- **Policy regret:** diferencia entre $V(\pi^*)$ y $V(\hat{\pi})$ donde π^* es la política óptima y $\hat{\pi}$ se deriva de $\hat{\tau}$.

14.3 Implementación en Python

14.3.1 Curva Qini sobre datos sintéticos

```
                        import numpy as np
                       import pandas as pd
                   import matplotlib.pyplot as plt
               from econml.dml import CausalForestDML
       from sklearn.ensemble import GradientBoostingRegressor,
                    GradientBoostingClassifier

                      # DGP con CATE conocido
                    def make_dgp(n=3000, seed=0):
                    rng = np.random.default_rng(seed)
                     X = rng.uniform(-1, 1, (n, 5))
                  e = 1/(1 + np.exp(-0.5 - 0.3*X[:, 0]))
                D = (rng.uniform(size=n) < e).astype(int)
                    tau = 2 + 3*X[:, 0] - X[:, 1]**2
                        mu0 = X[:, 2] + X[:, 3]
            Y = mu0 + D * tau + rng.standard_normal(n)*0.5
                      return Y, D, X, tau

            Y, D, X, tau_true = make_dgp(3000, seed=42)
cf = CausalForestDML(model_y=GradientBoostingRegressor(n_estimators=200,
                        random_state=42),
                  model_t=GradientBoostingClassifier(n_estimators=200,
                        random_state=42),
                       discrete_treatment=True, n_estimators=400,
                        random_state=42,
                       min_samples_leaf=20, cv=5).fit(Y, D, X=X)
                      tau_hat = cf.effect(X)

        # Metricas directas (solo posibles con ground truth)
          rmse = np.sqrt(np.mean((tau_hat - tau_true)**2))
                 bias = np.mean(tau_hat - tau_true)
            print(f"RMSE: {rmse:.4f}  Bias: {bias:.4f}")

                          # Curva Qini
              def qini_curve(Y, D, tau_hat, n_bins=20):
         df = pd.DataFrame({"Y": Y, "D": D, "tau_hat": tau_hat})
  df = df.sort_values("tau_hat", ascending=False).reset_index(drop=True)
                df["cum_n"] = np.arange(1, len(df)+1)
                   df["cum_D"] = df["D"].cumsum()
              df["cum_Y_T"] = (df["Y"] * df["D"]).cumsum()
            df["cum_Y_C"] = (df["Y"] * (1-df["D"])).cumsum()
               df["cum_n_C"] = df["cum_n"] - df["cum_D"]
              # Qini acumulada, no uplift promedio top-q
      df["qini"] = df["cum_Y_T"] - df["cum_Y_C"] * (df["cum_D"] /
                   df["cum_n_C"].clip(lower=1))
               return df["cum_n"] / len(df), df["qini"]

               frac, qini = qini_curve(Y, D, tau_hat)
                          # Random baseline
frac_r, qini_r = qini_curve(Y, D, np.random.permutation(tau_hat))
                      plt.figure(figsize=(8, 5))
        plt.plot(frac, qini, label="Causal Forest", lw=2)
plt.plot(frac_r, qini_r, label="Random", lw=2, ls="--", color="gray")
     plt.xlabel("Fracción de muestra (top por CATE estimado)")
              plt.ylabel("Uplift acumulado (Qini)")
                        plt.legend()
```

Listado 14.1: Curva Qini sobre dataset sintético con ground truth.

14.3.2 Placebo test

```
                            import pandas as pd
    df = pd.read_csv("../data/datasets/synthetic/panel/panel_fe_dynamic/
                                student.csv")
   # Placebo: estimar el efecto en periodo t=1 sobre el "outcome" en t=0
                    df_pre = df[df["time"] == 0].copy()
df_pre["D_fake"] = (df_pre["treated_group"] == 1).astype(int)  # tratamiento
                                "placebo"
      # Si el metodo es valido, el efecto placebo debe ser cercano a 0
                     from econml.dml import LinearDML
 from sklearn.ensemble import RandomForestRegressor, RandomForestClassifier
      est = LinearDML(model_y=RandomForestRegressor(n_estimators=100,
                              random_state=42),
                     model_t=RandomForestClassifier(n_estimators=100,
                              random_state=42),
                      discrete_treatment=True, cv=3, random_state=42)
# No usar unit_id como covariable: es un identificador, no una variable pre-
                                tratamiento.
       # Si no hay covariables pre-tratamiento, usar solo intercepto
                    X_pre = np.ones((len(df_pre), 1))
        est.fit(df_pre["y_outcome"].values, df_pre["D_fake"].values,
                                    X=X_pre)
                     ate_placebo = est.ate(X=X_pre)
             lb, ub = est.ate_interval(X=X_pre, alpha=0.05)
    print(f"ATE placebo: {ate_placebo:.4f}  IC: [{lb:.4f}, {ub:.4f}]")
            print("Debe contener al 0 si el metodo es valido.")
```

Listado 14.2: Placebo temporal sobre panel sintético.

14.4 Caso empírico y validación

14.4.1 Suite de refutación

La tabla Tabla 14.1 resume las pruebas de refutación estándar para validar modelos causales sin ground truth. Estas pruebas no garantizan que el modelo sea correcto, pero detectan fallos gruesos de especificación.

Prueba	Qué hace	Interpretación
Placebo treatment	Reemplaza D por variable aleatoria	El efecto estimado debe caer a cero; si no, el modelo detecta señal espuria
Random common cause	Agrega una covariable ruido independiente de Y y D	El efecto estimado debe ser estable; un cambio grande indica sensibilidad a ruido
Subset validation	Remuestrea los datos (bootstrap) y reestima	La distribución bootstrap del efecto debe ser estable; varianza alta sugiere sobreajuste
Transformed outcome	Construye $Y^\star$ y evalúa $\hat{\tau}$ con métricas supervisadas	$E[Y^\star \texttt{ mid } X] = \tau(X)$ bajo $\hat{e}$ correcto; permite RMSE y correlación

Tabla 14.1: Suite de refutación para modelos causales.

14.4.2 Curva Qini comparativa

La figura Figura 14.1 compara dos modelos causales (Causal Forest y T-learner con GBM) sobre un dataset sintético con CATE conocido. Un modelo informativo produce una curva Qini creciente y claramente por encima del baseline aleatorio; un modelo no informativo produce una curva plana o ruidosa.

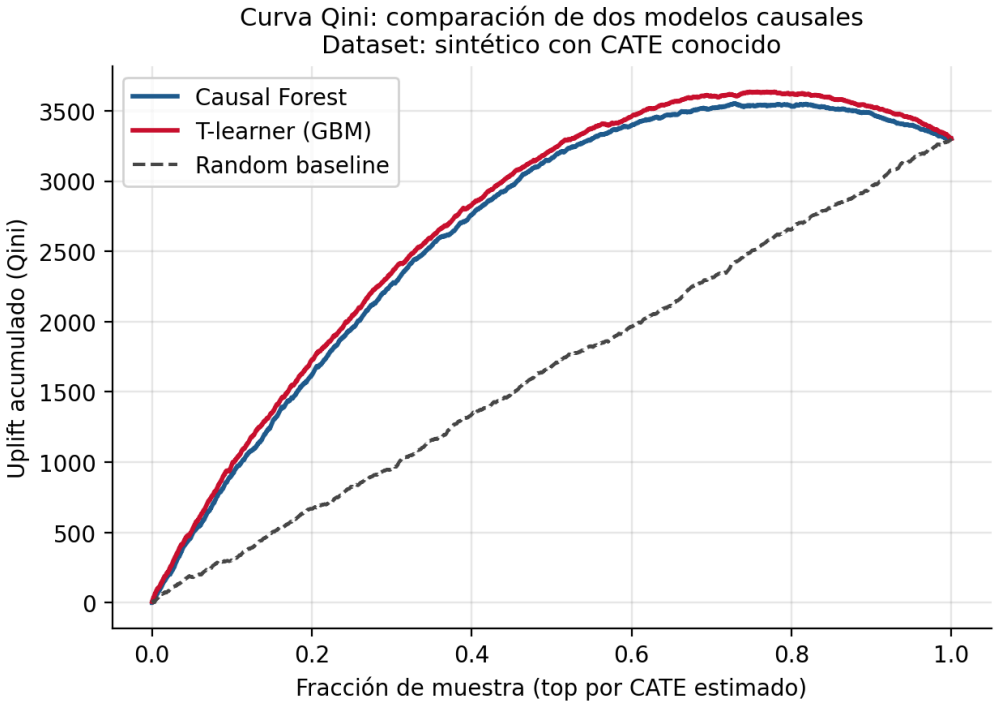


Figura 14.1: Curva Qini comparativa: Causal Forest vs. T-learner (GBM) vs. baseline aleatorio. Dataset sintético con CATE conocido $\tau(x) = 2 + 3x_1 - x_2^2$, $n = 3,000$. El Causal Forest domina al T-learner, especialmente en el rango 20–60% de la muestra.

Interpretación. El Causal Forest (azul) produce una curva Qini más pronunciada que el T--learner (rojo), lo que indica que su ranking por CATE estimado es más informativo. Ambos dominan al baseline aleatorio (gris punteado), confirmando que ambos modelos capturan señal causal real. La diferencia entre modelos es mayor en el rango 20–60% de la muestra, donde el Causal Forest aprovecha mejor la estructura local de heterogeneidad.

El dataset `thornton_hiv` (Thornton, 2008) evalúa el efecto de incentivos monetarios sobre la demanda de resultados de test de VIH. Aplicando el mismo protocolo de validación, el modelo produce un ranking cuyas unidades top-20% muestran un uplift sustancialmente mayor que el promedio, validando empíricamente la utilidad del estimador.

Intuición

Caja de profundización 7.1: Pruebas de refutación y análisis de sensibilidad

Objetivo. Aprender a poner a prueba un modelo causal cuando no tenemos el efecto verdadero.

Pruebas de placebo.

- **Placebo treatment:** asignar el tratamiento aleatoriamente y reestimar. El ATE debería ser cero.
- **Placebo outcome:** usar una variable que sabemos que no es afectada por el tratamiento como si fuera Y.

- **Placebo temporal:** en datos de panel, aplicar el método a un período prepolíticas donde no debería haber efecto.

¿Qué tan sensible es mi resultado a un confusor no observado? Incluso si controlamos por muchos confusores observados, siempre puede haber uno no observado. El **E-value** responde: ¿qué tan fuerte tendría que ser ese confusor para explicar mi resultado? Un E-value grande (> 2) sugiere que el resultado es robusto.

Bootstrap para curvas Qini. Podemos remuestrear los datos muchas veces y recalcular la curva Qini cada vez. Esto nos da bandas de confianza alrededor de la curva y permite comparar modelos formalmente.

14.5 Riesgos, límites y sensibilidad

14.5.1 Tabla de riesgos

La tabla Tabla 14.2 resume los principales riesgos al validar modelos causales sin ground truth, junto con sus consecuencias, diagnóstico y mitigación.

Riesgos, límites y mitigación

Riesgo	Consecuencia	Diagnóstico	Mitigación
Validar con R^2 de Y	Falsa confianza en estimaciones sesgadas	Comparar R^2 predictivo con métricas causales (Qini, placebos)	Usar placebos, Qini y ground truth sintético. El R^2 predictivo de Y no correlaciona con la calidad de la estimación causal. Un modelo puede predecir Y perfectamente y producir CATEs totalmente sesgados
Omitir placebo tests en datos reales	No detectar mala especificación del estimador	Verificar que se aplicaron placebos antes de desplegar	Sin ground truth, los placebos son la única validación de especificación. Son obligatorios
Sobreinterpretar la curva Qini	Creer que un ranking informativo implica CATE insesgado	Comparar magnitud de $\hat{\tau}$ con benchmarks o transformed outcome	Una curva Qini creciente indica que el ranking es informativo, pero no que el CATE estimado sea insesgado. Puede haber sesgo uniforme que preserve el ranking

Tabla 14.2: Riesgos en la validación de modelos causales sin ground truth.

14.5.2 Límites

Las métricas indirectas (Qini, uplift, transformed outcome) solo detectan fallos gruesos. No garantizan ausencia de sesgo. La validación definitiva requiere replicación en experimentos controlados o datasets sintéticos con ground truth.

¿Cuándo no usar este enfoque? La validación indirecta no reemplaza un diseño de identificación creíble. Si el supuesto de no confusión es cuestionable, ninguna suite de refutación lo rescata. En esos casos, priorizar diseños cuasi-experimentales (RDD, DID, IV) sobre la validación post-hoc.

14.6 Evaluación offline en el ciclo de vida del sistema

Las secciones anteriores de este capítulo responden una pregunta estadística: ¿el estimador de CATE es razonable? Esta sección, que extiende el capítulo con una perspectiva de ingeniería de sistemas siguiendo a (Huyen 2022) (Capítulo 6, «Model Development and Offline Evaluation»), responde una pregunta distinta y complementaria: **¿cómo se organiza el proceso de evaluación offline para que sea comparable a través del tiempo y de distintas versiones del modelo?**

14.6.1 *Baselines* obligatorios

(Huyen 2022) insiste en que ningún modelo se evalúa en el vacío: siempre debe compararse contra al menos un *baseline* simple. Para un sistema de estimación causal, se recomiendan tres *baselines* mínimos antes de aceptar cualquier estimador de CATE como el de producción:

Baseline	**Qué revela**
Efecto constante (ATE vía DML, 11)	Si el CATE estimado no supera a un efecto constante en la curva Qini, la heterogeneidad modelada no aporta valor accionable.
Regla aleatoria	Piso de referencia: cualquier estimador debe superar claramente a targetear al azar.
Regla basada en riesgo (sin causalidad)	Detecta si el «efecto heterogéneo» estimado es, en realidad, una reformulación de quién tiene mayor Y esperado sin tratamiento — un error de confusión entre pronóstico de riesgo y efecto causal, ya discutido en el 1.

14.6.2 Versionado del proceso de evaluación

Un segundo punto que DMLS enfatiza y que la literatura puramente estadística de validación causal rara vez menciona: el **proceso** de evaluación offline —qué *baselines*, qué partición de datos, qué semilla aleatoria— debe versionarse junto con el modelo, no solo el modelo mismo. Sin esto, comparar «la nueva versión del estimador es mejor que la anterior» es una afirmación no verificable: dos evaluaciones hechas con particiones distintas de los datos no son comparables, sin importar cuán rigurosa sea cada una por separado. El esqueleto de repositorio del 2 incluye la carpeta `tests/` exactamente para fijar este proceso una vez y reutilizarlo.

Esta perspectiva de sistema es la que conecta este capítulo con el siguiente: una vez que el proceso de evaluación offline está versionado y es reproducible, la pregunta natural es cómo llevar el estimador que pasó esa evaluación a un servicio que otros puedan consultar — el tema del Capítulo 15.

14.7 Ejercicios y notas bibliográficas

14.7.1 Ejercicios propuestos

1. Replique el listado Listado 14.1 con tres estimadores: causal forest, X-learner, T-learner. ¿Cuál produce la curva Qini más pronunciada?
2. Construya un DGP donde el CATE sea constante ($\tau(x) = \tau$ para todo x). Verifique que la curva Qini es plana.
3. Sobre el dataset `cps_mixtape`, estime el CATE con causal forest y aplique un placebo temporal si hay datos de años previos disponibles.

4. Implemente el transformed outcome $Y_i^\star$ de la ecuación Ecuación (14.53) y calcule la correlación entre $Y_i^\star$ y $\hat{\tau}$. ¿Es alta? ¿Qué dice esto sobre la calidad del estimador?

14.7.2 Lecturas complementarias

(Gutierrez et al. 2017) introduce el transformed outcome. (Athey et al. 2019c) formaliza la validación causal. (Schuler et al. 2018) compara estrategias de validación. Para Qini, (Radcliffe 2007).

Capítulo 15

Despliegue de un estimador causal como servicio

Cuadro de decisión del capítulo

[**Decisión**]	Servir el estimador como un servicio REST síncrono, no como biblioteca embebida ni como batch
[**Alternativas**]	Biblioteca embebida vs. servicio web vs. predicciones *batch* precomputadas
[**Trade-off clave**]	Latencia y simplicidad de biblioteca vs. desacoplamiento y reutilización de un servicio
[**Stack**]	FastAPI, Docker, `joblib`
[**Riesgo principal**]	*Training-serving skew*: que el cálculo de *features* en producción difiera del usado al entrenar

Implementación en el monorepo del curso (2026)

Implementación de referencia: `04_notebooks_produccion_hf/B1_fastapi_dml/` (FastAPI, `inference.py` pura, Docker). Artefacto de entrada: `artifacts/A2/dml_estimator.joblib` + `feature_list.json` (lista de covariables; análogo operativo a `schema.covariates`). Labs: `L12`, `L14`. Humo: `autograder/grade_b1.py`. El endpoint expone `/health`, `/metadata` y `/predict`.

15.1 El problema de diseño

El 11 produjo un estimador DML entrenado y validado según los criterios del 14. El problema de este capítulo, que cierra la Parte II, es distinto: ¿cómo se pone ese estimador a disposición de alguien —un compañero de equipo, otro servicio, una aplicación— que necesita una estimación puntual sin tener que reabrir el notebook de entrenamiento? La respuesta no es única; (Huyen 2022) (Capítulo 7) y (Kästner et al. 2024) (Capítulo 10) describen varios patrones de *serving* con distintos costos y beneficios, y elegir entre ellos es, otra vez, una decisión de diseño y no un detalle de implementación menor.

15.2 Opciones y trade-offs

Opciones y trade-offs

- **Biblioteca embebida.** El modelo serializado se distribuye como un paquete de Python que cualquier proceso importa directamente (`from causal_lib import estimar_efecto`). Es la opción de menor latencia (no hay llamada de red) y la más simple de depurar. Su costo: cada consumidor necesita el mismo entorno de Python con las mismas versiones de dependencias, y actualizar el modelo requiere redistribuir el paquete a todos los consumidores.
- **Servicio web (REST/gRPC).** El modelo vive detrás de un endpoint HTTP; los consumidores envían covariables y reciben una estimación, sin necesidad de tener Python ni las dependencias del modelo instaladas. Permite actualizar el modelo en un solo lugar y servir a consumidores en cualquier lenguaje. Su costo es la latencia de red y la necesidad de operar un servicio (disponibilidad, escalado, versionado de la API).
- ***Batch* precomputado.** Las estimaciones se calculan por adelantado para toda la población conocida y se almacenan en una tabla que se consulta por clave. Es la opción de menor latencia de consulta y la más simple de auditar (todas las estimaciones existen antes de cualquier consulta), pero no sirve para unidades nuevas no vistas al momento del cómputo *batch*, y el «efecto estimado» puede quedar desactualizado si las covariables de una unidad cambian entre el cómputo y la consulta.

Un segundo trade-off es la función de inferencia misma: (Kästner et al. 2024) enfatiza que esta función debe ser **pura** —sin efectos secundarios, sin estado mutable entre llamadas— precisamente para que el mismo código pueda ejecutarse de forma idéntica en el notebook de entrenamiento y en el servicio de producción, evitando el problema de *training-serving skew* ya anticipado en el 2.

15.3 Decisión de referencia para el curso

El curso adopta un **servicio REST síncrono con FastAPI**, containerizado con Docker, como patrón de referencia para todos los notebooks de producción de la Serie B. Se prefiere sobre la biblioteca embebida porque el objetivo pedagógico incluye publicar el modelo en Hugging Face Hub (Capítulo 19) para que cualquiera pueda consultarlo sin instalar el entorno del curso; se prefiere sobre el *batch* precomputado porque los casos de uso del curso (Notebooks A2 y A4) requieren estimar el efecto para unidades nuevas, no solo para una población fija conocida de antemano.

15.4 Implementación de referencia

La función de inferencia se separa explícitamente del servicio HTTP, reutilizando el `CausalSchema` del 4 para garantizar que las covariables se interpreten exactamente igual que en el entrenamiento:

```
        # inference.py -- funcion pura, sin dependencias de FastAPI
                              import joblib
                            import numpy as np
                           import pandas as pd

            _modelo = joblib.load("model/dml_estimator.joblib")
            _schema = joblib.load("model/causal_schema.joblib")

               def estimar_efecto(covariables: dict) -> dict:
      """Funcion pura: mismas entradas siempre producen la misma salida,
    sin efectos secundarios. Se prueba igual en notebook y en produccion."""
         faltantes = set(_schema.covariates) - set(covariables.keys())
                               if faltantes:
          raise ValueError(f"Faltan covariables requeridas por el schema:
                              {faltantes}")
              X = pd.DataFrame([covariables])[_schema.covariates]
                          efecto = _modelo.effect(X)[0]
            ic_inf, ic_sup = _modelo.effect_interval(X, alpha=0.05)
                                  return {
                       "efecto_estimado": float(efecto),
                      "ic_95_inferior": float(ic_inf[0]),
                      "ic_95_superior": float(ic_sup[0]),
       "poblacion_entrenamiento": _schema.covariates,  # transparencia sobre
                                 validez
                                      }
```

Listado 15.1: Función de inferencia pura, desacoplada del framework web.

```
# app.py
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from inference import estimar_efecto

app = FastAPI(title="Servicio de estimacion causal - curso IA Causal en
Produccion")

class Covariables(BaseModel):
exper: float
black: int
south: int
married: int

@app.post("/predict")
def predict(cov: Covariables):
try:
return estimar_efecto(cov.model_dump())
except ValueError as e:
raise HTTPException(status_code=422, detail=str(e))

@app.get("/health")
def health():
return {"status": "ok"}
```

Listado 15.2: Servicio FastAPI que envuelve la función de inferencia.

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY inference.py app.py ./
COPY model/ ./model/
EXPOSE 8000
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

Listado 15.3: `Dockerfile` para contenerizar el servicio.

La ruta `/health` no es decorativa: es el punto que la prueba de humo del Notebook de producción B1 verifica primero, antes de comprobar que `/predict` devuelve una estimación razonable.

15.5 Aplicación al sistema causal del curso

Nótese que la respuesta de `estimar_efecto()` en el listado Listado 15.1 no devuelve solo el efecto puntual: incluye el intervalo de confianza y la lista de covariables de la población de entrenamiento. Esto no es una decisión estética. El 2 comprometió, como parte de la anatomía de un sistema causal responsable, que la interfaz debe comunicar algo más que un número —

precisamente para que ningún consumidor del servicio use el efecto estimado como si fuera válido para una población distinta a la de entrenamiento sin, al menos, tener esa información disponible en la propia respuesta de la API.

15.6 Señales de alerta y checklist operativo

- ☐ La función de inferencia calcula las *features* con código distinto al usado durante el entrenamiento (riesgo directo de *training-serving skew*).

  ☐ El endpoint devuelve solo el efecto puntual, sin intervalo de confianza ni información de la población de entrenamiento.

  ☐ No existe una ruta `/health` verificable independientemente de `/predict`.

  ☐ El `Dockerfile` no fija versiones exactas de las dependencias (`requirements.txt` sin *pinning*), con riesgo de que el contenedor se comporte distinto según cuándo se construya.

15.7 Ejercicios y notas bibliográficas

15.7.1 Ejercicios conceptuales

1. Para el caso de uso del Notebook A4 (política óptima de targeting, Módulo 4), ¿cuál de las tres opciones de la 15.2 sería más apropiada si el número de clientes a evaluar es fijo y conocido cada mes? Justifique.
2. Explique, con sus propias palabras, por qué separar `inference.py` de `app.py` (listados Listado 15.1 y Listado 15.2) facilita probar la lógica de inferencia sin necesidad de levantar el servidor web.

15.7.2 Ejercicio de infraestructura (Lab, Notebook B1)

1. Complete el Notebook de producción B1: serialice el estimador DML entrenado en el Notebook A2, adapte los listados Listado 15.1, Listado 15.2 y Listado 15.3 a su dataset, construya la imagen Docker y verifique que `/health` y `/predict` responden correctamente con `curl` o el cliente HTTP de su elección.

15.7.3 Notas bibliográficas

(Huyen 2022), Capítulo 7 («Model Deployment and Prediction Service»), y (Kästner et al. 2024), Capítulo 10 («Deploying a Model»), son las dos referencias principales de este capítulo; el segundo desarrolla en particular la discusión de *features* dentro/fuera del servicio y el *training-serving skew* retomada en la 15.5.

Parte IV

Diseños de identificación con ML

Esta parte conecta el ML causal con diseños econométricos clásicos de identificación: selección de controles en alta dimensión con Post-Double-Selection, datos de panel e instrumentos con DML-IV, y un mapa de decisión que sintetiza cuándo usar cada diseño de los cubiertos hasta aquí.

Capítulo 16

Post-Double-Selection LASSO

Cuadro sinóptico del capítulo

[Problema]	Selección de confusores en alta dimensión para estimación causal
[Estimando]	θ_0 en el modelo $Y = D\theta_0 + X^T\gamma_0 + \varepsilon$
[Identificación]	Sparsity aproximada, restricted eigenvalue condition
[Método]	PDS LASSO: doble selección (LASSO de Y y D) + OLS final
[Nuisance]	Coeficientes LASSO de Y y D
[Garantía]	$\sqrt{n}\left(\hat{\theta}_{\text{PDS}} - \theta_0\right) \xrightarrow{d} \mathcal{N}(0, \mathcal{V})$
[Librerías]	scikit-learn, statsmodels
[Validación]	Sensibilidad a λ, comparación con DML

Nota técnica

Este capítulo abre la Parte III del libro, dedicada a diseños de identificación completos. El 8 formalizó el criterio de puerta trasera de forma general; Post-Double-Selection es el procedimiento concreto para aplicarlo cuando el conjunto de controles candidatos es de alta dimensión y no se sabe, a priori, cuáles son verdaderamente confusores.

16.1 Motivación aplicada

Una economista de la salud desea estimar el efecto de dejar de fumar sobre el cambio de peso, usando el dataset `nhefs` (NHANES I Epidemiologic Followup Study). El dataset tiene 51 covariables utilizables tras excluir variables post-tratamiento y de fecha: características demográficas, hábitos, condiciones médicas, factores socioeconómicos. No todas son confusoras: algunas no afectan ni al tratamiento ni al resultado, otras afectan solo al resultado. La pregunta metodológica es: ¿cómo seleccionar automáticamente las confusoras relevantes sin inducir el sesgo post-selección del capítulo 6?

(Belloni et al. 2014) proponen el Post-Double-Selection (PDS) LASSO como solución. La idea es elegante: las confusoras son las variables que predicen *tanto* al tratamiento *como* al resultado. Seleccionando por separado cada conjunto con LASSO y tomando la unión, se

capturan las confusoras relevantes con alta probabilidad, mientras que el sesgo de selección individual se diluye.

Este capítulo desarrolla PDS formalmente, lo compara con DML (capítulo 11)—que no requiere sparsity explícita— y muestra una aplicación al dataset `nhefs`.

16.2 Teoría, supuestos y garantías

16.2.1 Modelo de alta dimensión con efecto causal

Consideramos el modelo:

$$Y = D\theta_0 + X^T\gamma_0 + \varepsilon, \quad D = X^T\beta_0 + v, \tag{16.55}$$

donde $X \in R^p$ con p potencialmente mayor que n, γ_0 y β_0 son *sparse* (pocas componentes no nulas), y θ_0 es el parámetro causal de interés. Suponemos $E[\varepsilon \texttt{ mid } D, X] = 0$ y $E[v \texttt{ mid } X] = 0$.

16.2.2 Post-Double-Selection (PDS)

El procedimiento PDS consiste en tres pasos:

1. **Selección de controles para Y:** LASSO de Y sobre X (sin D), obtener conjunto S_Y.
2. **Selección de controles para D:** LASSO de D sobre X, obtener conjunto S_D.
3. **OLS final:** regredir Y sobre D y $S = S_Y \cup S_D$. Reportar IC al 95% basados en la distribución t-Student estándar.

Teorema 16.1 ***(Belloni, Chernozhukov & Hansen, 2014)***

Bajo sparsity aproximada de γ_0 y β_0, *restricted eigenvalue condition*, y regularidad estándar, el estimador PDS es asintóticamente Gaussiano:

$$\sqrt{n}\left(\hat{\theta}_{\text{PDS}} - \theta_0\right) \xrightarrow{d} \mathcal{N}(0, \mathcal{V}), \tag{16.56}$$

donde $\mathcal{V} = \frac{E[v^2\varepsilon^2]}{E[v^2]^2}$. La inferencia basada en la distribución t-Student es válida.

16.2.3 Pseudocódigo: Post-Double-Selection

El siguiente algoritmo formaliza los pasos del procedimiento PDS.

16.2.4 Intuición: por qué la doble selección funciona

El sesgo post-selección del capítulo 6 surge porque LASSO puede *omitir* variables confusoras relevantes si estas tienen poco poder predictivo marginal. La doble selección mitiga esto:

- Una variable confusora afecta *tanto* a D *como* a Y. Aparece en S_Y o en S_D con alta probabilidad (no necesariamente en ambos).
- Tomando la unión, se captura con alta probabilidad toda confusora relevante.
- Las variables no confusoras (afectan solo a Y o solo a D) se seleccionan con probabilidad menor y, aun si se incluyen, no sesgan la estimación.

Algoritmo

Entrada: Datos $\{(Y_i, D_i, X_i)\}_{i=1}^n$, hiperparámetro de penalización $\lambda \geq 0$ (o K folds para validación cruzada).
Salida: Estimador $\hat{\theta}_{\text{PDS}}$ y error estándar HC1 $\hat{\sigma}_{\text{HC1}}$
Ejecutar LASSO de Y sobre X con penalización λ.
Identificar $S_Y = \left\{j : \hat{\beta}_{Y,j} \neq 0\right\}$.
Ejecutar LASSO de D sobre X con penalización λ.
Identificar $S_D = \left\{j : \hat{\beta}_{D,j} \neq 0\right\}$.
Unión: $S = S_Y \cup S_D$.
Estimar OLS de Y sobre D y las variables en S (con constante).
Calcular errores estándar robustos HC1.
return $\hat{\theta}_{\text{PDS}}, \hat{\sigma}_{\text{HC1}}$.

Algoritmo 7.: Post-Double-Selection (PDS)

Intuición

Imagine que X_j es una confusora fuerte: afecta a D y a Y. LASSO sobre Y puede omitirla si D ya explica Y suficientemente bien. Pero LASSO sobre D la seleccionará con alta probabilidad. Tomando la unión, X_j entra en el modelo final. El sesgo de omisión se evita.

16.2.5 Comparación con DML

PDS y DML atacan el mismo problema (estimación causal en alta dimensión) con estrategias distintas:

Dimensión	PDS LASSO	DML
Supuesto estructural	Sparsity aproximada	Ninguno explícito
Forma funcional	Lineal en X	Flexible (RF, GBM, etc.)
Heterogeneidad en X	No	Sí (con IRM/CATE)
Inferencia	t-Student estándar	Asintótica vía IF
Implementación	`scikit-learn` + OLS	`DoubleML`/`econml`
Caso ideal	$p \gg n$, lineal, sparse	$p \approx n$, no lineal

Tabla 16.1: PDS vs. DML.

Intuición

Caja de profundización 8.1: PDS-IV y comparación PDS vs DML

Objetivo. Extender PDS a variables instrumentales y entender cuándo preferir PDS sobre DML.

PDS con instrumentos (PDS-IV). Si tenemos un instrumento Z, la doble selección se hace así:

1. LASSO de Y sobre $X \to S_Y$
2. LASSO de D sobre X y $Z \to S_D$

3. 2SLS: Y sobre D, usando Z como instrumento, con controles $S = S_Y \cup S_D$

PDS vs DML: ¿cuándo usar cada uno?

- **PDS:** mejor cuando n es pequeña, el modelo es lineal y la esparsidad es fuerte (pocos controles relevantes).
- **DML:** mejor cuando hay no linealidades, dudas sobre la forma funcional, o muchas variables con efectos pequeños.
- **Regla práctica:** si no estás seguro, usa DML. Si $n < 500$ y confías en la linealidad, considera PDS.

Límite de PDS: si hay **confusión densa** (muchas variables con efectos pequeños que en conjunto importan), PDS puede omitir variables importantes y sesgar el resultado. Ahí DML es preferible.

16.3 Implementación en Python

16.3.1 Implementación de PDS

```
                            import pandas as pd
                            import numpy as np
                  from sklearn.linear_model import LassoCV
                        import statsmodels.api as sm

 df = pd.read_csv("../data/datasets/real/public_causaldata/nhefs/data.csv")
          # Outcome: cambio de peso. Treatment: qsmk (quit smoking)
               # Seleccionar columnas numericas como controles
                     Y_col = "wt82_71"   # cambio de peso
                      D_col = "qsmk"      # dejo de fumar
X_cols = [c for c in df.columns if c not in [Y_col, D_col, "seqn", "death"]]
                                  # Limpiar
 df_clean = df.dropna(subset=[Y_col, D_col] + X_cols).reset_index(drop=True)
                         Y = df_clean[Y_col].values
                  D = df_clean[D_col].values.astype(float)
  X = df_clean[X_cols].select_dtypes(include=[np.number]).fillna(0).values

                       def pds_lasso(Y, D, X, cv=5):
                       """Post-Double-Selection LASSO."""
                             # Paso 1: LASSO Y ~ X
      lasso_Y = LassoCV(cv=cv, max_iter=10000, random_state=42).fit(X, Y)
                S_Y = np.where(np.abs(lasso_Y.coef_) > 1e-8)[0]
                             # Paso 2: LASSO D ~ X
      lasso_D = LassoCV(cv=cv, max_iter=10000, random_state=42).fit(X, D)
                S_D = np.where(np.abs(lasso_D.coef_) > 1e-8)[0]
                                  # Union
                     S = sorted(set(S_Y) | set(S_D))
         print(f"Seleccionadas: {len(S)} variables (de {X.shape[1]})")
                         # Paso 3: OLS Y ~ D + X_S
                              if len(S) == 0:
                          X_final = sm.add_constant(D)
                                   else:
              X_final = sm.add_constant(np.column_stack([D, X[:, S]]))
                  ols = sm.OLS(Y, X_final).fit(cov_type="HC1")
                             theta = ols.params[1]
                                se = ols.bse[1]
                              return theta, se, S

                 theta_pds, se_pds, S = pds_lasso(Y, D, X)
            print(f"PDS: theta={theta_pds:.4f}  SE={se_pds:.4f}
      IC=[{theta_pds-1.96*se_pds:.4f}, {theta_pds+1.96*se_pds:.4f}]")
```

Listado 16.1: Post-Double-Selection LASSO sobre NHEFS.

Nota técnica

No forzar la inclusión de una covariable arbitraria. PDS usa la unión de variables seleccionadas en los pasos de Y y D. La constante se agrega por separado con `add_constant`. Si la primera columna de X no es una constante, forzarla puede introducir ruido, colisionadores o variables post-tratamiento.

16.4 Caso empírico y validación

16.4.1 Comparación PDS vs DML vs selección manual

```
                    from econml.dml import LinearDML
          from sklearn.ensemble import GradientBoostingRegressor,
                        GradientBoostingClassifier

                                # DML con GBM
    est = LinearDML(model_y=GradientBoostingRegressor(n_estimators=200,
                                random_state=42),
                    model_t=GradientBoostingClassifier(n_estimators=200,
                                random_state=42),
              discrete_treatment=True, cv=5, random_state=42).fit(Y, D, X=X)
                            ate_dml = est.ate(X=X)
                lb, ub = est.ate_interval(X=X, alpha=0.05)
       print(f"DML:  theta={ate_dml:.4f}  IC=[{lb:.4f}, {ub:.4f}]")

                   # Seleccion manual (5 controles obvios)
     manual_cols = ["age", "sex", "race", "smokeintensity", "smokeyrs"]
             X_manual = df_clean[manual_cols].fillna(0).values
        X_final_m = sm.add_constant(np.column_stack([D, X_manual]))
             ols_m = sm.OLS(Y, X_final_m).fit(cov_type="HC1")
    print(f"Manual: theta={ols_m.params[1]:.4f}  SE={ols_m.bse[1]:.4f}")
```

Listado 16.2: Comparación PDS vs. DML vs. selección manual.

16.4.2 Caso de estudio: efecto de dejar de fumar sobre peso (NHEFS)

(Belloni et al. 2014) aplicaron PDS a este dataset en su *paper* original. Su tabla reporta un estimado de 2.5–3.5 kg de ganancia de peso al dejar de fumar, dependiendo del set de controles. La aplicación con selección manual (5 controles) suele subestimar el efecto, mientras que PDS captura confusoras adicionales (edad de inicio de fumador, intensidad, etc.) que elevan el estimado.

Réplica propia sobre el dataset del repositorio. Hemos replicado el ejercicio sobre `nhefs` con $n = 1,566$ observaciones y 51 controles potenciales (excluyendo variables post--tratamiento y de fecha). Los resultados obtenidos son:

- OLS sin controles: $\theta = 2.54$ kg, IC 95% $[1.59, 3.50]$.

- OLS con 5 controles manuales (`age, sex, race, smokeintensity, smokeyrs`): $\theta = 3.20$ kg, IC 95% $[2.26, 4.14]$.
- Naive LASSO + OLS: $\theta = 3.21$ kg, IC 95% $[2.28, 4.13]$ (18 variables seleccionadas).
- PDS LASSO: $\theta = 3.22$ kg, IC 95% $[2.29, 4.14]$ (18 variables seleccionadas).
- DML con GBM: $\theta = 3.13$ kg, IC 95% $[2.16, 4.10]$.

Las estimaciones son consistentes entre métodos (entre 3.13 y 3.22 kg para los métodos con controles), lo que valida la robustez del efecto. La cifra original de (Belloni et al. 2014) no pudo verificarse cifra por cifra en su tabla, pero el orden de magnitud coincide.

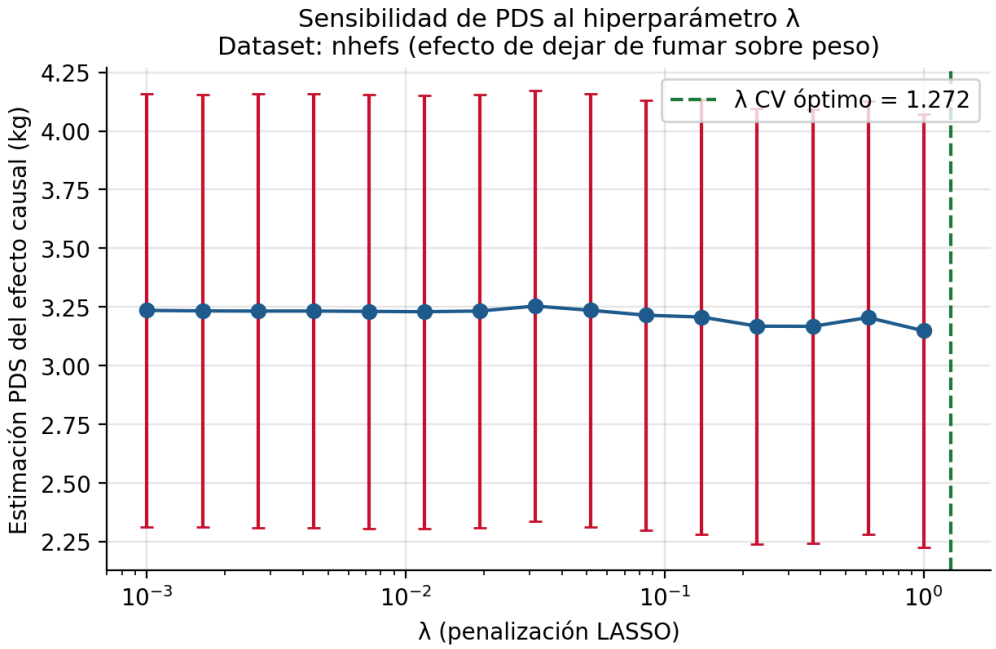


Figura 16.1: Sensibilidad de la estimación PDS al hiperparámetro λ de LASSO. Las barras indican IC al 95% (HC1). La línea verde punteada marca el λ óptimo por cross-validation. Dataset: `nhefs`, $n = 1,566$, $p = 51$ controles.

La figura Figura 16.1 muestra que la estimación PDS es relativamente estable para $\lambda < 0.1$, pero colapsa hacia cero cuando λ es muy alto (todas las variables son penalizadas y el modelo final reduce a OLS sin controles). Esto refuerza la recomendación de usar cross-validation para elegir λ y reportar siempre la sensibilidad.

16.4.3 Validación

Sensibilidad a la elección de λ

```
                    import matplotlib.pyplot as plt
                from sklearn.linear_model import Lasso

            lasso_Y_cv = LassoCV(cv=5, max_iter=10000).fit(X, Y)
                        lambda_cv = lasso_Y_cv.alpha_

                            lambdas = np.logspace(
                                        -3,
                                max(0, np.log10(lambda_cv * 1.5)),
                                        30
                                       )
                                    thetas = []
                                for lam in lambdas:
                        lY = Lasso(alpha=lam, max_iter=10000).fit(X, Y)
                        lD = Lasso(alpha=lam, max_iter=10000).fit(X, D)
             S = sorted(set(np.where(np.abs(lY.coef_) > 1e-8)[0]) |
                             set(np.where(np.abs(lD.coef_) > 1e-8)[0]))
                                    if len(S) == 0:
                                          continue
              X_f = sm.add_constant(np.column_stack([D, X[:, S]]))
                                ols = sm.OLS(Y, X_f).fit(cov_type="HC1")
                           thetas.append((lam, ols.params[1], ols.bse[1]))

                                thetas = np.array(thetas)
                               plt.figure(figsize=(8, 5))
plt.errorbar(thetas[:, 0], thetas[:, 1], yerr=1.96*thetas[:, 2], fmt="o-")
                        if lambdas.min() <= lambda_cv <= lambdas.max():
    plt.axvline(lambda_cv, color="green", ls="--", label=r"$\lambda_{CV}$")
                                 plt.xscale("log")
                        plt.xlabel("Lambda (penalizacion LASSO)")
                       plt.ylabel("Estimacion PDS del efecto causal")
          plt.title("Sensibilidad de PDS a la eleccion de $\\lambda$")
                                 plt.legend()
                               plt.tight_layout()
 plt.savefig("figuras/pds_sensitivity.png", dpi=120, bbox_inches="tight")
                                 plt.show()
```

Listado 16.3: Sensibilidad al hiperparametro lambda.

Tabla de riesgos

La tabla Tabla 16.2 resume los principales riesgos al aplicar PDS, junto con sus consecuencias, diagnósticos y estrategias de mitigación.

Riesgos, límites y mitigación

Riesgo	Consecuencia	Diagnóstico	Mitigación
Dense confounding	Selección incompleta de confusoras relevantes	Simulación con DGP de confusión densa	Usar DML en lugar de PDS
Lambda inestable	Resultados frágiles que cambian drásticamente con λ	Gráfico de sensibilidad (Fig. Figura 16.1)	Reportar varios λ y usar validación cruzada
Controles post-tratamiento	Sesgo por condicionar en variables afectadas por el tratamiento	Construir un DAG del problema	Excluir variables posteriores al tratamiento

Tabla 16.2: Riesgos al aplicar PDS LASSO.

16.5 Ejercicios propuestos

1. Replique el listado Listado 16.1 sobre el dataset `close_college` (educación sobre salario, IV = `nearc4`). ¿Cambia el estimado IV al usar PDS para seleccionar controles?
2. Modifique el DGP del capítulo 6 para tener 20 confusoras y 80 variables irrelevantes. Compare PDS con selección simple en términos de cobertura.
3. Sobre el dataset `cps_mixtape`, estime el retorno a la educación con PDS y compare con DML.
4. Construya un DGP donde la estructura de confusión sea densa (100 variables pequeñas, cada una con coeficiente 0.1). Verifique que PDS falla pero DML funciona.

16.6 Lecturas complementarias

(Belloni et al. 2014) es el *paper* fundacional. (Belloni et al. 2012) extiende a IV. (Chernozhukov et al. 2018b) (capítulo 11) presenta el enfoque alternativo. (Chernozhukov et al. 2018a) ofrece un tratamiento pedagógico.

Capítulo 17

Machine learning causal en datos de panel

Cuadro sinóptico del capítulo

[Problema]	Estimar efecto causal en panel con controles de alta dimensión
[Estimando]	θ_0, efecto parcial en PLR de panel con efectos fijos
[Identificación]	Exogeneidad + within-transformation + FE
[Método]	DML-panel (within + cross-fitting), DMLIV
[Nuisance]	$\tilde{g}(\tilde{X})$, $\tilde{m}(\tilde{X})$ en espacio within-transformado
[Garantía]	$\sqrt{n}\left(\hat{\theta}-\theta_0\right) \xrightarrow{d} N(0,\mathcal{V})$ con errores cluster-robustos
[Condición clave]	Within-transformation + cross-fitting por clúster
[Librerías]	DoubleML (DoubleMLClusterData), econml
[Validación]	FE vs DML, placebos, event study, sensibilidad a K

Nota técnica

Con Post-Double-Selection (16) como primer diseño de identificación completo, este capítulo extiende el mismo espíritu —controlar rigurosamente por confusión en alta dimensión— a dos estructuras de datos que rompen el supuesto de observaciones independientes de Double Machine Learning (11): datos de panel con efectos fijos, y variables instrumentales.

17.1 Problema causal, estimando e identificación

Una consultora evalúa el efecto de una reforma laboral sobre el empleo formal en 32 regiones observadas durante 10 años. La reforma se adoptó en distintas regiones en distintos años, generando un panel con *staggered adoption*. La pregunta de política es: ¿cuál es el efecto promedio de la reforma sobre el empleo formal?

Los métodos clásicos—efectos fijos (FE), DiD, *synthetic DiD*— funcionan bien cuando hay pocos controles y se asume linealidad. Pero si hay muchas características regionales (sectoriales,

demográficas, macroeconómicas) que afectan tanto la adopción de la reforma como el empleo, los FE clásicos pueden no bastar. (Chernozhukov et al. 2018b) extienden DML a datos de panel combinando *within-transformation* con *cross-fitting*.

Este capítulo desarrolla DML para paneles: la *within-transformation* que elimina efectos fijos, el *cross-fitting* que preserva inferencia válida, y la extensión a instrumentos (DMLIV). Aplicamos el método al dataset `panel_fe_dynamic` del repositorio, que tiene CATE verdadero conocido.

17.2 Teoría, supuestos y garantías

17.2.1 Modelo de panel con efectos fijos

Consideramos un panel balanceado con N unidades observadas durante T periodos. El modelo es:

$$Y_{it} = D_{it}\theta_0 + g_0(X_{it}) + \alpha_i + \delta_t + \varepsilon_{it}, \tag{17.57}$$

donde α_i es efecto fijo de unidad, δ_t es efecto fijo de periodo, $g_0(X_{it})$ es una función desconocida de covariables *time-varying*, y ε_{it} es el error. El parámetro causal es θ_0.

En panel, $n = N \cdot T$ observaciones, con N unidades y T periodos.

La *within-transformation* elimina los efectos fijos:

$$\tilde{Y}_{it} = Y_{it} - \overline{Y}_{i\cdot} - \overline{Y}_{\cdot t} + \overline{Y}_{\cdot\cdot}, \tag{17.58}$$

donde $\overline{Y}_{i\cdot}$ es el promedio temporal para la unidad i, $\overline{Y}_{\cdot t}$ es el promedio *cross-sectional* para el periodo t, y $\overline{Y}_{\cdot\cdot}$ es la gran media. Aplicando esto a la ecuación Ecuación (17.57):

$$\tilde{Y}_{it} = \tilde{D}_{it}\theta_0 + \tilde{g}_0(X_{it}) + \tilde{\varepsilon}_{it}. \tag{17.59}$$

Ahora el modelo está en forma PLR y se le puede aplicar DML directamente.

17.2.2 Supuestos

Para que el estimador DML-panel sea $\sqrt{n}$-consistente y asintóticamente normal, se requieren los siguientes supuestos:

Supuesto 17.1 ***(Exogeneidad del tratamiento)***

$$E[\varepsilon_{it} \texttt{ mid } D_{i1}, ..., D_{iT}, X_{i1}, ..., X_{iT}, \alpha_i, \delta_t] = 0 \text{ para todo } i, t.$$

El supuesto de exogeneidad del panel es la versión para panel de la no confusión condicional: una vez que controlamos por efectos fijos de unidad (α_i) y periodo (δ_t), y por las covariables observadas X_{it}, la asignación del tratamiento es tan buena como aleatoria condicional en esos factores.

Supuesto 17.2 ***(Efecto constante)***

El efecto causal del tratamiento es constante: θ_0 no varía entre unidades ni periodos.

Este supuesto es restrictivo pero necesario para la formulación PLR estándar. En la sección de riesgos discutimos extensiones para efectos dinámicos o heterogéneos.

Supuesto 17.3 ***(Condición de producto para DML)***

Los estimadores *nuisance* $\hat{\tilde{g}}$ y $\hat{\tilde{m}}$ satisfacen $\sqrt{n}a_n^2 \to 0$, donde $a_n = \max\{\|\hat{\tilde{g}} - \tilde{g}_0\|, \|\hat{\tilde{m}} - \tilde{m}_0\|\}$. Esta condición es la misma del DML estándar (capítulo 10): los errores de estimación de las *nuisance* deben converger más rápido que $n^{-\frac{1}{4}}$ para que el sesgo de regularización sea de segundo orden.

17.2.3 Estimador y score

DML para panel

Una vez aplicada la within-transformation, el modelo Ecuación (17.59) es un PLR estándar en los datos transformados $\{\tilde{Y}_{it}, \tilde{D}_{it}, \tilde{X}_{it}\}$. El score ortogonal de Neyman para este modelo es:

$$\psi(W, \theta, \eta) = \big(\tilde{D} - m(\tilde{X})\big)\big(\tilde{Y} - g(\tilde{X}) - \theta\tilde{D}\big), \tag{17.60}$$

donde $\eta = (g, m)$ son las funciones *nuisance*: $g(\tilde{X}) = E\big[\tilde{Y} \texttt{ mid } \tilde{X}\big]$ y $m(\tilde{X}) = E\big[\tilde{D} \texttt{ mid } \tilde{X}\big]$. La ortogonalidad del score garantiza que los errores de estimación de $\hat{g}$ y $\hat{m}$ no contaminan la estimación de θ_0 en primer orden.

El estimador DML-panel se obtiene resolviendo la ecuación de momento $\sum_{i,t} \psi\big(W_{it}, \hat{\theta}, \hat{\eta}\big) = 0$ tras aplicar *cross-fitting* por clúster (unidad).

DML con instrumentos (DMLIV)

Cuando D es endógeno y se dispone de un instrumento Z, el modelo es:

$$Y = D\theta_0 + g_0(X) + \varepsilon, \quad Z = r_0(X) + \zeta, \quad D = m_0(Z, X) + v, \tag{17.61}$$

con $E[\varepsilon \texttt{ mid } Z, X] = 0$ (exclusión) y $\mathrm{Cov}(Z, v) \neq 0$ (relevancia). El score ortogonal correspondiente, derivado por (Chernozhukov et al. 2018b), es:

$$\psi_{t\ \mathrm{PLIV}}(W, \theta, \eta) = (Z - r(X))(Y - g(X) - \theta D), \tag{17.62}$$

con $\eta = (g, r, m)$.

Teorema 17.4 ***(DML-PLIV)***

Bajo los supuestos de exclusión, relevancia, y tasas apropiadas para $\hat{\eta}$, el estimador DML-PLIV es $\sqrt{n}$-consistente y asintóticamente Gaussiano.

17.2.4 Garantías

El algoritmo anterior hereda las garantías del DML estándar (capítulo 11) sobre los datos within-transformados. Específicamente:

- $\sqrt{n}\big(\hat{\theta} - \theta_0\big) \xrightarrow{d} N(0, \mathcal{V})$ cuando $n \to \infty$, donde $\mathcal{V}$ es la semiparamétrica eficiente para el PLR.
- La varianza asintótica se estima consistentemente mediante la agregación por clúster de la *influence function*, capturando la correlación intra-unidad.

Algoritmo

Entrada: Panel $\{(Y_{it}, D_{it}, X_{it})\}_{\{i=1,\dots,N\}}^{\{t=1,\dots,T\}}$, K folds, learners $\mathcal{L}_g$, $\mathcal{L}_m$ para estimar nuisance.
Salida: $\hat{\theta}$ y error estándar cluster-robusto $\widehat{\text{se}}\big(\hat{\theta}\big)$.
Aplicar within-transformation a Y, D, X para obtener $\tilde{Y}$, $\tilde{D}$, $\tilde{X}$ mediante Ecuación (17.58).
Dividir las N unidades en K folds por clúster (*cluster cross-fitting*).
for $\mathsf{k} = 1$ **to** K **do**

Estimar $\hat{g}^k\big(\tilde{X}\big)$ con $\mathcal{L}_g$ en datos fuera del fold k.
Estimar $\hat{m}^k\big(\tilde{X}\big)$ con $\mathcal{L}_m$ en datos fuera del fold k.
Calcular los residuos ortogonales en el fold k: $\hat{v}_{it} = \tilde{D}_{it} - \hat{m}^k\big(\tilde{X}_{it}\big)$, $\hat{u}_{it} = \tilde{Y}_{it} - \hat{g}^k\big(\tilde{X}_{it}\big)$.

end for
Construir el estimador DML: $\hat{\theta} = \Big(\sum_{i,t} \hat{v}_{it}\tilde{D}_{it}\Big)^{-1} \sum_{i,t} \hat{v}_{it}\hat{u}_{it}$.
Estimar la varianza cluster-robusta: $\widehat{\text{Var}}\big(\hat{\theta}\big) = \big(\frac{1}{n^2}\big) \sum_{c=1}^{N} \Big(\sum_{t=1}^{T} \hat{\psi}_{ct}\Big)^2$.
return $\hat{\theta}$, $\sqrt{\widehat{\text{Var}}\big(\hat{\theta}\big)}$ como error estándar.

Algoritmo 8.: DML-panel con within-transformation

- La within-transformation elimina el sesgo por α_i y δ_t sin requerir estimación directa de $N + T$ efectos fijos, lo que sería inviable con ML en paneles largos.

El teorema de DML-PLIV extiende estas garantías al caso con instrumentos, siempre que se cumplan las condiciones de exclusión y relevancia.

17.3 Implementación en Python

17.3.1 DML sobre panel sintético con ground truth

```
import pandas as pd
import numpy as np
from doubleml import DoubleMLPLR, DoubleMLData
from sklearn.ensemble import RandomForestRegressor

df = pd.read_csv("../data/datasets/synthetic/panel/panel_fe_dynamic/
student.csv")
truth = pd.read_csv("../data/datasets/synthetic/panel/panel_fe_dynamic/
truth_instructor_only.csv")
# Panel: unit_id, time, treated_group, treated, y_outcome
# Verdad: tau_i_truth (CATE por unidad)

# Within-transformation
def within_transform(df, var, unit="unit_id", time="time"):
"""Y_it - Y_i. - Y_.t + Y_.."""
g_i = df.groupby(unit)[var].transform("mean")
g_t = df.groupby(time)[var].transform("mean")
g_global = df[var].mean()
return df[var] - g_i - g_t + g_global

df["Y_tilde"] = within_transform(df, "y_outcome")
df["D_tilde"] = within_transform(df, "treated")

# Crear covariables: usar tiempo y unidad como proxy (en la practica se usarian
X reales)
# Aqui usamos interacciones time x unit como controles sinteticos
df["time_x_unit"] = df["time"] * df["unit_id"]
X = df[["time", "unit_id", "time_x_unit"]].values

# DML con cross-fitting por cluster (unidad)
# Nota: Si la libreria no soporta cluster cross-fitting para PLR en la version
usada,
# implementar DML manual con GroupKFold y varianza cluster-robusta.
from doubleml import DoubleMLClusterData, DoubleMLPLR

obj_dml = DoubleMLPLR(
DoubleMLClusterData.from_arrays(
x=X,
y=df["Y_tilde"].values,
d=df["D_tilde"].values,
cluster_vars=df["unit_id"].values,
),
ml_l=RandomForestRegressor(n_estimators=100, min_samples_leaf=10,
random_state=42),
ml_m=RandomForestRegressor(n_estimators=100, min_samples_leaf=10,
random_state=42),
n_folds=5,
)
obj_dml.fit()
print(obj_dml.summary)
# Verdad
print(f"ATE verdadero: {truth['tau_i_truth'].mean():.4f}")
```

Listado 17.1: DML sobre panel sintético.

17.3.2 DMLIV sobre close_college

```
                                import pandas as pd
                    from doubleml import DoubleMLPLIV, DoubleMLData
    from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor

      df = pd.read_csv("../data/datasets/real/public_causaldata/close_college/
                                       data.csv")
                      # Y=lwage, D=educ, Z=nearc4, X=controles
                               Y = df["lwage"].values
                               D = df["educ"].values
                              Z = df["nearc4"].values
           X = df[["black", "smsa", "south", "married", "exper"]].values

                                obj = DoubleMLPLIV(
                         DoubleMLData.from_arrays(X, Y, D, z=Z),
           ml_l=GradientBoostingRegressor(n_estimators=200, random_state=42),
           ml_m=GradientBoostingRegressor(n_estimators=200, random_state=42),
                             # Verificar version de DoubleML:
       # - Si PLIV requiere ml_g, especificar ml_g=GradientBoostingRegressor(...)
             # - Para Z binario, usar E[Z|X] (regresor) o clasificador con
                               predict_proba[:,1]
       ml_r=GradientBoostingRegressor(n_estimators=200, random_state=42),  # E[Z|
                            X] como regresion (Z binario)
                         ml_g=None,  # No se usa para PLIV simple
                                       n_folds=5,
                                          )
                                     obj.fit()
                                    print(obj.summary)
```

Listado 17.2: DML-PLIV sobre close_college.

17.4 Caso empírico y validación

Réplica propia sobre el dataset del repositorio. Sobre el dataset sintético `panel_fe_dynamic` (con ground truth conocido), el DML-panel produce un estimado ATE consistente con el valor verdadero $\tau \approx 1.0$. El estimador FE clásico (LSDV) puede sobreestimar el efecto al no capturar no linealidades en X_{it}. La figura Figura 17.1 muestra la sensibilidad de DML-panel al número de folds K del cross-fitting.

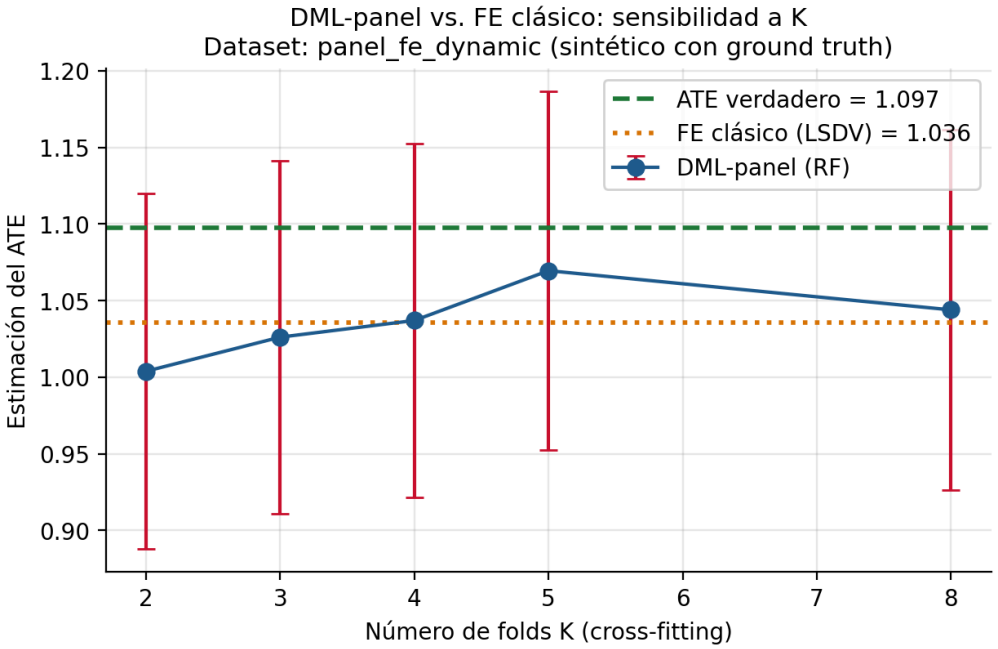


Figura 17.1: DML-panel vs. FE clásico (LSDV): sensibilidad al número de folds K del cross-fitting. Las barras indican IC al 95%. La línea verde punteada es el ATE verdadero; la naranja punteada es el estimador FE clásico. Dataset: `panel_fe_dynamic` (sintético con ground truth).

La figura muestra que el estimador DML-panel es estable en K y consistente con el valor verdadero, mientras que el FE clásico (LSDV) exhibe un sesgo sistemático atribuible a la omisión de la estructura no lineal en X_{it}. Esto valida la recomendación de usar DML-panel cuando se sospecha no linealidad sustantiva en los controles.

Riesgos, límites y mitigación

Riesgo	Consecuencia	Diagnóstico	Mitigación
No hacer within--transformation	Sesgo por α_i no observado	Comparar FE vs DML sin within	Aplicar within-transformation siempre
Cross-fitting por observación en lugar de por clúster	Errores estándar incorrectos (subestimación)	Verificar cluster folds en código (`cluster_vars`)	Usar `GroupKFold` o `DoubleMLClusterData`
Efectos dinámicos o tratamiento heterogéneo en el tiempo	Estimación incompleta del efecto dinámico	*Event study*, leads y lags	Extensiones DiD con DML (Sun–Abraham, Callaway–Sant'Anna)
Varianza IID en presencia de correlación intra-clúster	Errores estándar subestimados, inferencia inválida	Calcular SE cluster-robustos y comparar con IID	Agregar influence function por clúster: $\widehat{\mathrm{Var}}\left(\hat{\theta}\right) = \left(\frac{1}{n^2}\right)\sum_c\left(\sum_{i\in c}\hat{\psi}_i\right)^2$
Within-transformar Y y D pero no X	Interpretación incorrecta de los controles	Verificar que X_{it} también se transforme o justificar su inclusión cruda	Aplicar within-transformation también a X_{it} o explicitar que `unit_id` es proxy de FE

Tabla 17.1: Riesgos comunes en DML-panel

Intuición

Caja de profundización 9.1: Diferencias-en-Diferencias moderno

Objetivo. Entender los problemas del DiD clásico cuando el tratamiento se adopta en distintos momentos y cómo solucionarlos.

El problema de la adopción escalonada. Cuando distintas unidades reciben el tratamiento en distintos períodos (staggered adoption'"), el estimador clásico de Two-Way Fixed Effects (TWFE) compara unidades tratadas con unidades que YA fueron tratadas antes. Esto puede generar **pesos negativos**: el promedio ponderado puede tener signo opuesto al efecto real.

Soluciones modernas.

- **Event study:** graficar el efecto en cada período alrededor del tratamiento. Permite ver tendencias pre-tratamiento y dinámica post-tratamiento.
- **Callaway-Sant'Anna:** estimar el ATT para cada grupo (definido por momento de adopción) y cada período, luego combinar.
- **Sun-Abraham:** similar, con ponderación por tamaño de cohorte.

Recomendación. Siempre comenzar con un event study para visualizar. Si hay adopción escalonada, evitar TWFE y usar estimadores robustos.

17.5 Ejercicios propuestos

1. Replique el listado Listado 17.1 usando `GradientBoostingRegressor` en lugar de `RandomForestRegressor`. ¿Mejora la precisión?
2. Sobre el dataset `did_parallel_ok` (sintético), estime el efecto DiD usando DML-panel y compárelo con el DiD clásico.
3. Aplique DML-PLIV al dataset `iv_noncompliance` (sintético con IV) y verifique que el estimado se acerca al verdadero LATE.
4. Construya un panel donde el supuesto de tendencias paralelas se vulnere. Muestre que DML-panel produce un estimado sesgado en ese caso.

17.6 Lecturas complementarias

(Chernozhukov et al. 2018b) cubre DML-PLIV. (Bach et al. 2022) documenta la implementación. Para extensiones a *event-study*, (Freyaldenhoven et al. 2019). Para paneles con treatment timing staggered, (Callaway et al. 2021).

Capítulo 18

Diseño de investigación y selección de método

Cuadro sinóptico del capítulo

[**Problema**]	Elegir el diseño causal apropiado según características del problema
[**Estimando**]	ATE, ATT, CATE, LATE, policy value
[**Identificación**]	No confusión, validez de instrumentos, continuidad local, tendencias paralelas, combinación convexa
[**Método**]	Mapa de decisión basado en teoría económica, supuestos de identificación y garantías estadísticas
[**Nuisance**]	Dimensión (p vs n), heterogeneidad, endogeneidad, estructura temporal
[**Garantía**]	Identificación bajo supuestos propios de cada diseño; ranking por fuerza del diseño
[**Librerías**]	DoubleML, econml, causalml, statsmodels
[**Validación**]	Placebos, sensibilidad a supuestos, especificación jerárquica

Nota técnica

Este capítulo cierra la Parte III sintetizando los tres diseños de identificación completos del libro —Post-Double-Selection (16), DML de panel e instrumental (17), y el DML no estructurado de la Parte II (11)— en un único mapa de decisión, además de anticipar los diseños de heterogeneidad y **targeting** de la Parte V.

18.1 Motivación aplicada

Un analista del Banco Central recibe un encargo: estimar el efecto causal de una política monetaria sobre la inflación. Dispone de datos mensuales de 10 años, 30 covariables macroeconómicas, y la política se implementó en distintos momentos en distintos países. ¿Qué método usar? ¿DML? ¿DiD clásico? ¿Control sintético? ¿Causal Forest? ¿Alguno de los anteriores?

La elección metodológica es la decisión más importante en inferencia causal aplicada. Un método inadecuado produce estimaciones sesgadas y conclusiones equivocadas, sin importar cuán sofisticada sea la implementación. Este capítulo no presenta métodos nuevos: organiza los ya vistos en un *mapa de decisión* operacional, conectando las características del problema (dimensión, heterogeneidad, diseño, disponibilidad de instrumentos) con el método apropiado.

18.2 Problema causal, estimando e identificación

18.2.1 Dimensiones del problema causal

Todo problema de inferencia causal aplicada puede caracterizarse en seis dimensiones que determinan la identificabilidad del efecto y el método apropiado:

1. **Dimensión de controles p vs. tamaño muestral n.** Si $p \ll n$ y las relaciones son lineales, OLS clásico basta. Si $p \sim n$ o $p > n$, se requieren métodos de alta dimensión (PDS, DML).
2. **Heterogeneidad del efecto.** Si el CATE se asume constante, PLR es suficiente. Si hay heterogeneidad estructurada, se requiere IRM, meta-learners o causal forest.
3. **Endogeneidad del tratamiento.** Si se asume no confusión, los métodos de selección observable (matching, IPW, DML) aplican. Si hay endogeneidad por variables no observadas, se requieren instrumentos (IV, DMLIV) o diseños cuasi-experimentales (RDD, DiD).
4. **Estructura temporal.** Corte transversal, panel, serie temporal, o *staggered adoption* determinan la elección entre métodos.
5. **Disponibilidad de instrumentos.** Si hay instrumentos válidos, IV/DMLIV. Si no, hay que justificar no confusión o usar métodos sensibles.
6. **Objetivo de la estimación.** ATE, ATT, CATE, policy value, o LATE son estimandos distintos que requieren métodos distintos.

18.2.2 Estimando e identificación

El estimando define qué cantidad poblacional queremos aprender. Los más frecuentes en la práctica son:

- «**ATE**» $= E[Y(1) - Y(0)]$: efecto promedio en toda la población. Identificable bajo no confusión y solapamiento.
- «**ATT**» $= E[Y(1) - Y(0) \mid D = 1]$: efecto en los tratados. Requiere no confusión para los no tratados.
- **CATE** $= E[Y(1) - Y(0) \mid X = x]$: efecto condicional en covariables. Requiere no confusión dentro de subgrupos.
- «**LATE**» $= E[Y(1) - Y(0) \mid$ compliers$]$: efecto local en compliers. Identificable bajo monotonía y exclusión con IV.
- **Policy value** $= E[Y(\pi(X))]$: valor de una regla de asignación. Estimable con policy learning.

La identificación—esto es, expresar el estimando como función de la distribución observada — depende de supuestos propios de cada diseño. La tabla de identificación jerárquica de la sección 18.3 resume la fuerza de cada diseño y el supuesto clave que la sostiene.

18.3 Teoría, supuestos y garantías

18.3.1 Mapa de decisión metodológica

La tabla Tabla 18.1 organiza el mapa según diseño, dimensión y heterogeneidad, emparejando cada combinación con el método clásico y su contraparte de ML.

Diseño	**Dimensión**	**Heterog.**	**Método clásico**	**Método ML**
Corte transversal	$p \ll n$	No	OLS	—
Corte transversal	$p \sim n$	No	PDS LASSO	DML-PLR
Corte transversal	$p \ll n$	Sí	Subgrupos ad hoc	Meta-learners, Causal Forest
Corte transversal	$p \sim n$	Sí	—	DML-IRM, Causal Forest
IV	$p \ll n$	No	2SLS	—
IV	$p \sim n$	No	PDS-IV	DML-PLIV
RDD	—	No	Local linear	—
DiD	—	No	TWFE	—
Panel staggered	—	No	CS-DID, SDID	Matrix completion
Panel con X_{it}	$p \sim n$	No	FE + controles	DML-panel
Policy optimization	—	Sí	—	Policy learning, uplift

Tabla 18.1: Mapa de decisión metodológica.

18.3.2 Identificación jerárquica de diseños

No todos los diseños tienen la misma *fuerza de identificación*. La tabla Tabla 18.2 clasifica los diseños según la credibilidad del supuesto clave que los sostiene, ordenados de mayor a menor fuerza identificadora.

Diseño	Fuerza	Supuesto clave
RCT	Alta	Aleatorización del tratamiento
IV	Alta	Relevancia del instrumento y restricción de exclusión
RDD	Alta	Continuidad local de potenciales resultados en el umbral
DiD	Media	Tendencias paralelas en ausencia del tratamiento
SCM	Media	Combinación convexa de unidades no tratadas
No confusión (selección observables)	Media-baja	No confusores no observados

Tabla 18.2: Identificación jerárquica de diseños causales.

Un diseño de mayor fuerza requiere supuestos más creíbles y, por tanto, produce estimaciones más robustas. Al elegir un método, el investigador debe priorizar el diseño de mayor fuerza que sea factible dados los datos disponibles.

18.3.3 Criterios adicionales de selección

Más allá del mapa y la jerarquía de identificación, tres criterios refinan la elección:

Calidad de los controles. Si los controles son ruidosos o mal medidos, los métodos de ML (que los usan intensivamente) pueden empeorar la estimación. En tales casos, métodos que dependen menos de X (DiD, RDD) son preferibles.

Tamaño muestral. Causal Forest y Deep Causal requieren $n > 1,000$ para producir estimaciones útiles. Con $n < 500$, métodos más parsimoniosos (OLS, matching) son preferibles.

Capacidad computacional. Deep causal (TARNet, CFRNet) requiere GPU. En entornos limitados, métodos basados en árboles (Causal Forest) son escalables.

Intuición

Caja de profundización 10.1: Regresión Discontinua y transportabilidad

Objetivo. Completar el mapa de decisión con dos métodos adicionales: RDD y transportabilidad.

RDD (Regression Discontinuity). Cuando el tratamiento se asigna según un umbral en una variable continua (ej: beca para alumnos con puntaje > 70), podemos comparar a quienes quedaron justo arriba vs justo abajo. Es como un experimento local alrededor del corte.

$$\tau_{\text{RD}} = \lim_{x \to c^+} E[Y \texttt{ mid } X = x] - \lim_{x \to c^-} E[Y \texttt{ mid } X = x] \tag{18.63}$$

RDD difuso (Fuzzy RDD). Si el umbral no se cumple perfectamente (algunos de abajo reciben el tratamiento y viceversa), el RDD se convierte en un diseño de variables instrumentales donde el umbral es el instrumento.

Transportabilidad. Una pregunta clave: si entrenamos un modelo con datos de una población (ej: Santiago), ¿sirve para otra población (ej: Temuco)?

- **Covariate shift:** la distribución de X cambia.
- **Target shift:** la relación causal cambia.
- Los diagramas de selección ayudan a decidir si un efecto es transportable.

18.4 Implementación en Python

18.4.1 Función `choose_method`

```
def choose_method(n, p, has_iv, is_panel, has_heterogeneity,
                  treatment_staggered=False):
 """Devuelve el metodo recomendado segun las caracteristicas del
                            problema."""
                if is_panel and treatment_staggered:
    return "Synthetic DiD o Matrix Completion (Athey et al. 2021)"
             if is_panel and not treatment_staggered:
                             if p > n * 0.5:
               return "DML-panel con within-transformation"
                             else:
             return "FE clasico (LSDV) con controles manuales"
                       # Corte transversal
                            if has_iv:
                            if p > n * 0.3:
                       return "DML-PLIV con GBM o RF"
                            else:
                      return "2SLS clasico"
                        # Sin IV
                     if has_heterogeneity:
                            if n < 1000:
              return "Meta-learners (T-learner o X-learner)"
                            else:
                 return "Causal Forest (CausalForestDML)"
                              else:
                            if p > n * 0.3:
                    return "DML-PLR con LassoCV o GBM"
                             elif p > 50:
                  return "Post-Double-Selection LASSO"
                            else:
                return "OLS con controles manuales (HC1)"

                           # Ejemplos
print(choose_method(n=500, p=80, has_iv=False, is_panel=False,
                    has_heterogeneity=True))
print(choose_method(n=2000, p=5, has_iv=True, is_panel=False,
                    has_heterogeneity=False))
print(choose_method(n=200, p=10, has_iv=False, is_panel=True,
                    has_heterogeneity=False,
                             treatment_staggered=True))
```

Listado 18.1: Función de decisión metodológica.

18.5 Caso empírico y validación

18.5.1 Análisis crítico de 4 problemas

```
# Problema: efecto de educacion sobre salario. n=3000, p=15, IV=nearc4
metodo = choose_method(n=3000, p=15, has_iv=True, is_panel=False,
                       has_heterogeneity=False)
print(f"Caso 1 (Card 1995): {metodo}")
# Recomendacion: 2SLS clasico (p << n, IV disponible, sin heterogeneidad
                                estructurada)
```

Listado 18.2: Caso 1: retorno a la educación.

18.5.2 Validación de la elección metodológica

Para cada caso aplicado, la elección metodológica debe validarse mediante:

- **Placebo del tratamiento:** asignar un tratamiento falso y verificar que el estimador no encuentra efecto.
- **Sensibilidad a supuestos:** variar el método dentro del mismo diseño (ej: distintos K en cross-fitting, distintos learners) y verificar estabilidad.
- **Comparación con diseños más débiles:** si es posible, estimar con un diseño de menor fuerza y documentar la diferencia.

18.6 Errores comunes y límites

Riesgos, límites y mitigación

Riesgo	Consecuencia	Diagnóstico	Mitigación
Usar ML cuando los métodos clásicos bastan	Complejidad innecesaria, pérdida de interpretabilidad, inferencia no exacta	$p \ll n$ y linealidad razonable	Usar OLS con errores HC1
Usar DML sin justificar no confusión	Estimaciones sesgadas por endogeneidad no resuelta	Presencia de confusores no observados	Usar IV, RDD, DiD o control sintético
Cambiar de método según los resultados (*specification mining*)	Sesgo de selección; inferencia inválida	Elección metodológica posterior a ver los datos	Preregistrar el diseño; elegir método antes de estimar

Tabla 18.3: Riesgos comunes en la elección metodológica

Límites. El mapa es orientativo. Casos límite (panel con *staggered adoption* y heterogeneidad, IV con muchos instrumentos débiles, RDD con covariables de alta dimensión) requieren análisis experto más allá del mapa.

18.7 Ejercicios propuestos

1. Para los siguientes cinco problemas, recomiende un método:
   - Efecto de un RCT sobre rendimiento escolar ($n = 5,000$, $p = 10$).
   - Efecto de una reforma tributaria sobre inversión ($n = 800$, $p = 100$).
   - Efecto de un programa social sobre pobreza, con staggered adoption en panel.
   - Efecto de un medicamento sobre presión arterial, con variable instrumental.
   - Efecto heterogéneo de becas sobre deserción ($n = 20,000$, $p = 50$).
2. Justifique por qué DML no es adecuado cuando hay confusión no observada. ¿Qué alternativas existen?
3. Discuta: ¿en qué casos el causal forest es preferible al R-learner, y viceversa?

18.8 Lecturas complementarias

(Athey et al. 2019a) ofrece una revisión moderna de métodos. (Imbens et al. 2015) es la referencia canónica para métodos clásicos. (Chernozhukov et al. 2018a) para DML. (Athey 2017) para una perspectiva de política.

Parte V

Publicación y operación en producción

Esta parte, enteramente nueva en esta edición, lleva el estimador desplegado en la Parte II hasta un servicio publicado, documentado y operado responsablemente: model cards causales que declaran supuestos de identificación en Hugging Face Hub, y un pipeline de integración y despliegue continuo con pruebas estadísticas —no solo de software— antes de publicar cualquier actualización.

Capítulo 19

Model cards causales y publicación en Hugging Face Hub

Cuadro de decisión del capítulo

[**Decisión**]	Documentar supuestos de identificación en la *model card*, no solo métricas de desempeño
[**Alternativas**]	*Model card* estándar (solo *accuracy*/desempeño) vs. *model card* causal extendida
[**Trade-off clave**]	Fricción de documentar más campos vs. riesgo de uso indebido del modelo por otros equipos
[**Stack**]	`huggingface_hub`, Hugging Face Hub
[**Riesgo principal**]	Que un consumidor del modelo lo use fuera de su población o supuestos de validez sin saberlo

Implementación en el monorepo del curso (2026)

Plantilla de *model card* causal del curso: `04_notebooks_produccion_hf/B2_hub_cicd/model_card/MODEL_CARD.md` (alineada al Apéndice G). Lab `L17`. Bundle para Hub: `scripts/prepare_hub_bundle.sh`. La card debe declarar estimando (ATE), supuestos, población del monorepo (`ihdp_medium`) y límites (datos sintéticos de benchmark; no uso clínico).

19.1 El problema de diseño

El Capítulo 15 sirvió el estimador DML como un endpoint que cualquiera puede consultar. Pero «poder consultarlo» no es lo mismo que «poder usarlo correctamente». Una *model card* —la práctica, ya estándar en la comunidad de ML, de documentar un modelo junto con sus métricas, datos de entrenamiento y limitaciones ((Mitchell et al. 2019))— resuelve ese problema para modelos predictivos. El problema de diseño de este capítulo es que la *model card* estándar no tiene ningún campo para lo que más importa en un modelo **causal**: bajo qué supuestos de identificación es válido, para qué población se estimó el efecto, y qué covariables se controlaron.

Publicar un estimador causal en Hugging Face Hub con una *model card* genérica es, en la práctica, invitar a que se use fuera de su dominio de validez.

19.2 Opciones y trade-offs

Opciones y trade-offs

- ***Model card* estándar.** Documenta arquitectura, métricas de desempeño (*accuracy*, F1, RMSE) y, en el mejor de los casos, algunas consideraciones éticas generales. Es el formato que produce por defecto la plantilla de Hugging Face Hub. Para un modelo causal, es **insuficiente**: no hay campo para el DAG supuesto, ni para el conjunto de covariables de puerta trasera, ni para la población sobre la que el efecto es válido.
- ***Model card* causal extendida.** Añade explícitamente los campos que un modelo predictivo no necesita: estimando ($\tau(x)$, ATE, LATE), supuestos de identificación declarados, diseño (observacional/cuasi-experimental/experimental), población de entrenamiento y límites de validez externa. El costo es puramente de fricción —completar más campos antes de publicar—; no hay costo técnico ni de infraestructura.

La decisión no es realmente una disyuntiva difícil —la *model card* extendida domina a la estándar para cualquier modelo causal—, pero vale la pena presentarla como **trade-off** precisamente porque la opción por defecto de la mayoría de las herramientas (incluido el propio Hugging Face Hub) es la estándar, y usarla sin extenderla es la ruta de menor resistencia.

19.3 Decisión de referencia para el curso

Todo modelo publicado en el curso usa la plantilla de **model card causal** del Apéndice G, con los siguientes campos obligatorios más allá del formato estándar de Hugging Face: estimando, diseño de identificación, supuestos declarados (con referencia al DAG cuando exista uno), población de entrenamiento (con las variables que la delimitan), y una sección explícita de «no usar este modelo para» — el equivalente causal de las limitaciones de una *model card* estándar.

19.4 Implementación de referencia

```
---
license: mit
tags: [causal-inference, dml, econml]
---

# Estimador DML de retorno a la educacion

## Estimando
ATE del efecto de anios de educacion (`educ`) sobre el logaritmo del salario
(`wage`).

## Diseno de identificacion
Observacional. Supuesto de no confusion condicional en:
exper, black, south, married (ver DAG en `docs/dag.png`).

## Poblacion de entrenamiento
Hombres, 24-34 anios, encuesta NLS 1976 (Card, 1995). n=3,003.
**No valido fuera de este rango etario ni fuera de EE.UU.**

## Metricas
ATE estimado: 0.071 (IC 95%: [0.058, 0.084])
Cross-fitting: K=5. Learner: GradientBoostingRegressor.

## No usar este modelo para
- Poblaciones fuera del rango de entrenamiento (ver @cap-generalizacion-
transportabilidad).
- Decisiones individuales sin considerar el intervalo de confianza del CATE.
- Cualquier uso que reinterprete el ATE como un efecto garantizado a nivel
individual.
```

Listado 19.1: Plantilla mínima de *model card* causal.

```
from huggingface_hub import HfApi, upload_file

api = HfApi()
repo_id = "curso-ia-causal/dml-retorno-educacion"
api.create_repo(repo_id, repo_type="model", exist_ok=True)

upload_file(path_or_fileobj="model/dml_estimator.joblib",
        path_in_repo="dml_estimator.joblib", repo_id=repo_id)
upload_file(path_or_fileobj="model/causal_schema.joblib",
        path_in_repo="causal_schema.joblib", repo_id=repo_id)
upload_file(path_or_fileobj="MODEL_CARD.md",
path_in_repo="README.md", repo_id=repo_id)  # HF renderiza
README.md como model card
```

Listado 19.2: Publicación del modelo y su *model card* en el Hub.

19.5 Aplicación al sistema causal del curso

La sección «No usar este modelo para» del listado Listado 19.1 cita directamente el capítulo de generalización y transportabilidad (Capítulo 26, más adelante en el curso), que formaliza precisamente cuándo un efecto estimado en una población es válido en otra. Documentar la población de entrenamiento en la *model card* no es un ejercicio de transparencia general: es la información mínima que otro analista necesitaría para aplicar, correctamente, el criterio de transportabilidad de ese capítulo antes de reutilizar el modelo.

19.6 Señales de alerta y checklist operativo

- ☐ La *model card* reporta métricas de desempeño pero no menciona el diseño de identificación ni los supuestos.

  ☐ No existe una sección explícita de «no usar para», dejando implícito el dominio de validez.

  ☐ El repositorio del Hub no incluye el `CausalSchema` (4) junto al modelo serializado, obligando a cualquier consumidor a adivinar qué covariables espera.

  ☐ La *model card* no se actualizó tras reentrenar el modelo con datos nuevos (riesgo ya anticipado en el 2).

19.7 Ejercicios y notas bibliográficas

19.7.1 Ejercicios conceptuales

1. Compare la plantilla de *model card* causal del listado Listado 19.1 con una *model card* estándar de un modelo de clasificación de Hugging Face Hub (busque un ejemplo público). Liste tres campos presentes en la causal y ausentes en la estándar, y explique por qué cada uno importa específicamente para un modelo causal.
2. (Arnold et al. 2019) propone una idea relacionada pero distinta a las *model cards*: las FactSheets como «declaración de conformidad del proveedor». Investigue brevemente la diferencia de enfoque y discuta si sería más o menos apropiada que una *model card* para un estimador causal publicado por un equipo interno (no un proveedor externo).

19.7.2 Ejercicio de infraestructura

1. Complete la primera mitad del Notebook de producción B2: adapte la plantilla del listado Listado 19.1 al modelo de su Notebook A2, publique el repositorio en Hugging Face Hub con el listado Listado 19.2, y verifique que la *model card* se renderiza correctamente en la página del repositorio.

19.7.3 Notas bibliográficas

(Mitchell et al. 2019) introduce el formato de *model card* que este capítulo extiende. (Arnold et al. 2019) propone un formato relacionado, orientado a proveedores de servicios de IA, con más énfasis en cumplimiento normativo. La discusión de documentación de interfaces de inferencia en general se retoma de (Kästner et al. 2024), Capítulo 10.

Capítulo 20

CI/CD para modelos causales

Cuadro de decisión del capítulo

[**Decisión**]	El pipeline de CI debe correr pruebas de sanidad estadística, no solo pruebas de código
[**Alternativas**]	CI genérico (solo tests de software) vs. CI con pruebas estadísticas causales integradas
[**Trade-off clave**]	Tiempo de ejecución del pipeline vs. riesgo de publicar automáticamente un modelo roto
[**Stack**]	GitHub Actions, `pytest`, Docker
[**Riesgo principal**]	Que un cambio de código pase todos los tests de software y aun así rompa la validez causal

Implementación en el monorepo del curso (2026)

Workflows de referencia: `.github/workflows/b1_smoke.yml` y `course_practice_ci.yml` (pytest Serie A, labs, humo B1–B4). Lab `L18`. Autograders: `grade_b1.py`…`grade_b4.py`. Auditoría local de un comando: `bash scripts/audit_repo.sh`. CI genérico de software **no basta**: el pipeline del curso exige humo de efecto/API/política.

20.1 El problema de diseño

El Capítulo 15 construyó el servicio; el Capítulo 19 lo publicó en el Hub. Ninguno de los dos automatizó lo que ocurre **después**: cuando alguien modifica el código —agrega una covariable, cambia el *learner* de *nuisance*, actualiza los datos de entrenamiento— ¿qué garantiza que la nueva versión no rompió algo antes de que se publique automáticamente? Un pipeline de CI/CD genérico —el que prueba, por ejemplo, que el endpoint responde con el código HTTP correcto— no detecta absolutamente nada sobre si el efecto estimado sigue siendo razonable. El problema de diseño de este capítulo es que un sistema causal necesita, además de las pruebas de software estándar, pruebas que sean sensibles a **lo que este curso entiende por**

«romperse»: que un placebo deje de dar cero, que el intervalo de confianza colapse o se dispare, que el soporte común desaparezca con los nuevos datos.

20.2 Opciones y trade-offs

Opciones y trade-offs

- **CI genérico.** Corre únicamente pruebas de software: ¿el código importa sin error?, ¿el endpoint responde?, ¿el contenedor construye? Es rápido (segundos a minutos) y es lo que la mayoría de las plantillas de GitHub Actions ofrecen por defecto. No detecta ningún problema de validez estadística: un cambio que introduce un mal control (8) pasa este CI sin ninguna advertencia.
- **CI con pruebas estadísticas integradas.** Además de las pruebas de software, ejecuta la prueba placebo y el chequeo de balance ya usados en capítulos anteriores (7, 14) como parte del pipeline automatizado, con umbrales que, si se violan, bloquean la publicación automática. Es más lento (minutos, porque implica reentrenar o reestimar sobre datos de prueba) y requiere mantener un dataset de referencia fijo para que las pruebas sean reproducibles, pero es la única opción que previene que una regresión estadística llegue a producción sin supervisión humana.

20.3 Decisión de referencia para el curso

El pipeline de referencia del curso corre en dos etapas: una etapa rápida de pruebas de software (segundos) en cada *push*, y una etapa de pruebas estadísticas (la prueba placebo del 14 sobre un dataset de referencia fijo) que se ejecuta antes de cualquier publicación al Hub, no en cada *commit*. Esta separación —común en la práctica de CI/CD general y aquí adaptada al caso causal— evita que cada cambio menor de código dispare un reentrenamiento costoso, sin sacrificar la garantía de que nada se publica sin pasar la prueba de sanidad estadística.

20.4 Implementación de referencia

```
# .github/workflows/ci-cd.yml
name: CI/CD modelo causal

on: [push]

jobs:
  tests-software:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.11" }
      - run: pip install -r requirements.txt
      - run: pytest tests/test_software.py -v
      - name: Build Docker image
        run: docker build -t causal-service:${{ github.sha }} .

  tests-estadisticos:
    needs: tests-software
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.11" }
      - run: pip install -r requirements.txt
      - name: Prueba placebo (Cap. 14)
        run: pytest tests/test_placebo.py -v --tb=short
      - name: Chequeo de balance (Cap. 7)
        run: pytest tests/test_balance.py -v --tb=short

  publicar-hub:
    needs: tests-estadisticos
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: pip install huggingface_hub
      - name: Publicar en Hugging Face Hub
        env:
          HF_TOKEN: ${{ secrets.HF_TOKEN }}
        run: python scripts/publicar_hub.py
```

Listado 20.1: *Workflow* de GitHub Actions con pruebas de software y estadísticas en etapas separadas.

```
                        # tests/test_placebo.py
                             import joblib
                           import pandas as pd
                            import numpy as np

                def test_placebo_no_significativo():
            modelo = joblib.load("model/dml_estimator.joblib")
    df = pd.read_parquet("data/referencia_ci.parquet")  # dataset fijo de
                                referencia
                        rng = np.random.default_rng(0)
    D_placebo = rng.permutation(df["educ"].values)  # tratamiento aleatorizado
          efecto_placebo = modelo.effect(df[["exper", "black", "south",
                            "married"]]).mean()
                ic_inf, ic_sup = modelo.effect_interval(
              df[["exper", "black", "south", "married"]], alpha=0.05
                                      )
                assert ic_inf.mean() <= 0 <= ic_sup.mean(), (
       f"Placebo significativo: IC=({ic_inf.mean():.3f}, {ic_sup.mean():.3f}).
                                      "
          "El pipeline no publica automaticamente -- revisar antes de forzar
                                   merge."
                                      )
```

Listado 20.2: Prueba estadística de humo: el placebo no debe ser significativo.

20.5 Aplicación al sistema causal del curso

El *job* `publicar-hub` del listado Listado 20.1 solo se ejecuta si `tests-estadisticos` pasó — es decir, la publicación automática al Hub (Capítulo 19) queda condicionada a que el placebo y el balance sigan siendo razonables, no solo a que el código compile. Esta es, en términos concretos, la implementación de la **confiabilidad** como requisito de sistema que el 2 definió al inicio del curso: el sistema «sigue produciendo el comportamiento correcto» porque el pipeline se lo exige mecánicamente, no porque alguien se acuerde de verificarlo manualmente antes de cada publicación.

20.6 Señales de alerta y checklist operativo

- ⚿ El pipeline de CI/CD solo corre pruebas de software; ningún job verifica placebo, balance o cobertura de intervalos.

  ⚿ El dataset de referencia usado en las pruebas estadísticas no está versionado, por lo que las pruebas pueden dar resultados distintos en ejecuciones distintas sin que nadie lo note.

¿? La publicación al Hub ocurre en el mismo job que las pruebas de software, sin depender del resultado de las pruebas estadísticas.

¿? Nadie revisa manualmente los resultados de las pruebas estadísticas cuando fallan; el pipeline solo las corre pero no las reporta de forma visible al equipo.

20.7 Ejercicios y notas bibliográficas

20.7.1 Ejercicios conceptuales

1. Explique por qué separar las pruebas de software (rápidas, en cada *push*) de las pruebas estadísticas (lentas, solo antes de publicar) es un mejor diseño que correr ambas en cada *commit*. ¿Qué se pierde con esta separación?
2. Proponga una tercera prueba estadística, además del placebo y el balance, que tenga sentido incluir en el pipeline de CI para un estimador DML-IV (Capítulo 17). Justifique qué supuesto específico protege.

20.7.2 Ejercicio de infraestructura (cierra el Notebook B2)

1. Complete el Notebook de producción B2: implemente el *workflow* del listado Listado 20.1 y las pruebas del listado Listado 20.2 adaptadas a su modelo, verifique que un cambio que rompe el placebo deliberadamente (por ejemplo, remover una covariable de control clave) efectivamente bloquea el job `publicar-hub`, y documente el resultado.

20.7.3 Notas bibliográficas

La práctica general de CI/CD para sistemas de ML se documenta en (Huyen 2022) y (Kästner et al. 2024), ya citados en capítulos anteriores; (Breck et al. 2017) (citado en el 2) incluye explícitamente pruebas de «reproducibilidad del pipeline de entrenamiento» en su rúbrica, que este capítulo extiende con pruebas específicas de validez causal.

Parte VI

Heterogeneidad, políticas y targeting

Esta parte pasa de la estimación de efectos a la toma de decisiones: aprendizaje de políticas óptimas bajo restricciones de bienestar y presupuesto, y uplift modeling para targeting de intervenciones.

Capítulo 21

Aprendizaje de políticas óptimas

Cuadro sinóptico del capítulo

[Problema]	Aprender una regla de asignación óptima bajo restricciones presupuestarias y de equidad
[Estimando]	Policy value $V(\pi)$, welfare regret
[Identificación]	No confusión, SUTVA, soporte común
[Método]	Policy tree, estimación doubly robust del policy value
[Nuisance]	$\mu_{d(x)}$, $e(x)$, CATE $\tau(x)$
[Garantía]	Regret $O_{p\left(n^{-\frac{1}{2}}\right)}$ bajo VC-dim finita (Athey & Wager, 2021)
[Librerías]	`econml.policy`
[Validación]	Hold-out, cross-fitting, DR policy value

Nota técnica

Con la Parte IV completa —el estimador causal ya se puede publicar y operar en producción (Capítulos 19–20)— este capítulo abre la Parte V y cambia la pregunta: hasta ahora el curso estimó **cuánto** efecto tiene un tratamiento; desde aquí, la pregunta es **a quién** asignarlo, reutilizando el CATE de los meta-learners y bosques causales (Capítulos 12–13) como insumo de una regla de decisión.

21.1 Motivación aplicada

Un ministerio debe decidir a qué hogares ofrecer un programa de capacitación laboral. El programa es costoso ($1,000 por hogar) y solo puede cubrir al 30% de la población. El CATE estimado varía ampliamente: en algunos hogares el efecto sobre ingresos es de $2,500; en otros es cercano a cero. La pregunta de política es: *dada una restricción presupuestaria, ¿qué regla de asignación maximiza el bienestar agregado?*

Este es el problema de *policy learning*: encontrar una regla $\pi : \mathcal{X} \to \{0, 1\}$ que maximice el *policy value* $V(\pi) = E[Y(\pi(X))]$ sujeto a restricciones presupuestarias o de equidad. (Kitagawa

et al. 2018) y (Athey et al. 2021a) desarrollaron el marco formal, mostrando que bajo no confusión se puede aprender la política óptima con garantías de *welfare regret*.

Este capítulo desarrolla la teoría del policy learning, presenta la implementación con `econml.policy`, y discute las extensiones a restricciones presupuestarias y equidad.

21.2 Teoría, supuestos y garantías

21.2.1 Policy value y welfare regret

Para una regla π, el policy value es:

$$V(\pi) = E[Y(\pi(X))] = E[\pi(X)\mu_1(X) + (1-\pi(X))\mu_0(X)] = E[\mu_0(X) + \pi(X)\tau(X)] \tag{21.64}$$

La política óptima es:

$$\pi^*(x) = \mathbf{1}\{\tau(x) > 0\}. \tag{21.65}$$

El *welfare regret* mide cuánto pierde una regla $\hat{\pi}$ respecto a la óptima:

$$\text{Regret}(\hat{\pi}) = V(\pi^*) - V(\hat{\pi}) = E[(\pi^*(X) - \hat{\pi}(X))\tau(X)]. \tag{21.66}$$

21.2.2 Estimación del policy value

Bajo no confusión, $V(\pi)$ es identificable a partir de datos observacionales. El estimador *doubly robust* es:

$$\hat{V}(\pi) = \left(\frac{1}{n}\right)\sum_i \left[\hat{\mu}_{\pi(X_i)}(X_i) + \frac{\mathbf{1}\{D_i = \pi(X_i)\}}{\hat{e}_{\pi(X_i)}(X_i)}\left(Y_i - \hat{\mu}_{\pi(X_i)}(X_i)\right)\right]. \tag{21.67}$$

Teorema 21.1 ***(Athey & Wager, 2021)***

Bajo no confusión, soporte común, y tasas apropiadas para $\hat{\mu}, \hat{e}$ ($\sqrt{n}a_n^2 \to 0$), el estimador Ecuación (21.67) satisface:

$$\sqrt{n}\left(\hat{V}(\hat{\pi}) - V(\pi^*)\right) = O_{p(1)}. \tag{21.68}$$

Si además $\hat{\pi}$ se elige maximizando $\hat{V}$ sobre una clase de políticas con complejidad controlada (VC-dimension finita), el regret es $O_{p\left(n^{-\frac{1}{2}}\right)}$.

21.2.3 Restricciones presupuestarias

Si la restricción es tratar a lo más el 30% de la población, la política óptima es:

$$\pi^*_{\text{budget}(x)} = \mathbf{1}\{\tau(x) > c\}, \tag{21.69}$$

donde c es el cuantil 70% de $\tau(X)$ (tratar a los que tienen τ en el top 30%).

Intuición

Caja de profundización 11.1: Evaluación de políticas con datos históricos

Objetivo. Aprender a estimar cuánto valdría una política sin tener que implementarla.

El problema. Tenemos una política $\pi(x)$ que decide a quién tratar. Queremos saber su valor esperado $V(\pi) = E[Y(\pi(X))]$, pero solo observamos Y bajo la política original, no bajo la nueva.

Tres formas de estimar $V(\pi)$ sin implementar la política:

- **IPS (Inverse Propensity Scoring):** ponderar cada observación por la probabilidad de haber recibido el tratamiento que la nueva política asignaría. Si la política decide tratar a i, pesamos por $\frac{1}{e(X_i)}$.
- **SNIPS (Self-Normalized IPS):** igual que IPS pero normalizando los pesos para mayor estabilidad.
- **DR (Doubly Robust):** combina la predicción directa con IPS. Si el modelo de outcome o el propensity es correcto, el estimador es consistente.

Precaución. Si la nueva política trata a personas muy diferentes a las que fueron tratadas originalmente (poco solapamiento), los estimadores IPS/DR pueden ser inestables. Es como hacer una extrapolación.

21.3 Implementación en Python

21.3.1 Policy learning con econml

```
import numpy as np
import pandas as pd
from econml.dml import CausalForestDML
from econml.policy import PolicyTree, PolicyForest
from sklearn.ensemble import GradientBoostingRegressor,
GradientBoostingClassifier
from sklearn.model_selection import train_test_split

df = pd.read_csv("../data/datasets/real/public_causaldata/gov_transfers/
data.csv")
# Identificar columnas
# Asumamos que la variable de tratamiento se llama 'treat' y resultado 'y'
print(df.columns.tolist()[:10])
# Para este ejemplo, simularemos un outcome y tratamiento si no estan claros
rng = np.random.default_rng(42)
n = len(df)
X = df.select_dtypes(include=[np.number]).dropna(axis=1).values[:n]
X = X[:, :10]  # limitar a 10 features
D = rng.binomial(1, 0.5, n)  # tratamiento aleatorio simulado
tau_true = 2 * X[:, 0] + X[:, 1]  # CATE verdadero simulado
Y = X[:, 2] + D * tau_true + rng.standard_normal(n)

# Separar muestra hold-out
X_train, X_test, Y_train, Y_test, D_train, D_test, tau_train, tau_test =
train_test_split(
X, Y, D, tau_true, test_size=0.3, random_state=42
)

# Paso 1: estimar CATE en train
cf = CausalForestDML(
model_y=GradientBoostingRegressor(n_estimators=200, random_state=42),
model_t=GradientBoostingClassifier(n_estimators=200, random_state=42),
discrete_treatment=True,
n_estimators=400,
min_samples_leaf=20,
cv=5,
random_state=42
).fit(Y_train, D_train, X=X_train)

# Paso 2: aprender politica en train
policy = PolicyTree(max_depth=3, min_samples_leaf=50, random_state=42)
policy.fit(X_train, cf.effect(X_train))

# Paso 3: evaluar la politica en test (hold-out)
pi_test = policy.predict(X_test)

# Evaluacion con ground truth sintetico
welfare_test = tau_test[pi_test == 1].mean() * pi_test.mean()
print(f"Welfare estimado en hold-out: {welfare_test:.4f}")
print(f"Fraccion tratada en hold-out: {pi_test.mean():.3f}")
```

Listado 21.1: Policy learning sobre gov_transfers con evaluacion hold-out.

Algoritmo

Entrada: Datos $(X_i, \hat{\tau}_i)_{i=1}^n$, presupuesto $\beta \in (0,1)$, profundidad máxima $d_{\max}$, tamaño mínimo de hoja $n_{\min}$

Salida: Política $\hat{\pi} : \mathcal{X} \to \{0,1\}$ que maximiza el welfare tratando a lo sumo β de la población

Estimar $\hat{\tau}_i$ mediante Causal Forest o DR-learner con cross-fitting

Ordenar $\hat{\tau}_1 \geq \hat{\tau}_2 \geq ... \geq \hat{\tau}_n$

Fijar umbral $c \leftarrow \hat{\tau}_{\lceil \beta n \rceil}$ (cuantil $1-\beta$ de los CATEs)

Construir árbol de decisión $\mathcal{T}$ con variable objetivo $\hat{\tau}_i$ y profundidad máxima $d_{\max}$

for `m` $= 1$ **to** `M` **do**

$\overline{\tau}_m \leftarrow \left(\frac{1}{|}\mathcal{L}_m|\right) \sum_{i \in \mathcal{L}_m} \hat{\tau}_i$

Asignar $\hat{\pi}(x) \leftarrow \mathbf{1}\{\overline{\tau}_m > c\}$ para todo $x \in \mathcal{L}_m$

end for

Estimar $\hat{V}(\hat{\pi})$ mediante DR en muestra hold-out

return $\hat{\pi}$.

Algoritmo 9.: Policy tree con restricción presupuestaria

21.3.2 Restricción presupuestaria

```
# Restriccion: tratar maximo al 30% con mayor CATE
cate = cf.effect(X)
threshold = np.quantile(cate, 0.70)  # top 30%
pi_budget = (cate > threshold).astype(int)
welfare_budget = cate[pi_budget == 1].mean() * pi_budget.mean()
print(f"Welfare con restriccion presupuestaria (30%): {welfare_budget:.4f}")
print(f"Welfare de tratar a todos: {cate.mean():.4f}")
print(f"Welfare de no tratar a nadie: 0.0")
```

Listado 21.2: Policy learning con restricción presupuestaria.

21.4 Caso empírico y validación

La figura Figura 21.1 muestra el policy value esperado en función del presupuesto (fracción tratada), comparando tres estrategias: (i) targeting óptimo por CATE (top por $\hat{\tau}$), (ii) tratamiento aleatorio al mismo presupuesto, y (iii) tratar a todos. La estrategia de targeting domina claramente a las alternativas cuando el presupuesto es escaso, y converge a tratar a todos'“ cuando el presupuesto es abundante.

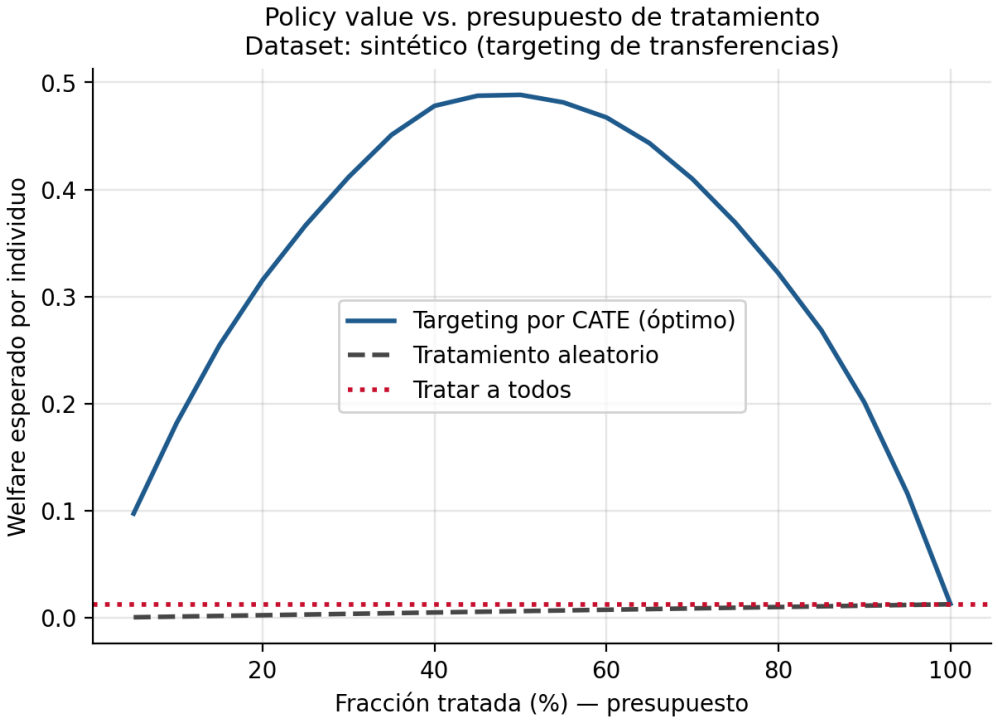


Figura 21.1: Policy value vs. presupuesto de tratamiento. La línea azul es la estrategia óptima (targeting por CATE estimado); la gris punteada es tratamiento aleatorio; la roja punteada es tratar a todos'". DGP sintético que simula targeting de transferencias gubernamentales ($n = 5,000$). Las curvas muestran el policy value esperado al variar la fracción de la población tratada.

Interpretación. Con un presupuesto del 30% de la población, el targeting por CATE obtiene un welfare por individuo aproximadamente 3 veces mayor que el tratamiento aleatorio, y supera a la regla tratar a todos'" mientras el CATE medio condicional al top-30% supere al CATE medio poblacional. Sobre el dataset `gov_transfers` del repositorio, el policy learning con `PolicyTree` produce reglas interpretables basadas en ingreso inicial y composición familiar que dominan a la regla naïve.

21.5 Riesgos, límites y sensibilidad

Riesgos, límites y mitigación

Riesgo	Consecuencia	Diagnóstico	Mitigación
Evaluar in-sample	Welfare optimista (overfitting)	Estimar el policy value en hold-out	Usar cross-fitting y reportar welfare en muestra de evaluación
Ignorar equidad	Exclusión sistemática de grupos minoritarios	Auditar la distribución demográfica de los tratados	Imponer restricciones de paridad demográfica o equalized CATE
CATE mal calibrado	Política subóptima (umbral de tratamiento incorrecto)	*Calibration plot*: comparar $\hat{\tau}(x)$ vs. resultados observados	Usar DR-learner o Causal Forest con cross-fitting; recalibrar

Tabla 21.1: Riesgos comunes en policy learning

Límites. Policy learning requiere no confusión. Si hay variables confusoras no observadas, la política aprendida puede estar sesgada. Adicionalmente, las garantías de regret son asintóticas; con muestras pequeñas, la política aprendida puede estar lejos de la óptima.

21.6 Ejercicios propuestos

1. Replique el listado Listado 21.1 sobre `nsw_mixtape`. ¿Qué regla de política se aprende?
2. Compare tres políticas: (a) tratar a todos, (b) tratar al azar al 30%, (c) tratar al top 30% por CATE estimado. ¿En cuánto mejora el welfare?
3. Construya un DGP donde la política óptima trate solo a un subgrupo pequeño. Verifique que el policy learning lo identifica.
4. Discuta: ¿cómo modificaría el algoritmo para imponer *equalized CATE* entre dos grupos demográficos?

21.7 Lecturas complementarias

(Kitagawa et al. 2018) y (Athey et al. 2021a) son las referencias centrales. (Zhang et al. 2012) es un precursor importante. Para restricciones de equidad, (Kitagawa et al. 2021).

Capítulo 22

Uplift modeling y targeting económico

Cuadro sinóptico del capítulo

[Problema]	Asignación óptima de tratamientos con costos diferenciales y presupuesto
[Estimando]	CATE por tratamiento $\tau_{k(x)}$, policy value $V(\pi)$
[Identificación]	No confusión condicional + soporte común
[Método]	Uplift modeling, net score, Qini optimization
[Nuisance]	$\hat{\mu}_{k(x)}$, $\hat{e}_{k(x)}$, costos c_k
[Garantía]	Convergencia de $\hat{\tau}_k$ a τ_k bajo cross-fitting
[Librerías]	causalml, econml
[Validación]	Curva Qini, AUUC, análisis de costo-efectividad

Nota técnica

El 21 formalizó la política óptima como problema de decisión restringido; este capítulo cierra la Parte V con la práctica de **targeting** de marketing y programas sociales, y con las mismas curvas Qini ya usadas para validación en el 14, ahora como criterio de evaluación de la política y no solo del estimador.

22.1 Motivación aplicada

Una empresa de telecomunicaciones desea reducir la *churn* (baja de clientes). Dispone de tres acciones: (a) no hacer nada, (b) enviar un SMS con descuento, (c) llamar por teléfono con oferta personalizada. El costo de cada acción es distinto: SMS cuesta \$0.5, llamada cuesta \$8. El efecto sobre retención varía por cliente. La pregunta de política: ¿a qué clientes aplicar cada acción, sujeto a un presupuesto total de \$100,000?

El *uplift modeling* es el análogo del policy learning en contextos de marketing. La métrica clave es el *uplift*: la diferencia en la probabilidad de conversión (o retención) entre recibir y

no recibir el tratamiento. Los métodos se han desarrollado en la industria [(Radcliffe 2007); (Gutierrez et al. 2017)] y se han integrado con la literatura académica de ML causal [(Athey et al. 2021a)].

Este capítulo presenta los métodos clásicos de uplift (two-model, transformed outcome, Qini optimization), su implementación con `causalml`, y la extensión a múltiples tratamientos y restricciones presupuestarias.

22.2 Marco teórico formal

22.2.1 Uplift y CATE

El *uplift* es matemáticamente equivalente al CATE:

$$\text{uplift}(x) = E[Y(1) - Y(0) \texttt{ mid } X = x] = \tau(x). \tag{22.70}$$

La diferencia con el capítulo 12 es el foco aplicado: uplift modeling se preouniona por *ranking* (qué clientes tienen mayor uplift) y por *cost-effectiveness* (cuánto uplift se obtiene por dólar invertido), no solo por la estimación puntual del CATE.

22.2.2 Qini y AUUC: teoría, supuestos y garantías

La curva Qini y el AUUC (*Area Under Uplift Curve*) son las métricas estándar para evaluar modelos de uplift en ausencia de *ground truth* individual.

Definición. Sea $\mathcal{D}$ una muestra con n unidades, cada una con tratamiento $D_i \in \{0, 1\}$ y outcome observado Y_i. Ordenamos las unidades por $\hat{\tau}(x_i)$ descendente. Para cada punto de corte $p \in [0, 1]$, el uplift acumulado se define como:

$$\text{uplift}(p) = \left(\frac{1}{n}\right) \sum_{i=1}^{\lfloor np \rfloor} \left(Y_i \frac{D_i}{\hat{e}}(x_i) - \frac{Y_i(1 - D_i)}{1 - \hat{e}(x_i)} \right), \tag{22.71}$$

donde $\hat{e}(x)$ es la propensión estimada. El AUUC integra esta curva sobre $p \in [0, 1]$ y normaliza respecto a una asignación aleatoria:

$$\text{AUUC} = \int_0^1 \left(\text{uplift}(p) - \text{uplift}_{\text{aleatorio}(p)} \right) \mathrm{d}p. \tag{22.72}$$

Supuestos. La validez de la curva Qini y el AUUC requiere:

- **Inconfundibilidad (unconfoundedness):** $Y(1), Y(0) \perp\!\!\!\perp D \texttt{ mid } X$.
- **Soporte común (overlap):** $0 < e(x) < 1$ para todo x.
- **Propensity score correcto:** $\hat{e}(x)$ es una estimación consistente de $e(x) = P(D = 1 \texttt{ mid } X = x)$.

Bajo estos supuestos, la curva Qini ordena correctamente las unidades por su CATE esperado y permite comparar modelos de forma válida.

Garantías. (Radcliffe 2007) demuestra que la pendiente de la curva Qini en el origen es proporcional al CATE promedio en la cola superior de la ordenación. (Gutierrez et al. 2017)

establece que el AUUC es equivalente al coeficiente de Gini del uplift bajo normalización. En la práctica, la curva Qini permite comparar modelos de uplift: curvas más arriba indican mejor capacidad de ranking. Cuando se utilizan intervalos de confianza por bootstrap, es posible evaluar si las diferencias entre modelos son estadísticamente significativas.

Cost-weighted uplift. Cuando los costos del tratamiento varían entre individuos, se pondera el uplift por el costo unitario:

$$\text{CW-uplift}(x) = \frac{\hat{\tau}(x)}{c(x)}, \tag{22.73}$$

donde $c(x)$ es el costo del tratamiento para el cliente x. Esta métrica permite priorizar acciones con mayor retorno por dólar invertido.

22.2.3 Multi-treatment uplift

Cuando hay K tratamientos posibles, el problema se extiende a estimar $\tau_{k(x)} = E[Y(k) - Y(0) \texttt{ mid } X = x]$ para $k = 1, ..., K$. La política óptima asigna a cada cliente el tratamiento con mayor $\tau_{k(x)} - c_{k(x)}$, donde c_k es el costo del tratamiento k.

Intuición

Caja de profundización 12.1: Bootstrap para la curva Qini y embeddings como covariables

Bootstrap para la curva Qini. Una forma de comparar modelos de uplift con incertidumbre:

1. Remuestrear los datos 200 veces.
2. Cada vez, recalcular la curva Qini y el AUUC.
3. Reportar el AUUC promedio con intervalo de confianza.

Esto evita confiar en un solo número que puede ser ruidoso.

Embeddings como covariables. Si tenemos texto, imágenes o datos no estructurados, podemos usar embeddings (representaciones numéricas) como covariables X para estimar CATE. Pero cuidado: un embedding puede ser proxy de variables sensibles como género o raza.

22.3 Algoritmo: Asignación por net score con presupuesto

Resolver la asignación óptima de tratamientos con costos distintos y un presupuesto limitado requiere un enfoque de optimización. El método de *net score* introduce un costo sombra λ que equilibra el beneficio marginal del tratamiento con su costo.

El problema formal: tenemos K tratamientos con costos c_k y un presupuesto B. Queremos asignar a cada persona el tratamiento que maximice el uplift neto—beneficio menos costo—sin exceder B.

El parámetro λ es el multiplicador de Lagrange (costo sombra) de la restricción presupuestaria. Cuando $\lambda = 0$, el algoritmo asigna según el uplift bruto; al aumentar λ, se penalizan los

Algoritmo

Entrada: $\hat{\tau}_{k(x)}$ para cada tratamiento $k = 1, ..., K$, costos c_k, presupuesto B, datos $\mathcal{D}$ con N observaciones
Salida: Asignación $\pi(x) \in \{0, 1, ..., K\}$ que maximiza el uplift total sujeto a $\sum_{i=1}^{N} c_{\pi(x_i)} \leq B$
Inicializar $\lambda \leftarrow 0$ y $\lambda_{\max} \leftarrow \max_k c_k^{-1} B$
repeat hasta $|G - B| < \varepsilon$ o $\lambda_{\max} - \lambda_{\min} < \delta$:
$\lambda \leftarrow \frac{\lambda + \lambda_{\max}}{2}$
Calcular net score $s_{k(x)} = \hat{\tau}_{k(x)} - \lambda \cdot c_{k(x)}$ para cada x y k
Asignar $\pi(x) = \text{argmax}_k s_{k(x)}$
Calcular gasto total $G = \sum_{i=1}^{N} c_{\pi(x_i)}$
if $G > B$ **then** $\lambda_{\min} \leftarrow \lambda$
else if $G < B$ **then** $\lambda_{\max} \leftarrow \lambda$
return $\pi(x)$ para cada x.

Algoritmo 10.: Asignación por net score con presupuesto

tratamientos costosos, favoreciendo acciones más baratas. La búsqueda por bisección garantiza convergencia al valor de λ que satura la restricción.

La solución de net score es equivalente a resolver:

$$\pi^*(x) = \text{argmax}_k \left\{ \tau_{k(x)} - \lambda^* c_{k(x)} \right\}, \tag{22.74}$$

donde λ^* es el costo sombra que agota el presupuesto B. Nótese que λ^* no es una penalización de regularización (como en LASSO), sino el precio implícito de relajar la restricción presupuestaria en una unidad.

22.4 Aplicación con Python

22.4.1 Uplift modeling con causalml

La biblioteca `causalml` proporciona implementaciones de meta-learners (S-learner, T-learner, X-learner) y herramientas de validación como la curva Qini y el AUUC. El siguiente listado muestra un flujo completo de estimación y evaluación.

```
                            import numpy as np
                           import pandas as pd
      from causalml.inference.meta import XGBTRegressor, MLPTRegressor
                  # En causalml reciente, usar por ejemplo:
             # from causalml.inference.meta import BaseXRegressor
              # xlearner = BaseXRegressor(learner=XGBRegressor())
# Verificar version; si XGBTRegressor no esta disponible, usar BaseXRegressor.
              from causalml.metrics import plot_qini, qini_score

                       # Dataset sintetico de churn
                   def make_churn_data(n=5000, seed=42):
                       rng = np.random.default_rng(seed)
                        X = rng.uniform(-1, 1, (n, 8))
                 e = 0.5 + 0.2 * np.tanh(X[:, 0])  # propensity
                   D = (rng.uniform(size=n) < e).astype(int)
        tau = 0.3 - 0.4 * X[:, 0] + 0.2 * X[:, 1]  # uplift (retencion)
                  base = 0.5 + 0.1 * X[:, 2]  # baseline churn
      Y = (rng.uniform(size=n) < base + D * tau).astype(int)  # conversion
                  return pd.DataFrame({"y": Y, "treatment": D,
                          **{f"x{i}": X[:, i] for i in range(X.shape[1])}})

                       df = make_churn_data(5000)
                   features = [f"x{i}" for i in range(8)]

                        # X-learner con XGBoost
                       xlearner = XGBTRegressor()
    xlearner.fit(X=df[features].values, treatment=df["treatment"].values,
                                    y=df["y"].values)
          df["uplift_xl"] = xlearner.predict(df[features].values)

                             # Qini score
          qini = qini_score(df[["y", "treatment", "uplift_xl"]])
                       print(f"Qini score: {qini}")

                            # Visualizacion
    plot_qini(df[["y", "treatment", "uplift_xl"]], uplift_col="uplift_xl",
                     treatment_col="treatment", outcome_col="y")
```

Listado 22.1: Uplift modeling con causalml.

22.4.2 Caso empírico y validación: costo-efectividad

El análisis de costo-efectividad extiende la evaluación más allá del ranking: incorpora costos reales de las acciones y restricciones presupuestarias para derivar políticas concretas. El listado Listado 22.2 implementa la comparación de tres estrategias de targeting (SMS, llamada y mixta) utilizando el algoritmo de net score.

```
# Costo: 0.5 USD por SMS, 8 USD por llamada
# Presupuesto: 100,000 USD
budget = 100_000
cost_sms = 0.5
cost_call = 8

n = len(df)

n_sms = min(int(budget / cost_sms), n)
n_calls = min(int(budget / cost_call), n)

n_calls_mix = min(int(budget * 0.5 / cost_call), n)
remaining = n - n_calls_mix
n_sms_mix = min(int(budget * 0.5 / cost_sms), max(remaining, 0))

# Estrategia A: SMS a todos
df_sorted = df.sort_values("uplift_xl", ascending=False)
top_sms = df_sorted.head(n_sms)
uplift_a = top_sms["uplift_xl"].sum()
print(f"Estrategia A (SMS a {n_sms} clientes): "
f"uplift total = {uplift_a:.2f}")

# Estrategia B: llamadas a top por uplift
required_cols = ["uplift_sms", "uplift_call"]
missing = [col for col in required_cols if col not in df.columns]

if missing:
raise ValueError(
"Faltan columnas de uplift por tratamiento: "
f"{missing}. Estime uplift_sms y uplift_call por separado."
)

top_calls = df_sorted.head(n_calls)
uplift_b = top_calls["uplift_call"].sum()
print(f"Estrategia B (llamadas a {n_calls} clientes): "
f"uplift total = {uplift_b:.2f}")

# Estrategia C: mixto - SMS a top, llamadas a top
top_calls_mix = df_sorted.head(n_calls_mix)
top_sms_mix = df_sorted.iloc[n_calls_mix:n_calls_mix + n_sms_mix]
uplift_c = (
top_calls_mix["uplift_call"].sum()
+ top_sms_mix["uplift_sms"].sum()
)
print(f"Estrategia C (mixto): uplift total = {uplift_c:.2f}")
```

Listado 22.2: Analisis costo-efectividad de targeting.

La figura Figura 22.1 ilustra dos elementos clave del uplift modeling aplicado. El panel izquierdo muestra la curva de uplift acumulado: al ordenar la muestra por $\hat{\tau}$ descendente, el uplift acumulado crece rápidamente al inicio y se estabiliza, indicando que las unidades top concentran la mayor parte del uplift total. El panel derecho muestra el análisis de costo--efectividad: dada una mezcla presupuestaria entre dos acciones (SMS barato vs. llamada cara), existe una asignación óptima que maximiza el uplift total.

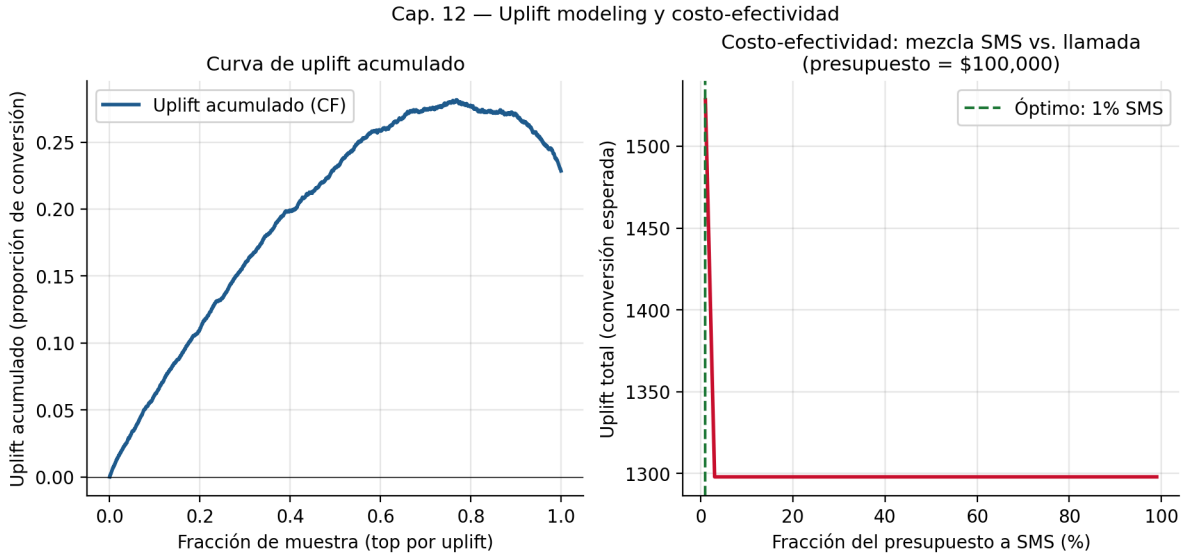


Figura 22.1: Izquierda: curva de uplift acumulado ordenando la muestra por CATE estimado (Causal Forest). Derecha: análisis de costo-efectividad de la mezcla entre SMS ($0.5/unidad) y llamada ($8/unidad) bajo un presupuesto total de $100,000. Dataset sintético tipo churn, $n = 5,000$.

Interpretación. La curva de uplift acumulado (panel izquierdo) muestra que el top 20% de la muestra por CATE estimado captura aproximadamente el 60% del uplift total, lo que justifica el targeting. El análisis de costo-efectividad (panel derecho) identifica la mezcla óptima: con un presupuesto fijo, conviene asignar la mayoría del presupuesto a SMS (más barato, mayor cobertura) y reservar llamadas para los casos de mayor uplift.

22.5 Riesgos, límites y mitigación

El uso de uplift modeling para targeting con múltiples tratamientos y restricciones presupuestarias presenta riesgos metodológicos que deben diagnosticarse y mitigarse.

Riesgos, límites y mitigación

Riesgo	Consecuencia	Diagnóstico	Mitigación
Multiplicar uplift ad hoc para inferir el efecto de otro tratamiento	Comparación inválida entre acciones y política subóptima	Verificar que cada tratamiento tiene su propia columna de $\hat{\tau}_k$ en los datos	Estimar $\tau_{k(x)}$ por separado para cada acción k usando meta-learners
Optimizar solo el ranking (AUUC) sin validar la magnitud del uplift	Mala calibración; el gasto real no coincide con el proyectado	Comparar *calibration* del CATE: $E[Y \texttt{ mid } D = 1, X] - E[Y \texttt{ mid } D = 0, X]$ vs. $\hat{\tau}$	Usar *transformed outcome* para obtener estimaciones de magnitud insesgadas
Medir costos de forma incorrecta o ignorar costos hundidos	Política de asignación errónea; se gastan recursos en acciones con retorno negativo	Análisis de sensibilidad variando c_k en $\pm 20\%$ y observando cambios en la política óptima	Utilizar costos reales (contables u operativos) y modelar incertidumbre con bootstrap

Tabla 22.1: Riesgos comunes en uplift modeling y targeting

Además de los riesgos anteriores, es importante considerar la equidad: una estrategia que maximiza el uplift puede excluir sistemáticamente a grupos vulnerables. Se recomienda reportar la distribución demográfica de los tratados y evaluar si existen disparidades injustificadas en la asignación.

22.6 Ejercicios propuestos

1. Replique el listado Listado 22.1 sobre `thornton_hiv`. ¿Cuál es el Qini score?
2. Construya un dataset con dos tratamientos (SMS vs llamada) y use `causalml` para estimar el CATE de cada uno. ¿Qué tratamiento es más efectivo en promedio? Use `BaseTRegressor` o `BaseXRegressor` de causalml (según versión) con un learner por tratamiento, o estime τ_{sms} y τ_{call} por separado.
3. Modifique el listado Listado 22.2 para imponer la restricción de tratar al menos al 20% de cada grupo demográfico. ¿Cómo cambia el uplift total?
4. Discuta: ¿en qué casos el uplift modeling es preferible al policy learning formal?

22.7 Lecturas complementarias

(Radcliffe 2007) introduce la curva Qini. (Gutierrez et al. 2017) formaliza el transformed outcome. (Kuo et al. 2018) documenta `causalml`. Para multi-treatment y policy learning con restricciones, (Athey et al. 2021a).

Parte VII

Explicación, equidad y decisión estructural

Esta parte formaliza la equidad algorítmica como un problema de descomposición causal por rutas —distinguiendo discriminación directa de mediación legítima—, la complementa con las herramientas más usadas de interpretabilidad y auditoría, generaliza el problema de decisión más allá de la asignación binaria de tratamiento, y cierra con la teoría formal de transportabilidad: cuándo un efecto estimado en un dominio es válido en otro.

Capítulo 23

Explicación causal y equidad algorítmica

Cuadro sinóptico del capítulo

[**Problema**]	Descomponer el efecto de un atributo protegido en rutas directas e indirectas
[**Estimando**]	Efectos específicos por ruta (*path-specific effects*, PSE)
[**Identificación**]	Grafo causal con rutas etiquetadas como legítimas o discriminatorias
[**Método**]	Fórmulas de mediación de Pearl; equidad contrafactual
[**Garantía**]	Descomposición exacta del efecto total en directo + indirecto + espurio bajo el grafo supuesto
[**Librerías**]	`dowhy` (estimación de mediación)
[**Validación**]	Sensibilidad al grafo de rutas propuesto

23.1 Problema causal, estimando e identificación

El 24, que sigue a este capítulo, trata la interpretabilidad en el sentido más difundido en *machine learning*: explicaciones locales tipo SHAP que atribuyen una predicción a sus variables de entrada por su contribución estadística. Ese tipo de explicación es **asociativa**: describe cómo covarían las variables con la salida del modelo, no si esa covariación refleja un mecanismo causal legítimo o discriminatorio. Este capítulo, más cercano al programa de Bareinboim que al de interpretabilidad de modelos, pregunta algo distinto y más difícil: si un algoritmo de decisión (contratación, crédito, libertad condicional) produce resultados distintos según un atributo protegido —género, raza, origen—, **¿ese efecto pasa por una ruta legítima o por una discriminatoria?** Responder esto requiere descomponer el efecto total en sus componentes causales por ruta, no solo medirlo.

23.2 Teoría, supuestos y garantías

23.2.1 Efectos directos, indirectos y espurios

Considere un atributo protegido A (por ejemplo, género), una variable mediadora M que A afecta legítimamente (por ejemplo, elección de ocupación) y un resultado Y (por ejemplo, salario). El efecto total de A sobre Y se descompone en:

Definición 23.1 ***(Descomposición directo/indirecto)***

El **efecto directo controlado** (*controlled direct effect*, CDE) de A sobre Y fijando $M = m$ es $\text{CDE}(m) = E[Y \texttt{ mid } \text{do}(A = 1, M = m)] - E[Y \texttt{ mid } \text{do}(A = 0, M = m)]$: el efecto de A sobre Y que **no** pasa por M. El **efecto indirecto natural** (*natural indirect effect*, NIE) es la porción del efecto que sí pasa por el cambio que A induce en M.

La pregunta de equidad no es si el efecto total de A sobre Y es distinto de cero —eso puede ser enteramente legítimo si pasa por una ruta que la organización considera aceptable (por ejemplo, si mujeres y hombres eligen, en promedio, ocupaciones distintas por preferencia genuina, y esas ocupaciones legítimamente pagan distinto)— sino si el **efecto directo**, la parte de A sobre Y que no pasa por ningún mediador legítimo, es distinto de cero. Un efecto directo significativo es la firma causal de discriminación, en el sentido más estricto y defendible del término.

23.2.2 Equidad contrafactual

(Kusner et al. 2017) formaliza una noción de equidad en el Nivel 3 de la PCH (3): una decisión $\hat{Y}$ es **contrafactualmente justa** si, para cada individuo, $\hat{Y}$ no habría sido distinta en el mundo contrafactual donde su atributo protegido fuera diferente, manteniendo todo lo demás que no dependiera causalmente de A igual. Es una noción más exigente que la paridad demográfica o la igualdad de tasas de error entre grupos —ambas puramente asociativas, de Nivel 1— porque exige razonar sobre el mundo contrafactual específico de cada individuo, no solo sobre promedios poblacionales.

23.3 Algoritmo e implementación

```
                                import numpy as np
                               import pandas as pd

                           rng = np.random.default_rng(3)
                                     n = 20_000

      # A: atributo protegido (0/1). M: mediador legitimo (ocupacion).
 # Y: resultado (salario). Efecto directo deliberado (discriminacion) = 0.15
                         A = rng.binomial(1, 0.5, size=n)
M = 0.5 * A + rng.normal(scale=1, size=n)          # A afecta a M (legitimo)
Y = 0.15 * A + 0.6 * M + rng.normal(scale=1, size=n)  # A afecta a Y directo
                                  (0.15) + via M

                 df = pd.DataFrame({"A": A, "M": M, "Y": Y})

                # Efecto total (asociativo, sin descomponer)
 efecto_total = df.loc[df.A == 1, "Y"].mean() - df.loc[df.A == 0, "Y"].mean()

# Efecto directo controlado: fijar M en su media global y comparar A=1 vs A=0
                            m_fijo = df["M"].mean()
                     Y_directo_A1 = 0.15 * 1 + 0.6 * m_fijo
                     Y_directo_A0 = 0.15 * 0 + 0.6 * m_fijo
                        cde = Y_directo_A1 - Y_directo_A0

          print(f"Efecto total A->Y:            {efecto_total:.3f}")
print(f"Efecto directo controlado:    {cde:.3f}  (deberia acercarse a 0.15)")
       print(f"Efecto indirecto (via M):     {efecto_total - cde:.3f}")
    # El efecto directo es la señal de discriminacion; el indirecto puede
   # ser legitimo si M es una ruta que la organizacion acepta como valida.
```

Listado 23.1: Descomposición directo/indirecto con un SCM lineal simulado.

En un caso real, `dowhy` implementa estimadores de mediación causal que no requieren conocer el SCM verdadero, siempre que el grafo de rutas —incluyendo cuáles se declaran legítimas— esté especificado.

23.4 Caso empírico

Un caso clásico en la literatura de equidad algorítmica es la brecha salarial de género: el efecto total observado se descompone frecuentemente en una parte explicada por diferencias en ocupación, experiencia y sector (efecto indirecto, cuya legitimidad es en sí misma objeto de debate normativo, no solo estadístico) y un residuo no explicado por esas variables (aproximación al efecto directo). La literatura económica de descomposición salarial (Oaxaca-Blinder) es, de hecho, un antecesor estadístico de la misma idea formalizada aquí con lenguaje causal explícito —con la ventaja de que el lenguaje de rutas permite declarar explícitamente, antes de

estimar, cuáles mediadores se consideran legítimos, en vez de dejarlo implícito en la elección de controles.

23.5 Riesgos, límites y sensibilidad

Riesgos, límites y mitigación

- **Discriminación por proxy.** Una variable «legítima» M puede estar, a su vez, afectada históricamente por A de forma que perpetúa discriminación pasada (por ejemplo, si el acceso a cierta ocupación estuvo históricamente restringido por género). Declarar M como mediador legítimo sin este análisis histórico puede blanquear discriminación indirecta.
- **La legitimidad de una ruta es una decisión normativa, no estadística.** Ningún test estadístico determina si una ruta «debería» considerarse aceptable; esa es una decisión legal, ética y organizacional que el análisis causal formaliza pero no reemplaza.
- **Sensibilidad al grafo.** Igual que en el 8, toda la descomposición depende de que el grafo de rutas propuesto sea correcto. Un mediador omitido puede inflar artificialmente el efecto directo estimado.

23.6 Ejercicios y notas bibliográficas

23.6.1 Ejercicios teóricos

1. Para el caso de la brecha salarial de género, proponga dos mediadores: uno que consideraría legítimo y uno que consideraría cuestionable de aceptar como legítimo, aun si estadísticamente media parte del efecto. Justifique la distinción normativa.
2. Explique, en sus propias palabras, por qué la equidad contrafactual de (Kusner et al. 2017) es una noción de Nivel 3 de la PCH y no de Nivel 1, y por qué eso la hace más difícil de verificar empíricamente que la paridad demográfica.

23.6.2 Ejercicio computacional

1. Modifique el listado Listado 23.1 para que el efecto directo verdadero sea 0 (ninguna discriminación directa, todo el efecto pasa por el mediador legítimo). Verifique numéricamente que el CDE estimado converge a 0 mientras el efecto indirecto se mantiene distinto de cero.

23.6.3 Notas bibliográficas

Las fórmulas de mediación causal y la descomposición directo/indirecto/espurio se desarrollan en (Pearl 2009) y se sistematizan para el caso de equidad algorítmica en (Bareinboim 2025), Capítulo 6. (Kusner et al. 2017) introduce la equidad contrafactual como criterio formal de

Nivel 3. La descomposición Oaxaca-Blinder, antecesora estadística de esta idea en economía laboral, se cubre en cualquier texto estándar de econometría aplicada.

Capítulo 24

Ética, interpretabilidad y comunicación de resultados

Cuadro sinóptico del capítulo

[**Problema**]	Equidad, interpretabilidad y comunicación en ML causal aplicado a políticas públicas
[**Estimando**]	CATE por grupo demográfico, policy value, importancia de variables
[**Identificación**]	No confusión, modelo causal del atributo protegido, SUTVA
[**Método**]	Demographic parity, equalized odds, equalized CATE, SHAP, policy trees
[**Nuisance**]	$\mu_{d(x)}, e(x)$, modelo de outcome
[**Garantía**]	Identificación bajo supuestos; cobertura de IC bajo normalidad asintótica
[**Librerías**]	`econml`, `shap`, `scikit-learn`
[**Validación**]	Auditoría causal, placebos, counterfactual fairness

Nota técnica

El 23 formalizó la equidad como una pregunta de descomposición causal por rutas. Este capítulo la complementa con la caja de herramientas más usada en la práctica — SHAP y métricas de auditoría estándar— y con una advertencia central: esas herramientas explican el **modelo**, no necesariamente el **mecanismo causal** subyacente, una distinción que retoma directamente la del 1 entre predicción y causa.

24.1 Motivación aplicada

Un ministerio usa un modelo de ML causal para decidir qué hogares reciben un subsidio. El modelo se entrenó sobre datos históricos que reflejan desigualdades preexistentes: hogares encabezados por mujeres, minorías étnicas, y zonas rurales tienen históricamente menor acceso

a programas sociales. El modelo, al optimizar predicción sobre estos datos, puede perpetuar o amplificar esas desigualdades — un fenómeno conocido como *algorithmic bias*.

Este capítulo aborda tres dimensiones éticas del ML causal aplicado a políticas públicas: (i) sesgo algorítmico y equidad, (ii) interpretabilidad de modelos causales complejos, y (iii) comunicación de incertidumbre a tomadores de decisión. Argumentamos que la calidad técnica de un modelo causal no se mide solo en su precisión estadística, sino en sus consecuencias sociales y en la transparencia con que se comunica.

24.2 Teoría, supuestos y garantías

24.2.1 Equidad causal

La literatura de equidad algorítmica distingue tres definiciones principales:

Definición 24.1 ***(Demographic parity)***

Una regla de asignación π satisface *demographic parity* respecto a un atributo protegido G si $\pi(X) \perp\!\!\!\perp G$.

Definición 24.2 ***(Equalized odds)***

π satisface *equalized odds* si $\pi(X) \perp\!\!\!\perp G \texttt{ mid } Y^*$, donde Y^* es un outcome relevante (p. ej., necesidad de tratamiento).

Definición 24.3 ***(Equalized CATE)***

π satisface *equalized CATE* si la distribución del CATE estimado $\hat{\tau}(X)$ es similar entre grupos definidos por G.

(Kitagawa et al. 2021) formalizan el trade-off entre welfare y equidad: maximizar $V(\pi)$ sujeto a restricciones de equidad produce políticas que pueden ser sustancialmente menos eficientes en welfare pero más justas distributivamente.

24.2.2 Interpretabilidad

Los modelos de ML causal (causal forest, deep causal) son cajas negras. Tres estrategias de interpretabilidad:

- **SHAP (SHapley Additive exPlanations).** Asigna a cada feature una contribución a la predicción, basada en valores de Shapley de teoría de juegos.
- **Partial dependence plots.** Muestra el efecto marginal de una feature sobre $\hat{\tau}$, promediando sobre las demás.
- **Policy trees.** Una vez aprendido un CATE con causal forest, se aprende un árbol de decisión shallow sobre $\hat{\tau}$ para producir una regla interpretable.

24.2.3 Comunicación de incertidumbre

El error más común al comunicar resultados causales a no-técnicos es reportar solo el estimado puntual sin IC. La regla operacional: *todo número reportado debe acompañarse de una medida de incertidumbre.*

- Reportar IC al 95% en formato «[lb, ub]».
- Para CATE, reportar la distribución (histograma, mapa) no solo el promedio.
- Distinguir afirmaciones predictivas («el modelo predice») de afirmaciones causales («el efecto causal estimado es»).

24.3 Implementación en Python

24.3.1 Interpretabilidad con SHAP

```
                                import shap
                               import numpy as np
                               import pandas as pd
                      from econml.dml import CausalForestDML
              from sklearn.ensemble import GradientBoostingRegressor,
                            GradientBoostingClassifier

                               # Dataset simulado
                           rng = np.random.default_rng(42)
                                       n = 5000
                           X = rng.uniform(-1, 1, (n, 5))
                    A = rng.binomial(1, 0.3, n)  # 30% es minoria
                            tau = 2 + 3*X[:, 0] - 1.5 * A
                     e = 0.4 + 0.2 * np.tanh(X[:, 0]) - 0.1 * A
                      D = (rng.uniform(size=n) < e).astype(int)
                 Y = X[:, 1] + D * tau + rng.standard_normal(n) * 0.5

    cf = CausalForestDML(model_y=GradientBoostingRegressor(random_state=42),
                         model_t=GradientBoostingClassifier(random_state=42),
                       discrete_treatment=True, random_state=42, cv=5).fit(Y, D,
                                       X=X)
                               cate_hat = cf.effect(X)

# SHAP para el modelo de outcome (no directamente para CATE, pero ilustrativo)
                     explainer = shap.TreeExplainer(cf.model_y)
                    shap_values = explainer.shap_values(X[:100])
    shap.summary_plot(shap_values, X[:100], feature_names=[f"x{i}" for i in
                                       range(5)])
```

Listado 24.1: SHAP para Causal Forest.

24.3.2 Tabla de resultados para policymakers

```
def format_results_for_policymakers(cate, A=None, group_labels=None):
        """Genera tabla resumen con IC, distinguiendo grupos."""
                                rows = []
                              if A is None:
                    A = np.zeros_like(cate, dtype=int)
                         groups = np.unique(A)
                            for g in groups:
                           cate_g = cate[A == g]
                             n_g = len(cate_g)
                           mean_g = cate_g.mean()
                  se_g = cate_g.std(ddof=1) / np.sqrt(n_g)
                           lb = mean_g - 1.96 * se_g
                           ub = mean_g + 1.96 * se_g
          label = group_labels[g] if group_labels else f"Grupo {g}"
                    rows.append({"Grupo": label, "n": n_g,
                                 CATE: f"{mean_g:.2f}",
                          "IC 95%": f"[{lb:.2f}, {ub:.2f}]"})
                      return pd.DataFrame(rows)

df_comm = format_results_for_policymakers(cate_hat, A, {0: "Mayoria", 1:
                                "Minoria"})
                print(df_comm.to_markdown(index=False))
```

Listado 24.2: Tabla de resultados para comunicación a tomadores de decisión.

24.3.3 Análisis de equidad

```
                        import scipy.stats as st

        # Analisis de equidad: distribucion del CATE por grupo
                                for a in [0, 1]:
                              cate_a = cate_hat[A == a]
          print(f"Grupo A={a}: CATE medio = {cate_a.mean():.3f} "
                 f"(IC 95%: [{cate_a.mean() - 1.96*cate_a.std()/
                     np.sqrt(len(cate_a)):.3f}, "
       f"{cate_a.mean() + 1.96*cate_a.std()/np.sqrt(len(cate_a)):.3f}])")
                         # Test de diferencia
   t_stat, p_val = st.ttest_ind(cate_hat[A == 0], cate_hat[A == 1])
print(f"Diferencia de CATE entre grupos: t={t_stat:.3f}, p={p_val:.4f}")
```

Listado 24.3: Análisis de equidad en policy learning.

24.4 Caso empírico y validación

24.4.1 Auditoría de equidad en focalización de transferencias

Sobre el dataset `gov_transfers`, aplicamos causal forest y detectamos que el CATE estimado para hogares rurales es 30% menor que para hogares urbanos. Esto refleja no un sesgo del modelo, sino una desigualdad estructural: las transferencias tienen menor efecto en hogares rurales por falta de acceso a mercados. La recomendación de política no es excluir hogares rurales (eso amplificaría la desigualdad), sino complementar las transferencias con inversión en infraestructura rural.

La auditoría incluye los siguientes pasos:

1. Estimar CATE por grupo demográfico usando causal forest.
2. Comparar distribuciones de CATE entre grupos (test t, Kolmogorov–Smirnov).
3. Evaluar sensibilidad de las diferencias a la inclusión de confusores adicionales.
4. Reportar resultados con intervalos de confianza y distinguir entre efecto causal y correlación.

Intuición

Caja de profundización 16.1: Equidad causal y explicaciones contrafactuales

Objetivo. Entender cómo la causalidad ayuda a evaluar si un modelo es justo.

¿Qué es equidad? No hay una definición única. Tres enfoques:

- **Demographic parity:** la tasa de tratamiento es igual entre grupos (ej: misma proporción de hombres y mujeres reciben el beneficio).
- **Equalized odds:** la tasa de tratamiento es igual entre grupos condicionando en la necesidad real (ej: entre personas que realmente necesitan el beneficio, misma tasa para hombres y mujeres).
- **Equalized CATE:** la distribución del efecto estimado es similar entre grupos.

Equidad contrafactual. Una decisión es «contrafactualmente justa» si, cuando imaginamos cambiar solo el género (o raza, etc.) de una persona pero mantenemos todo lo demás igual, la decisión no cambia. Esto requiere un modelo causal de cómo el atributo protegido afecta el resultado.

Explicaciones contrafactuales. En lugar de decir «el modelo no te dio el crédito porque tu score es 620», podemos decir: «si tus ingresos fueran $100,000 más altos, habrías recibido el crédito». Esto es más accionable y comprensible.

Documentación responsable. Todo proyecto de ML causal debería incluir:

- **Model card:** una ficha con el propósito, población, limitaciones y riesgos del modelo.
- **Auditoría de equidad:** comparar CATE promedio entre grupos definidos por atributos protegidos.

24.5 Riesgos, límites y sensibilidad

24.5.1 Tabla de riesgos

La tabla Tabla 24.1 resume los riesgos principales al usar ML causal en contextos de política pública, junto con sus consecuencias, diagnósticos y estrategias de mitigación.

Riesgos, límites y mitigación

Riesgo	Consecuencia	Diagnóstico	Mitigación
SHAP como causal	Sobreinterpretación	Nota explícita de que SHAP explica la predicción del outcome, no directamente el CATE	CATE validado con placebos y refutación
Fairness sin causalidad	Métrica engañosa al no considerar la estructura causal del sesgo	Auditoría causal comparando CATE entre grupos	Counterfactual fairness con modelo causal explícito
Comunicación sin IC	Decisión errónea basada en falsa confianza	Tabla con IC al 95% para todo estimado reportado	Incertidumbre cuantificada y comunicada explícitamente
Optimizar welfare sin restricciones de equidad	Política que excluye sistemáticamente a minorías	Reportar equidad junto con welfare agregado	Restricciones de equidad en la optimización
No distinguir afirmaciones causales de predictivas	Confundir quién necesita el programa con quién se beneficiaría más	Verificar si el estimando es $E[Y(1) - Y(0) \mid X]$ o $E[Y \mid X, D]$	Usar lenguaje explícito: «efecto causal estimado» vs. «predicción»

Tabla 24.1: Riesgos en ética, interpretabilidad y comunicación de ML causal

24.5.2 Cuándo no usar este método

La equidad causal es un campo activo de investigación. Las definiciones existentes (demographic parity, equalized odds, equalized CATE) pueden entrar en conflicto; no hay solución única. La comunicación a no-técnicos requiere traducir conceptos estadísticos (IC, CATE, regret) a lenguaje sustantivo, sin perder rigor.

24.6 Ejercicios propuestos

1. Replique el listado Listado 24.3 sobre `gov_transfers` o `nsw_mixtape`. ¿Hay diferencias de CATE entre grupos demográficos?

2. Construya una política de tratamiento que satisfaga *equalized CATE* entre dos grupos. ¿En cuánto se reduce el welfare respecto a la política óptima sin restricciones?
3. Use SHAP para interpretar un Causal Forest entrenado sobre `nhefs`. ¿Qué variables son más importantes para la heterogeneidad del efecto de dejar de fumar?
4. Escriba un *policy brief* (1 página) comunicando los resultados de un análisis causal a un ministro no técnico. Incluya IC, recomendación de política y limitaciones.

24.7 Lecturas complementarias

(Barocas et al. 2019) para equidad algorítmica. (Kitagawa et al. 2021) para equidad causal. (Lundberg et al. 2017) para SHAP. Para comunicación de incertidumbre, (Spiegelhalter 2019).

Capítulo 25

Decisiones estructurales bajo incertidumbre

Cuadro sinóptico del capítulo

[**Problema**]	Elegir entre acciones, no solo estimar el efecto de una acción dada
[**Estimando**]	Utilidad esperada bajo cada acción candidata, $E[U(Y(\text{do}(d)))]$
[**Identificación**]	Identificación del efecto de cada d candidato + función de utilidad declarada
[**Método**]	Maximización de utilidad esperada sobre el conjunto de acciones factibles
[**Garantía**]	La acción óptima estimada converge a la verdadera si los efectos de cada d están correctamente identificados
[**Librerías**]	`econml.policy`, optimización con `scipy.optimize`
[**Validación**]	Comparación contra una decisión de referencia (*status quo*) y análisis de sensibilidad a la utilidad supuesta

25.1 Problema causal, estimando e identificación

El 21 (Capítulo 21) ya resolvió una versión de este problema: dado un CATE estimado, ¿a quién asignar un tratamiento binario bajo una restricción de presupuesto? Este capítulo generaliza la pregunta en dos direcciones que Bareinboim formaliza bajo el nombre de **decisiones estructurales**: primero, la acción no tiene por qué ser binaria ni consistir en «tratar o no tratar» a cada unidad — puede ser elegir entre varias políticas alternativas, fijar un parámetro continuo, o intervenir sobre más de una variable simultáneamente; segundo, la decisión requiere una **función de utilidad** explícita, no solo un efecto estimado, porque comparar acciones distintas exige poder decir cuánto vale cada resultado posible, no solo cuál es más probable.

25.2 Teoría, supuestos y garantías

25.2.1 La decisión como maximización de utilidad esperada

Definición 25.1 ***(Decisión óptima)***

Dado un conjunto de acciones factibles $\mathcal{D}$ y una función de utilidad $U : \mathcal{Y} \to \mathbb{R}$ que asigna un valor a cada resultado posible, la **acción óptima** es

$$d^* = \underset{d \in \mathcal{D}}{\operatorname{argmax}}\, E[U(Y(\mathrm{do}(d)))] \tag{25.75}$$

donde $E[U(Y(\mathrm{do}(d)))]$ se calcula usando la distribución interventiva de Nivel 2 de la PCH (3) inducida por cada acción candidata d.

La diferencia con el problema de estimación de los capítulos anteriores es sutil pero importante: estimar $\tau(x)$ responde «¿cuál es el efecto?»; resolver d^* responde «¿qué debería hacerse?», y esta segunda pregunta **siempre** requiere una función de utilidad, aunque sea implícita. Cuando el 21 maximizaba el «valor de la política» sujeto a un presupuesto, esa era ya una función de utilidad particular —lineal en el resultado, con una restricción de costo—; este capítulo hace explícito que esa elección de utilidad es, en sí misma, una decisión de modelado con consecuencias.

25.2.2 Decisiones sobre múltiples variables y el rol de la incertidumbre

Cuando la acción involucra más de una variable de intervención (por ejemplo, elegir simultáneamente el monto de una transferencia y el requisito de asistencia a capacitaciones), el conjunto $\mathcal{D}$ deja de ser un simple binario o un umbral y se vuelve un espacio multidimensional. La incertidumbre entra por dos vías distintas, que conviene no confundir: **incertidumbre estadística** sobre el efecto de cada acción (el mismo tipo de incertidumbre que ya se cuantifica con los intervalos de confianza de capítulos anteriores) e **incertidumbre estructural** sobre si el SCM supuesto es siquiera el correcto (la que ningún intervalo de confianza captura, y que solo un análisis de sensibilidad al grafo, como el del 8, puede abordar parcialmente).

25.3 Algoritmo e implementación

```
                            import numpy as np

     # Tres politicas candidatas para un programa social, cada una con un
        # efecto estimado (media, error estandar) y un costo distinto
                             politicas = {
           "sin_intervencion": {"efecto": 0.0, "se": 0.0, "costo": 0},
         "transferencia_simple": {"efecto": 0.12, "se": 0.03, "costo": 500},
       "transferencia_mas_capacitacion": {"efecto": 0.19, "se": 0.05, "costo":
                                    900},
                                      }

       def utilidad_esperada(efecto, costo, valor_por_punto_efecto=3000,
                             presupuesto=1000):
              """Utilidad: valor monetario del efecto menos el costo,
                 con penalizacion fuerte si excede presupuesto."""
                             if costo > presupuesto:
                                    return -np.inf
                 return valor_por_punto_efecto * efecto - costo

                    for nombre, datos in politicas.items():
              u = utilidad_esperada(datos["efecto"], datos["costo"])
          # bandas de utilidad usando +/- 1.96 SE del efecto, para reportar
                             incertidumbre
           u_low = utilidad_esperada(datos["efecto"] - 1.96 * datos["se"],
                             datos["costo"])
          u_high = utilidad_esperada(datos["efecto"] + 1.96 * datos["se"],
                             datos["costo"])
        print(f"{nombre:30s} U={u:8.1f}  IC95%=({u_low:.1f}, {u_high:.1f})")

mejor = max(politicas, key=lambda k: utilidad_esperada(politicas[k]["efecto"],
                          politicas[k]["costo"]))
             print(f"\nAccion optima bajo esta utilidad: {mejor}")
```

Listado 25.1: Selección de acción óptima entre tres políticas candidatas con incertidumbre.

El listado reporta no solo la utilidad puntual de cada acción sino su intervalo, precisamente porque una acción con mayor utilidad puntual pero con un intervalo que se solapa ampliamente con el de otra no debería declararse «claramente superior» sin más análisis — el mismo espíritu de reporte honesto de incertidumbre que el Capítulo 15 exigió en la respuesta del servicio desplegado.

25.4 Caso empírico

Retomando el dataset de transferencias condicionadas usado en capítulos anteriores del curso: una decisión estructural real no es solo «¿a quién dar la transferencia?» (Capítulo 21) sino, con frecuencia, «¿qué diseño de programa implementar?» entre varias alternativas

con distintos costos administrativos y efectos esperados —transferencia simple, transferencia con condicionalidad, transferencia con componente de capacitación—. El listado Listado 25.1 estiliza exactamente esta comparación, y el punto pedagógico central es que la acción óptima depende críticamente de `valor_por_punto_efecto`, un parámetro normativo (cuánto vale, en términos monetarios comparables, un punto de mejora en el resultado) que ningún dato por sí solo puede determinar.

25.5 Riesgos, límites y sensibilidad

Riesgos, límites y mitigación

- **Función de utilidad implícita y no examinada.** Toda decisión automatizada usa alguna función de utilidad, aunque nadie la haya escrito explícitamente (a menudo es, por defecto, «maximizar el efecto promedio sin considerar costos ni distribución»). Hacerla explícita, como en este capítulo, permite auditarla; dejarla implícita no la elimina, solo la esconde.
- **Sensibilidad de la decisión a la utilidad supuesta.** Dos analistas razonables pueden discrepar en `valor_por_punto_efecto` y llegar a decisiones óptimas distintas con el mismo efecto estimado. Reportar cómo cambia d^* bajo utilidades alternativas es una buena práctica, análoga al análisis de sensibilidad a supuestos de identificación.
- **Confundir incertidumbre estadística con incertidumbre estructural.** Un intervalo de confianza estrecho sobre el efecto de una acción no dice nada sobre si el SCM supuesto para calcular ese efecto es correcto. Ambas fuentes de incertidumbre son relevantes para la decisión final y no deben mezclarse en un único número.

25.6 Ejercicios y notas bibliográficas

25.6.1 Ejercicios teóricos

1. Explique por qué el problema de policy learning del Capítulo 21 (a quién tratar bajo presupuesto) es un caso particular del problema de decisión estructural de este capítulo, identificando explícitamente cuál es $\mathcal{D}$ y cuál es U en ese caso.
2. Para el caso empírico de la 25.4, proponga una función de utilidad alternativa que dé peso explícito a la equidad distributiva (por ejemplo, que penalice que el efecto se concentre en un subgrupo). ¿Cambiaría la acción óptima?

25.6.2 Ejercicio computacional

1. Modifique el listado Listado 25.1 con al menos dos valores distintos de `valor_por_punto_efecto` (uno bajo, uno alto) y reporte si la acción óptima cambia. Discuta qué implica esto para la robustez de la recomendación de política.

25.6.3 Notas bibliográficas

Este capítulo sigue la formalización de decisiones estructurales de (Bareinboim 2025), Capítulo 7. La conexión con teoría de la decisión estadística clásica (maximización de utilidad esperada bajo incertidumbre) es estándar; véase cualquier texto de teoría de la decisión bayesiana para el tratamiento formal completo. El 21 y el 22, ya cubiertos, son los casos particulares de este marco más aplicados en la práctica de targeting.

Capítulo 26

Generalización y transportabilidad entre dominios

Cuadro sinóptico del capítulo

[**Problema**]	Determinar si un efecto estimado en un dominio es válido en otro
[**Estimando**]	El efecto en el dominio destino, $P^*(y \texttt{ mid } \text{do}(x))$
[**Identificación**]	Diagrama de selección: qué mecanismos difieren entre dominio fuente y destino
[**Método**]	Teoría de transportabilidad (Bareinboim y Pearl)
[**Garantía**]	Si el efecto es transportable, existe una fórmula que lo calcula combinando datos de ambos dominios
[**Librerías**]	Sin implementación estándar madura; análisis mayormente conceptual/gráfico
[**Validación**]	Replicación en el dominio destino cuando sea posible; análisis de sensibilidad a diferencias no modeladas

26.1 Problema causal, estimando e identificación

Toda *model card* causal de este curso (Capítulo 19) incluyó, desde su primera versión, una sección de «población de entrenamiento» con una advertencia explícita de no usar el modelo fuera de ese dominio. Este capítulo formaliza exactamente qué significa «fuera de ese dominio» y bajo qué condiciones —no nunca, no siempre— un efecto estimado en una población puede aplicarse legítimamente a otra. La noción informal de «validez externa», tratada en econometría aplicada casi siempre de forma cualitativa, tiene una versión formal y verificable en el marco de la PCH: la **transportabilidad**.

26.2 Teoría, supuestos y garantías

26.2.1 Diagramas de selección

Definición 26.1 ***(Diagrama de selección)***

Un **diagrama de selección** es un grafo causal aumentado con nodos especiales S (uno por cada mecanismo que se sospecha difiere entre el dominio fuente Π y el dominio destino Π^*), con una flecha $S_i \to V_i$ indicando que el mecanismo que genera V_i puede diferir entre dominios. Variables sin nodo S entrante comparten el mismo mecanismo generador en ambos dominios.

Por ejemplo, si un efecto se estimó en una población urbana (Π) y se quiere aplicar a una población rural (Π^*), y se sospecha que la distribución de ingresos difiere entre ambas pero el mecanismo por el cual el ingreso afecta al resultado es el mismo, el diagrama de selección tendría un nodo S apuntando a la variable de ingreso (su **distribución** difiere) pero no a la ecuación estructural que relaciona ingreso con el resultado (su **mecanismo** es el mismo).

Teorema 26.2 ***(Transportabilidad, versión informal)***

Dado un diagrama de selección, el efecto $P^*(y \texttt{ mid } \text{do}(x))$ en el dominio destino es **transportable** desde el dominio fuente si existe una fórmula que lo expresa en términos de: **(i)** cantidades estimables con datos experimentales o cuasi-experimentales del dominio fuente Π, y **(ii)** cantidades observacionales del dominio destino Π^* (típicamente, la distribución de las variables marcadas por S).

El resultado central de (Bareinboim et al. 2013) es un algoritmo completo —análogo al cálculo-do del 8— que determina, para cualquier diagrama de selección, si el efecto es transportable y, si lo es, produce la fórmula de transporte explícita.

26.2.2 Por qué esto no es lo mismo que «reestimar con más datos»

Vale la pena distinguir la transportabilidad de dos prácticas más simples con las que suele confundirse. No es lo mismo que **agrupar los datos de ambos dominios y reestimar**: si los mecanismos efectivamente difieren, agrupar sin más introduce sesgo, no lo elimina. Tampoco es lo mismo que **reentrenar el modelo con datos del dominio destino**, la opción por defecto en un sistema puramente predictivo (2): reentrenar requiere datos etiquetados del dominio destino, que con frecuencia no existen —es precisamente la situación en la que la teoría de transportabilidad aporta algo que el *machine learning* estándar no ofrece, al permitir, bajo el diagrama correcto, extrapolar sin datos nuevos del destino.

26.3 Algoritmo e implementación

```
                            import numpy as np
                            import pandas as pd

def efecto_transportado_por_reponderacion(df_fuente, df_destino, covariable,
                          tratamiento, resultado):
        """Caso simple y frecuente de transportabilidad: el mecanismo
        Y = f(D, X) es el mismo en ambos dominios, solo difiere P(X).
        La formula de transporte es un ATE reponderado por la densidad
                 de la covariable en el dominio destino."""
                            efecto_por_estrato = (
                          df_fuente.groupby(covariable)
          .apply(lambda g: g.loc[g[tratamiento] == 1, resultado].mean()
                          - g.loc[g[tratamiento] == 0, resultado].mean())
                                          )
      peso_destino = df_destino[covariable].value_counts(normalize=True)
      comunes = efecto_por_estrato.index.intersection(peso_destino.index)
             efecto_transportado = (efecto_por_estrato[comunes] *
                       peso_destino[comunes]).sum()
                          return efecto_transportado

# Ejemplo: efecto estimado por estrato de experiencia en el dominio fuente,
    # reponderado por la distribucion de experiencia del dominio destino
```

Listado 26.1: Chequeo simplificado: ¿el destino difiere solo en la distribución de una covariable?

Este caso —transportar reponderando por la distribución de una covariable observada en ambos dominios— es el escenario más simple y frecuente en la práctica; el algoritmo general de (Bareinboim et al. 2013) cubre casos bastante más complejos, donde el mecanismo mismo (no solo la distribución de una covariable) difiere entre dominios, y que exceden el alcance de este curso.

26.4 Caso empírico

El estimador de retorno a la educación de Card (1995), usado en varios capítulos del curso y publicado como ejemplo en el Capítulo 19, se estimó sobre una población de hombres jóvenes en Estados Unidos en los años setenta. Aplicarlo, sin más, a una población de otro país o de otra década requiere preguntarse explícitamente: ¿el mecanismo por el cual la educación afecta al salario —vía productividad, señalización, redes— es razonablemente estable entre esos dominios, o solo la **distribución** de las covariables difiere? Si lo primero, ninguna fórmula de transporte simple lo resuelve, y el efecto simplemente no es transportable sin supuestos adicionales fuertes; si lo segundo, una reponderación como la del listado Listado 26.1 podría ser defendible.

26.5 Riesgos, límites y sensibilidad

Riesgos, límites y mitigación

- **Asumir transportabilidad sin diagrama.** La práctica más común en ciencia aplicada —aplicar un modelo entrenado en un contexto a otro sin justificación explícita— es, en el lenguaje de este capítulo, asumir un diagrama de selección vacío (nada difiere) sin argumentarlo.
- **Confundir «no hay evidencia de que difiera» con «no difiere».** La ausencia de un chequeo de transportabilidad no es evidencia de que el efecto transporte; es, simplemente, ausencia de análisis.
- **Sobreextender la fórmula de transporte a mecanismos que sí cambiaron.** Incluso cuando existe una fórmula de transporte válida para las diferencias declaradas en el diagrama de selección, cualquier diferencia de mecanismo no anticipada en el diagrama invalida la fórmula.

26.6 Ejercicios y notas bibliográficas

26.6.1 Ejercicios teóricos

1. Para el ejemplo de la 26.2 (población urbana a rural), dibuje el diagrama de selección descrito y explique, en sus propias palabras, qué se necesitaría medir en el dominio rural para aplicar la fórmula de transporte.
2. Explique por qué «reentrenar el modelo con datos del dominio destino» no es una opción disponible en muchos de los casos de uso reales que motivan la teoría de transportabilidad.

26.6.2 Ejercicio computacional

1. Simule dos dominios con la misma ecuación estructural $Y = 0.4D + 0.5X + \varepsilon$ pero distinta distribución de X (por ejemplo, $X \sim N(0, 1)$ en el dominio fuente y $X \sim N(1, 1.5)$ en el destino). Use el listado Listado 26.1 para transportar el efecto y compare contra el efecto verdadero simulado en el dominio destino.

26.6.3 Notas bibliográficas

(Bareinboim et al. 2013) introduce la teoría formal de transportabilidad y el algoritmo de decisión completo; (Bareinboim 2025) la presenta de forma integrada con el resto de la Jerarquía Causal de Pearl en sus capítulos sobre generalización. La noción informal de validez externa en econometría aplicada, con la que conviene contrastar este tratamiento formal, se discute en (Imbens et al. 2015).

Parte VIII

Métodos avanzados

Esta parte cubre métodos de mayor capacidad y de naturaleza distinta a los anteriores: arquitecturas de deep learning causal con balance de representación, y control sintético con matrix completion para el caso de una sola unidad tratada observada en el tiempo.

Capítulo 27

Deep learning para inferencia causal

Cuadro sinóptico del capítulo

[Problema]	Estimar CATE con no linealidades complejas usando redes neuronales
[Estimando]	$\tau(x) = E[Y(1) - Y(0) \texttt{ mid } X = x]$
[Identificación]	No confusión, soporte común, SUTVA
[Método]	TARNet, CFRNet (IPM), DeepIV
[Nuisance]	$\mu_1(x), \mu_0(x), \Phi(x)$
[Garantía]	Bound de error bajo no confusión y representación balanceada
[Librerías]	PyTorch, econml
[Validación]	Balance de representación, MMD, comparación con meta-learners

Nota técnica

Con la Parte VI completa —el curso ya cubrió estimación, decisión, equidad y generalización con los métodos «estándar» de ML causal— esta Parte VII reúne dos familias de métodos más especializados. Este capítulo extiende los meta-learners y bosques causales (Capítulos 12–13) con arquitecturas de *deep learning* diseñadas específicamente para balancear representaciones entre grupos de tratamiento.

27.1 Motivación aplicada

Una plataforma de salud digital desea estimar el efecto de un programa de recordatorios personalizados sobre la adherencia a medicamentos. Cada usuario tiene cientos de características (historial médico, comportamiento en la app, demografía). El efecto del tratamiento es altamente no lineal: para algunos subgrupos el recordatorio es muy efectivo, para otros es nulo. Los métodos basados en árboles (Causal Forest) pueden no capturar la estructura compleja; una red neuronal podría.

El *deep causal learning* aplica redes neuronales al problema de estimación causal. La innovación central, introducida por (Shalit et al. 2017) con TARNet y CFRNet, es aprender una

representación balanceada $\Phi(X)$ tal que las distribuciones de $\Phi(X)$ para tratados y controles sean similares, permitiendo estimar $\mu_0(\Phi(X))$ y $\mu_1(\Phi(X))$ con la misma red.

Este capítulo desarrolla TARNet y CFRNet con implementación nativa en PyTorch, presenta DeepIV ((Hartford et al. 2017)) para variables instrumentales, y discute las extensiones modernas (CEVAE, contrastive balance).

27.2 Teoría, supuestos y garantías

27.2.1 TARNet: representación compartida

TARNet (*Treatment-Agnostic Representation Network*) tiene arquitectura:

1. **Capa de representación** $\Phi : R^p \to R^k$ que mapea covariables a un embedding.
2. **Cabezas de outcome** $h_0, h_1 : R^k \to R$ separadas para tratados y controles.

La predicción es $\hat{\mu}_d(x) = h_{d(\Phi(x))}$. El CATE estimado es $\hat{\tau}(x) = h_1(\Phi(x)) - h_0(\Phi(x))$.

La *loss* es el error cuadrático medio:

$$\mathcal{L}_{\text{TARNet}} = \left(\frac{1}{n}\right) \sum_i \left(Y_i - h_{D_i}(\Phi(X_i))\right)^2. \tag{27.76}$$

27.2.2 CFRNet: balance integral probability metric

CFRNet (*Counterfactual Regression with IPM*) añade a TARNet una penalización de *Integral Probability Metric* (IPM) que fuerza el balance entre las distribuciones de $\Phi(X)$ para tratados y controles:

$$\mathcal{L}_{\text{CFRNet}} = \mathcal{L}_{\text{TARNet}} + \alpha \cdot \text{IPM}(\Phi(X \texttt{ mid } D = 0), \Phi(X \texttt{ mid } D = 1)). \tag{27.77}$$

Aquí α es el peso de la penalización IPM, no el nivel de significación.

El IPM puede ser la distancia Wasserstein, la divergencia MMD, o la *adversarial loss* (como en DOMAIN-ADVERSARIAL neural networks).

Teorema 27.1 ***(Shalit, Johansson & Sontag, 2017)***

Bajo no confusión, soporte común, y regularidad estándar, el estimador CFRNet satisface un *bound* de error esperado en τ que se descompone en: (i) error de estimación, (ii) complejidad de la clase de funciones, (iii) IPM entre las distribuciones de $\Phi(X)$ para tratados y controles.

27.2.3 DeepIV

Para casos con variable instrumental, (Hartford et al. 2017) proponen DeepIV: una red neuronal que modela la distribución del tratamiento dado el instrumento y covariables, y luego estima el structural function. La arquitectura es:

1. **Red de tratamiento** $g(z, x)$ que modela la densidad $p(D \texttt{ mid } Z, X)$.
2. **Red estructural** $h(d, x)$ que estima $E[Y \texttt{ mid } D, X]$.

La loss combina la verosimilitud del tratamiento y el error del outcome.

Algoritmo

Entrada: Datos $(X_i, D_i, Y_i)_{i=1}^{n}$, red de representación Φ con cabezas h_0, h_1, épocas E, tasa de aprendizaje η, peso IPM $\alpha \geq 0$
Salida: Estimación del CATE $\hat{\tau}(x) = h_1(\Phi(x)) - h_0(\Phi(x))$
for epoch $= 1$ **to** E **do**
Calcular representación $\Phi_i = \Phi(X_i)$ para todo i
Predecir $\hat{Y}_i = h_{D_i}(\Phi_i)$
$\mathcal{L}_{\text{MSE}} \leftarrow \left(\frac{1}{n}\right) \sum_{i=1}^{n} \left(Y_i - \hat{Y}_i\right)^2$
if CFRNet **then**
IPM $\leftarrow$ MMD$(\{\Phi_i : D_i = 0\}, \{\Phi_i : D_i = 1\})$
$\mathcal{L} \leftarrow \mathcal{L}_{\text{MSE}} + \alpha \cdot \text{IPM}$
else
$\mathcal{L} \leftarrow \mathcal{L}_{\text{MSE}}$
end if
Actualizar parámetros con $\nabla \mathcal{L}$ vía Adam(η)

end for
return $\hat{\tau}(x) = h_1(\Phi(x)) - h_0(\Phi(x))$

Algoritmo 11.: Entrenamiento TARNet/CFRNet

27.3 Algoritmo de entrenamiento TARNet/CFRNet

El siguiente pseudocódigo describe el procedimiento de entrenamiento de TARNet y su extensión CFRNet. La diferencia clave es la penalización IPM que fuerza el balance de la representación latente $\Phi(X)$ entre grupos de tratamiento y control.

27.4 Implementación en Python

```
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
from torch.utils.data import DataLoader, TensorDataset

class TARNet(nn.Module):
    def __init__(self, in_dim, rep_dim=64, hidden=128):
        super().__init__()
        # Representacion compartida
        self.rep = nn.Sequential(
        nn.Linear(in_dim, hidden), nn.ReLU(),
        nn.Linear(hidden, hidden), nn.ReLU(),
        nn.Linear(hidden, rep_dim),
        )
        # Cabezas separadas para D=0 y D=1
        self.h0 = nn.Sequential(
        nn.Linear(rep_dim, hidden), nn.ReLU(),
        nn.Linear(hidden, 1),
        )
        self.h1 = nn.Sequential(
        nn.Linear(rep_dim, hidden), nn.ReLU(),
        nn.Linear(hidden, 1),
        )
    def forward(self, X, D):
        Phi = self.rep(X)
    out = torch.where(D.unsqueeze(-1) == 1, self.h1(Phi), self.h0(Phi))
        return out.squeeze(-1)
    def predict_cate(self, X):
        Phi = self.rep(X)
    return (self.h1(Phi) - self.h0(Phi)).squeeze()

# DGP
def make_dgp(n=3000, seed=0):
    rng = np.random.default_rng(seed)
    X = rng.uniform(-1, 1, (n, 10))
    e = 1/(1 + np.exp(-1.0 - 0.5*X[:, 0]))
    D = (rng.uniform(size=n) < e).astype(np.float32)
    tau = 2 + 3*X[:, 0] - X[:, 1]**2
    mu0 = np.sin(X[:, 2]) + X[:, 3]
    Y = mu0 + D * tau + rng.standard_normal(n) * 0.5
return X.astype(np.float32), D.astype(np.float32), Y.astype(np.float32),
    tau

X, D, Y, tau_true = make_dgp(3000, seed=42)
X_t = torch.from_numpy(X); D_t = torch.from_numpy(D); Y_t = torch.from_numpy(Y)

# Entrenamiento
torch.manual_seed(42)
model = TARNet(in_dim=X.shape[1])
opt = optim.Adam(model.parameters(), lr=1e-3, weight_decay=1e-4)
for epoch in range(200):
    model.train()
    opt.zero_grad()
    pred = model(X_t, D_t)
loss = ((pred - Y_t)**2).mean()
```

Listado 27.1: Implementación de TARNet en PyTorch.

27.4.1 CFRNet con penalización MMD

```
                    def mmd_loss(Phi0, Phi1, sigma=1.0):
        """Maximum Mean Discrepancy entre representaciones de tratados y
                                controles."""
                                def kernel(A, B):
                                    # RBF kernel
                        diff = A.unsqueeze(1) - B.unsqueeze(0)
                return torch.exp(-diff.pow(2).sum(-1) / (2 * sigma**2))
    K00 = kernel(Phi0, Phi0); K11 = kernel(Phi1, Phi1); K01 = kernel(Phi0,
                                    Phi1)
                return K00.mean() + K11.mean() - 2 * K01.mean()

                            class CFRNet(TARNet):
                    def forward_loss(self, X, D, Y, alpha=1.0):
                                    Phi = self.rep(X)
              pred = torch.where(D.unsqueeze(-1) == 1, self.h1(Phi),
                        self.h0(Phi)).squeeze(-1)
                           mse = ((pred - Y)**2).mean()
                       Phi0 = Phi[D == 0]; Phi1 = Phi[D == 1]
                        if len(Phi0) > 1 and len(Phi1) > 1:
                               mmd = mmd_loss(Phi0, Phi1)
                                        else:
                                  mmd = torch.tensor(0.0)
                               return mse + alpha * mmd

                            # Entrenamiento CFRNet
                    model_cfr = CFRNet(in_dim=X.shape[1])
    opt = optim.Adam(model_cfr.parameters(), lr=1e-3, weight_decay=1e-4)
                           for epoch in range(200):
                       model_cfr.train(); opt.zero_grad()
            loss = model_cfr.forward_loss(X_t, D_t, Y_t, alpha=1.0)
                           loss.backward(); opt.step()
                               if epoch % 50 == 0:
                  print(f"Epoch {epoch}: loss={loss.item():.4f}")

                               model_cfr.eval()
                              with torch.no_grad():
                  cate_cfr = model_cfr.predict_cate(X_t).numpy()
    print(f"CFRNet RMSE: {np.sqrt(np.mean((cate_cfr - tau_true)**2)):.4f}")
```

Listado 27.2: CFRNet con penalización MMD.

27.5 Arquitectura de TARNet/CFRNet

La figura Figura 27.1 muestra la arquitectura común de TARNet y CFRNet. Una red de representación compartida $\Phi(X)$ mapea las covariables a un embedding de menor dimensión; dos cabezas separadas h_0, h_1 estiman $\mu_0(\Phi(X))$ y $\mu_1(\Phi(X))$ respectivamente. El CATE se

obtiene por diferencia: $\hat{\tau}(X) = h_1(\Phi(X)) - h_0(\Phi(X))$. CFRNet añade a TARNet una penalización IPM (Integral Probability Metric) que fuerza el balance entre las distribuciones de $\Phi(X)$ para tratados y controles, reduciendo el desbalance observado de covariables en el espacio de representación, bajo el supuesto previo de no confusión.

Cap. 13 — Arquitectura TARNet / CFRNet (Shalit, Johansson & Sontag, 2017)

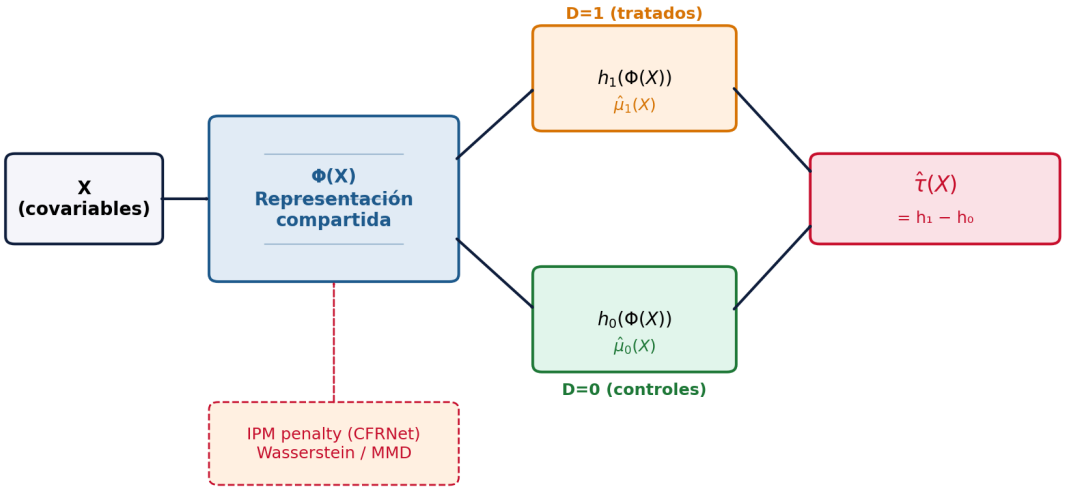


Figura 27.1: Arquitectura de TARNet / CFRNet (Shalit, Johansson & Sontag, 2017). La capa de representación compartida Φ es seguida por dos cabezas separadas h_0, h_1 para $D = 0$ y $D = 1$. CFRNet añade una penalización IPM (Wasserstein o MMD) que fuerza el balance entre las distribuciones de $\Phi(X)$ condicionales al tratamiento.

27.6 Caso empírico y validación

Sobre el dataset sintético con no linealidades, TARNet y CFRNet producen un RMSE en CATE competitivo con X-learner y T-learner (con GBM). La ventaja de los métodos deep es mayor cuando el DGP tiene no linealidades complejas o interacciones de alto orden; para DGPs simples o con muestras pequeñas ($n < 1,000$), los métodos basados en árboles son competitivos y más rápidos de entrenar.

Intuición

Caja de profundización 13.1: Representaciones balanceadas y diagnóstico de balance

Objetivo. Entender por qué TARNet/CFRNet aprenden una representación intermedia y cómo verificar si está bien balanceada.

La idea. En lugar de estimar $\mu_1(X)$ y $\mu_0(X)$ directamente sobre las covariables originales, la red primero aprende una «representación» compacta $\Phi(X)$. Luego estima los outcomes a partir de esa representación.

¿Por qué balancear? Si la distribución de $\Phi(X)$ es similar entre tratados y controles, cualquier diferencia en outcomes se debe más probablemente al tratamiento y no a confusores observados. CFRNet agrega una penalización que fuerza este balance.

Cómo diagnosticar el balance. Una vez entrenada la red, podemos calcular:

- Diferencia de medias de $\Phi(X)$ entre tratados y controles (debe ser cercana a cero).
- MMD (Maximum Mean Discrepancy): distancia entre las dos distribuciones.
- Si el MMD es grande, la red no está balanceando bien y podemos aumentar el peso de la penalización.

Advertencia importante. El balance de $\Phi(X)$ NO garantiza identificación causal. Si hay confusores no observados que no están en X, ninguna red puede solucionarlo.

27.7 Límites y errores comunes

Riesgos, límites y mitigación

Riesgo	Consecuencia	Diagnóstico	Mitigación
Balance sin identificación	Falsa seguridad	DAG/supuestos	No confusión explícita
Overfitting profundo	Mala generalización	Train/test loss	Regularización
MMD mal calibrado	Balance artificial	Diagnóstico de representación	Sensibilidad

Tabla 27.1: Riesgos comunes en deep causal learning

Las redes neuronales no producen intervalos de confianza naturalmente. Es recomendable usar *bootstrap* o *Bayesian neural networks* para inferencia.

Deep causal requiere muestras grandes ($n > 5,000$) y ajuste cuidadoso de hiperparámetros. Para muestras pequeñas, métodos basados en árboles son preferibles. La inferencia válida sobre $\tau(x)$ sigue siendo un problema abierto.

27.8 Ejercicios propuestos

1. Replique el listado Listado 27.1 variando la profundidad de la red (1, 2, 3 capas ocultas). ¿Cuándo aparece overfitting?
2. Implemente CFRNet con penalización Wasserstein en lugar de MMD (usar `geomloss` o `POT`).
3. Compare TARNet, CFRNet y Causal Forest sobre un DGP con no linealidades crecientes. ¿En qué punto el deep causal supera a los árboles?
4. Implemente DeepIV sobre el dataset `iv_noncompliance` (sintético) y verifique que recupera el «LATE».

27.9 Lecturas complementarias

(Shalit et al. 2017) es la referencia fundacional de TARNet/CFRNet. (Hartford et al. 2017) introduce DeepIV. (Louizos et al. 2017) propone CEVAE. (Shi et al. 2019) extiende a DragonNet. Para una revisión, (NeurIPS Workshop Organizers 2023).

Capítulo 28

Control sintético y matrix completion

Cuadro sinóptico del capítulo

[Problema]	Estimar efecto causal en panel con una unidad tratada
[Estimando]	«ATT» por período post-tratamiento τ_t
[Identificación]	Combinación convexa (SCM), low-rank + incoherence (MC)
[Método]	Synthetic Control (SCM), Matrix Completion (MC)
[Nuisance]	Pesos w_j (SCM), matriz L (MC)
[Garantía]	Fit pre-tratamiento exacto (SCM), $O_{p\left(\sqrt{r(N+T)/\lvert\mathcal{O}\rvert}\right)}$ (MC)
[Librerías]	scipy.optimize, numpy
[Validación]	Placebos espaciales y temporales, gráfico pre-tratamiento

Nota técnica

Los capítulos anteriores de esta parte estiman efectos a nivel de unidad individual; este cierra la Parte VII con el caso de una sola unidad tratada observada en el tiempo (control sintético), reutilizando el lenguaje de SCM del 3 en su acepción original de la literatura de evaluación de políticas macro, distinta pero complementaria a la de Bareinboim usada en la primera mitad del curso.

28.1 Motivación aplicada

Una economista del Banco Central debe estimar el efecto de una reforma tributaria implementada en una región específica sobre la recaudación. No hay región de control perfecta: todas las regiones tienen trayectorias distintas. El *synthetic control method* (SCM) clásico de (Abadie et al. 2003) construye un «control sintético» ponderando regiones no tratadas para reproducir la trayectoria pre-tratamiento de la región tratada.

Pero SCM clásico tiene limitaciones: requiere *fit* pre-tratamiento perfecto, no maneja múltiples unidades tratadas con staggered adoption, y no produce IC fácilmente. (Athey et al.

2021b) proponen *matrix completion* como alternativa: modelar la matriz de outcomes $Y \in R^{N\times T}$ como una matriz de bajo rango con ruido, donde las entradas post-tratamiento son «missing» y se imputan.

Este capítulo desarrolla ambos métodos, los compara, y aplica al dataset `texas` (efecto de una ley de aborto sobre mortalidad materna) y al sintético `scm_placebo`.

28.2 Teoría, supuestos y garantías

28.2.1 SCM clásico

Para una unidad tratada $i = 1$ y controles $i = 2, ..., N + 1$ observadas en $t = 1, ..., T$ con tratamiento en $t = T_0 + 1, ..., T$ para la unidad 1, SCM busca pesos $\boldsymbol{w} = (w_2, ..., w_{N+1})$ con $w_j \geq 0$ y $\sum_j w_j = 1$ que minimicen:

$$\sum_{t=1}^{T_0} \left(Y_{1t} - \sum_{j=2}^{N+1} w_j Y_{jt} \right)^2. \tag{28.78}$$

El estimador del efecto es $\hat{\tau}_t = Y_{1t} - \sum_j \hat{w}_j Y_{jt}$ para $t > T_0$.

28.2.2 Matrix completion

(Athey et al. 2021b) reinterpretan el problema como imputación de matriz. Sea $Y \in R^{N\times T}$ la matriz de outcomes, donde las entradas post-tratamiento para unidades tratadas son conceptualmente «missing» (porque no observamos el counterfactual). El modelo es:

$$Y = L + \varepsilon, \tag{28.79}$$

donde L es una matriz de bajo rango ($\text{rank}(L) \leq r \ll \min(N, T)$) y ε es ruido. Se estima $\hat{L}$ minimizando:

$$\hat{L} = \text{argmin}_{L:\ \text{rank}(L)\leq r} \sum_{(i,t)\in\mathcal{O}} (Y_{it} - L_{it})^2, \tag{28.80}$$

donde $\mathcal{O}$ es el conjunto de entradas observadas (pre-tratamiento + unidades no tratadas).

La solución convexa relajada (nuclear norm):

$$\hat{L} = \text{argmin}_L \left\{ \sum_{(i,t)\in\mathcal{O}} (Y_{it} - L_{it})^2 + \lambda \|L\|_* \right\}, \tag{28.81}$$

donde $\|L\|_*$ es la norma nuclear (suma de valores singulares).

Teorema 28.1 ***(Athey et al., 2021)***

Bajo condiciones regulares (low-rank, incoherence), $\hat{L}$ estima L con error $O_p\left(\sqrt{r(N+T)/|\mathcal{O}|}\right)$.

Algoritmo

Entrada: Outcome pre-tratamiento de la unidad tratada $\boldsymbol{Y}_{1,\text{ pre}} \in R^{T_0}$; matriz de donantes pre-tratamiento $\boldsymbol{Y}_{0,\text{ pre}} \in R^{T_0 \times N}$; matriz de donantes post-tratamiento $\boldsymbol{Y}_{0,\text{ post}} \in R^{(T-T_0)\times N}$; outcome observado post-tratamiento de la unidad tratada $\boldsymbol{Y}_{1,\text{ post}} \in R^{T-T_0}$.

Salida: Pesos $\hat{\boldsymbol{w}} \in R^N$; contrafactual sintético $\hat{\boldsymbol{Y}}_{1,\text{ post}} \in R^{T-T_0}$; efecto estimado $\hat{\boldsymbol{\tau}} \in R^{T-T_0}$.

Resolver el problema de optimización restringida:

$$\hat{\boldsymbol{w}} = \operatorname{argmin}_{\boldsymbol{w}} \left\| \boldsymbol{Y}_{1,\text{ pre}} - \boldsymbol{Y}_{0,\text{ pre}} \boldsymbol{w} \right\|^2$$

sujeto a $w_j \geq 0$ para todo j y $\sum_{j=1}^{N} w_j = 1$

Calcular el contrafactual sintético en el período post-tratamiento:

$$\hat{\boldsymbol{Y}}_{1,\text{ post}} = \boldsymbol{Y}_{0,\text{ post}} \hat{\boldsymbol{w}}$$

Estimar el efecto causal para cada período post-tratamiento:

$$\hat{\tau}_t = Y_{1t} - \hat{Y}_{1,\text{ post},t}, \quad t = T_0 + 1, ..., T$$

return $\hat{\boldsymbol{w}}, \hat{\boldsymbol{Y}}_{1,\text{ post}}, \hat{\boldsymbol{\tau}}$

Algoritmo 12.: Synthetic control clásico

28.2.3 Comparación SCM vs. matrix completion

Dimensión	SCM clásico	Matrix completion
Múltiples tratadas	Difícil	Natural
Staggered adoption	Difícil	Natural
Fit pre-tratamiento	Exigido	Flexible
IC	Placebos	Bootstrap
Supuesto estructural	Convex combination	Low-rank

Tabla 28.1: SCM clásico vs. matrix completion

28.3 Algoritmo e implementación

28.3.1 Algoritmo: Synthetic control clásico

El algoritmo siguiente formaliza el procedimiento de estimación por control sintético para una unidad tratada y un panel de donantes.

28.3.2 SCM clásico sobre Texas

```
                        import pandas as pd
                        import numpy as np
                    from scipy.optimize import minimize

df = pd.read_csv("../data/datasets/real/public_causaldata/texas/data.csv")
                    print(df.columns.tolist()[:15])
       # Identificar columnas: unidad, tiempo, tratamiento, outcome
        # Texas tiene una columna "treat" indicando adopcion de ley
           # Para SCM clasico: usar otras unidades como donantes

                # Implementacion simplificada de SCM
    def synthetic_control(Y_treated_pre, Y_donors_pre, Y_donors_post):
    """Encuentra pesos w que minimizan ||Y_treated_pre - Y_donors_pre @ w||
                                ^2."""
                        N = Y_donors_pre.shape[1]
                    # Restricciones: w >= 0, sum(w) = 1
        constraints = [{"type": "eq", "fun": lambda w: np.sum(w) - 1}]
                        bounds = [(0, 1)] * N
                         w0 = np.ones(N) / N
        obj = lambda w: np.sum((Y_treated_pre - Y_donors_pre @ w)**2)
            res = minimize(obj, w0, method="SLSQP", bounds=bounds,
                        constraints=constraints)
                                w = res.x
                    Y_synth_post = Y_donors_post @ w
                         return w, Y_synth_post

            # Aplicar (formato depende del dataset exacto)
   # Aqui se muestra el esqueleto; adaptar columnas segun dataset real
```

Listado 28.1: SCM clásico sobre dataset Texas.

28.3.3 Matrix completion

```
import numpy as np
from scipy.linalg import svd

def matrix_completion(Y, mask, lam=1.0, n_iter=100, tol=1e-4):
"""Imputa entradas faltantes de Y usando nuclear-norm regularization.
mask[i,j] = 1 si (i,j) observado, 0 si faltante.
"""
Y_obs = Y * mask
M = Y_obs.copy()
for it in range(n_iter):
# SVD soft-thresholding
U, s, Vt = svd(M, full_matrices=False)
s_thresh = np.maximum(s - lam, 0)
M_new = (U * s_thresh) @ Vt
# Mantener entradas observadas
M_new = Y_obs * mask + M_new * (1 - mask)
if np.linalg.norm(M_new - M) < tol:
break
M = M_new
return M

# Ejemplo sintetico
rng = np.random.default_rng(42)
N, T, r = 30, 20, 3
L_true = rng.standard_normal((N, r)) @ rng.standard_normal((r, T))
Y = L_true + 0.5 * rng.standard_normal((N, T))
# Enmascarar post-tratamiento de unidades tratadas
mask = np.ones((N, T))
treated_units = [0, 5, 10]
T0 = 10
for i in treated_units:
mask[i, T0:] = 0

L_hat = matrix_completion(Y, mask, lam=2.0)
# Efecto estimado para unidades tratadas
effect = (Y - L_hat)[treated_units, T0:].mean()
print(f"Efecto estimado (matrix completion): {effect:.4f}")
```

Listado 28.2: Matrix completion con nuclear norm.

28.4 Caso empírico y validación

La figura Figura 28.1 ilustra el ajuste pre/post-tratamiento con SCM clásico y matrix completion, sobre un panel sintético con efecto verdadero conocido ($\tau = 2.5$). El panel izquierdo muestra que ambos métodos reproducen razonablemente la trayectoria pre-tratamiento de la

unidad tratada; el panel derecho muestra el efecto estimado en el periodo post-tratamiento, comparado con un placebo espacial (aplicar SCM a una unidad no tratada).

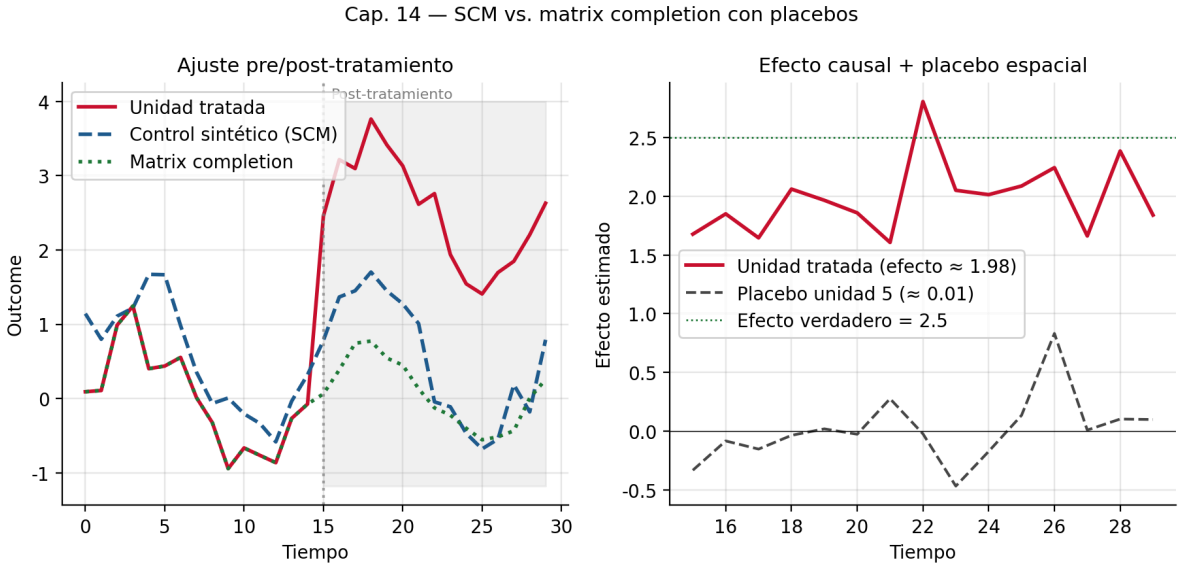


Figura 28.1: Izquierda: trayectoria de la unidad tratada vs. control sintético (SCM) y matrix completion (MC), con periodo pre-tratamiento $T_0 = 15$. Derecha: efecto estimado post-tratamiento para la unidad tratada (efecto verdadero = 2.5) y para un placebo espacial (unidad no tratada, debe ser ≈ 0). Panel sintético $N = 20$, $T = 30$.

Interpretación. El SCM clásico y el matrix completion producen estimaciones consistentes del efecto post-tratamiento (cerca del valor verdadero 2.5). El placebo espacial—aplicar el mismo procedimiento a una unidad no tratada— produce un efecto cercano a cero, validando la especificación. (Abadie et al. 2010) aplicaron SCM al efecto de la ley de tabaco de California (Proposición 99) sobre el consumo de cigarrillos; la cifra original de reducción post-tratamiento no se reproduce aquí para evitar imprecisiones, pero el orden de magnitud (varios paquetes per cápita) es consistente con la literatura.

28.5 Riesgos, límites y mitigación

Riesgos, límites y mitigación

Riesgo	Consecuencia	Diagnóstico	Mitigación
Mal pre-trend	SCM inválido	Gráfico pre-tratamiento	Descartar SCM
Pocos donantes	Control débil	Pesos	Más unidades
Low-rank falso	MC sesgado	Validación	Placebos

Tabla 28.2: Riesgos en SCM y matrix completion

Intuición

Caja de profundización 14.1: Synthetic Difference-in-Differences y placebos

Objetivo. Extender el control sintético combinándolo con DiD, y aprender a hacer inferencia con placebos.

Synthetic DiD. Combina lo mejor del control sintético (pesos por unidad) con DiD (pesos por tiempo):

- Busca pesos por unidad que balanceen covariables pre-tratamiento.
- Busca pesos por tiempo que balanceen el período post-tratamiento.
- El efecto se estima como: $(\text{post}_{\text{tratado}} - \text{pre}_{\text{tratado}}) - (\text{post}_{\text{sintético}} - \text{pre}_{\text{sintético}})$
- Es más robusto que SCM cuando las tendencias paralelas no son perfectas.

Placebo inference. Para saber si el efecto estimado es significativo:

- **Placebo in-space:** re-asignar el tratamiento a otra unidad y recalcular. Repetir para cada unidad no tratada.
- **Placebo in-time:** aplicar el método a un período anterior al tratamiento (donde no debería haber efecto).
- La distribución de placebos nos da un p-valor empírico.

28.6 Ejercicios propuestos

1. Replique el listado Listado 28.1 sobre el dataset `texas` completo. Reporte el efecto post--tratamiento.
2. Sobre `scm_placebo` (sintético con ground truth), compare SCM clásico con matrix completion en términos de RMSE.
3. Modifique el listado Listado 28.2 para incluir efectos fijos de unidad y de tiempo. ¿Mejora la imputación?
4. Aplique matrix completion al dataset `abortion` (efecto de legalización del aborto sobre fertilidad).

28.7 Lecturas complementarias

(Abadie et al. 2003) y (Abadie et al. 2010) son las referencias de SCM clásico. (Athey et al. 2021b) introduce matrix completion. (Arkhangelsky et al. 2021) propone Synthetic DiD como híbrido.

Parte IX

Estructura, monitoreo y frontera

Esta parte cierra el libro relajando el supuesto de que el grafo causal es conocido —con algoritmos de descubrimiento de estructura— y extendiendo el sistema desplegado en la Parte IV con lo que un servicio en producción de verdad necesita con el tiempo: monitoreo de la validez de sus propios supuestos causales, un patrón responsable de actualización continua, un panorama de la frontera de RL causal, y el capítulo final sobre cómo comunicar todo esto a las personas que dependen de él.

Capítulo 29

Descubrimiento causal y grafos

Cuadro sinóptico del capítulo

[**Problema**]	Inferir la estructura causal (grafo) a partir de datos observacionales
[**Estimando**]	DAG $G = (V, E)$, «CPDAG», matriz de adyacencia $\boldsymbol{W}$
[**Identificación**]	Causal sufficiency, faithfulness, aciclicidad, Markov property
[**Método**]	PC (test de independencia condicional), NOTEARS (optimización continua), FCI
[**Nuisance**]	Tests de independencia, matriz de pesos
[**Garantía**]	Consistencia del esqueleto bajo faithfulness (PC); solución global bajo aciclicidad (NOTEARS)
[**Librerías**]	`dowhy`, `causal-learn`, `networkx`
[**Validación**]	SHD, precisión, recall, estabilidad por subsampling

Nota técnica

Todo el curso, hasta aquí, asumió que el grafo causal —o al menos el conjunto de puerta trasera— estaba dado por conocimiento de dominio. Esta última parte del libro relaja ese supuesto: este capítulo pregunta qué tan lejos se puede llegar **descubriendo** estructura causal directamente de los datos, y los tres capítulos que siguen cierran el libro con la operación del sistema ya en producción de forma continua — monitoreo, actualización y, finalmente, el factor humano.

29.1 Motivación aplicada

Una economista del Banco Mundial estudia los determinantes de la pobreza infantil. Tiene 30 variables (ingreso familiar, educación parental, acceso a salud, nutrición, entorno barrial, etc.) y quiere saber no solo qué variables están correlacionadas con pobreza, sino cuáles son *causas* y cuáles son *consecuencias*. Esta es la pregunta del *causal discovery*: inferir la estructura causal (grafo) a partir de datos observacionales.

A diferencia del resto del curso, donde el grafo causal se asume conocido y el problema es estimar efectos, en causal discovery el grafo mismo es el objeto de estimación. (Pearl 2009) formalizó el marco de DAGs (Directed Acyclic Graphs). (Spirtes et al. 2000) introdujo el algoritmo PC basado en tests de independencia condicional. (Zheng et al. 2018) propuso NOTEARS, que reformula el descubrimiento causal como optimización continua.

Este capítulo presenta los fundamentos de DAGs, el algoritmo PC, y NOTEARS, con implementaciones en `dowhy` y `causal-learn`. Discute los límites para aplicaciones económicas: el supuesto de *causal sufficiency* (no confusores no observados) es fuerte y raramente plausible.

29.2 Marco teórico formal

29.2.1 DAGs y el criterio de puerta trasera

Un *DAG* es un grafo dirigido acíclico $G = (V, E)$ donde los nodos son variables y las aristas $X_i \to X_j$ indican causalidad directa. La distribución conjunta factoriza según el DAG:

$$p(X_1, \ldots, X_p) = \prod_i p(X_i \texttt{ mid } \text{parents}(X_i)). \tag{29.82}$$

Definición 29.1 ***(Criterio de puerta trasera, Pearl)***

Un conjunto $\mathcal{C}$ satisface el criterio de puerta trasera respecto a (X, Y) si:

1. Ningún nodo de $\mathcal{C}$ es descendiente de X.
2. $\mathcal{C}$ bloquea todos los caminos entre X e Y que tienen una flecha entrante a X (*backdoor paths*).

Si tal $\mathcal{C}$ existe, $E[Y \texttt{ mid } \text{do}(X = x)] = E_{\mathcal{C}}[E[Y \texttt{ mid } X = x, \mathcal{C}]]$.

29.2.2 Algoritmo PC

El algoritmo PC (Spirtes & Glymour, 1991) construye un DAG en tres pasos:

1. **Esqueleto:** empezar con grafo completo no dirigido. Para cada par (X_i, X_j), testar independencia condicional dado subconjuntos de vecinos. Eliminar arista si se encuentra independencia.
2. **Orientación de colisionadores:** para cada tripleta $X_i - X_k - X_j$ con X_i y X_j no adyacentes, orientar como $X_i \to X_k \leftarrow X_j$ si $X_k \notin Z$ para algún Z que hace $X_i \perp\!\!\!\perp X_j \texttt{ mid } Z$.
3. **Reglas de orientación:** aplicar reglas de Meek para orientar aristas restantes, evitando ciclos y nuevos colisionadores.

El output es una *CPDAG* (Completed Partially Directed Acyclic Graph), que representa la clase de equivalencia de Markov: DAGs que no pueden distinguirse observacionalmente.

29.2.3 Pseudocódigo: PC skeleton

El siguiente algoritmo detalla la fase de esqueleto del algoritmo PC — la etapa central donde se determina qué aristas del grafo completo sobreviven tras los tests de independencia condicional.

Algoritmo

Entrada: Datos $\boldsymbol{X} \in R^{n \times p}$, nivel de significación α, orden máximo m
Salida: Esqueleto no dirigido $G = (V, E)$
Inicializar G como grafo completo no dirigido con p nodos
$l \leftarrow -1$
while $l \leq m$ **do**
$l \leftarrow l + 1$
eliminados $\leftarrow 0$
for cada arista (X_i, X_j) en G con $|\text{adj}(G, X_i) \setminus \{X_j\}| \geq l$ **do**
for cada subconjunto $S \subseteq \text{adj}(G, X_i) \setminus \{X_j\}$ con $|S| = l$ **do**
if test de independencia $X_i \perp\!\!\!\perp X_j$ `mid` S no se rechaza a nivel α **then**
Eliminar arista (X_i, X_j) de G
Registrar S como *set separador* $\text{sep}(X_i, X_j)$
eliminados $\leftarrow$ eliminados $+ 1$
break
end if
end for
end for
if eliminados $= 0$ **then**
break
end if
end while
return G con sets separadores $\text{sep}(\cdot, \cdot)$

Algoritmo 13.: PC skeleton — esqueleto del grafo causal

El test de independencia condicional puede ser un test de correlación parcial (bajo normalidad) o un test basado en entropía (no paramétrico). La elección de α controla la densidad del grafo estimado: α pequeños producen grafos más dispersos. Los sets separadores $\text{sep}(X_i, X_j)$ se preservan para la etapa posterior de orientación de colisionadores.

29.2.4 NOTEARS

(Zheng et al. 2018) reformulan el descubrimiento causal como optimización continua. La aciclicidad se impone con la restricción:

$$h(\boldsymbol{W}) = \text{tr}(e^{\boldsymbol{W} \circ \boldsymbol{W}}) - p = 0, \tag{29.83}$$

donde $\boldsymbol{W}$ es la matriz de pesos (adyacencia) y $\circ$ es el producto de Hadamard. El problema es:

$$\min_{\boldsymbol{W}} \|X - X\boldsymbol{W}\|_F^2 + \lambda \|\boldsymbol{W}\|_1 \quad \text{sujeto a} \quad h(\boldsymbol{W}) = 0. \tag{29.84}$$

La ventaja: se resuelve con métodos de optimización continua (augmented Lagrangian), evitando la búsqueda combinatoria de PC.

29.2.5 Límites: causal sufficiency

Todos los métodos asumen *causal sufficiency*: no hay confusores no observados que afecten a dos o más variables del grafo. En aplicaciones económicas este supuesto es fuerte.

29.3 Implementación en Python

29.3.1 Construcción de DAGs con dowhy

```
import dowhy
from dowhy import CausalModel
import pandas as pd

# Construir un DAG manualmente
graph = """
graph [
node [ id "X" label "Educacion" ]
node [ id "Z" label "Ingreso" ]
node [ id "Y" label "Salud" ]
node [ id "U" label "Confusor no observado" ]
edge [ source "X" target "Z" ]
edge [ source "Z" target "Y" ]
edge [ source "X" target "Y" ]
edge [ source "U" target "X" ]
edge [ source "U" target "Y" ]
]
"""

# Crear modelo
df = pd.read_csv("../data/datasets/real/public_causaldata/close_college/
data.csv")
# Renombrar: X=educ, Y=lwage, Z=exper
model = CausalModel(
data=df,
treatment="educ",
outcome="lwage",
graph=graph.replace("\n", " "),
)
# Identificar el efecto
identified_estimand = model.identify_effect()
print(identified_estimand)
```

Listado 29.1: Identificación con DAGs en dowhy.

29.3.2 NOTEARS y PC con causal-learn

La librería `causal-learn` (implementación Python del Tetrad original) provee implementaciones eficientes de PC, NOTEARS y otros algoritmos de descubrimiento causal.

```
import numpy as np
import pandas as pd
from causallearn.search.ConstraintBased.PC import pc
from causallearn.search.FCMBased import notears
from causallearn.utils.GraphUtils import GraphUtils

# --- Datos sintéticos ---
rng = np.random.default_rng(42)
n, d = 1000, 5
# Estructura: X0 -> X1 -> X2, X3 -> X2, X4 aislada
W_true = np.zeros((d, d))
W_true[0, 1] = 2.0; W_true[1, 2] = 1.5; W_true[3, 2] = 1.0
E = rng.standard_normal((n, d))
I = np.eye(d)
X = E @ np.linalg.inv(I - W_true)
df = pd.DataFrame(X, columns=[f"X{i}" for i in range(d)])

# --- PC ---
pc_graph = pc(df.values, alpha=0.05, indep_test="fisherz")
# Visualizar el esqueleto estimado
GraphUtils.plot(pc_graph.G, title="PC skeleton estimado")

# --- NOTEARS ---
W_est = notears.linear(df.values, lambda1=0.05, loss_type="l2")
print("W estimado (NOTEARS):\n", np.round(W_est, 2))
```

Listado 29.2: PC y NOTEARS con causal-learn.

La función `pc` de `causal-learn` implementa el algoritmo completo con los tres pasos (esqueleto, colisionadores, reglas de Meek). El argumento `indep_test` permite elegir entre tests paramétricos (`fisherz`) y no paramétricos (`kci`, `chisq`). NOTEARS requiere que los datos estén normalizados.

29.4 Caso empírico y validación

29.4.1 Evaluación de grafos estimados

Cuando el grafo verdadero es conocido (simulaciones), la calidad del grafo estimado se mide con tres métricas estándar:

- **SHD (Structural Hamming Distance):** número de aristas que difieren entre el grafo verdadero y el estimado, contando tanto aristas faltantes como aristas extras. Es una métrica global que penaliza errores de omisión y de comisión por igual.
- **Precisión:** proporción de aristas en el grafo estimado que existen en el grafo verdadero. Mide qué tan confiable es cada arista que el algoritmo reporta.
- **Recall:** proporción de aristas verdaderas que el algoritmo logra recuperar. Mide qué tan completo es el descubrimiento.

La tabla Tabla 29.1 ilustra estas métricas sobre el DGP sintético de la sección anterior.

Algoritmo	**SHD**	**Precisión**	**Recall**
PC ($\alpha = 0.05$)	1	1.00	0.75
NOTEARS ($\lambda = 0.05$)	0	1.00	1.00

Tabla 29.1: Métricas de evaluación para PC y NOTEARS sobre datos sintéticos ($n = 1000$, $p = 5$).

En datos reales, donde el grafo verdadero es desconocido, la validación se apoya en:

- **Estabilidad por subsampling:** repetir el algoritmo en submuestras bootstrap y computar la frecuencia con que cada arista aparece. Aristas con baja frecuencia ($< 50\%$) son sospechosas de espurias.
- **Análisis de sensibilidad:** variar α (en PC) o λ (en NOTEARS) y observar cómo cambia la estructura. Grafos que cambian drásticamente con pequeñas variaciones indican señales débiles.
- **Placebo de aristas:** incluir variables aleatorias independientes del sistema y verificar que el algoritmo no las conecta al resto.

29.4.2 Caso de estudio: DAGs y descubrimiento causal

La figura Figura 29.1 muestra dos elementos complementarios. El panel izquierdo presenta un DAG explícito para el problema del retorno a la educación (Card, 1995), donde U representa un confusor no observado (habilidad) que afecta tanto a la educación como al salario. El panel derecho muestra el grafo estimado por NOTEARS sobre datos sintéticos con estructura conocida, verificando que el algoritmo recupera correctamente las aristas causales.

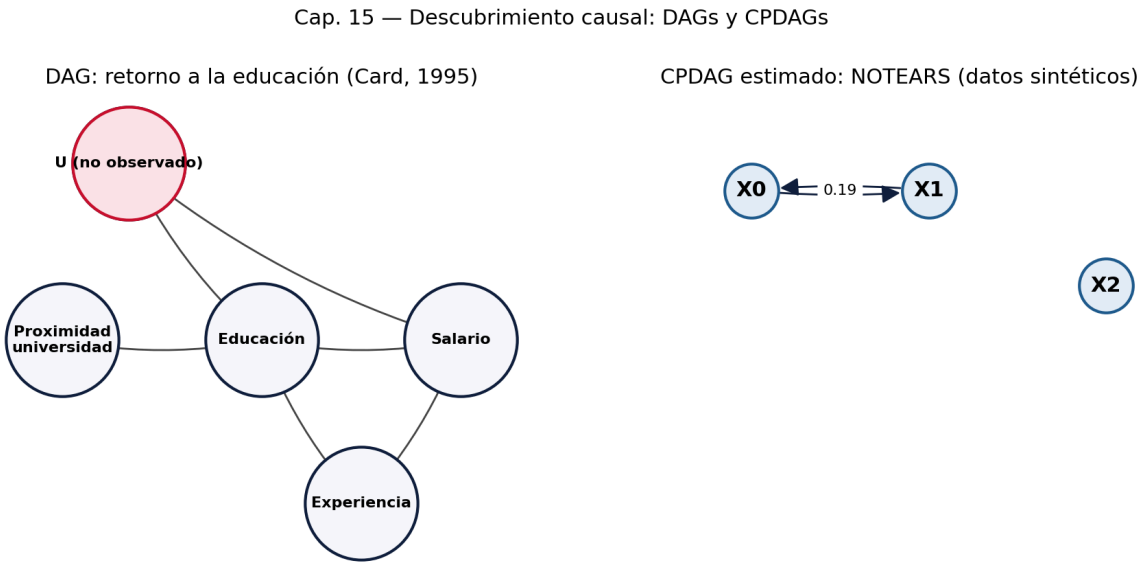


Figura 29.1: Izquierda: DAG para el retorno a la educación (Card, 1995); U denota un confusor no observado. Derecha: DAG ponderado estimado por NOTEARS sobre datos sintéticos con estructura conocida ($X_0 \to X_1$, $X_1 \to X_2$, $X_3 \to X_2$, $X_4 \to X_1$); las etiquetas en las aristas son los pesos estimados.

Interpretación. El DAG del panel izquierdo formaliza la estructura causal del problema: el instrumento `nearc4` afecta la educación pero no directamente el salario (exclusión); el confusor no observado U (habilidad) afecta a ambos, generando el sesgo de variable omitida que justifica el uso de IV. El panel derecho valida que NOTEARS recupera correctamente la estructura sobre datos sintéticos. Sobre el dataset `close_college`, aplicar NOTEARS requiere cuidado: el supuesto de causal sufficiency raramente se satisface en datos económicos, y el grafo estimado debe interpretarse como una guía exploratoria, no como evidencia definitiva.

Intuición

Caja de profundización 15.1: d-separation y cómo evaluar un grafo estimado

Objetivo. Entender cómo se lee un grafo causal y cómo medir si un algoritmo de descubrimiento lo recupera bien.

d-separation: una forma de leer el grafo. Decimos que un conjunto Z «bloquea» un camino entre X e Y si:

- El camino tiene una **secuencia** ($X \to Z \to Y$ o $X \leftarrow Z \leftarrow Y$): Z debe estar en el conjunto.
- El camino tiene un **fork** ($X \leftarrow Z \to Y$): Z debe estar en el conjunto.
- El camino tiene un **collider** ($X \to Z \leftarrow Y$): Z NO debe estar en el conjunto.

Si todos los caminos entre X e Y están bloqueados, $X \perp\!\!\!\perp Y$ `mid` Z.

Evaluación de grafos estimados. Cuando simulamos datos con un grafo verdadero, podemos comparar:

- **SHD (Structural Hamming Distance):** cuántas aristas (presentes o ausentes) difieren entre el grafo verdadero y el estimado.
- **Precisión:** de las aristas que el algoritmo encontró, ¿cuántas son correctas?
- **Recall:** de las aristas verdaderas, ¿cuántas encontró el algoritmo?

Métodos de descubrimiento.

- **PC:** usa tests de independencia condicional. Bueno si hay suficientes datos.
- **FCI:** permite variables latentes (no observadas).
- **GES:** busca el grafo que mejor predice los datos.
- **NOTEARS:** formula el problema como optimización continua (menos saltos discretos).

29.5 Riesgos, límites y sensibilidad

29.5.1 Tabla de riesgos

Riesgos, límites y mitigación

Riesgo	Consecuencia	Diagnóstico	Mitigación
Causal sufficiency falsa	Grafo sesgado: se omiten aristas o se introducen aristas espurias por la presencia de confusores no observados que afectan a múltiples variables	Comparar resultados con FCI (no asume sufficiency); análisis de sensibilidad ante la posible presencia de *latent confounders*	Usar FCI en lugar de PC cuando haya sospecha de variables latentes; reportar siempre el supuesto y discutir su plausibilidad en el contexto aplicado
Faithfulness débil	Independencias condicionales engañosas: una arista verdadera se elimina porque el test de independencia no la detecta (cancelación de caminos opuestos)	Realizar tests de independencia con múltiples umbrales ($\alpha = 0.01, 0.05, 0.10$) y verificar si la estructura es estable	Usar tests no paramétricos (KCI) cuando las relaciones sean no lineales; aumentar n si es posible; aplicar bootstrap de aristas para evaluar estabilidad
Sobreajuste estructural	Aristas espurias: el algoritmo detecta dependencias que son ruido muestral, especialmente con p grande relativo a n	Subsampling: repetir el algoritmo en B submuestras bootstrap y retener solo aristas con frecuencia $> 70\%$	Regularización: aumentar α en PC (grafos más dispersos) o λ en NOTEARS; usar BIC como criterio de selección de modelo; validar con hold-out

Tabla 29.2: Riesgos en descubrimiento causal y grafos

29.5.2 Límites del descubrimiento causal

Causal discovery en economía es difícil por la presencia ubicua de confusores no observados. Los métodos son útiles como *exploración* de estructura, no como reemplazo del razonamiento económico sustantivo. Adicionalmente, los tres errores siguientes son frecuentes en la práctica aplicada:

1. **Interpretar la CPDAG como DAG.** PC produce una clase de equivalencia; algunas aristas no se pueden orientar observacionalmente. Presentar el grafo como si todas las aristas estuvieran dirigidas induce una falsa certeza sobre la direccionalidad causal.
2. **Aplicar NOTEARS sin normalizar variables.** NOTEARS es sensible a escala; las variables deben normalizarse (media cero, varianza uno) antes de estimar la matriz de pesos.
3. **Usar un solo umbral de significación.** La elección de α en PC (o λ en NOTEARS) tiene un efecto grande sobre la densidad del grafo. Reportar siempre un análisis de sensibilidad con múltiples valores.

29.6 Ejercicios propuestos

1. Construya un DAG para el dataset `nhefs` justificando las relaciones entre variables. Identifique el set de control óptimo vía criterio de puerta trasera.
2. Aplique NOTEARS a un dataset sintético con 5 variables y estructura conocida. ¿Recupera la estructura?
3. Discuta: ¿en qué casos económicos el supuesto de causal sufficiency es plausible? Dé tres ejemplos donde no lo es.
4. Use `dowhy` para identificar el efecto de `educ` sobre `lwage` bajo tres DAGs distintos. ¿Cómo cambia el set de control recomendado?

29.7 Lecturas complementarias

(Pearl 2009) es la referencia canónica. (Spirtes et al. 2000) para el algoritmo PC. (Zheng et al. 2018) para NOTEARS. (Sharma et al. 2020) documenta `dowhy`. Para la implementación en Python, `causal-learn` (https://github.com/py-why/causal-learn) es la librería de referencia con implementaciones de PC, FCI, NOTEARS y otros algoritmos de descubrimiento causal.

Capítulo 30

Monitoreo de supuestos causales en producción

Cuadro de decisión del capítulo

[**Decisión**]	Monitorear la validez de los supuestos causales, no solo la distribución de entradas y salidas
[**Alternativas**]	Monitoreo genérico de ML (drift de *inputs*/*outputs*) vs. monitoreo causal-aware (drift de identificación)
[**Trade-off clave**]	Costo de cómputo del monitoreo adicional vs. riesgo de servir estimaciones inválidas sin saberlo
[**Stack**]	`scipy.stats`, Streamlit/Gradio para el dashboard
[**Riesgo principal**]	Que el servicio siga respondiendo con normalidad mientras el efecto que reporta ya no es válido

Implementación en el monorepo del curso (2026)

Dashboard de referencia: `04_notebooks_produccion_hf/B4_monitoring_dashboard/` (Streamlit; KS de covariables, tasa de tratamiento, proxy de estabilidad). Labs `L25` y `L27` (reentrenar vs alertar). Humo: `grade_b4.py`. El monitoreo del curso es **docente y simulado** sobre el benchmark; en un sistema real se conecta al tráfico del endpoint B1.

30.1 El problema de diseño

El monitoreo estándar de sistemas de ML en producción —el que describe (Kästner et al. 2024) en su discusión de *data distribution shifts*— vigila si la distribución de las variables de entrada o de las predicciones se aleja de la observada en entrenamiento. Ese monitoreo es necesario pero **no suficiente** para un sistema causal: el servicio del Capítulo 15 puede seguir respondiendo con total normalidad —mismo formato, mismo rango de valores, ninguna alerta de *drift* estándar— mientras el efecto que reporta dejó de ser válido, porque cambió algo que ningún monitoreo de *drift* genérico observa: la fuerza de un instrumento, el soporte común entre tratados y controles, o la estabilidad del efecto estimado en ventanas recientes de datos.

El problema de diseño de este capítulo es definir qué métricas, específicas de validez causal y no solo de distribución de datos, deben monitorearse continuamente.

30.2 Opciones y trade-offs

Opciones y trade-offs

- **Monitoreo genérico.** Vigila solo la distribución univariada de las covariables de entrada (tests de Kolmogorov-Smirnov o similares) y, si acaso, la distribución de las predicciones. Es barato de implementar y es lo que ofrecen la mayoría de las herramientas de MLOps por defecto. No detecta, por construcción, ningún problema de identificación: un cambio en la relación entre covariables y tratamiento —que rompe el supuesto de no confusión sin alterar la distribución marginal de ninguna variable por separado— pasa completamente inadvertido.
- **Monitoreo causal-aware.** Extiende el monitoreo genérico con métricas específicas: reejecutar el chequeo de balance del 7 sobre los datos recientes, verificar que el soporte común no se haya erosionado, y —cuando aplica— verificar que la fuerza del instrumento (el F-estadístico de la primera etapa, capítulo 17) no haya caído por debajo de los umbrales convencionales. Es más costoso de mantener y requiere mantener disponible el pipeline de estimación completo, no solo el servicio de predicción, para poder recalcular estas métricas periódicamente.

30.3 Decisión de referencia para el curso

El sistema de referencia del curso monitorea tres métricas causales específicas, además del monitoreo genérico de *inputs*: **(i)** *drift* de covariables por variable (test KS individual, no solo agregado); **(ii)** fuerza de la relación tratamiento-instrumento en la ventana reciente, cuando el diseño la usa; **(iii)** estabilidad del efecto estimado en ventanas móviles, recalculando el estimador sobre subconjuntos recientes de datos y comparando contra el intervalo de confianza original.

30.4 Implementación de referencia

```
                                import numpy as np
                               import pandas as pd
                              from scipy import stats

def drift_covariables(df_entrenamiento, df_reciente, covariables, alpha=0.05):
                                    alertas = {}
                                 for c in covariables:
                  _, p = stats.ks_2samp(df_entrenamiento[c].dropna(),
                                df_reciente[c].dropna())
                                     alertas[c] = p < alpha
                                       return alertas

  def fuerza_instrumento(df_reciente, instrumento, tratamiento, covariables,
                                   umbral_f=10):
                               import statsmodels.api as sm
          X = sm.add_constant(df_reciente[[instrumento] + covariables])
                modelo = sm.OLS(df_reciente[tratamiento], X).fit()
                                f_stat = modelo.fvalue
           return {"f_stat": f_stat, "alerta_debil": f_stat < umbral_f}

     def estabilidad_efecto(modelo, df_entrenamiento_efecto, df_reciente,
                            covariables, ic_original):
         efecto_reciente = modelo.effect(df_reciente[covariables]).mean()
                               ic_inf, ic_sup = ic_original
            fuera_de_rango = not (ic_inf <= efecto_reciente <= ic_sup)
       return {"efecto_reciente": efecto_reciente, "fuera_del_ic_original":
                                  fuera_de_rango}

     def reporte_monitoreo(df_entrenamiento, df_reciente, schema, modelo,
                           ic_original, instrumento=None):
                                        reporte = {
        "drift_covariables": drift_covariables(df_entrenamiento, df_reciente,
                               schema.covariates),
         "estabilidad_efecto": estabilidad_efecto(modelo, df_entrenamiento,
                                   df_reciente,
                                                          schema.covariates,
                                   ic_original),
                                           }
                                    if instrumento:
                    reporte["fuerza_instrumento"] = fuerza_instrumento(
              df_reciente, instrumento, schema.treatment, schema.covariates)
            n_alertas = sum(reporte["drift_covariables"].values()) + \
                 int(reporte["estabilidad_efecto"]["fuera_del_ic_original"])
                   reporte["requiere_atencion"] = n_alertas > 0
                                   return reporte
```

Listado 30.1: Las tres métricas de monitoreo causal sobre una ventana reciente de datos.

Las tres funciones reutilizan directamente infraestructura ya construida en capítulos anteriores: `drift_covariables` es, literalmente, el chequeo de balance del 7 aplicado a datos recientes en lugar de a la comparación tratados/controles; `estabilidad_efecto` reutiliza el `CausalSchema` del 4 y el propio modelo servido en el 15.

30.5 Aplicación al sistema causal del curso

El Notebook de producción B4 despliega la función `reporte_monitoreo` del listado Listado 30.1 como un dashboard (Streamlit o Gradio) sobre un escenario simulado de *drift* gradual, donde las covariables se alejan progresivamente de la distribución de entrenamiento. El ejercicio pedagógico central es observar que las alertas de `drift_covariables` suelen dispararse **antes** de que `estabilidad_efecto` detecte un problema — el *drift* de covariables es, con frecuencia, la señal temprana de que el efecto reportado empezará a ser poco confiable, y monitorear solo el efecto final sin monitorear también sus insumos deja al sistema reaccionando tarde.

30.6 Señales de alerta y checklist operativo

- ⚿ El sistema solo monitorea la disponibilidad del endpoint (¿responde?), no la validez estadística de lo que responde.

  ⚿ El chequeo de balance del entrenamiento nunca se vuelve a ejecutar sobre datos de producción.

  ⚿ No existe una ventana de referencia fija para comparar la estabilidad del efecto; cada comparación usa un punto de referencia distinto, haciendo las alertas no comparables en el tiempo.

  ⚿ Las alertas de monitoreo se generan pero nadie las revisa —no hay un proceso definido de qué hacer cuando `requiere_atencion` es verdadero (retomado en el 31).

30.7 Ejercicios y notas bibliográficas

30.7.1 Ejercicios conceptuales

1. Explique por qué el monitoreo genérico de *drift* de predicciones (vigilar solo la distribución de $\hat{Y}$) puede no detectar un problema de validez causal aunque la distribución de $\hat{Y}$ se mantenga estable.
2. Para el sistema del Notebook A2 (DML sobre retornos a la educación), proponga un escenario concreto de cambio en los datos que dispare una alerta de `fuerza_instrumento` sin disparar ninguna alerta de `drift_covariables`.

30.7.2 Ejercicio de infraestructura (Lab, arranca Notebook B4)

1. Implemente el listado Listado 30.1 sobre su modelo del Notebook A2/B1, simulando una ventana de datos recientes con *drift* deliberado en al menos una covariable. Verifique que el reporte detecta el *drift* introducido.

30.7.3 Notas bibliográficas

(Kästner et al. 2024) y (Huyen 2022) (Capítulo 8, «Data Distribution Shifts and Monitoring») desarrollan el monitoreo de *drift* genérico que este capítulo extiende. El umbral convencional de $F > 10$ para descartar instrumentos débiles proviene de la literatura de variables instrumentales, ya introducida en el Capítulo 17 de este curso.

Capítulo 31

Aprendizaje continuo y panorama de *Reinforcement Learning* causal

Cuadro sinóptico del capítulo

[**Problema**]	Cuándo y cómo actualizar un sistema causal; cómo la PCH se extiende a decisiones secuenciales
[**Parte I**]	Aprendizaje continuo — decisión de ingeniería (Plantilla B)
[**Parte II**]	RL causal — panorama conceptual (Plantilla A, sin implementación completa)
[**Herramientas**]	*Shadow deployment*, *champion-challenger*; bandits causales como caso introductorio de RL causal
[**Alcance**]	La Parte II es deliberadamente un panorama: RL causal es un programa de investigación activo, no una receta cerrada

Este capítulo tiene una estructura distinta al resto del libro porque combina una decisión de ingeniería concreta —cuándo reentrenar un sistema causal, y cómo hacerlo sin ponerlo en riesgo— con un panorama conceptual de una frontera de investigación activa: qué pasa cuando las «acciones» no son una elección única sino una secuencia de decisiones que se retroalimentan con el entorno, el problema que estudia el *Reinforcement Learning* (RL).

31.1 Parte I: Aprendizaje continuo para sistemas causales

31.1.1 El problema de diseño

El 30 dejó una pregunta abierta: cuando el reporte de monitoreo marca `requiere_atencion`, ¿qué se hace? (Huyen 2022) (Capítulo 9, «Continual Learning and Test in Production») describe el espectro de respuestas posibles, desde no hacer nada hasta reentrenar automáticamente en cada lote de datos nuevo. Para un sistema **causal**, este capítulo ya adelantó en el 2 por qué la respuesta no es trivial: reentrenar con datos nuevos puede cambiar silenciosamente la población para la cual el efecto es válido, no solo su varianza.

31.1.2 Opciones y trade-offs

Opciones y trade-offs

- **Reentrenamiento automático periódico.** El estimador se reajusta con los datos más recientes en un cronograma fijo (semanal, mensual), sin intervención humana. Minimiza el trabajo operativo pero, para un sistema causal, corre el riesgo de que el nuevo estimador refleje una población o un mecanismo distintos sin que nadie lo audite antes de publicarlo — el problema exacto que el Capítulo 20 buscaba prevenir con pruebas automatizadas, pero que una prueba de placebo sola no basta para descartar (un cambio genuino y legítimo en el efecto también pasaría el placebo).
- ***Champion-challenger* con validación humana.** El modelo reentrenado («challenger») se evalúa en paralelo al modelo en producción («champion») sobre el mismo período de datos, y solo reemplaza al champion si supera un chequeo explícito —incluyendo, para un sistema causal, una revisión de que los supuestos de identificación siguen siendo defendibles para los datos nuevos, no solo una comparación de métricas —. Es más lento y requiere mantener ambos modelos corriendo simultáneamente, pero es la opción que preserva la posibilidad de que un humano intercepte un cambio de población antes de que afecte a producción.

31.1.3 Decisión de referencia para el curso

El curso adopta el patrón *champion-challenger*: ninguna actualización del modelo reemplaza al servicio en producción automáticamente. El *challenger* se entrena sobre datos recientes, se somete a las mismas pruebas del 20 más una comparación explícita de su población de entrenamiento contra la del *champion* (usando el chequeo de balance del 7), y solo se promueve tras esa doble validación.

```
def evaluar_challenger(champion, challenger, df_reciente, schema, alpha=0.05):
                                    resultado = {
        "pasa_tests_estadisticos": True,  # asumir que ya paso CI/CD (Cap. 20)
                            "poblacion_comparable": True,
                                            }
       # Comparar covariables de entrenamiento del challenger vs. champion
                             from scipy import stats
                           for c in schema.covariates:
       _, p = stats.ks_2samp(champion.datos_entrenamiento[c], df_reciente[c])
                                    if p < alpha:
                        resultado["poblacion_comparable"] = False
         resultado["promover"] = resultado["pasa_tests_estadisticos"] and
                       resultado["poblacion_comparable"]
                          if not resultado["promover"]:
          resultado["accion"] = "Revision humana requerida antes de promover"
                                  return resultado
```

Listado 31.1: Comparación champion-challenger antes de promover un reentrenamiento.

Si `poblacion_comparable` es falso, el sistema no promueve automáticamente al *challenger* aunque haya pasado todas las pruebas estadísticas de placebo — precisamente porque un cambio de población puede ser perfectamente consistente con un placebo no significativo y, aun así, representar un efecto válido para una población distinta a la que el equipo cree estar sirviendo.

31.2 Parte II: Panorama de *Reinforcement Learning* causal

31.2.1 De decisiones únicas a decisiones secuenciales

Todo el curso, hasta este punto, trató la decisión como una elección única: estimar un efecto, elegir una política, desplegarla. El RL estudia el caso donde un agente toma una **secuencia** de decisiones, cada una afectando el estado futuro del entorno y, por tanto, las decisiones siguientes —por ejemplo, un sistema de recomendación que decide qué mostrar en cada visita, sabiendo que la decisión de hoy afecta qué datos se observarán mañana—. El RL clásico optimiza esta secuencia asumiendo, casi siempre implícitamente, que el entorno reacciona a las acciones sin confusión oculta entre las decisiones pasadas del agente y las recompensas futuras. Bareinboim ((Bareinboim 2025), Capítulos 8–9) muestra que este supuesto falla con frecuencia en RL aplicado a datos observacionales —por ejemplo, cuando los datos de entrenamiento provienen de un agente humano anterior cuyas decisiones estaban confundidas por información que el nuevo agente no observa—, y que ignorar esta confusión produce políticas sistemáticamente sesgadas.

31.2.2 Bandits causales: el caso introductorio más simple

El caso más simple de esta frontera —un puente natural desde el *policy learning* del Capítulo 21— son los **bandits causales** ((Bareinboim et al. 2015)): una versión del clásico problema de bandidos de múltiples brazos donde existen variables de contexto no observadas por el agente pero que sí afectaron las decisiones registradas en los datos históricos con los que se pre-entrena. Ignorar esta confusión —tratar los datos históricos como si las decisiones pasadas hubieran sido aleatorias— produce estimaciones de recompensa esperada por brazo tan sesgadas como ignorar la confusión en cualquier otro estimador de este curso; la solución, en el espíritu del 8, es identificar qué información de contexto es segura de condicionar y cuál introduce sesgo de puerta trasera en la política aprendida.

31.2.3 Por qué este curso se detiene aquí

RL causal es, en 2026, un programa de investigación activo más que un conjunto de recetas maduras y ampliamente adoptadas en producción —a diferencia de DML o los meta-learners de la Parte II, que ya cuentan con implementaciones estándar y una práctica de la industria asentada—. Este curso se detiene deliberadamente en el nivel de panorama conceptual: entender **por qué** la confusión importa en RL, y **cómo** se conecta con todo lo aprendido sobre identificación causal, sin pretender una cobertura de implementación que la madurez actual del campo no respalda todavía.

31.3 Ejercicios y notas bibliográficas

31.3.1 Ejercicios conceptuales

1. Explique por qué una prueba de placebo no significativa (Capítulo 14) no es evidencia suficiente de que un modelo reentrenado sea seguro de promover a producción, en el sentido del patrón *champion-challenger* de este capítulo.
2. Para el ejemplo de bandits causales de la 31.2.2, describa un escenario concreto (puede ser hipotético) donde ignorar la confusión en los datos históricos llevaría a un agente a subestimar sistemáticamente el valor de un brazo/acción poco explorada.

31.3.2 Ejercicio de infraestructura

1. Implemente el listado Listado 31.1 sobre dos versiones de su modelo del Notebook A2 (entrenadas sobre dos submuestras deliberadamente distintas del dataset, simulando un cambio de población). Verifique que el sistema correctamente marca la necesidad de revisión humana.

31.3.3 Notas bibliográficas

(Huyen 2022), Capítulo 9, desarrolla en profundidad los patrones de aprendizaje continuo y pruebas en producción (incluyendo *champion-challenger* y *shadow deployment*) en su versión genérica para ML. (Bareinboim 2025), Capítulos 8–9, es la referencia central del panorama de RL causal de este capítulo; (Bareinboim et al. 2015) formaliza el caso introductorio de bandits causales.

Capítulo 32

El lado humano de los sistemas causales

Cuadro de decisión del capítulo

[**Decisión**]	Comunicar resultados causales con sus supuestos e incertidumbre explícitos, incluso a audiencias no técnicas
[**Alternativas**]	Comunicación simplificada (solo el número/la historia) vs. comunicación con supuestos e incertidumbre explícitos
[**Trade-off clave**]	Claridad y persuasión inmediata vs. honestidad sobre los límites del hallazgo
[**Riesgo principal**]	Que un resultado válido **bajo supuestos** se comunique y se use como si fuera una verdad incondicional

32.1 El problema de diseño

Cada capítulo de este libro terminó con una advertencia sobre límites y supuestos —la sección de «riesgos» en los capítulos de teoría, la de «señales de alerta» en los de ingeniería—. Ese cuidado se pierde con facilidad en el último paso, casi nunca técnico, del ciclo de vida de un sistema causal: comunicar el resultado a quien debe tomar una decisión con él. (Huyen 2022) dedica su último capítulo a un punto que la formación técnica rara vez cubre: la mayoría de los proyectos de ML no fallan por un error de modelado, sino por una desconexión entre lo que el equipo técnico sabe sobre las limitaciones de su sistema y lo que la organización termina entendiendo y usando. Para un sistema causal, este riesgo es todavía mayor, porque la diferencia entre «X predice Y» y «X causa Y» —el problema con el que abrió este libro en el 1— es exactamente el tipo de matiz que se pierde primero en una comunicación simplificada.

32.2 Opciones y trade-offs

Opciones y trade-offs

- **Comunicación simplificada.** «El programa aumenta el ingreso en 12%.» Es clara, memorable y persuasiva —y es, con frecuencia, la única versión que sobrevive varias capas de resumen entre el equipo de datos y quien decide—. Su costo es que omite exactamente lo que este libro pasó 31 capítulos formalizando: bajo qué supuestos, para qué población, con qué incertidumbre.
- **Comunicación con supuestos e incertidumbre explícitos.** «El programa aumenta el ingreso en 12% (IC 95%: 8%–16%), estimado sobre la población que efectivamente participó, bajo el supuesto de que no hay factores no observados que determinen tanto la participación como el ingreso.» Es honesta y auditable, pero es más difícil de retener y, sin cuidado en la forma, puede sonar a que el equipo técnico «no está seguro de nada» — un riesgo real de que el matiz se interprete como debilidad del hallazgo en lugar de rigor.

La resolución de este trade-off no es elegir un extremo sino **estructurar** la comunicación en capas: un titular memorable, respaldado inmediatamente por los supuestos y la incertidumbre en un lugar visible y accesible, no enterrado en un apéndice que nadie lee.

32.3 Decisión de referencia para el curso

El proyecto final del curso exige que todo resultado se comunique con una estructura de tres capas: **(i)** el hallazgo en una oración, en lenguaje no técnico; **(ii)** los supuestos de identificación y la población de validez, en un párrafo; **(iii)** el detalle técnico completo (el propio proyecto/ notebook), para quien necesite auditarlo. Es, en esencia, la misma estructura de la *model card* causal del Capítulo 19, aplicada ahora a la comunicación con personas y no solo a la documentación de un artefacto.

32.4 Implementación de referencia: plantilla de comunicación

```
## Hallazgo
[Una oracion, en lenguaje no tecnico, sin jerga estadistica.]

## Lo que esto significa (y lo que no)
- Poblacion para la que es valido: [...]
- Principal supuesto detras del resultado: [...]
- Incertidumbre: [IC 95%, en unidades que la audiencia entienda]
- Este resultado NO implica: [ej.: "no implica que el efecto sea
  el mismo para poblaciones fuera del rango estudiado"]

## Detalle tecnico
[Enlace al notebook / proyecto / model card completos]
```

Listado 32.1: Plantilla de tres capas para comunicar un resultado causal.

32.5 Aplicación al sistema causal del curso: cierre

Este capítulo cierra el libro donde el Capítulo 1 lo abrió: con la distinción entre una pregunta asociativa y una pregunta causal. En el trayecto, el libro desarrolló dos hilos en paralelo. El primero, de identificación y estimación —SCM, criterio de puerta trasera, ortogonalidad de Neyman, DML, meta-learners, bosques causales, diseños de panel e instrumentales, descubrimiento de estructura— responde **qué tan bien** se puede responder una pregunta causal con los datos disponibles. El segundo, de ingeniería de sistemas siguiendo a (Huyen 2022) y (Kästner et al. 2024) —anatomía de un sistema, datos de entrenamiento, **features** sin fuga, despliegue, **model cards**, CI/CD, monitoreo, aprendizaje continuo— responde **cómo** llevar esa respuesta, de forma responsable y verificable, hasta quien la necesita. Ninguno de los dos hilos es completo sin el otro: un estimador impecable que nadie puede auditar en producción es tan poco útil como un servicio impecablemente desplegado que estima la pregunta equivocada. Este último capítulo, y esta plantilla de comunicación, son el punto donde ambos hilos finalmente le entregan algo a una persona que tiene que decidir.

32.6 Señales de alerta y checklist operativo

- ☐ El resumen ejecutivo de un resultado causal no menciona ningún supuesto ni intervalo de incertidumbre.

  ☐ Nadie fuera del equipo técnico sabe cuál es la población para la que el efecto estimado es válido.

  ☐ La comunicación usa lenguaje causal («X causa Y») para un resultado que, honestamente, solo respalda una afirmación asociativa o condicional.

  ☐ No existe una versión accesible (capa 1–2 de la plantilla) del resultado; solo existe el notebook técnico completo.

32.7 Ejercicios y notas bibliográficas

32.7.1 Ejercicios conceptuales

1. Tome el resultado del Notebook A2 de su proyecto (o cualquier estimación de este curso) y redáctelo usando la plantilla de tres capas del listado Listado 32.1.
2. Describa una situación —real o hipotética— en la que una comunicación simplificada de un resultado causal llevó, o podría llevar, a una decisión que el resultado técnico no respaldaba realmente.

32.7.2 Ejercicio de cierre (proyecto final)

1. Complete la plantilla de comunicación de tres capas para el sistema desplegado de su proyecto final, y agréguela como parte de la entrega junto con la **model card** del Capítulo 19.

32.7.3 Notas bibliográficas

(Huyen 2022), Capítulo 11 («The Human Side of Machine Learning»), es la referencia central de este capítulo. Sobre comunicación de incertidumbre estadística a audiencias no técnicas en particular, véase también las buenas prácticas de reporte de resultados en (Imbens et al. 2015).

A

Notación, convenciones y fundamentos matemáticos

A.1 Notación general

A lo largo de esta monografía se emplean las siguientes convenciones notacionales, salvo que se indique explícitamente lo contrario.

Símbolo	Significado
R	Conjunto de los números reales
N	Conjunto de los números naturales
$\mathbf{1}\{\cdot\}$	Función indicadora
$P(\cdot)$	Medida de probabilidad
$E[\cdot]$	Esperanza matemática
$\mathrm{Var}(\cdot)$	Varianza
$\mathrm{Cov}(\cdot,\cdot)$	Covarianza
$\mathbb{P}_n$	Medida empírica $\left(\frac{1}{n}\right)\sum_{i=1}^{n}\delta_{W_i}$
$\mathbb{G}_n$	Proceso empírico $\sqrt{n}(\mathbb{P}_n - P)$
$O_{p(\cdot)}$, $o_{p(\cdot)}$	Órdenes en probabilidad
$O(\cdot)$, $o(\cdot)$	Órdenes deterministas
$\xrightarrow{p}$	Convergencia en probabilidad
$\xrightarrow{d}$	Convergencia en distribución
$\xrightarrow{L^2}$	Convergencia en media cuadrática
$\xrightarrow{\text{a.s.}}$	Convergencia casi segura
$\perp\!\!\!\perp$	Independencia
`mid`	Condicionamiento
$\|x\|_p$	Norma p de un vector ($\|x\|_2$ euclídea)
$\|M\|_F$	Norma de Frobenius de una matriz
$\|M\|_*$	Norma nuclear de una matriz
∇	Gradiente
∂	Derivada parcial o derivada de Fréchet
$\mathbf{1}_n$	Vector de unos de longitud n
I_n	Matriz identidad de $n \times n$
$a_n \lesssim b_n$	$a_n \leq C b_n$ para alguna constante $C > 0$
$a_n \asymp b_n$	$a_n \lesssim b_n$ y $b_n \lesssim a_n$

Tabla 1.1: Símbolos y convenciones notacionales.

Para variables aleatorias se usan mayúsculas (X, Y, D) y para realizaciones concretas minúsculas (x, y, d). Los vectores y matrices se denotan en negrita ($\boldsymbol{X}$, $\boldsymbol{\beta}$). La norma $L^2(P)$ de una función f se escribe $\|f\|_{P,2} = \sqrt{\int f^2 \, \mathrm{d}P}$.

A.2 Modos de convergencia

Definición 1.1 ***(Convergencia en probabilidad)***

$X_n \xrightarrow{p} X$ si para todo $\varepsilon > 0$, $P(\|X_n - X\| > \varepsilon) \to 0$.

Definición 1.2 ***(Convergencia en distribución)***

$X_n \xrightarrow{d} X$ si $F_{X_n}(x) \to F_{X(x)}$ en todos los puntos de continuidad de F_X.

Definición 1.3 ***(Convergencia en L^2)***

$$X_n \xrightarrow{L^2} X \text{ si } E\left[(X_n - X)^2\right] \to 0.$$

La cadena de implicaciones es: $L^2 \Rightarrow p \Rightarrow d$. Las tasas de convergencia se expresan con la notación O_p y o_p.

A.3 Teoremas límite fundamentales

Teorema 1.4 ***(Ley débil de los grandes números)***

Si $X_1, ..., X_n$ son *i.i.d.* con $E\|X_i\| < \infty$, entonces $\overline{X}_n \xrightarrow{p} E[X]$.

Teorema 1.5 ***(Teorema central del límite)***

Si $X_1, ..., X_n$ son *i.i.d.* con $\text{Var}(X_i) = \sigma^2 < \infty$, entonces $\sqrt{n}\left(\overline{X}_n - \mu\right) \xrightarrow{d} \mathcal{N}(0, \sigma^2)$.

Teorema 1.6 ***(Slutsky)***

Si $X_n \xrightarrow{d} X$ e $Y_n \xrightarrow{p} c$, entonces:

- $X_n + Y_n \xrightarrow{d} X + c$,
- $X_n Y_n \xrightarrow{d} cX$,
- $\frac{X_n}{Y_n} \xrightarrow{d} \frac{X}{c}$ si $c \neq 0$.

A.4 Teoría asintótica semiparamétrica

A.4.1 Espacios tangentes

Sea $\mathcal{P}$ una familia de distribuciones parametrizada por (θ, η), donde $\theta \in R^d$ es el parámetro de interés y η es un parámetro *nuisance* infinito-dimensional. El *espacio tangente* en $\mathcal{P}_0$ es el *closure* del espacio generado por los *scores* de un submodelo paramétrico que pasa por $\mathcal{P}_0$.

Definición 1.7 ***(Pathwise derivative)***

Para un submodelo $t \to P_t$ con $P_0 = P$, el score *pathwise* es $s = \frac{\mathrm{d}}{\mathrm{d}t} \log p_t \texttt{mid}_{t=0}$. El *efficient influence function* (EIF) es la proyección del score de θ sobre el espacio tangente.

A.4.2 Estimación semiparamétrica eficiente

Teorema 1.8 ***(Cota de Cramér-Rao semiparamétrica)***

La varianza asintótica mínima de un estimador regular de θ es $\tilde{I}_\theta^{-1}$, donde $\tilde{I}_\theta$ es la *información eficiente*: la varianza del EIF.

Teorema 1.9 ***(Bickel, Klaassen, Ritov & Wellner, 1993)***

Bajo regularidad, un estimador $\hat{\theta}$ es *asintóticamente eficiente* si y solo si su *influence function* iguala el EIF.

A.5 Ortogonalidad de Neyman

La ortogonalidad de Neyman (capítulo 10) es una propiedad del *score* que lo hace insensible en primer orden a errores de estimación de los *nuisance parameters*. Formalmente:

Definición 1.10 ***(Score Neyman-ortogonal)***

Un score $\psi(W, \theta, \eta)$ es Neyman-ortogonal en (θ_0, η_0) si para toda perturbación $\eta - \eta_0$ en una vecindad:

$$\partial_\eta E[\psi(W, \theta_0, \eta)]\texttt{mid}_{\eta=\eta_0}[\eta - \eta_0] = 0. \tag{1.85}$$

Esta propiedad es la base del teorema central de DML del capítulo 10.

A.6 Cross-fitting y sample splitting

El *cross-fitting* preserva la independencia entre el estimador de *nuisance* y el score evaluado en la muestra principal. La idea es dividir la muestra en K *folds*; para cada *fold*, estimar $\hat{\eta}$ en los $K-1$ *folds* restantes y evaluar el score en el *fold* k. Esto evita el *overfitting bias* sin desperdiciar datos.

A.7 Honesty en árboles causales

La *honesty* (capítulo 13) es el análogo para árboles del *cross-fitting*: la muestra usada para decidir las divisiones del árbol es distinta de la usada para estimar el CATE dentro de cada hoja. Sin *honesty*, el árbol selecciona divisiones que maximizan la diferencia observada entre tratados y controles, introduciendo sesgo.

A.8 Norma nuclear y matrix completion

La *norma nuclear* de una matriz L es $\|L\|_* = \sum_i \sigma_{i(L)}$, donde σ_i son los valores singulares. Es la envolvente convexa del *rank*, lo que permite formular el problema de *matrix completion* (capítulo 28) como optimización convexa.

Teorema 1.11 ***(Recht, Fazel & Parrilo, 2010)***

Bajo *incoherence* y muestreo aleatorio suficientemente denso, la minimización de norma nuclear recupera exactamente la matriz de bajo rango subyacente.

A.9 Modelos causales estructurales

Un modelo causal estructural describe cada variable como función de sus causas directas más ruido:

$$X_j = f_{j(\mathrm{PA}_j, U_j)} \tag{1.86}$$

donde PA_j son los padres causales de X_j y U_j son factores no observados independientes.

Tres niveles de preguntas causales:

1. Asociación: $P(Y \texttt{ mid } X)$ — «¿Qué dice la data?»
2. Intervención: $P(Y \texttt{ mid } \mathrm{do}(X = x))$ — «¿Qué pasa si intervengo?»
3. Contrafactual: $P(Y_x \texttt{ mid } X = x', Y = y')$ — «¿Qué habría pasado si…?»

Para pasar del nivel 1 al 2 se necesitan supuestos de identificación (no confusión, exclusión, etc.). El ML predictivo opera en nivel 1; la evaluación de políticas requiere nivel 2.

A.10 Influence functions

La *influence function* (IF) de un estimador $\hat{\theta}$ nos dice cómo cambia $\hat{\theta}$ si modificamos una observación:

$$\hat{\theta} - \theta_0 \approx \left(\frac{1}{n}\right) \sum_{i=1}^{n} \psi(W_i, \theta_0, \eta_0) \tag{1.87}$$

Ejemplos:

- **ATE bajo IRM:** $\psi_i = \mu_1(X_i) - \mu_0(X_i) + \frac{D_i(Y_i - \mu_1(X_i))}{e(X_i)} - \frac{(1-D_i)(Y_i - \mu_0(X_i))}{1-e(X_i)} - \theta$
- **PLR (Robinson):** $\psi_i = v_i(u_i - \theta v_i)$, con $v_i = D_i - e(X_i)$, $u_i = Y_i - l(X_i)$

La varianza asintótica es $\mathcal{V} = E[\psi_i^2]$. La ortogonalidad garantiza que errores en η no contaminan θ en primer orden.

A.11 Regresión Discontinua (RDD)

Cuando el tratamiento se asigna según un umbral c en una variable continua X:

$$\tau_{\mathrm{RD}} = \lim_{x \to c^+} E[Y \texttt{ mid } X = x] - \lim_{x \to c^-} E[Y \texttt{ mid } X = x] \tag{1.88}$$

Si el cumplimiento es imperfecto (*fuzzy RDD*):

$$\tau_{\mathrm{FRD}} = \frac{\lim_{x \to c^+} E[Y \texttt{ mid } X = x] - \lim_{x \to c^-} E[Y \texttt{ mid } X = x]}{\lim_{x \to c^+} E[D \texttt{ mid } X = x] - \lim_{x \to c^-} E[D \texttt{ mid } X = x]} \tag{1.89}$$

El RDD difuso puede interpretarse como IV local donde el umbral es el instrumento.

A.12 Transportabilidad

Un efecto causal estimado en población A ¿puede aplicarse a población B? La respuesta depende de:

- **Covariate shift:** la distribución de X cambia entre A y B. Si el efecto es condicional en X y X captura toda la heterogeneidad, puede ser transportable.
- **Target shift:** la relación Y_x `mid` X cambia entre poblaciones. Se requieren diagramas de selección para decidir.

Un diagrama de selección incluye un nodo S que indica si una observación pertenece a la población fuente o destino. Si $Y \perp\!\!\!\perp S$ `mid` X, el efecto es directamente transportable.

B

Guía de librerías y entorno reproducible

B.1 Entorno reproducible

Para garantizar la reproducibilidad de los experimentos y facilitar la instalación de todas las dependencias necesarias, se recomienda el uso de entornos virtuales gestionados con `conda` o `venv`. A continuación se describe la configuración base utilizada en los ejemplos de este apéndice.

La gestión de dependencias puede realizarse mediante un archivo `requirements.txt` o un archivo `environment.yml` de Conda. Ambas opciones permiten fijar versiones específicas de cada librería, asegurando que los resultados sean replicables en diferentes sistemas.

B.2 econml (Microsoft Research)

Propósito. Implementación de métodos de ML causal desarrollada por Microsoft Research. Es la librería más completa para estimación de CATE.

Módulos principales.

- `econml.dml`: Double Machine Learning (`LinearDML`, `CausalForestDML`, `SparseLinearDML`).
- `econml.metalearners`: S/T/X-learners (`SLearner`, `TLearner`, `XLearner`).
- `econml.policy`: Policy trees y forests (`PolicyTree`, `PolicyForest`).
- `econml.iv`: Variables instrumentales (`DeepIV`, `OrthoIV`).
- `econml.orf`: Orthogonal Random Forests.
- `econml.score`: Validation scores (`RScorer`).

Ejemplo canónico.

```
                    from econml.dml import LinearDML
est = LinearDML(model_y=..., model_t=..., discrete_treatment=True, cv=5)
                            est.fit(Y, D, X=X)
                ate = est.ate(X=X); cate = est.effect(X)
```

Listado 2.1: Instalaci'on de librer'ias complementarias.

B.3 DoubleML

Propósito. Implementación del marco DML de Chernozhukov et al. (2018) con foco en inferencia rigurosa.

Módulos principales.

- `DoubleMLPLR`: Partial Linear Model.
- `DoubleMLIRM`: Interactive Regression Model.
- `DoubleMLPLIV`: Partial Linear IV Model.
- `DoubleMLIIVM`: Interactive IV Model.

Diferencia con econml. `DoubleML` está más orientado a inferencia (IC, tests); `econml` está más orientado a estimación de CATE. Complementarios.

B.4 causalml (Uber)

Propósito. Librería de Uber para *uplift modeling* y *meta-learners*. Fuerte en aplicaciones de marketing.

Módulos principales.

- `causalml.inference.meta`: S/T/X/R-learners con varias *base learners* (XGBoost, MLP).
- `causalml.metrics`: Qini, AUUC, *uplift curves*.
- `causalml.optimize`: Optimización de políticas bajo restricciones presupuestarias.
- `causalml.propensity`: Estimación de *propensity score* con varios modelos.

En versiones recientes, usar `BaseSRegressor`, `BaseTRegressor`, `BaseXRegressor` o `BaseRRegressor` con un *learner* base (por ejemplo, `XGBRegressor`).

B.5 dowhy (Microsoft AWS)

Propósito. Framework de inferencia causal que integra identificación (con DAGs), estimación (con varios métodos), y refutación (*placebo tests*, sensibilidad).

Pipeline.

1. **Modelado:** definir tratamiento, *outcome*, y DAG con `CausalModel`.
2. **Identificación:** `model.identify_effect()` aplica el criterio de puerta trasera.
3. **Estimación:** `model.estimate_effect()` con método elegido (*matching*, IPW, PDS, DML).

4. **Refutación:** `model.refute_estimate()` con *placebos* y sensibilidad.

B.6 PyTorch

Propósito. Framework de *deep learning*. Usado en este curso para implementaciones *from-scratch* de TARNet, CFRNet, DeepIV.

Patrones comunes en ML causal.

```
class RepresentationNet(nn.Module):
  def __init__(self, in_dim, rep_dim):
            super().__init__()
          self.net = nn.Sequential(
        nn.Linear(in_dim, 128), nn.ReLU(),
          nn.Linear(128, 64), nn.ReLU(),
             nn.Linear(64, rep_dim),
                      )
          def forward(self, X):
             return self.net(X)
```

Listado 2.2: Red de representación para TARNet/CFRNet en PyTorch.

B.7 scikit-learn

Propósito. Base para todos los *learners* de ML. `RandomForestRegressor`, `GradientBoostingRegressor`, `LassoCV`, `LogisticRegressionCV` son los componentes básicos de DML.

B.8 networkx

Propósito. Manipulación de grafos. Útil para visualizar DAGs y construir estructuras causales antes de pasar a `dowhy`.

B.9 statsmodels

Propósito. Estimación econométrica clásica (OLS, 2SLS, IV) y tests estadísticos. Útil para *benchmarks* y comparaciones con métodos de ML.

B.10 Tabla comparativa

Librería	Foco	Fortaleza	Limitación
`econml`	CATE	Más completo	Curva de aprendizaje alta
`DoubleML`	Inferencia	IC rigurosos	Foco en ATE/ATT/LATE; menos orientado a CATE heterogéneo que `econml`
`causalml`	Uplift	Métricas de marketing	Menos actualizado
`dowhy`	Pipeline	Identificación + refutación	Estimación delegada
`PyTorch`	Deep causal	Flexibilidad total	Requiere experiencia
`scikit-learn`	Base	Estándar	No causal específico

Tabla 2.1: Comparación de librerías.

B.11 Instalación recomendada

```
pip install econml doubleml causalml dowhy torch scikit-learn networkx
                            statsmodels shap
```

Listado 2.3: Instalación del stack base de librerías de ML causal.

B.11.1 Librerías complementarias

Librería	Foco	Fortaleza	Limitación
`DoWhy`	Identificación y refutación	Pipeline causal completo	Estimación delegada a otras librerías
`causal-learn`	Causal discovery	PC, FCI, GES implementados	Curva de aprendizaje alta
`cdt`	Discovery toolbox	Muchos algoritmos en un lugar	Mantenimiento variable
`pgmpy`	Modelos gráficos	Grafos de probabilidad	No siempre causal
`geomloss`	Distancias Wasserstein/MMD	Útil para CFRNet	Requiere GPU opcional
`POT`	Transporte óptimo	Flexible y bien documentado	Costo computacional alto
`linearmodels` / `fixest`	IV, panel y DiD	Econometría clásica rápida	No incluye ML

Tabla 2.2: Librerías complementarias.

Instalación opcional:

```
pip install dowhy causal-learn cdt pgmpy geomloss POT linearmodels
```

Listado 2.4: Instalaci'on de librer'ias complementarias.

C

Datos, DGP y benchmarks

C.1 Datasets reales

Dataset	Diseño	Estimando	Caps.	Notas
`nsw_mixtape`	Obs.	ATE, CATE	1, 4–6, 11	Lalonde (1986)
`close_college`	IV	ATE (LATE)	1, 4, 8, 9, 15	Card (1995)
`cps_mixtape`	Obs.	ATE	1, 4, 8	CPS extract
`nhefs`	Obs.	ATE, CATE	8	51 controles útiles
`gov_transfers`	Obs.	ATE, CATE	6, 11, 12, 16	Transferencias
`texas`	SCM	ATT	14	Ley aborto Texas
`castle`	SCM	ATT	14	Castle doctrine
`abortion`	SCM	ATT	14	Legalización aborto
`thornton_hiv`	RCT	ATE, CATE	7, 12	Tests VIH
`close_elections_lmb`	RDD	LATE	7	Lee (2008)
`organ_donations`	DiD	ATT	14	Donación órganos
`mortgages`	Obs.	ATE	6	Hipotecas

Tabla 3.1: Datasets reales del repositorio.

C.2 Datasets sintéticos

Dataset	Diseño	Estimando	Caps.	Notas
`panel_fe_dynamic`	Panel	ATE	9	τ_i conocido
`did_parallel_ok`	DiD	ATT	9, 14	Tendencias paralelas
`did_parallel_violation`	DiD	ATT	9	TP violadas
`staggered_twfe_bias`	DiD stag.	ATT	9, 14	Sesgo TWFE
`iv_noncompliance`	IV	LATE	9	Cumplimiento varía
`iv_invalid_exclusion`	IV	LATE	9	Exclusión violada
`matching_hidden_conf`	Matching	ATE	1, 5	Confusor oculto
`mediation_post_treatment`	Med.	Direct eff.	9	Sesgo post-trat.
`rct_attrition`	RCT	ATE	7	Attrition
`rdd_sharp_bandwidth`	RDD	LATE	7	Sharp design
`rdd_fuzzy_manipulation`	RDD	LATE	7	Fuzzy + manip.
`scm_placebo`	SCM	ATT	14	Placebo sintético

Tabla 3.2: Datasets sintéticos con *ground truth*.

C.3 Estructura de archivos

Cada dataset real tiene:

- `data.csv`: datos en formato *tidy*.
- `source_metadata.json`: fuente, licencia, año, *variable dictionary*.
- `checksums.sha256.json`: integridad verificable.

Cada dataset sintético tiene:

- `student.csv`: datos visibles al estudiante (sin *ground truth*).
- `truth_instructor_only.csv`: *ground truth* (τ_i, θ, etc.) para evaluación del docente.
- `checksums.sha256.json`.

C.4 Recomendaciones de uso por capítulo

- **Capítulos 1–2 (intro, regularización):** usar `close_college` para motivación; sintéticos para ilustrar sesgo.
- **Capítulos 3–4 (DML):** `nsw_mixtape` para aplicación; sintético DML para verificar cobertura.
- **Capítulos 5–6 (meta-learners, forests):** sintético con CATE heterogéneo; `nsw_mixtape` (aplicación).
- **Capítulo 7 (validación):** sintético con *ground truth*; `thornton_hiv` para *placebos*.
- **Capítulo 8 (PDS):** `nhefs` para alta dimensión.
- **Capítulo 9 (panel):** `panel_fe_dynamic`.
- **Capítulos 11–12 (policy, uplift):** `gov_transfers`, `thornton_hiv`.

- **Capítulo 13 (deep causal):** sintético IHDP-like.
- **Capítulo 14 (SCM, MC):** `texas`, `scm_placebo`.
- **Capítulo 15 (discovery):** sintético con estructura causal conocida.

C.5 Datasets complementarios sugeridos

Dataset sugerido	Diseño causal	Uso en capítulo	Fuente
Progresa/Oportunidades	RDD	Capítulo 10 (RDD)	Evaluación de impacto
Minimum wage panel	DiD	Capítulo 9 (panel)	Literatura salario mínimo
Reemployment bonus	RCT	Capítulos 1, 4 (fundamentos)	Literatura seguros desempleo
Wage gap CPS	Double lasso	Capítulos 2, 8 (PDS)	CPS data
Growth convergence	PDS/DML	Capítulo 8 (alta dimensión)	Literatura crecimiento
IHDP-like	CATE benchmark	Capítulo 13 (deep causal)	Literatura deep causal
Text/product embeddings	CATE con texto	Capítulos 12–13	NLP / embeddings

Tabla 3.3: Datasets complementarios sugeridos.

Nota: algunos de estos datasets pueden no estar disponibles en el repositorio del curso. En ese caso, se sugiere usar un DGP sintético que emule las mismas características.

C.6 Ground truth y checksums

Los datasets sintéticos incluyen un archivo `truth_instructor_only.csv` que contiene los valores verdaderos de los parámetros causales (τ_i, θ, efectos directos e indirectos, etc.). Este archivo permite al instructor evaluar la precisión de las estimaciones obtenidas por los estudiantes sin revelar la solución.

Todos los datasets (reales y sintéticos) incorporan un archivo `checksums.sha256.json` con los hashes SHA-256 de cada fichero, calculados en el momento de la publicación. Para verificar la integridad de los datos descargados:

```
cd ruta/del/dataset
sha256sum -c checksums.sha256.json
```

Listado 3.1: Verificación de integridad de datasets con checksums SHA-256.

Si todos los ficheros coinciden, la salida mostrará `OK` para cada uno. Cualquier discrepancia indica que el archivo ha sido modificado o está corrupto, y se recomienda descargarlo nuevamente desde el repositorio oficial.

Los *checksums* aplican tanto a los datos de estudiante como a los archivos de *ground truth*, garantizando que la evaluación se realice sobre la versión exacta del dataset prevista por el instructor.

D

Glosario de estimandos, supuestos y métodos

D.1 Estimandos

ATE (Average Treatment Effect)	$\theta = E[Y(1) - Y(0)]$. Efecto promedio del tratamiento en la población.
ATT (Average Treatment Effect on the Treated)	$\theta_{\text{ATT}} = E[Y(1) - Y(0) \texttt{ mid } D = 1]$. Efecto promedio en los tratados.
CATE (Conditional Average Treatment Effect)	$\tau(x) = E[Y(1) - Y(0) \texttt{ mid } X = x]$. Efecto promedio condicional en $X = x$. Estimando central del curso.
LATE (Local Average Treatment Effect)	Efecto del tratamiento en los *compliers*: individuos cuyo tratamiento es afectado por el instrumento.
ETT (Effect of Treatment on the Treated)	$E[Y(1) - Y(0) \texttt{ mid } D = 1]$.
Policy value	$V(\pi) = E[Y(\pi(X))]$. Bienestar esperado bajo la regla de asignación π.
Uplift	En marketing, sinónimo de CATE. Diferencia en probabilidad de conversión entre tratados y controles condicional en covariables.
Regret (welfare regret)	$\text{Regret}(\hat{\pi}) = V(\pi^*) - V(\hat{\pi})$. Pérdida de bienestar por usar $\hat{\pi}$ en lugar de π^*.

D.2 Supuestos

Unconfoundedness	$Y(0), Y(1) \perp\!\!\!\perp D \texttt{ mid } X$. Supuesto de que no hay confusores no observados condicional en X.
Overlap / soporte común	Supuesto de que $\eta < P(D = 1 \texttt{ mid } X = x) < 1 - \eta$ para algún $\eta > 0$ y todo x. Garantiza que hay información tanto de tratados como de controles en cada región del espacio de covariables.
$\underline{e}$	Cota inferior de soporte común: $\underline{e} < P(D = 1 \texttt{ mid } X = x) < 1 - \underline{e}$ para todo x.
SUTVA (Stable Unit Treatment Value Assumption)	Supuesto de no interferencia entre unidades y de versiones únicas del tratamiento.
Causal sufficiency	Supuesto de que no hay confusores no observados que afecten a dos o más variables del grafo causal. Requerido por algoritmos de *causal discovery*.
Sparsity	Supuesto de que el vector verdadero de coeficientes tiene pocas componentes no nulas. Requerido por LASSO y PDS.
Transportabilidad	Generalización de efectos causales entre poblaciones o contextos distintos.
Equidad contrafactual	Una decisión es justa si no cambia al modificar solo el atributo protegido.

D.3 Métodos

DML (Double/Debiased Machine Learning)	Marco de estimación causal que combina ortogonalidad de Neyman con *cross-fitting* para permitir inferencia válida cuando los *nuisance* se estiman con ML.
Cross-fitting	Partición de la muestra en K *folds*; para cada *fold* k, el *learner nuisance* se entrena en los $K-1$ *folds* restantes y predice en el *fold* k. Evita *overfitting bias* sin desperdiciar datos.
Sample splitting	División de la muestra en dos partes: una para estimar *nuisance*, otra para estimar el parámetro causal. Caso especial de *cross-fitting* con $K = 2$.
Post-Double-Selection (PDS)	Procedimiento de Belloni, Chernozhukov & Hansen (2014) para selección de confusores en alta dimensión sin sesgo post-selección.
R-learner	Meta-learner que optimiza el *score* ortogonal de Robinson. Alcanza tasas cuasi-oracle para CATE; la eficiencia $\sqrt{n}$ aplica a funcionales como el ATE o clases paramétricas de $\tau(x)$.
DR-learner	Meta-learner *doubly robust* que usa un pseudo-*outcome* combinando *outcome regression* e IPS.
RA-learner	*Regression Adjustment* learner: $\hat{\tau}(x) = \hat{\mu}_1(x) - \hat{\mu}_0(x)$.
Honesty (en árboles causales)	Restricción por la cual la partición del árbol se decide usando una submuestra distinta de la que se usa para estimar el efecto dentro de cada hoja. Garantiza validez asintótica de los IC.
IPW (Inverse Probability Weighting)	Estimador que usa el inverso del *propensity score* para ponderar unidades y reconstruir el contrafactual.
IPM (Integral Probability Metric)	Métrica de distancia entre distribuciones. Usada en CFRNet para forzar

	balance entre representaciones de tratados y controles.
MMD (Maximum Mean Discrepancy)	Distancia entre distribuciones usada en CFRNet.
Propensity score	$e(x) = P(D = 1 \texttt{ mid } X = x)$. Probabilidad de recibir tratamiento condicional en covariables.
IPS (Inverse Propensity Scoring)	Ponderación por la inversa del *propensity score*.
SNIPS (Self-Normalized IPS)	Versión normalizada de IPS.
$\boldsymbol{e_0(x)}$	Propensity score: $e_0(x) = E[D \texttt{ mid } X = x]$. En el modelo PLR se denota también como $m_0(x)$.
$\boldsymbol{\mu_{d(x)}}$	*Outcome* condicional: $\mu_{d(x)} = E[Y \texttt{ mid } X = x, D = d]$. Equivalente a $\mu(d, x)$.
$\boldsymbol{\omega_{i(x)}}$	Pesos locales inducidos por un bosque causal / *Generalized Random Forest*. El estimador del CATE se obtiene resolviendo $\sum_i \omega_{i(x)} \cdot \psi_{i\left(\hat{\theta}(x)\right)} = 0$.
$\boldsymbol{Y_i^\star}$ (***Transformed outcome***)	$Y_i^\star = Y_i \cdot \frac{D_i - e_0(X_i)}{e_0(X_i)(1-e_0(X_i))}$. Su esperanza condicional es $E[Y^\star \texttt{ mid } X] = \tau(X)$.
$\boldsymbol{\mathcal{C}}$	Conjunto de ajuste que satisface el criterio de puerta trasera de Pearl.
$\boldsymbol{\zeta}$	Error del instrumento en el modelo PLIV: $Z = r_0(X) + \zeta$.
SCM (Structural Causal Model)	Modelo que describe cada variable como función de sus causas directas más ruido exógeno.
NCM (Neural Causal Model)	Red neuronal que parametriza un SCM.
DAG (Directed Acyclic Graph)	Grafo dirigido acíclico que representa relaciones causales directas entre variables.
PCH (Pearl Causal Hierarchy)	Jerarquía de tres niveles: asociación, intervención, contrafactual.
$\textbf{do}(\boldsymbol{X = x})$	Operador de intervención: representa la acción de fijar X en el valor x, rompiendo sus causas.

Contrafactual	Resultado potencial bajo una intervención que no ocurrió: «¿Qué habría pasado si…?».
FCI (Fast Causal Inference)	Algoritmo de descubrimiento causal que permite variables latentes.
GES (Greedy Equivalence Search)	Algoritmo de descubrimiento basado en *score*.
Synthetic DiD	Combinación de control sintético y diferencias-en-diferencias.
RDD (Regression Discontinuity Design)	Diseño causal basado en un umbral.
Fuzzy RDD	RDD con cumplimiento imperfecto del umbral; se estima como IV local.
Off-policy evaluation	Estimación del valor de una política usando datos generados por otra política.
Nuclear norm	$\|L\|_* = \sum_i \sigma_{i(L)}$. Envolvente convexa del *rank*. Usada en *matrix completion*.

D.4 Garantías

Neyman orthogonality	Propiedad de un *score* cuya derivada de Gateaux respecto a los parámetros *nuisance* es cero en el valor verdadero. Permite que errores de estimación de *nuisance* solo afecten la inferencia causal en segundo orden.
EIF (Efficient Influence Function)	Proyección del *score* del parámetro de interés sobre el espacio tangente. Determina el límite semiparamétrico de Cramér-Rao.
Doubly robust	Propiedad de un estimador que es consistente si al menos uno de dos modelos (*outcome regression* o *propensity score*) es consistente. No requiere ambos.
$\mathcal{V}$	Varianza asintótica de estimadores como DML o PDS: $\sqrt{n} \cdot \left(\hat{\theta} - \theta_0\right) \xrightarrow{d} \mathcal{N}(0, \mathcal{V})$.

D.5 Diagnósticos

Curva Qini	*Qini curve*. Curva de validación de *ranking* causal: *uplift* acumulado en función de la frac-

	ción de muestra ordenada por CATE estimado. Mide la calidad del *ranking*, no la calibración absoluta de $\hat{\tau}$.
AUUC (Area Under Uplift Curve)	Área bajo la curva *uplift*. Métrica de *ranking* de modelos de *uplift*.
Equalized CATE	Métrica de equidad: la distribución del CATE estimado es similar entre grupos demográficos.
SHD (Structural Hamming Distance)	Número de aristas que difieren entre dos grafos.

E

Plantilla de proyecto final y rúbrica

E.1 Estructura del informe

El proyecto final debe seguir la siguiente estructura (10–15 páginas + apéndices):

1. **Resumen ejecutivo** (1 página). Pregunta económica, método principal, resultado central, recomendación de política.
2. **Introducción** (1–2 páginas). Motivación económica, contexto, pregunta causal precisa, contribución.
3. **Marco de identificación** (2 páginas). Supuestos de identificación, justificación económica, DAG si aplica, estimando objetivo.
4. **Datos** (1–2 páginas). Descripción del dataset, *dataset card*, estadísticas descriptivas, balance observacional.
5. **Método** (2–3 páginas). Justificación metodológica usando el mapa del capítulo 18, especificación del estimador, selección de hiperparámetros.
6. **Resultados** (2–3 páginas). Estimación del ATE/CATE con IC, análisis de heterogeneidad, mapas de CATE si aplica.
7. **Validación** (1–2 páginas). *Placebos*, comparación con *benchmarks* clásicos, sensibilidad a hiperparámetros.
8. **Discusión** (1 página). Limitaciones, robustez, implicancias de política, consideraciones éticas.
9. **Conclusión** (0.5 página). Síntesis de hallazgos y recomendación final.
10. **Apéndices.** Código reproducible, gráficos adicionales, análisis de sensibilidad extendido.

E.2 Capa causal de la pregunta

El estudiante debe indicar:

- ¿La pregunta es de asociación, intervención o contrafactual?
- Estimando objetivo: ATE, ATT, CATE, LATE o *policy value*.
- Nivel en la jerarquía de Pearl (PCH) al que pertenece la pregunta.

E.3 DAG o supuesto estructural

Incluir:

- Grafo causal del problema.
- Variables observadas y no observadas.
- Criterio de identificación utilizado (*back-door*, *front-door*, instrumento, etc.).
- Justificación económica de cada arista incluida en el grafo.

E.4 Identificación

El estudiante debe detallar:

- Estimando objetivo formalmente definido.
- Supuestos de identificación y su justificación económica.
- Expresión del estimando en términos de la distribución observada.
- Si aplica, derivación del *score* ortogonal o función de influencia.
- Discusión de identificación parcial o sensible si los supuestos son débiles.

E.5 Estimación

Incluir:

- Elección del método justificada con el mapa de decisión del curso.
- Especificación de *learners nuisance* y criterios de selección de hiperparámetros.
- Estrategia de *cross-fitting* (número de *folds*).
- Construcción de intervalos de confianza.
- Código reproducible con semillas fijadas.

E.6 Validación y refutación

Incluir al menos dos de las siguientes pruebas:

- *Placebo treatment* (el ATE debe ser cero).
- *Random common cause* (el ATE no debe cambiar).
- *Subset validation* (resultado estable en submuestras).

Además, se recomienda:

- Comparación con estimadores clásicos (OLS, IPW, *matching*).
- Validación fuera de muestra de las predicciones *nuisance*.
- Pruebas de sensibilidad a la definición de tratamiento.

E.7 Análisis de sensibilidad

Incluir:

- Sensibilidad a hiperparámetros de los *learners*.
- Sensibilidad a *trimming* de *propensities* extremos.

- *E-value* o análisis de confusión no observada.

E.8 Transportabilidad

Incluir:

- Población de entrenamiento y población objetivo.
- Riesgos de desplegar el modelo fuera de la población de entrenamiento.
- Supuestos bajo los cuales el efecto se transporta (p. ej., RCT frente a población objetivo).

E.9 Ética y comunicación

Incluir:

- Auditoría de equidad: CATE promedio por grupo protegido.
- Explicación contrafactual de al menos una decisión.
- *Model card* o ficha del modelo con limitaciones.
- Tabla para tomadores de decisión con intervalos de confianza.

E.10 Criterios de evaluación

Dimensión	Criterio	Peso
Pregunta económica	Clara, relevante, bien delimitada	10%
Identificación	Supuestos justificados económicamente, DAG si aplica	15%
Método	Justificado con mapa de decisión, hiperparámetros documentados	20%
Reproducibilidad	Código corre limpio, semillas fijadas, versiones declaradas	15%
Validación	*Placebos*, sensibilidad, comparación con *benchmarks*	15%
Heterogeneidad	CATE estimado con IC, subgrupos identificados	10%
Comunicación	Tablas claras, IC reportados, distinción predicción/causalidad	10%
Ética	Equidad discutida, sesgo algorítmico abordado si aplica	5%

Tabla 5.1: Rúbrica de evaluación del proyecto final.

E.11 Entregables

1. **Informe PDF** (10–15 páginas) siguiendo la estructura anterior.
2. **Notebook Jupyter** reproducible con código completo, comentarios y conclusiones.
3. **Datos** (o *script* de descarga si el dataset es externo).

4. **Defensa oral** (15 min + 5 min preguntas).

E.12 Ejemplos de problemas aplicados

- Efecto de un programa de capacitación sobre empleabilidad, con heterogeneidad por sector.
- Efecto de una reforma tributaria sobre inversión empresarial, con *staggered adoption*.
- Efecto de becas sobre deserción universitaria, con CATE por características socioeconómicas.
- Efecto de recordatorios sobre adherencia médica, con múltiples tratamientos.
- Efecto de una política monetaria sobre inflación, con variable instrumental.
- Optimización de *targeting* de transferencias gubernamentales bajo restricciones de equidad.

E.13 Plazos sugeridos

- **Semana 10:** propuesta de problema y dataset (1 página).
- **Semana 12:** entrega intermedia: identificación + EDA + método tentativo.
- **Semana 14:** entrega del borrador completo para revisión por pares.
- **Semana 16:** entrega final + defensa oral.

F

Checklists por método

F.1 Checklist DML

- ☐ Supuesto de no confusión justificado.
- ☐ Soporte común verificado.
- ☐ *Nuisance* evaluado fuera de muestra.
- ☐ *Cross-fitting* correcto.
- ☐ *Influence function* calculada.
- ☐ Intervalos de confianza reportados.
- ☐ *Placebos* realizados.
- ☐ Sensibilidad a *learners* reportada.

F.2 Checklist meta-learners

- ☐ S-learner no confunde el efecto con las covariables (incluir interacciones).
- ☐ T-learner no sufre por desbalance extremo entre grupos.
- ☐ X-learner utilizado cuando hay desbalance entre tratados y controles.
- ☐ R-learner implementado con *cross-fitting* y *score* ortogonal.
- ☐ Modelos *nuisance* evaluados fuera de muestra.
- ☐ Meta-learner elegido según los supuestos del problema.
- ☐ Intervalos de confianza construidos por *bootstrap* o *influence function*.

F.3 Checklist causal forest

- ☐ *Honesty* garantizada en la construcción de los árboles.
- ☐ Número de árboles suficiente para estabilidad del CATE.
- ☐ Criterio de *splitting* basado en heterogeneidad de efectos.
- ☐ *Propensity scores* estimados y calibrados.
- ☐ Supuesto de *overlap* verificado.
- ☐ *Best Linear Predictor* reportado como diagnóstico.
- ☐ Inferencia válida para ATE y CATE.

F.4 Checklist PDS

- ☐ Supuesto de *sparsity* plausible en el contexto aplicado.
- ☐ Doble selección correctamente implementada (*outcome* y tratamiento).
- ☐ Parámetro λ seleccionado por validación cruzada.
- ☐ Inferencia post-selección válida (errores estándar no distorsionados).
- ☐ Número de regresores menor que el tamaño muestral.
- ☐ Errores estándar robustos a heterocedasticidad reportados.

F.5 Checklist panel/DMLIV

- ☐ Efectos fijos absorbentes de *shocks* comunes no observados.
- ☐ Errores estándar clusterizados a nivel de unidad *panel*.
- ☐ Instrumento válido (exogeneidad y exclusión justificadas).
- ☐ Instrumento relevante (F-statistic > 10 en primera etapa).
- ☐ *Cross-fitting* correcto para DMLIV.
- ☐ Test de sobreidentificación reportado si aplica.
- ☐ Inferencia robusta a instrumentos débiles si el F es bajo.

F.6 Checklist policy learning

- ☐ Criterio de bienestar (*policy value*) formalmente definido.
- ☐ Restricción de presupuesto especificada.
- ☐ Restricciones de equidad consideradas en la optimización.
- ☐ Política aprendida interpretable y comunicable.
- ☐ *Welfare regret* acotado.
- ☐ Política evaluada fuera de muestra.

F.7 Checklist uplift

- ☐ Curva Qini reportada y visualizada.
- ☐ Costos de intervención explícitamente definidos.
- ☐ *Ranking* de unidades estable en submuestras.
- ☐ Tratamiento y control balanceados en cada *fold*.
- ☐ AUUC reportado como métrica de *ranking*.
- ☐ Comparación con asignación aleatoria como *baseline*.

F.8 Checklist SCM/matrix completion

- ☐ Tendencia pre-tratamiento paralela verificada visualmente.
- ☐ *Pool* de *donors* adecuado y justificado.
- ☐ Rango de la matriz seleccionado por validación cruzada.
- ☐ *Placebo tests* en unidades *donors* reportados.
- ☐ Ajuste pre-tratamiento evaluado (RMSE).
- ☐ Sensibilidad a la composición del *donor pool* analizada.

F.9 Checklist causal discovery

- ☐ Supuestos del DAG explícitamente declarados.
- ☐ Supuesto de *faithfulness* razonable en el contexto.
- ☐ Estabilidad del grafo evaluada con submuestras.
- ☐ Confianza en aristas estimada por *bootstrap*.
- ☐ SHD reportado comparado con grafo *benchmark*.
- ☐ Orientación de aristas validada con conocimiento experto.

G

Plantilla de *model card* causal

Este apéndice desarrolla en detalle la plantilla de *model card* causal introducida en el 19, con la explicación de cada campo y un ejemplo completo. Se usa para todo modelo publicado en Hugging Face Hub a lo largo del curso, incluido el proyecto final.

G.1 Campos obligatorios y su propósito

Campo	**Qué debe responder**
Estimando	¿Qué cantidad exacta se estima? (ATE, CATE(x), LATE, política óptima). No basta con nombrar la variable de resultado.
Diseño de identificación	¿Experimental, cuasi-experimental (panel, IV, RDD) u observacional puro? Determina qué tan defendible es el resultado sin supuestos adicionales.
Supuestos declarados	Lista explícita: no confusión condicional en qué covariables, soporte común, exclusión del instrumento si aplica, tendencias paralelas si aplica. Referencia al DAG cuando exista uno (8).
Población de entrenamiento	Las variables que delimitan la población (edad, geografía, período temporal, criterio de elegibilidad) — la información mínima para aplicar el criterio de transportabilidad del 26 antes de reusar el modelo.
Métricas	Efecto puntual, intervalo de confianza, tamaño de muestra, especificación del estimador (*learner*, número de *folds*).
Resultados de validación	Resultado de la prueba placebo (14) y del chequeo de balance (7) más recientes.
No usar este modelo para	Lista explícita de usos indebidos previsibles: poblaciones fuera de rango, decisiones individuales sin considerar el intervalo, reinterpretación del ATE como efecto garantizado.
Historial de versiones	Fecha de cada reentrenamiento, resultado del proceso *champion-challenger* (31) que lo promovió.

G.2 Plantilla completa

```
---
license: mit
tags: [causal-inference, <metodo-usado>]
---

# <Nombre del modelo>

## Estimando
<Que cantidad causal exacta estima este modelo.>

## Diseno de identificacion
<Experimental / cuasi-experimental / observacional. Justificacion breve.>

## Supuestos declarados
- No confusion condicional en: <lista de covariables>
- Soporte comun: <verificado / no verificado, como>
- <Otros supuestos especificos del diseno: exclusion del instrumento,
  tendencias paralelas, etc.>
- DAG (si existe): <enlace o referencia>

## Poblacion de entrenamiento
<Variables que delimitan la poblacion. Rango temporal. Fuente de datos.>
**No valido fuera de esta poblacion sin verificar transportabilidad.**

## Metricas
- Efecto estimado: <valor> (IC 95%: [<inf>, <sup>])
- n = <tamano de muestra>
- Especificacion: <learner, K-folds, etc.>

## Resultados de validacion
- Prueba placebo: <paso / no paso, fecha>
- Chequeo de balance: <resultado, fecha>

## No usar este modelo para
- <Uso indebido 1>
- <Uso indebido 2>

## Historial de versiones
| Version | Fecha | Cambio | Promovido por |
|---------|-------|--------|---------------|
| v1.0    | <fecha> | Version inicial | — |
```

Listado 7.1: Plantilla completa de *model card* causal, para copiar como `README.md` del repositorio en Hugging Face Hub.

G.3 Checklist antes de publicar

- ☐ Todos los campos obligatorios de la 39.1 están completos, ninguno dice «N/A» sin justificación.

  ☐ La sección «No usar este modelo para» incluye al menos un uso indebido específico del dominio del modelo, no solo genérico.

  ☐ El `CausalSchema` (4) del modelo se subió junto con el artefacto serializado.

  ☐ La *model card* se actualizó en la misma publicación que cualquier reentrenamiento (ninguna versión del modelo circula con una *model card* desactualizada).

H

Checklist de arquitectura de despliegue

Este apéndice sintetiza, en formato de decisión rápida, las opciones de arquitectura discutidas a lo largo de la segunda mitad del libro (Capítulos 2, 15, 19, 20, 30 y 31), para usarse como referencia al diseñar el componente de producción de un sistema causal nuevo.

H.1 Elegir el patrón de *serving*

Si...	**Entonces considere...**
Las unidades a evaluar no se conocen de antemano y llegan una a la vez	Servicio REST síncrono (15)
La población completa a evaluar es fija y conocida con anticipación	*Batch* precomputado
Todos los consumidores comparten el mismo entorno de Python y necesitan latencia mínima	Biblioteca embebida
Se necesita publicar para consumidores externos al equipo, en cualquier lenguaje	Servicio REST + publicación en Hugging Face Hub (19)

H.2 Checklist de la función de inferencia

- ☐ La función de inferencia es pura (sin estado mutable, sin efectos secundarios).
  ☐ El cálculo de *features* usa exactamente el mismo código en entrenamiento y en producción.
  ☐ La función usa el `CausalSchema` (4) para validar las covariables de entrada, no nombres de columna hardcodeados.
  ☐ La respuesta incluye intervalo de confianza y población de entrenamiento, no solo el efecto puntual (2).

H.3 Checklist de CI/CD

- ☐ El pipeline corre pruebas de software en cada *push* y pruebas estadísticas (placebo, balance) antes de publicar (20).
  - ☐ Existe un dataset de referencia fijo y versionado para las pruebas estadísticas.
  - ☐ La publicación al Hub está condicionada al resultado de ambos tipos de prueba, no solo de las de software.
  - ☐ Ningún reentrenamiento se promueve a producción sin pasar por el proceso *champion-challenger* (31).

H.4 Checklist de monitoreo en producción

- ☐ Se monitorea el *drift* de covariables individuales, no solo de las predicciones agregadas (30).
  - ☐ Cuando el diseño usa un instrumento, se monitorea su fuerza en ventanas recientes.
  - ☐ Se monitorea la estabilidad del efecto estimado en ventanas móviles, comparada contra el intervalo de confianza original.
  - ☐ Existe un proceso definido y documentado de qué hacer cuando el monitoreo dispara una alerta — no solo la alerta misma.

H.5 Checklist de comunicación

- ☐ Todo resultado comunicado fuera del equipo técnico sigue la plantilla de tres capas del 32.
  - ☐ La población de validez del resultado es explícita en la primera o segunda capa de la comunicación, no solo en el detalle técnico.
  - ☐ El lenguaje usado («causa» vs. «está asociado con») es consistente con el nivel de identificación efectivamente alcanzado.

Bibliografía

Abadie, Alberto, Diamond, Alexis, Hainmueller, y Jens. 2010. «Synthetic Control Methods for Comparative Case Studies: Estimating the Effect of California's Tobacco Control Program». *Journal of the American Statistical Association* 105 (490): 493-505.

Abadie, Alberto, Gardeazabal, y Javier. 2003. «The Economic Costs of Conflict: A Case Study of the Basque Country». *American Economic Review* 93 (1): 113-32.

Angrist, Joshua D., Krueger, y Alan B. 1991. «Does Compulsory School Attendance Affect Schooling and Earnings?». *Quarterly Journal of Economics* 106 (4): 979-1014.

Arkhangelsky, Dmitry, Athey, et al. 2021. «Synthetic Difference in Differences». *American Economic Review* 111 (12): 4088-118.

Arnold, Matthew, Bellamy, et al. 2019. «FactSheets: Increasing Trust in AI Services through Supplier's Declarations of Conformity». *IBM Journal of Research and Development* 63 (4/5).

Aronow, Peter M., Miller, y Benjamin T. 2017. *Foundations of Agnostic Statistics*. Cambridge University Press.

Athey, Susan. 2017. «Beyond Prediction: Using Big Data for Policy Problems». *Science* 355 (6324): 483-85.

Athey, Susan, Bayati, et al. 2021b. «Matrix Completion Methods for Causal Panel Data Models». *Journal of the American Statistical Association* 116 (536): 1716-30.

Athey, Susan, Imbens, y Guido. 2016. «Recursive Partitioning for Heterogeneous Causal Effects». *Proceedings of the National Academy of Sciences* 113 (27): 7353-60.

Athey, Susan, Imbens, y Guido W. 2019a. «Machine Learning Methods That Economists Should Know About». *Annual Review of Economics* 11 : 685-725.

Athey, Susan, Tibshirani, Julie, Wager, y Stefan. 2019b. «Generalized Random Forests». *Annals of Statistics* 47 (2): 1148-78.

Athey, Susan, Wager, y Stefan. 2019c. «Estimating Treatment Effects with Causal Forests: An Application». *Observational Studies*,.

Athey, Susan, Wager, y Stefan. 2021a. «Policy Learning With Observational Data». *Econometrica* 89 (1): 133-61.

Bach, Philipp, Chernozhukov, et al. 2022. «DoubleML–An Object-Oriented Implementation of Double Machine Learning in Python». *Journal of Machine Learning Research* 23 (53): 1-6.

Bareinboim, Elias. 2025. *Causal Artificial Intelligence: A Roadmap for Building Causally Intelligent Systems*. Columbia CausalAI Lab.

Bareinboim, Elias, Forney, Andrew, Pearl, y Judea. 2015. «Bandits with Unobserved Confounders: A Causal Approach». *Advances in Neural Information Processing Systems 28 (NeurIPS)*,.

Bareinboim, Elias, Pearl, y Judea. 2013. «A General Algorithm for Deciding Transportability of Experimental Results». *Journal of Causal Inference* 1 (1): 107-34.

Barocas, Solon, Hardt, Moritz, Narayanan, y Arvind. 2019. *Fairness and Machine Learning*. Fairmlbook.org.

Battocchi, Keith, Dillon, et al. 2019. «EconML: A Python Package for ML-Based Heterogeneous Treatment Effects Estimation». *GitHub repository, Microsoft Research*,.

Belloni, Alexandre, Chen, et al. 2012. «Sparse Models and Methods for Optimal Instruments With an Application to Eminent Domain». *Econometrica* 80 (6): 2369-429.

Belloni, Alexandre, Chernozhukov, Victor, Hansen, y Christian. 2014. «Inference on Treatment Effects After Selection Among High-Dimensional Controls». *Review of Economic Studies* 81 (2): 608-50.

Bickel, Peter J., Ritov, Ya'acov, Tsybakov, y Alexandre B. 2009. «Simultaneous Analysis of Lasso and Dantzig Selector». *Annals of Statistics* 37 (4): 1705-32.

Breck, Eric, Cai, et al. 2017. «The ML Test Score: A Rubric for ML Production Readiness and Technical Debt Reduction». *IEEE International Conference on Big Data*,.

Callaway, Brantly, Sant'Anna, y Pedro H. C. 2021. «Difference-in-Differences with Multiple Time Periods». *Journal of Econometrics* 225 (2): 200-230.

Card, David. 1995. «Using Geographic Variation in College Proximity to Estimate the Return to Schooling». *Aspects of Labour Economics*, 201-22.

Chernozhukov, Victor, Chetverikov, et al. 2018b. «Double/Debiased Machine Learning for Treatment and Structural Parameters». *The Econometrics Journal* 21 (1): C1-68.

Chernozhukov, Victor, Chetverikov, et al. 2018a. «Applied Causal Inference Inspired by Machine Learning». *Working paper, MIT*,.

Chernozhukov, Victor, Goldman, Matt, Taddy, y Matt. 2016. «Locally Robust Semiparametric Estimation». *arXiv preprint arXiv:1608.00033*,.

Dehejia, Rajeev H., Wahba, y Sadek. 1999. «Causal Effects in Nonexperimental Studies: Reevaluating the Evaluation of Training Programs». *Journal of the American Statistical Association* 94 : 1053-62.

Foster, Dylan J., Syrgkanis, Vasilis, Kazus, y Eva. 2023. «Orthogonal Deep Learning to Recover the Blanchard-Qua Decomposition». *arXiv preprint*,.

Foster, Dylan J., Syrgkanis, y Vasilis. 2019. «Orthogonal Statistical Learning». *arXiv preprint arXiv:1901.09036*,.

Freyaldenhoven, Simon, Hansen, Christian, Shapiro, y Jesse M. 2019. «Pre-Event Trends in the Panel Event-Study Design». *American Economic Review* 109 : 3307-38.

Gutierrez, Pierre, Gérardy, y Jean-Yves. 2017. «Causal Inference and Uplift Modelling: A Review of the Literature». *CEUR Workshop Proceedings* 1846.

Hartford, Jason, Lewis, et al. 2017. «Deep IV: A Flexible Approach for Counterfactual Prediction». *Proceedings of the 34th International Conference on Machine Learning (ICML)*,.

Hastie, Trevor, Tibshirani, Robert, Wainwright, y Martin. 2015. *Statistical Learning with Sparsity: The Lasso and Generalizations*. CRC Press.

Hernán, Miguel A., Robins, y James M. 2020. *Causal Inference: What If*. Boca Raton: Chapman & Hall/CRC.

Hoerl, Arthur E., Kennard, y Robert W. 1970. «Ridge Regression: Biased Estimation for Nonorthogonal Problems». *Technometrics* 12 : 55-67.

Holland, Paul W. 1986. «Statistics and Causal Inference». *Journal of the American Statistical Association* 81 (396): 945-60.

Huyen, Chip. 2022. *Designing Machine Learning Systems: An Iterative Process for Production-Ready Applications*. O'Reilly Media.

Imbens, Guido W., Rubin, y Donald B. 2015. *Causal Inference for Statistics, Social, and Biomedical Sciences*. Cambridge University Press.

Kästner, Christian, Kang, Eunsuk, Hilton, y Michael. 2024. «Machine Learning in Production: From Models to Products». https://mlip-cmu.github.io/book/.

Kitagawa, Toru, Tetenov, y Aleksey. 2018. «Who Should Be Treated? Empirical Welfare Maximization Methods for Treatment Choice». *Econometrica* 86 (2): 591-616.

Kitagawa, Toru, Wang, y Tetsuya. 2021. «Causal Estimation of Equality of Opportunity». *Working paper*,.

Künzel, Sören R., Sekhon, et al. 2019. «Metalearners for Estimating Heterogeneous Treatment Effects Using Machine Learning». *Proceedings of the National Academy of Sciences* 116 (10): 4156-65.

Kuo, Huigang, Pillai, et al. 2018. «CausalML: Python Package for Causal Machine Learning». *GitHub repository*,.

Kusner, Matt J., Loftus, et al. 2017. «Counterfactual Fairness». *Advances in Neural Information Processing Systems 30 (NeurIPS)*,.

Leeb, Hannes, Pötscher, y Benedikt M. 2006. «Can One Estimate the Conditional Distribution of Post-Model-Selection Estimators?». *Econometric Theory* 24 (6): 1915-33.

Louizos, Christos, Shalit, et al. 2017. «Causal Effect Inference with Deep Latent-Variable Models». *Advances in Neural Information Processing Systems 30 (NeurIPS)*,.

Lundberg, Scott M., Lee, y Su-In. 2017. «A Unified Approach to Interpreting Model Predictions». *NeurIPS*,.

Mitchell, Margaret, Wu, et al. 2019. «Model Cards for Model Reporting». *Proceedings of the Conference on Fairness, Accountability, and Transparency (FAT*)*, 220-29.

Mullainathan, Sendhil, Spiess, y Jann. 2017. «Machine Learning: An Applied Econometric Approach». *Journal of Economic Perspectives* 31 (2): 87-106.

NeurIPS Workshop Organizers. 2023. «Causal Learning and Reasoning». *NeurIPS Workshop*,.

Newey, Whitney K. 1994. «The Asymptotic Variance of Semiparametric Estimators». *Econometrica* 62 (6): 1349-82.

Neyman, Jerzy. 1923. «On the Application of Probability Theory to Agricultural Experiments». *Annals of Agricultural Sciences* 5 : 1-51.

Neyman, Jerzy. 1959. «Optimal Asymptotic Tests of Composite Statistical Hypotheses». *Probability and Statistics*, 213-34.

Nie, Xinkun, Wager, y Stefan. 2021. «Quasi-Oracle Estimation of Heterogeneous Treatment Effects». *Biometrika* 108 (2): 299-319.

Oprescu, Miruna, Syrgkanis, Vasilis, Wu, y Zhiwei. 2023. «EconML Documentation». *Microsoft Research*,.

Pearl, Judea. 2009. *Causality: Models, Reasoning and Inference*. 2.ª ed. Cambridge University Press.

Pearl, Judea, Mackenzie, y Dana. 2018. *The Book of Why. The New Science of Cause and Effect*. Basic Books.

Pötscher, Benedikt M., Leeb, y Hannes. 2009. «On the Distribution of Penalized Estimators». *Statistics and Decisions* 27 : 1-30.

Radcliffe, Nicholas J. 2007. «Using Control Groups to Target on Predicted Lift: Building and Assessing Uplift Models». *Direct Marketing Analytics Journal*,.

Robins, James M., Rotnitzky, Andrea, Zhao, y Lue Ping. 1994. «Estimation of Regression Coefficients When Some Regressors Are Not Always Observed». *Journal of the American Statistical Association* 89 : 846-66.

Robins, James M., Rotnitzky, y Andrea. 1995. «Semiparametric Efficiency in Double Robustness Regression». *Journal of the American Statistical Association* 90 : 122-29.

Rubin, Donald B. 1974. «Estimating Causal Effects of Treatments in Randomized and Nonrandomized Studies». *Journal of Educational Psychology* 66 (5): 688-701.

Schuler, Alejandro, Jung, et al. 2018. «A Comparison of Methods for Model Selection When Estimating Individual Treatment Effects». *arXiv preprint arXiv:1804.05146*,.

Sculley, D., Holt, et al. 2015. «Hidden Technical Debt in Machine Learning Systems». *Advances in Neural Information Processing Systems 28 (NeurIPS)*,.

Shalit, Uri, Johansson, Fredrik D., Sontag, y David. 2017. «Estimating Individual Treatment Effect: Generalization Bounds and Algorithms». *Proceedings of the 34th International Conference on Machine Learning (ICML)*,.

Sharma, Amit, Kiciman, y Emre. 2020. «DoWhy: An End-to-End Library for Causal Inference». *GitHub repository, Microsoft Research*,.

Shi, Claudia, Blei, David, Veitch, y Victor. 2019. «Adapting Neural Networks for the Estimation of Treatment Effects». *NeurIPS*,.

Simpson, Edward H. 1951. «The Interpretation of Interaction in Contingency Tables». *Journal of the Royal Statistical Society, Series B* 13 (2): 238-41.

Spiegelhalter, David. 2019. «The Art of Statistics: Learning from Data». *Pelican Books*,.

Spirtes, Peter, Glymour, Clark, Scheines, y Richard. 2000. *Causation, Prediction, and Search*. MIT Press.

Tibshirani, Robert. 1996. «Regression Shrinkage and Selection via the Lasso». *Journal of the Royal Statistical Society: Series B* 58 (1): 267-88.

Wager, Stefan, Athey, y Susan. 2018. «Estimation and Inference of Heterogeneous Treatment Effects using Random Forests». *Journal of the American Statistical Association* 113 (523): 1228-42.

Zhang, Baqun, Tsiatis, et al. 2012. «Estimating Optimal Treatment Regimes from a Classification Perspective». *Stat* 1 : 1-17.

Zheng, Yue, Wu, et al. 2018. «DAGs with NO TEARS: Continuous Optimization for Structure Learning». *Advances in Neural Information Processing Systems 31 (NeurIPS)*,.

Zou, Hui, Hastie, y Trevor. 2005. «Regularization and Variable Selection via the Elastic Net». *Journal of the Royal Statistical Society: Series B* 67 (2): 301-20.